

---

## Chapter: C as a High-Level Language and Its History

---

### 1. Introduction

Programming languages are developed to bridge the gap between **human thinking** and **machine execution**.

Among all programming languages, C holds a unique position because it combines the **efficiency of low-level languages** with the **readability and structure of high-level languages**.

C is often described as a **middle-level language**, but in practical and academic classification, it is treated as a **high-level language with low-level capabilities**.

---

### 2. What is a High-Level Language?

A **high-level language (HLL)** is a programming language that:

- Is closer to human language than machine language
- Is easy to read, write, and understand
- Is machine-independent
- Uses English-like keywords and structured syntax

#### Characteristics of High-Level Languages

- Portability across platforms
- Automatic memory management (partially, depending on language)
- Easier debugging and maintenance
- Reduced development time

#### Example (C code – human readable)

```
int sum = a + b;
```

Compared to assembly language:

```
ADD AX, BX
```

---

### 3. Is C a High-Level Language?

Yes, **C is considered a high-level language**, but with special characteristics.

#### Why C is High-Level

- Uses English-like keywords (if, while, return)
- Supports structured programming
- Machine-independent source code
- Supports functions, loops, and data structures

#### Why C is Sometimes Called Middle-Level

- Supports pointers and direct memory access
- Allows low-level manipulation
- Can interact closely with hardware

#### Conclusion

C is a **high-level language with low-level features**, making it extremely powerful and flexible.

---

### 4. Need for the C Programming Language

Before C, programming was done using:

- Machine language
- Assembly language

#### Problems with Early Languages

- Highly machine-dependent
- Difficult to write and debug
- Poor portability
- Complex syntax

There was a strong need for a language that:

- Is efficient like assembly
- Is portable like high-level languages
- Can be used to write system software

This need led to the development of C.

---

## 5. History of the C Programming Language

---

### 5.1 Origin of C

The C programming language was developed by **Dennis Ritchie** in **1972** at **Bell Laboratories**.

C was developed to implement the **UNIX** operating system.

---

### 5.2 Evolution Before C

C did not appear suddenly. It evolved through earlier languages.

#### BCPL (1967)

- Developed by Martin Richards
- Used for system programming
- Lacked data types

#### B Language (1970)

- Developed by Ken Thompson
- Simplified version of BCPL
- Used for early UNIX development
- Lacked structured data types

C was developed as an improvement over B, adding:

- Data types
- Structured programming

- Better performance
- 

## 6. Development of C Language

Dennis Ritchie added several important features to B, including:

- Data types (int, char, float)
- Structures
- Pointers
- Control statements
- Function support

### Early C Code Example

```
#include <stdio.h>
int main(void)
{
    // This prints "Hello World"
    printf("Hello World");
    return 0;
}
```

This simplicity made C extremely popular.

[#include <stdio.h>]

The first component is the [Header files](#) in a C program. A header file is a file with extension .h which contains C function declarations and macro definitions to be shared between several source files. All lines that start with # are processed by a preprocessor which is a program invoked by the compiler.

Some of the C Header files:

- `stddef.h` - Defines several useful types and macros.
- `stdint.h` - Defines exact width integer types.
- `stdio.h` - Defines core input and output functions

- `stdlib.h` - Defines numeric conversion functions, pseudo-random number generator, and memory allocation
- `string.h` - Defines string handling functions
- `math.h` - Defines common mathematical functions.

```
[int main()]
```

The next part of a C program is the [main\(\) function](#). It is the entry point of a C program and the execution typically begins with the first line of the `main()`.

```
[enclosed in {}]
```

The body of the main method in the C program refers to statements that are a part of the main function. It can be anything like manipulations, searching, sorting, printing, etc. A pair of curly brackets define the body of a function. All functions must start and end with curly brackets.

```
[// This prints "Hello World"]
```

The [comments](#) are used for the documentation of the code or to add notes in your program that are ignored by the compiler and are not the part of executable program.

```
[printf("Hello World");]
```

Statements are the instructions given to the compiler. In C, a statement is always terminated by a **semicolon (;)**. In this particular case, we use [printf\(\)](#) function to instruct the compiler to display "Hello World" text on the screen.

```
[return 0;]
```

The last part of any C function is the [return](#) statement. The return statement refers to the return values from a function. This return statement and return

value depend upon the return type of the function. The return statement in our program returns the value from `main()`. The returned value may be used by an operating system to know the termination status of your program. The value 0 typically means successful termination

---

## 7. Standardization of C

Initially, different versions of C existed, which caused portability issues.

To solve this, C was standardized.

---

### 7.1 K&R C (1978)

- Documented in the book *The C Programming Language*
  - Written by Brian Kernighan and Dennis Ritchie
  - Known as **K&R C**
- 

### 7.2 ANSI C (C89 / C90)

- Standardized by ANSI in 1989
  - Improved portability
  - Added function prototypes and standard libraries
- 

### 7.3 ISO C Standards

| Standard  | Year      |
|-----------|-----------|
| C89 / C90 | 1989–1990 |
| C99       | 1999      |
| C11       | 2011      |
| C17       | 2018      |

Each version improved safety, performance, and clarity.

---

## 8. Features of C as a High-Level Language

---

### 8.1 Simple and Efficient

C has a small set of keywords but powerful constructs.

```
for (int i = 0; i < 5; i++)  
{  
    printf("%d ", i);  
}
```

---

### 8.2 Structured Programming Language

C supports structured programming through:

- Functions
- Loops
- Conditional statements

```
if (marks >= 40)  
    printf("Pass");  
else  
    printf("Fail");
```

---

### 8.3 Portability

A C program written on one system can run on another with minimal or no changes.

```
#include <stdio.h>
```

---

### 8.4 Rich Set of Operators

C provides arithmetic, relational, logical, bitwise, and assignment operators.

```
result = (a > b) ? a : b;
```

---

### 8.5 Low-Level Access Through Pointers

Pointers allow direct memory manipulation.

```
int x = 10;  
int *p = &x;
```

This makes C suitable for system programming.

---

### 8.6 Extensibility

C allows user-defined functions and libraries.

```
int add(int a, int b)  
{  
    return a + b;  
}
```

---

## 9. Applications of C Language

C is widely used in:

- Operating systems (UNIX, Linux)
  - Embedded systems
  - Device drivers
  - Compilers and interpreters
  - Database systems
  - Game engines
- 

## 10. Advantages of C as a High-Level Language

1. Fast execution



2. Portability
  3. Hardware interaction
  4. Modular programming
  5. Reusability of code
- 

## 11. Limitations of C

1. No automatic garbage collection
  2. No built-in object-oriented support
  3. Manual memory management
  4. Less secure compared to modern languages
- 

## 12. C Compared with Other Languages

| Feature         | C    | Assembly  | Python    |
|-----------------|------|-----------|-----------|
| Level           | High | Low       | Very High |
| Speed           | High | Very High | Low       |
| Portability     | Yes  | No        | Yes       |
| Hardware access | Yes  | Yes       | No        |

---

## 13. Sample Program Demonstrating High-Level Nature of C

```
#include <stdio.h>
```

```
int main()
{
    int a = 10, b = 20;
    int sum = a + b;
```

```
    printf("Sum = %d", sum);  
    return 0;  
}
```

This program is:

- Easy to read
  - Portable
  - Independent of hardware
- 

#### **14. Why C is Still Relevant Today**

- Forms the base of C++, Java, and many modern languages
  - Used in performance-critical applications
  - Essential for understanding operating systems and compilers
  - Teaches strong fundamentals of programming
- 

#### **15. Summary**

- C is a high-level language with low-level capabilities
  - Developed by Dennis Ritchie at Bell Labs in 1972
  - Created for UNIX system development
  - Combines efficiency, portability, and flexibility
  - Still widely used in academia and industry
  - Forms the foundation of modern programming languages
-

---

## Chapter: Variables, Constants, and Keywords in C Programming

---

### 1. Introduction

In C programming, **variables**, **constants**, and **keywords** are the basic building blocks of a program.

- **Variables** store data that can change during program execution.
- **Constants** store fixed values that do not change.
- **Keywords** are reserved words with predefined meanings used to form program structure.

Understanding these concepts is essential for writing correct, efficient, and readable C programs.

---

### 2. Variables in C

#### 2.1 Definition of Variable

A **variable** is a named memory location used to store data whose value can be changed during program execution.

Each variable has:

- A **data type**
  - A **name (identifier)**
  - A **value**
- 

#### 2.2 Syntax for Declaring Variables

```
data_type variable_name;
```

**Example**

```
int age;
```

```
float salary;
```

```
char grade;
```

---

## 2.3 Variable Initialization

Initialization means assigning an initial value to a variable.

```
int marks = 85;  
float pi = 3.14;  
char ch = 'A';
```

---

## 2.4 Rules for Naming Variables (Identifiers)

1. Must begin with a letter or underscore (\_)
2. Cannot start with a digit
3. Can contain letters, digits, and underscore
4. Cannot be a keyword
5. Case-sensitive

### Valid

```
int total_marks;  
int _count;
```

### Invalid

```
int 2sum;      // starts with digit  
int float;     // keyword
```

---

## 3. Types of Variables

### 3.1 Local Variables

Declared inside a function or block.

```
void display()  
{  
    int x = 10;
```

```
    printf("%d", x);  
}
```

✓ Accessible only inside the function

✓ Stored in stack memory

---

### 3.2 Global Variables

Declared outside all functions.

```
int count = 5;
```

```
void show()  
{  
    printf("%d", count);  
}
```

✓ Accessible throughout the program

✓ Stored in static memory

---

### 3.3 Static Variables

Retain their value between function calls.

```
void counter()  
{  
    static int c = 0;  
    c++;  
    printf("%d\n", c);  
}
```

---

### 3.4 Automatic Variables

By default, all local variables are automatic.

```
void test()
```

```
{  
    auto int x = 5;  
}
```

(auto keyword is optional and rarely used.)

---

### 3.5 Register Variables

Stored in CPU registers for faster access.

```
register int i;
```

⚠ Address of register variable cannot be accessed using &.

---

## 4. Scope of Variables

Scope defines **where a variable can be accessed**.

### 4.1 Block Scope

```
{  
    int x = 10;  
}
```

x is accessible only inside the block.

---

### 4.2 Function Scope

Variables declared inside a function.

---

### 4.3 Global Scope

Accessible throughout the program.

---

## 5. Lifetime of Variables

| Variable Type | Lifetime              |
|---------------|-----------------------|
| Local         | Until block execution |
| Global        | Entire program        |
| Static        | Entire program        |
| Register      | Until block execution |

---

## 6. Constants in C

### 6.1 Definition of Constant

A **constant** is a fixed value that does not change during program execution.

---

### 6.2 Types of Constants

---

#### 6.2.1 Integer Constants

10

-25

0

---

#### 6.2.2 Floating Point Constants

3.14

-0.5

2.0

---

#### 6.2.3 Character Constants

'A'

'9'

'\n'

---

### 6.2.4 String Constants

"Hello World"

---

### 7. Symbolic Constants using #define

Defined using preprocessor directives.

```
#define PI 3.14159
```

```
float area = PI * r * r;
```

✓ Value cannot be changed

✓ No memory allocation

---

### 8. Constants using const Keyword

```
const int MAX = 100;
```

Attempting to modify:

```
MAX = 50;    // Error
```

Difference from #define:

- const follows type checking
  - Stored in memory
- 

### 9. Difference between Variable and Constant

| Feature      | Variable    | Constant        |
|--------------|-------------|-----------------|
| Value change | Allowed     | Not allowed     |
| Memory       | Allocated   | Allocated / Not |
| Usage        | Stores data | Fixed values    |

---



## 10. Keywords in C

### 10.1 Definition

**Keywords** are reserved words that have special meaning to the compiler and cannot be used as identifiers.

---

### 10.2 List of Keywords in C (32 Keywords)

|          |          |          |        |
|----------|----------|----------|--------|
| auto     | break    | case     | char   |
| const    | continue | default  | do     |
| double   | else     | enum     | extern |
| float    | for      | goto     | if     |
| int      | long     | register | return |
| short    | signed   | sizeof   | static |
| struct   | switch   | typedef  | union  |
| unsigned | void     | volatile | while  |

---

## 11. Classification of Keywords

---

### 11.1 Data Type Keywords

int, char, float, double, void

Example:

```
int number;
```

---

### 11.2 Storage Class Keywords

auto, static, extern, register

Example:

```
static int count;
```

---

### 11.3 Control Statement Keywords

if, else, switch, for, while, do

Example:

```
if (x > 0)
{
    printf("Positive");
}
```

---

### 11.4 Loop Control Keywords

break, continue

---

### 11.5 Structure Keywords

struct, union, enum, typedef

Example:

```
struct student
{
    int id;
    char name[20];
};
```

---

### 11.6 Type Qualifier Keywords

const, volatile, signed, unsigned

---

## 12. Difference between Keywords and Identifiers

| Feature | Keyword    | Identifier   |
|---------|------------|--------------|
| Meaning | Predefined | User-defined |

| Feature      | Keyword     | Identifier |
|--------------|-------------|------------|
| Modification | Not allowed | Allowed    |
| Example      | int         | total      |

---

### 13. Common Programming Errors

1. Using keywords as variable names
  2. Not initializing variables
  3. Changing constant values
  4. Declaring variables without scope awareness
  5. Confusing #define and const
- 

### 14. Practical Example Program

```
#include <stdio.h>

#define PI 3.14

int global = 10;

int main()
{
    const int MAX = 100;
    int radius = 5;
    float area;

    area = PI * radius * radius;

    printf("Area = %.2f\n", area);
```

```
printf("Global = %d\n", global);
printf("Max = %d\n", MAX);

return 0;
}
```

---

## 15. Summary

- Variables store changeable data
  - Constants store fixed values
  - Keywords form the grammar of C
  - Scope and lifetime determine accessibility
  - Correct usage improves program reliability
- 

### //Creating an Identifier for a Variable

```
#include <stdio.h>

int main()
{
    // Creating an integer variable and
    // assign it the identifier 'a'
    int a; // variable declaration

    // Assigning value to the variable
    // using assigned name
    a = 6; // variable initialization

    // Referring to same variable using
```

```
        // assigned name
        printf("the output of the program is %d\n", a);
        return 0;
}
```

---

### **//Creating an Identifier for a Function**

```
#include <stdio.h>

// Function declaration which contains user
// defined identifier as its name
int sum(int a, int b)
{
    return a + b;
}

int main()
{

    // Calling the function using its name
    printf("%d", sum(10, 20));
    return 0;
}
```

---

### **//Variables in C**

```
#include <stdio.h>
```

```
int main()
{
    // integer variable
    int age = 20;

    // floating-point variable
    float height = 5.7;

    // character variable
    char grade = 'A';

    printf("Age: %d\n", age);
    printf("Height: %.1f\n", height);
    printf("Grade: %c\n", grade);

    return 0;
}
```

---

## **//C Variable Initialization**

```
#include <stdio.h>

int main()
{
    // 1. Initialization at the time of declaration
    int age = 20;
    float height = 5.7;
```

```
// 2. Initialization after declaration
char grade;
grade = 'A';

printf("Age: %d\n", age);
printf("Height: %.1f\n", height);
printf("Grade: %c\n", grade);

return 0;
}
```

---

### **//Accessing Variables**

```
#include <stdio.h>

int main() {

    // Create integer variable
    int num = 3;

    // Access the value stored in
    // variable
    printf("%d", num);
    return 0;
}
```

---

### **//Changing Stored Values in a variable**

```
#include <stdio.h>

int main()
{

    // initial value
    int number = 10;
    printf("Initial value: %d\n", number);

    // updating value
    number = 25;
    printf("Updated value: %d\n", number);

    // updating again using expression
    number = number + 5;
    printf("After adding 5: %d\n", number);

    return 0;
}
```

---

### **//Memory Allocation of C Variables**

```
#include <stdio.h>

int main() {

    // Create integer variable
    int num = 22;
```



```
    // Finding size of num
    printf("%d bytes", sizeof(num));
    return 0;
}
```

---

## **//Scope rules in C**

```
#include <stdio.h>
```

```
int main()
{
    // Scope of this variable is
    // within main() function only.
    int var = 34;

    printf("%d", var);
    return 0;
}
```

```
// function where we try to access
// the var defined in main()
void func()
{
    // This will throw an error as var is not
    // defined in this function.
    printf("%d", var);
}
```

---

## **// Global Scope in C**

```
#include <stdio.h>
```

```
// variable declared in global scope
```

```
int global = 5;
```

```
// global variable accessed from
```

```
// within a function
```

```
void display()
```

```
{
```

```
    printf("%d\n", global);
```

```
}
```

```
int main()
```

```
{
```

```
    printf("Before change within main: ");
```

```
    display();
```

```
    // changing value of global
```

```
    // variable from main function
```

```
    printf("After change within main: ");
```

```
    global = 10;
```

```
    display();
```

```
}
```

---

**//Global Variables in C**

**// C program to update global variables**

```
#include <stdio.h>
```

```
int a, b; // global variables
```

```
void add()
```

```
{ // we are adding values of global a and b i.e. 10+15
```

```
    printf("%d", a + b);
```

```
}
```

```
int main()
```

```
{
```

```
    // we are now updating the values of global variables
```

```
    // as you can see we dont need to redeclare a and b
```

```
    // again
```

```
    a = 10;
```

```
    b = 15;
```

```
    add();
```

```
    return 0;
```

```
}
```

---

**//Local Scope in C**

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    {
```

```

int x = 10, y = 20;
{
    // The outer block contains
    // declaration of x and
    // y, so following statement
    // is valid and prints
    // 10 and 20
    printf("x = %d, y = %d\n", x, y);
    {
        // y is declared again,
        // so outer block y is
        // not accessible in this block
        int y = 40;

        // Changes the outer block
        // variable x to 11
        x++;

        // Changes this block's
        // variable y to 41
        y++;

        printf("x = %d, y = %d\n", x, y);
    }

    // This statement accesses
    // only outer block's
    // variables

```

```
        printf("x = %d, y = %d\n", x, y);
    }
}
return 0;
}
```

---

## **//Local Variable in C**

```
// C Program to demonstrate
// Function SCoped Local Variables
#include <stdio.h>

// Function declared
void gfg()
{
    // localVar is a local variable of
    // gfg function
    int localVar = 10;

    printf("The value of localVar is %d\n", localVar);
}

// Main Function
int main()
{
    gfg();
    return 0;
}
```

---

## //Block Scoped Local Variables

```
// C Program to demonstrate
// Block Scoped Local variables
#include <stdio.h>

// Driver Function
int main()
{
    // declaration of x variable outside the block
    int x = 15;

    // Block is defined here
    {
        // Block scoped variable "x"
        int x = 25;
        printf("x was declared in the block: %d \n", x);
    }

    // prints 15 that was declared outside
    printf("x outside the block: %d", x);

    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a C program to calculate area of a rectangle:  
// a. Using hard coded inputs.  
// b. Using inputs supplied by the user.
```

```
int main(){  
    //int length = 15;  
    //int breadth = 20;  
    int length , breadth;  
    printf("Enter the length of the rectangle: \n");  
    scanf("%d" , &length);  
    printf("Enter the breadth of the rectangle: \n");  
    scanf("%d" , &breadth);  
  
    printf("the area of this rectangle is %d\n" , length*breadth);  
  
    return 0;  
}
```

---

```
#include <stdio.h>
```

```
// Calculate the area of a circle and modify the same program  
to calculate the  
// volume of a cylinder given its radius and height.
```

```
int main(){  
    int radius = 15;  
    //printf("the area of this circle is %f\n" ,  
3.14*radius*radius);  
    int height = 20;
```

```
    float volume;

    printf("the volume of this cylinder is %f\n" ,
3.14*radius*radius*height);

    return 0;

}
```

---

```
#include <stdio.h>
```

```
// Write a program to convert Celsius (Centigrade degrees
temperature to Fahrenheit).
```

```
int main(){
    int c = 37;

    float f;
    f = ((9.0/5.0)*c) + 32;

    printf("the value in Fahrenheit is %f" , f);

    return 0;

}
```

---

```
#include <stdio.h>
```



```
// Write a program to calculate simple interest for a set of  
values representing
```

```
// principal, number of years and rate of interest.
```

```
int main(){
```

```
    int p = 1200;
```

```
    float r = 0.06;
```

```
    int t = 2;
```

```
    float Simple_Interest = (p*r*t)/100;
```

```
    printf("the value in simple interest is %f" ,  
Simple_Interest);
```

```
    return 0;
```

```
}
```

---

---

## Chapter: Data Types in C Programming

---

### 1. Introduction

In C programming, **data types** specify:

- What kind of data a variable can store
- How much memory is allocated
- What operations can be performed on the data

Every variable in C must be declared with a **data type** before it is used.

Correct use of data types ensures **efficient memory usage**, **data accuracy**, and **program correctness**.

---

### 2. What is a Data Type?

A **data type** is a classification that tells the compiler:

- The **size** of data
- The **range of values**
- The **operations allowed**

#### Example

```
int age = 20;
```

Here:

- `int` → data type
  - `age` → variable name
  - `20` → value
- 

### 3. Classification of Data Types in C

C data types are broadly classified into **four categories**:

1. Basic (Primary) Data Types

2. Derived Data Types
3. User-Defined Data Types
4. Void Data Type

---

## 4. Basic (Primary) Data Types

Basic data types are the **fundamental building blocks** of C.

### List of Basic Data Types

- `int`
- `char`
- `float`
- `double`

---

## 5. Integer Data Type (int)

### 5.1 Description

The `int` data type is used to store **whole numbers** without decimal points.

---

### 5.2 Declaration

```
int number;
```

---

### 5.3 Example

```
int marks = 85;  
printf("%d", marks);
```

---

### 5.4 Variants of int

| Type | Size (bytes)* | Range |
|------|---------------|-------|
|------|---------------|-------|

| Type         | Size (bytes)* | Range                           |
|--------------|---------------|---------------------------------|
| short int    | 2             | -32,768 to 32,767               |
| int          | 4             | -2,147,483,648 to 2,147,483,647 |
| long int     | 4 or 8        | Large range                     |
| unsigned int | 4             | 0 to 4,294,967,295              |

\* Size depends on compiler and system architecture.

---

### Example

```
unsigned int count = 300;  
long int population = 8000000000;
```

---

## 6. Character Data Type (char)

### 6.1 Description

The char data type stores **a single character** and occupies **1 byte** of memory.

---

### 6.2 Declaration

```
char grade;
```

---

### 6.3 Example

```
char grade = 'A';  
printf("%c", grade);
```

---

### 6.4 ASCII Concept

Characters are stored as **ASCII values** internally.

```
char ch = 'A';  
printf("%d", ch);    // Output: 65
```

---

## 6.5 Signed and Unsigned char

```
signed char a;  
unsigned char b;
```

---

## 7. Floating-Point Data Types

Used to store **real numbers** (numbers with decimal points).

---

### 7.1 Float (float)

#### Description

Stores single-precision decimal numbers.

---

#### Example

```
float pi = 3.14;  
printf("%f", pi);
```

---

#### Precision

- Typically 6 decimal places
- 

### 7.2 Double (double)

#### Description

Stores double-precision floating-point numbers.

---

#### Example

```
double value = 123.456789;
printf("%lf", value);
```

---

## Comparison

| Type   | Size    | Precision |
|--------|---------|-----------|
| float  | 4 bytes | 6 digits  |
| double | 8 bytes | 15 digits |

---

## 8. Derived Data Types

Derived data types are formed using **basic data types**.

---

### 8.1 Arrays

An array stores **multiple values of the same data type**.

```
int arr[5] = {1, 2, 3, 4, 5};
```

---

### 8.2 Pointers

A pointer stores the **address of another variable**.

```
int x = 10;
int *p = &x;
```

---

### 8.3 Structures

A structure groups **different data types**.

```
struct student
{
    int roll;
    float marks;
```

```
};
```

---

## 8.4 Unions

A union shares memory among members.

```
union data
{
    int i;
    float f;
};
```

---

## 9. User-Defined Data Types

User-defined data types allow programmers to define their own types.

---

### 9.1 Structure (struct)

Used to represent complex records.

```
struct employee
{
    int id;
    float salary;
};
```

---

### 9.2 Union (union)

Used for memory-efficient storage.

```
union number
{
    int i;
    float f;
};
```

```
};
```

---

### 9.3 Enumeration (enum)

Used to define named integer constants.

```
enum day {Mon, Tue, Wed, Thu, Fri};
```

---

#### Example

```
enum day today = Wed;
```

---

### 9.4 Typedef

Creates an alias for an existing data type.

```
typedef unsigned int uint;  
uint age = 25;
```

---

## 10. Void Data Type

### 10.1 void

The void data type represents **no value**.

---

### 10.2 Uses of void

#### Function with no return value

```
void display()  
{  
    printf("Hello");  
}
```

---

#### Function with no parameters

```
void show(void)
```



```
{  
    printf("No parameters");  
}
```

---

### **Void Pointer**

```
void *ptr;  
int x = 10;  
ptr = &x;
```

---

## **11. Sizeof Operator**

The sizeof operator returns memory size.

---

### **Example**

```
printf("%d", sizeof(int));  
printf("%d", sizeof(float));
```

---

## **12. Type Modifiers**

Type modifiers alter the range or size of data types.

---

### **Modifiers**

- short
  - long
  - signed
  - unsigned
- 

### **Example**

```
unsigned long int value;
```

---

## 13. Type Conversion

---

### 13.1 Implicit Type Conversion

Automatically done by compiler.

```
int a = 10;  
float b = a;
```

---

### 13.2 Explicit Type Conversion (Type Casting)

Manually done by programmer.

```
float x = (float)5 / 2;
```

---

## 14. Common Errors with Data Types

1. Using incorrect format specifier
  2. Overflow and underflow
  3. Precision loss
  4. Wrong type casting
  5. Ignoring signed vs unsigned difference
- 

## 15. Example Program

```
#include <stdio.h>
```

```
int main()  
{  
    int a = 10;  
    float b = 3.5;  
    char c = 'X';
```

```
    printf("Integer: %d\n", a);  
    printf("Float: %.2f\n", b);  
    printf("Character: %c\n", c);  
  
    return 0;  
}
```

---

## 16. Best Practices

- Choose smallest suitable data type
  - Use double for high precision
  - Avoid unnecessary type casting
  - Use meaningful type aliases
  - Be careful with signed and unsigned types
- 

## 17. Summary

- Data types define memory size and value range
  - C supports basic, derived, user-defined, and void types
  - Modifiers control size and sign
  - Correct data type usage improves efficiency and reliability
  - Data types form the foundation of all C programs
-

---

## Chapter: Input and Output (I/O) Operations in C Programming

---

### 1. Introduction

A computer program becomes meaningful only when it can **interact with the user or external devices**.

This interaction is achieved using **Input and Output (I/O) operations**.

- **Input:** Supplying data to the program
- **Output:** Displaying or sending results from the program

C provides a rich set of **standard input/output functions** through its library.

---

### 2. Standard Input and Output in C

C uses three standard streams:

| Stream | Purpose         | Device   |
|--------|-----------------|----------|
| stdin  | Standard input  | Keyboard |
| stdout | Standard output | Screen   |
| stderr | Error output    | Screen   |

These streams are defined in the header file:

```
#include <stdio.h>
```

---

### 3. Types of I/O Operations in C

I/O operations in C can be classified as:

1. Console I/O operations
2. Formatted I/O operations

3. Unformatted I/O operations
4. File I/O operations

This chapter focuses on **console and formatted/unformatted I/O**.

---

## 4. Formatted Input and Output

Formatted I/O allows data to be **read or printed in a specific format** using format specifiers.

---

## 5. Output Using printf()

### 5.1 Purpose

The printf() function is used to **display formatted output** on the screen.

---

### 5.2 Syntax

```
printf("format string", variable_list);
```

---

### 5.3 Example

```
int age = 20;  
printf("Age = %d", age);
```

---

## 5.4 Common Format Specifiers

### 1. Introduction

In C programming, **format specifiers** are used with input/output functions such as printf() and scanf() to **define the type and format of data** being displayed or read.

They act as **placeholders** in format strings, instructing the compiler how to interpret or display data.

---

## 2. What is a Format Specifier?

A **format specifier** is a special symbol that begins with % and is used to specify:

- The **type of data**
  - The **format of output/input**
- 

### Example

```
int a = 10;  
printf("Value = %d", a);
```

Here:

- %d → format specifier for integer
  - a → variable whose value is printed
- 

## 3. General Syntax of Format Specifiers

%[flags][width][.precision][length]specifier

---

### Components Explained

| Component | Meaning                        |
|-----------|--------------------------------|
| %         | Start of format specifier      |
| flags     | Alignment, padding             |
| width     | Minimum field width            |
| precision | Number of digits after decimal |
| length    | Data size modifier             |
| specifier | Data type                      |

---

## 4. Common Format Specifiers

---

## 4.1 Integer Format Specifiers

| Specifier | Meaning                 |
|-----------|-------------------------|
| %d        | Signed integer          |
| %i        | Integer                 |
| %u        | Unsigned integer        |
| %o        | Octal                   |
| %x        | Hexadecimal (lowercase) |
| %X        | Hexadecimal (uppercase) |

---

### Example

```
int num = 255;  
printf("%d %o %x %X", num, num, num, num);  
Output:  
255 377 ff FF
```

---

## 4.2 Floating Point Format Specifiers

| Specifier | Meaning                         |
|-----------|---------------------------------|
| %f        | Float                           |
| %lf       | Double                          |
| %e        | Scientific notation             |
| %E        | Scientific notation (uppercase) |
| %g        | Shortest representation         |

---

### Example

```
float x = 3.14159;  
printf("%f", x);
```

---

### Scientific Notation

```
printf("%e", x);
```

---

## 4.3 Character and String Specifiers

| Specifier | Meaning   |
|-----------|-----------|
| %c        | Character |
| %s        | String    |

---

### Example

```
char ch = 'A';  
char name[] = "Shamik";  
  
printf("%c %s", ch, name);
```

---

## 4.4 Pointer Format Specifier

| Specifier | Meaning        |
|-----------|----------------|
| %p        | Memory address |

---

### Example

```
int x = 10;  
printf("%p", &x);
```



---

## 5. Format Specifiers in scanf()

scanf( ) also uses format specifiers to read input.

---

### Example

```
int a;  
scanf("%d", &a);
```

---

### Important Difference

| Function | Usage  |
|----------|--------|
| printf() | Output |
| scanf()  | Input  |

---

## 6. Field Width Specifier

Specifies **minimum number of characters to print**.

---

### Example

```
printf("%5d", 10);
```

Output:

```
  10
```

---

## 7. Precision Specifier

Used mainly with floating-point numbers.

---

### Syntax

`%.nf`

---

### Example

```
printf("%.2f", 3.14159);
```

Output:

3.14

---

## 8. Flags in Format Specifiers

Flags modify the output format.

---

| Flag  | Meaning                             |
|-------|-------------------------------------|
| -     | Left align                          |
| +     | Show sign                           |
| 0     | Zero padding                        |
| #     | Alternate form                      |
| space | Insert space before positive number |

---

### Example

```
printf("%+d", 10);    // +10
```

```
printf("%05d", 10);   // 00010
```

```
printf("%-5d", 10);   // 10   (left aligned)
```

---

## 9. Length Modifiers

Used to specify the size of data type.

---

| Modifier | Meaning     |
|----------|-------------|
| h        | short       |
| l        | long        |
| ll       | long long   |
| L        | long double |

---

#### Example

```
long int x = 100000;  
printf("%ld", x);
```

---

### 10. Combining Width and Precision

---

#### Example

```
printf("%10.2f", 3.14159);
```

Output:

3.14

---

### 11. Escape Sequences with Format Specifiers

Escape sequences are used along with format specifiers.

---

| Escape | Meaning        |
|--------|----------------|
| \n     | New line       |
| \t     | Horizontal Tab |
| \\     | Backslash      |
| \v     | Vertical Tab   |

| Escape          | Meaning       |
|-----------------|---------------|
| <code>\a</code> | Alarm or Beep |
| <code>\"</code> | Double quote  |

---

### Example

```
printf("Hello\nWorld");
```

---

## 12. Format Specifiers for scanf() vs printf()

---

### Example Difference

```
float x;  
scanf("%f", &x);    // float
```

```
double y;  
scanf("%lf", &y);    // double
```

---

### Important Rule

- `printf()` uses `%f` for both float and double
  - `scanf()` requires `%lf` for double
- 

## 13. Common Errors with Format Specifiers

1. Mismatched data type

```
printf("%d", 3.14); // Wrong
```

2. Missing `&` in `scanf()`

```
scanf("%d", a); // Wrong
```

3. Wrong specifier for double

```
scanf("%f", &d); // Wrong for double
```

#### 4. Incorrect width or precision

---

### 14. Example Program

```
#include <stdio.h>
```

```
int main()
{
    int a = 10;
    float b = 3.14159;
    char c = 'X';

    printf("Integer: %d\n", a);
    printf("Float: %.2f\n", b);
    printf("Character: %c\n", c);

    return 0;
}
```

---

### 15. Advanced Example

```
#include <stdio.h>
```

```
int main()
{
    int num = 255;
    float pi = 3.14159;

    printf("Decimal: %d\n", num);
}
```

```
    printf("Hex: %x\n", num);  
    printf("Octal: %o\n", num);  
    printf("Float (2 decimal): %.2f\n", pi);  
  
    return 0;  
}
```

---

## 5.5 Precision and Width Control

```
float x = 3.14159;  
printf("%.2f", x);    // Output: 3.14
```

---

## 6. Input Using scanf()

### 6.1 Purpose

The `scanf()` function is used to **read formatted input** from the keyboard.

---

### 6.2 Syntax

```
scanf("format string", &variable);
```

---

### 6.3 Example

```
int num;  
scanf("%d", &num);
```

---

### 6.4 Multiple Inputs

```
int a, b;  
scanf("%d %d", &a, &b);
```

---

### 6.5 Important Points

- Address operator (&) is mandatory for variables
- Not required for arrays and strings

```
char name[20];  
scanf("%s", name);
```

---

## 7. Problems with scanf()

1. Stops reading string input at whitespace
  2. Difficult to handle mixed data types
  3. Buffer overflow risk for strings
- 

## 8. Unformatted Input and Output

Unformatted I/O functions handle **single characters or strings without formatting**.

---

## 9. Character Output: putchar()

### Purpose

Prints a single character.

---

### Example

```
char ch = 'A';  
putchar(ch);
```

---

## 10. Character Input: getchar()

### Purpose

Reads a single character from input.

---

### Example

```
char ch;  
ch = getchar();
```

---

## 11. String Output: puts()

### Purpose

Prints a string followed by a newline.

---

### Example

```
char msg[] = "Hello C";  
puts(msg);
```

---

## 12. String Input: gets() (Deprecated)

```
gets(str);
```

⚠ **Unsafe** due to buffer overflow.

✗ Not recommended.

---

## 13. Safe String Input: fgets()

### Purpose

Reads a line of text safely.

---

### Syntax

```
fgets(string, size, stdin);
```

---

### Example

```
char name[30];  
fgets(name, 30, stdin);
```



- ✓ Reads spaces
- ✓ Prevents buffer overflow

---

#### 14. Comparison of String Input Functions

| Function    | Safe | Reads Spaces |
|-------------|------|--------------|
| scanf("%s") | No   | No           |
| gets()      | No   | Yes          |
| fgets()     | Yes  | Yes          |

---

#### 15. Formatted vs Unformatted I/O

| Feature    | Formatted     | Unformatted   |
|------------|---------------|---------------|
| Formatting | Yes           | No            |
| Complexity | Higher        | Lower         |
| Speed      | Slower        | Faster        |
| Examples   | printf, scanf | getchar, puts |

---

#### 16. Error Output Using stderr

Error messages can be printed using stderr.

---

##### Example

```
fprintf(stderr, "Error occurred");
```

This separates error output from normal output.

---

#### 17. Reading Until End of File (EOF)

The constant EOF indicates end of input.

---

### Example

```
int ch;
while ((ch = getchar()) != EOF)
{
    putchar(ch);
}
```

---

### 18. Input Buffer and Buffer Issues

Input buffer stores characters before processing.

---

#### Common Issues

- Newline character (\n) left in buffer
  - Mixing scanf() and fgets()
- 

#### Solution Example

```
getchar(); // clears newline from buffer
```

---

### 19. Example Program Using Multiple I/O Functions

```
#include <stdio.h>
```

```
int main()
{
    int age;
    char name[20];

    printf("Enter name: ");
```

```
fgets(name, 20, stdin);

printf("Enter age: ");
scanf("%d", &age);

printf("Name: %s", name);
printf("Age: %d", age);

return 0;
}
```

---

## 20. Common Errors in I/O Operations

1. Missing & in scanf()
  2. Wrong format specifier
  3. Buffer overflow
  4. Mixing formatted and unformatted input incorrectly
  5. Ignoring newline characters
- 

## 21. Best Practices

- Use fgets() for string input
  - Match format specifiers with data types
  - Validate user input
  - Handle buffer properly
  - Separate error output using stderr
- 

## 22. Summary

- I/O operations enable interaction with users

- `printf()` and `scanf()` provide formatted I/O
  - `getchar()` and `putchar()` handle character I/O
  - `fgets()` is safest for string input
  - Proper I/O handling avoids runtime errors
  - I/O forms the foundation of interactive programs
-

---

## Chapter: Instructions and Operators in C Programming

---

### 1. Introduction

A C program is essentially a **sequence of instructions** written using **operators, operands, and keywords**.

Instructions tell the computer **what to do**, while operators specify **how operations are performed on data**.

This chapter explains:

- Types of instructions in C
- Different categories of operators
- Operator precedence and associativity
- Common pitfalls and best practices

---

### 2. Instructions in C

#### 2.1 Definition of Instruction

An **instruction** is a command given to the computer to perform a specific task. Every instruction in C ends with a **semicolon (;)**.

```
x = 10;  
printf("Hello");
```

---

### 3. Types of Instructions in C

C instructions are broadly classified into:

1. Type Declaration Instructions
  2. Arithmetic Instructions
  3. Control Instructions
-

## 4. Type Declaration Instructions

Type declaration instructions are used to **declare variables and their data types** before using them.

### 4.1 Purpose

- Reserve memory
- Define type of data stored

#### Syntax

```
data_type variable_name;
```

#### Example

```
int a;  
float b;  
char ch;
```

---

### 4.2 Declaration with Initialization

```
int x = 5;  
float pi = 3.14;
```

---

### 4.3 Multiple Declarations

```
int a, b, c;
```

---

## 5. Arithmetic Instructions

Arithmetic instructions perform **mathematical operations** on variables and constants.

#### General Form

```
variable = operand1 operator operand2;
```

---

### 5.1 Arithmetic Operators Used

| Operator | Meaning        |
|----------|----------------|
| +        | Addition       |
| -        | Subtraction    |
| *        | Multiplication |
| /        | Division       |
| %        | Modulus        |

---

## 5.2 Examples

```
int a = 10, b = 3;  
int sum = a + b;  
int mod = a % b;
```

---

## 5.3 Integer vs Floating Division

```
int a = 5, b = 2;  
float result = a / b;    // result = 2.0 (integer division)  
Correct way:  
float result = (float)a / b; // result = 2.5
```

---

## 6. Control Instructions

Control instructions determine the **flow of execution** of a program.

**Types:**

1. Decision Control Instructions
  2. Loop Control Instructions
  3. Jump Control Instructions
- 

### 6.1 Decision Control Instructions

Used to make decisions based on conditions.

**Keywords:**

if, else, switch

**Example**

```
if (x > 0)
{
    printf("Positive number");
}
else
{
    printf("Negative number");
}
```

---

## 6.2 Loop Control Instructions

Used to repeat a block of code.

**Keywords:**

for, while, do-while

**Example**

```
for (int i = 1; i <= 5; i++)
{
    printf("%d ", i);
}
```

---

## 6.3 Jump Control Instructions

Used to transfer control abruptly.

**Keywords:**

break, continue, goto, return



### Example

```
if (x == 0)
    return;
```

---

## 7. Operators in C

### 7.1 Definition of Operator

An **operator** is a symbol that tells the compiler to perform a specific operation on operands.

```
c = a + b;
```

Here:

- + → operator
  - a, b → operands
- 

## 8. Types of Operators in C

---

### 8.1 Arithmetic Operators

+, -, \*, /, %

#### Example

```
int a = 10, res;
```

```
    // post-incrementing a res is assigned 10, a is not updated
yet
```

```
    res = a++;
```

```
    printf("a is %d, res is %d\n", a, res);
```

```
    // post-decrementing res is assigned 11, a is not updated
yet
```

```
res = a--;

printf("a is %d, res is %d\n", a, res);

res = ++a;

// a and res have same values = 11
printf("a is %d, res is %d\n", a, res);

res = --a;

// a and res have same values = 10
printf("a is %d, res is %d\n", a, res);

printf("+a is %d\n", +a);
printf("-a is %d", -a);

return 0;
```

### Output

```
a is 11, res is 10
a is 10, res is 11
a is 11, res is 11
a is 10, res is 10
+a is 10
-a is -10
```

---

## 8.2 Relational Operators

Used to compare values.

| Operator | Meaning               |
|----------|-----------------------|
| <        | Less than             |
| >        | Greater than          |
| <=       | Less than or equal    |
| >=       | Greater than or equal |
| ==       | Equal to              |
| !=       | Not equal             |

### Example

```
int a = 10, b = 4;
```

```
// greater than example
```

```
if (a > b)
```

```
    printf("a is greater than b\n");
```

```
else
```

```
    printf("a is less than or equal to b\n");
```

```
// greater than equal to
```

```
if (a >= b)
```

```
    printf("a is greater than or equal to b\n");
```

```
else
```

```
    printf("a is lesser than b\n");
```

```
// less than example
```

```
if (a < b)
```

```
    printf("a is less than b\n");
```

```
else
```

```
    printf("a is greater than or equal to b\n");
```

```
// lesser than equal to
if (a <= b)
    printf("a is lesser than or equal to b\n");
else
    printf("a is greater than b\n");

// equal to
if (a == b)
    printf("a is equal to b\n");
else
    printf("a and b are not equal\n");

// not equal to
if (a != b)
    printf("a is not equal to b\n");
else
    printf("a is equal b\n");

return 0;
```

### **Output**

```
a is greater than b
a is greater than or equal to b
a is greater than or equal to b
a is greater than b
a and b are not equal
a is not equal to b
```

---

### 8.3 Logical Operators

Used to combine conditions.

| Operator | Meaning     |
|----------|-------------|
| &&       | Logical AND |
|          | Logical OR  |
| !        | Logical NOT |

#### Example

```
int a = 10, b = 20;
```

```
if (a > 0 && b > 0) //Logical AND Operator ( && )

{
    printf("Both values are greater than 0\n");
}
else
{
    printf("Both values are less than 0\n");
}
return 0;
```

#### Output

Both values are greater than 0

```
int a = -1, b = 20;
```

```
if (a > 0 || b > 0) //Logical OR Operator ( || )

{
    printf("Any one of the given value is "
           "greater than 0\n");
```

```

    }
    else
    {
        printf("Both values are less than 0\n");
    }
    return 0;

```

### Output

Any one of the given value is greater than 0

```
int a = 10, b = 20;
```

```

    if (!(a > 0 && b > 0)) //Logical NOT Operator ( ! )
    {
        // condition returned true but logical NOT operator
changed it to false
        printf("Both values are greater than 0\n");
    }
    else
    {
        printf("Both values are less than 0\n");
    }
    return 0;

```

### Output

Both values are less than 0

---

```

int main()
{
    int a = 10, b = 5, c = 0;

    // c is 0(false), so (a / b) is not evaluated and division will
not be done

```

```

        if (c && (a = a / b))
            printf("This won't be printed\n");
        printf("%d", a);
        return 0;
}

```

### Output

10

```

int main()
{
    int a = 10, b = 5, c = 1;
    // c is 1(true), so (a / b) is not evaluated and division will
    not be done
    if (c || (a = a / b))
        printf("This will be printed\n");
    printf("%d", a);
    return 0;
}

```

### Output

This will be printed

10

```

void main()
{
    int a = 1, b = 0, c = 5;
    int d = a && b || c++;
    printf("%d", c);
}

```

### Output

6

```

int main()

```

```

{
    int i = 1;
    if (i++ && (i == 1))
        printf("GeeksforGeeks\n");
    else
        printf("Coding\n");
}

```

### Output

Coding

---

## 8.4 Assignment Operators

Used to assign values.

| Operator                   | Meaning                       |
|----------------------------|-------------------------------|
| <code>a = b</code>         | <code>a = b</code>            |
| <code>a += b</code>        | <code>a = a + b</code>        |
| <code>a -= b</code>        | <code>a = a - b</code>        |
| <code>a *= b</code>        | <code>a = a * b</code>        |
| <code>a /= b</code>        | <code>a = a / b</code>        |
| <code>a %= b</code>        | <code>a = a % b</code>        |
| <code>a &amp;= b</code>    | <code>a = a &amp; b</code>    |
| <code>a  = b</code>        | <code>a = a   b</code>        |
| <code>a ^= b</code>        | <code>a = a ^ b</code>        |
| <code>a &gt;&gt;= b</code> | <code>a = a &gt;&gt; b</code> |
| <code>a &lt;&lt;= b</code> | <code>a = a &lt;&lt; b</code> |

### Example

```
// Assigning value 10 to a using "=" operator
```



```

    int a = 10;
    printf("%d", a);
    return 0;

int a = 5;
// Equivalent to a = a + 3
    a += 3;
    printf("%d", a);
    return 0;

int a = 10, b = 5;
// a = a - b
    a -= b;
    printf("%d", a);
    return 0;

int a = 10, b = 5;
// a = a * b
    a *= b;
    printf("%d", a);
    return 0;

int a = 10, b = 5;
// a = a / b
    a /= b;
    printf("%d", a);
    return 0;

int a = 10, b = 5;
// a = a % b
    a %= b;
    printf("%d", a);
    return 0;

```

```

// 60 = 0011 1100 in binary
    int a = 60;
// 13 = 0000 1101 in binary
    int b = 13;
// Bitwise AND Assignment a = a & b -> 60 & 13 = 12 (0000 1100
in binary)
    a &= b;
    printf("%d", a);
    return 0;
// 60 = 0011 1100 in binary
    int a = 60;
// 13 = 0000 1101 in binary
    int b = 13;
// Bitwise OR Assignment a = a | b -> 60 | 13 = 61 (0011 1101
in binary)
    a |= b;
    printf("a |= b: %d\n", a);
    return 0;
// 60 = 0011 1100 in binary
    int a = 60;
// 13 = 0000 1101 in binary
    int b = 13;
// a = a ^ b -> 60 ^ 13 = 49 (0011 0001 in binary)
    a ^= b;
    printf("%d", a);
    return 0;
// 60 = 0011 1100 in binary
    int a = 60;
// 13 = 0000 1101 in binary

```

```

    int b = 13;
// Bitwise Left Shift Assignment a = a << 2 -> 60 << 2 = 240
(1111 0000 in binary)
    a <<= 2;
    printf("%d", a);
    return 0;
// 60 = 0011 1100 in binary
    int a = 60;
// 13 = 0000 1101 in binary
    int b = 13;
// Bitwise Right Shift Assignment a = a >> 2 -> 60 >> 2 = 15
(0000 1111 in binary)
    a >>= 2;
    printf("%d", a);
    return 0;

```

---

## 8.5 Increment and Decrement Operators

| Operator | Meaning   |
|----------|-----------|
| ++       | Increment |
| --       | Decrement |

### Pre-increment

```
++x;
```

### Post-increment

```
x++;
```

---

### Difference Example

```

int x = 5;
printf("%d", x++); // prints 5

```

```
printf("%d", ++x); // prints 7
```

---

## 8.6 Conditional (Ternary) Operator

### Syntax

```
condition ? expression1 : expression2;
```

If the (**condition**) is *True* then we will execute and return the result of **expression1** otherwise if the (**condition**) is *false* then we will execute and return the result of **expression2**.

### Example

```
int max = (a > b) ? a : b;
```

---

## 8.7 Bitwise Operators

Operate at **bit level**.

| Operator | Meaning             |
|----------|---------------------|
| &        | Bitwise AND         |
|          | Bitwise OR          |
| ^        | Bitwise XOR         |
| ~        | Bitwise Complement  |
| <<       | Bitwise Left shift  |
| >>       | Bitwise Right shift |

### Example

```
int main()  
{  
  
    // a = 5 (00000101 in 8-bit binary)
```

```

// b = 9 (00001001 in 8-bit binary)
unsigned int a = 5, b = 9;

// The result is 00000001
printf("a&b = %u\n", a & b);

// The result is 00001101
printf("a|b = %u\n", a | b);

// The result is 00001100
printf("a^b = %u\n", a ^ b);

// The result is 11111111111111111111111111111010 (assuming
32-bit unsigned int)
printf("~a = %u\n", a = ~a);

// The result is 00010010
printf("b<<1 = %u\n", b << 1);

// The result is 00000100
printf("b>>1 = %u\n", b >> 1);
return 0;
}

```

## Output

```

a&b = 1
a|b = 13
a^b = 12
~a = 4294967290
b<<1 = 18

```

```
b>>1 = 4
```

---

### 8.8 sizeof Operator

Returns size of data type or variable in bytes.

```
printf("%d", sizeof(int));
```

---

### 8.9 Comma Operator

Executes multiple expressions.

```
int a = (x = 3, y = 4, x + y);
```

---

### 8.10 Pointer Operators

| Operator | Purpose          |
|----------|------------------|
| &        | Address of       |
| *        | Value at address |
| ->       | Arrow operator   |
| .        | Dot operator     |

#### Example

```
int x = 10;  
int *p = &x;  
printf("%d", *p);
```

---

## 9. Operator Precedence and Associativity

#### Example

```
int result = 10 + 5 * 2;
```

Result:

```
10 + (5 * 2) = 2
```

**Operator Precedence Table (High → Low) with Pre/Post Operators Focus**

| Precedence | Operator            | Description                       | Associativity |
|------------|---------------------|-----------------------------------|---------------|
| 1          | ()                  | Parentheses (function call)       | Left-to-Right |
|            | []                  | Array Subscript (Square Brackets) |               |
|            | .                   | Dot Operator                      |               |
|            | ->                  | Structure Pointer Operator        |               |
|            | expr++ ,<br>expr--  | Postfix increment, decrement      |               |
| 2          | ++expr / --<br>expr | Prefix increment, decrement       | Right-to-Left |
|            | + / -               | Unary plus, minus                 |               |
|            | ! , ~               | Logical NOT, Bitwise complement   |               |
|            | (type)              | Cast Operator                     |               |

| Precedence | Operator | Description                                       | Associativity |
|------------|----------|---------------------------------------------------|---------------|
|            | *        | Dereference Operator                              |               |
|            | &        | Address of Operator                               |               |
|            | sizeof   | Determine size in bytes                           |               |
| 3          | *,/,%    | Multiplication, division, modulus                 | Left-to-Right |
| 4          | +/-      | Addition, subtraction                             | Left-to-Right |
| 5          | << , >>  | Bitwise shift left, Bitwise shift right           | Left-to-Right |
| 6          | < , <=   | Relational less than, less than or equal to       | Left-to-Right |
|            | > , >=   | Relational greater than, greater than or equal to |               |
| 7          | == , !=  | Relational is equal to, is not equal to           | Left-to-Right |
| 8          | &        | Bitwise AND                                       | Left-to-Right |
| 9          | ^        | Bitwise exclusive OR                              | Left-to-Right |



| Precedence | Operator | Description                                | Associativity |
|------------|----------|--------------------------------------------|---------------|
| 10         |          | Bitwise inclusive OR                       | Left-to-Right |
| 11         | &&       | Logical AND                                | Left-to-Right |
| 12         |          | Logical OR                                 | Left-to-Right |
| 13         | ?:       | Ternary conditional                        | Right-to-Left |
| 14         | =        | Assignment                                 | Right-to-Left |
|            | += , -=  | Addition, subtraction assignment           |               |
|            | *= , /=  | Multiplication, division assignment        |               |
|            | %= , &=  | Modulus, bitwise AND assignment            |               |
|            | ^= ,  =  | Bitwise exclusive, inclusive OR assignment |               |
|            | <<=, >>= | Bitwise shift left, right assignment       |               |
| 15         | ,        | comma (expression                          | Left-to-Right |

| Precedence | Operator | Description | Associativity |
|------------|----------|-------------|---------------|
|            |          | separator)  |               |

---

## 2. Pre vs Post Operators

---

### 2.1 Post-Increment / Post-Decrement (Highest Priority among ++/--)

#### Operators

`x++`    `x--`

#### Behavior

- Use value first
- Then update

---

#### Example

```
int x = 5;
```

```
int y = x++;
```

Step-by-step:

```
y = 5
```

```
x = 6
```

---

### 2.2 Pre-Increment / Pre-Decrement

#### Operators

`++x`    `--x`

#### Behavior

- Update first

- Then use value

---

### Example

```
int x = 5;
```

```
int y = ++x;
```

Step-by-step:

```
x = 6
```

```
y = 6
```

---

### 3. Key Precedence Rule (Very Important)

👉 **Postfix (x++) has higher precedence than Prefix (++x)**

---

### Example

```
int x = 5;
```

```
int y = ++x * x++;
```

Step-by-step:

1. ++x → x becomes 6

2. x++ → uses 6, then becomes 7

```
y = 6 * 6 = 36
```

Final:

```
x = 7, y = 36
```

---

### 4. Detailed Evaluation Examples

---

#### Example 1

```
int x = 5;
```

```
int y = x++ + ++x;
```

Step-by-step:

1.  $x++ \rightarrow$  uses 5, then  $x = 6$

2.  $++x \rightarrow x = 7$ , uses 7

$y = 5 + 7 = 12$

$x = 7$

⚠ Note: This is **undefined behavior in C standard**, but often asked in exams.

---

### Example 2

```
int x = 10;
```

```
int y = x-- - --x;
```

Step-by-step:

1.  $x-- \rightarrow$  uses 10, then  $x = 9$

2.  $--x \rightarrow x = 8$ , uses 8

$y = 10 - 8 = 2$

$x = 8$

---

### Example 3

```
int x = 3;
```

```
int y = x++ * ++x;
```

Step-by-step:

1.  $x++ \rightarrow$  uses 3, then  $x = 4$

2.  $++x \rightarrow x = 5$ , uses 5

$y = 3 * 5 = 15$

$x = 5$

---

## 5. Associativity with Pre/Post Operators

Operator

Associativity

| Operator | Associativity |
|----------|---------------|
| x++, x-- | Left → Right  |
| ++x, --x | Right → Left  |

---

### Example

```
int x = 5;
int y = ++(++x);
Evaluation:
x = 6 → then 7
y = 7
```

---

## 6. Pre vs Post: Quick Comparison

| Feature       | Pre (++x)     | Post (x++)     |
|---------------|---------------|----------------|
| Execution     | First         | Later          |
| Value used    | Updated value | Original value |
| Precedence    | Lower         | Higher         |
| Associativity | Right → Left  | Left → Right   |

---

## 7. Important Rules for Exams

---

### Rule 1

Post-increment has **higher precedence** than pre-increment

---

### Rule 2

Never rely on multiple increments in one expression

```
x++ + ++x    // risky
```

---

### Rule 3

Use parentheses to avoid confusion

```
y = (x++) + (++x);
```

---

### Rule 4

Compiler behaviour may vary → undefined behaviour

---

## 8. Common Mistakes

1. Assuming pre and post have same precedence
  2. Ignoring evaluation order
  3. Writing complex expressions
  4. Using multiple increments in one statement
  5. Expecting consistent output across compilers
- 

## 9. Best Practice

✓ Always write simple expressions:

```
x++;
```

```
y = x;
```

Instead of:

```
y = x++;
```

---

## 10. Expressions in C

An **expression** is a combination of operands and operators that produces a value.

```
a + b * c
```

---

## 11. Common Programming Errors

1. Using = instead of ==
2. Division by zero
3. Confusing pre and post increment
4. Ignoring operator precedence
5. Misusing logical and bitwise operators

---

## 12. Example Program Using Multiple Operators

```
#include <stdio.h>
```

```
int main()
{
    int a = 10, b = 5;
    int sum, max;

    sum = a + b;
    max = (a > b) ? a : b;

    printf("Sum = %d\n", sum);
    printf("Max = %d\n", max);

    return 0;
}
```

---

## 13. Summary

- Instructions define program actions

- Operators perform computations
  - Control instructions manage program flow
  - Operator precedence affects results
  - Proper understanding avoids logical errors
- 

//Type Conversion in C

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int main() {
```

```
    bool x = true;
```

```
    // Automatic type conversion from bool to int
```

```
    int y = x;
```

```
    // Manual type conversion from bool to int
```

```
    bool z = (bool)y;
```

```
    printf("x: %d\n", x);
```

```
    printf("y: %d\n", y);
```

```
    printf("z: %d", z);
```

```
    return 0;
```

```
}
```

---

// Arithmetic Operators

```
#include <stdio.h>
```



```

int main()
{

    int a = 25, b = 5;

    // using operators and printing results
    printf("a + b = %d\n", a + b); // addition
    printf("a - b = %d\n", a - b); // subtraction
    printf("a * b = %d\n", a * b); // multiplication
    printf("a / b = %d\n", a / b); // division
    printf("a %% b = %d\n", a % b); // modulus
    printf("+a = %d\n", +a);        // unary plus
    printf("-a = %d\n", -a);        // unary minus
    printf("++a = %d\n", ++a);      // pre-increment
    printf("a = %d\n", a);          // value of a after pre-
increment
    printf("a++ = %d\n", a++);      // post-increment
    printf("--a = %d\n", --a);      // pre-decrement
    printf("a = %d\n", a);          // value of a after pre-
decrement
    printf("a++ = %d\n", a++);      // post-increment
    printf("a = %d\n", a);          // value of a after post-
increment
    printf("a-- = %d\n", a--);      // post-decrement
    printf("a = %d\n", a);          // value of a after post-
decrement
    printf("--a = %d\n", --a);      // pre-decrement
    printf("a = %d\n", a);          // value of a after pre-
decrement

```

```
        printf("++a = %d\n", ++a);        // pre-increment
        printf("a = %d\n", a);            // value of a after pre-
increment
    return 0;
}
```

---

// Relational Operators

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 25, b = 5;
```

```
    // using operators and printing results
```

```
    printf("a < b : %d\n", a < b); // less than
```

```
    printf("a > b : %d\n", a > b); // greater than
```

```
    printf("a <= b : %d\n", a <= b); // less than or equal to
```

```
    printf("a >= b : %d\n", a >= b); // greater than or equal
to
```

```
    printf("a == b : %d\n", a == b); // equivalence operator
```

```
    printf("a != b : %d\n", a != b); // not equal to
```

```
    return 0;
```

```
}
```

---

// Logical Operator

```

#include <stdio.h>

int main()
{
    int a = 25, b = 5;

    // using operators and printing results
    printf("a && b : %d\n", a && b); // Logical AND
    printf("a || b : %d\n", a || b); // Logical OR
    printf("!a: %d\n", !a);          // Logical NOT

    return 0;
}

```

---

// Bitwise Operators

```

#include <stdio.h>

int main()
{
    int a = 16, b = 8;

    // using operators and printing results
    printf("a & b: %d\n", a & b); // Bitwise AND
    printf("a | b: %d\n", a | b); // Bitwise OR
    printf("a ^ b: %d\n", a ^ b); // Bitwise XOR
    printf("~a: %d\n", ~a);        // Bitwise NOT
    printf("a >> b: %d\n", a >> b); // Right shift

```

```

    printf("a << b: %d\n", a << b); // Left shift

    return 0;
}

```

---

```

// Assignment Operators

#include <stdio.h>

int main()
{
    int a = 25, b = 5;

    // using operators and printing results
    printf("a = b: %d\n", a = b); // Assignment
    // a=b assigns the value of b to a, so a becomes 5
    printf("a : %d\n", a);          // Print current value of a
    printf("a += b: %d\n", a += b); // Addition assignment
    // a+=b is equivalent to a = a + b
    printf("a : %d\n", a);          // Print current value of a
    printf("a -= b: %d\n", a -= b); // Subtraction assignment
    // a-=b is equivalent to a = a - b
    printf("a : %d\n", a);          // Print current value of a
    printf("a *= b: %d\n", a *= b); // Multiplication
assignment
    // a*=b is equivalent to a = a * b
    printf("a : %d\n", a);          // Print current value of a
    printf("a /= b: %d\n", a /= b); // Division assignment
    // a/=b is equivalent to a = a / b

```

```

    printf("a : %d\n", a);          // Print current value of
a
    printf("a %= b: %d\n", a %= b); // Modulus assignment
    // a%=b is equivalent to a = a % b
    printf("a : %d\n", a);          // Print current value of a
    printf("a &= b: %d\n", a &= b); // Bitwise AND assignment
    // a&=b is equivalent to a = a & b
    printf("a |= b: %d\n", a |= b); // Bitwise OR assignment
    // a|=b is equivalent to a = a | b
    printf("a ^= b: %d\n", a ^= b); // Bitwise XOR assignment
    // a^=b is equivalent to a = a ^ b
    printf("a >>= b: %d\n", a >>= b); // Right shift assignment
    // a>>=b is equivalent to a = a >> b
    printf("a <<= b: %d\n", a <<= b); // Left shift assignment
    // a<<=b is equivalent to a = a << b

    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Which of the following is invalid in C?
// a. int a=1; int b = a;
// b. int v = 3*3;
// c. char dt = '21 dec 2020';

```

```

int main()
{
    // which of the following is invalid in C?

```

```
int a = 1;
int b = a;
int v = 3 * 3;
char dt = '21 dec 2020'; // wrong.

printf(" the value of dt %c", dt); // error.

return 0;
}
```

---

```
#include <stdio.h>
```

```
// What data type will 3.0/8 - 2 return
```

```
int main(){
    float a = 3.0;
    int b = 8;
    int c = 2;
    float evaluation = 3.0/8-2;
    printf("the value of evaluation is %f" , evaluation);

    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program to check whether a number is divisible by
97 or not.
```

```

int main(){
    int a = 1067;
    int b = 97;
    int evaluation = a%b;
    int c = 7893;
    int solution = a%b;

    printf("the value of evaluation is %d\n" , evaluation);
    printf("the value of solution is %d\n" , solution);

    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Explain step by step evaluation of  $3*x/y - z+k$ , where  $x=2$ ,
 $y=3$ ,  $z=3$ ,  $k=1$ 

```

```

int main(){
    //explain step by step the evaluation of  $3*x/y-z+k$ , where
 $x=2$ ,  $y=3$ ,  $z=3$  ,  $k=1$ .
    int x = 2 , y = 3 , z = 3 , k = 1;
    float g =  $3*x/y - z+k$ ;

    printf("the value of g is %f" , g);

    // now if i do the evaluation in c then :-
    //  $3*x/y - z+k$ ;

```

```
    // 6/y - z+k;  
    // 2 - z+k;  
    // -1+k;  
    // 0;  
  
    return 0;  
}
```

---

```
#include <stdio.h>
```

```
// 3.0 + 1 will be:  
// a. Integer.  
// b. Floating point number.  
// c. Character.
```

```
int main(){  
    int s = 1;  
    float d = 3.0;  
    float m = d-s;  
  
    printf("the value of m is %f" , m);  
  
    return 0;  
}
```

---

```
// Logical Operators in C
```

```
#include <stdio.h>
```



```

int main()
{
    int a = 4, b = 20;

    if (a > 5 && b > 5) //Logical AND Operator ( && )
    {
        printf("Both values are greater than 5\n");
    }
    if (a > 0 || b > 0) //Logical OR Operator ( || )
    {
        printf("At least one value is greater than 5\n");
    }
    if (!(a > 0 && b > 0)) // condition returned true but
    Logical NOT operator changed it to false
    {
        printf("Both values are greater than 5\n");
    }
    else
    {
        printf("Both values are less than 5\n");
    }
    return 0;
}

```

---

// Bitwise Operators in C

```

#include <stdio.h>

```

```

int main()

```

```

{

    // a = 5 (00000101 in 8-bit binary)
    // b = 9 (00001001 in 8-bit binary)
    unsigned int a = 5, b = 9;

    // The result is 00000001
    printf("a&b = %u\n", a & b); // AND operator

    // The result is 00001101
    printf("a|b = %u\n", a | b); // OR operator

    // The result is 00001100
    printf("a^b = %u\n", a ^ b); // XOR operator

    // The result is 11111111111111111111111111111010
    // (assuming 32-bit unsigned int)
    printf("~a = %u\n", a = ~a); // NOT operator

    // The result is 00010010
    printf("b<<1 = %u\n", b << 1); // Left shift operator

    // The result is 00000100
    printf("b>>1 = %u\n", b >> 1); // Right shift operator
    return 0;
}

```

---

// Assignment Operators in C

```

#include <stdio.h>

int main()
{
    int a = 25, b = 5;

    // using operators and printing results
    printf("a = b: %d\n", a = b); // simple assignment the
value of b is assigned to a
    printf("a: %d\n", a);
    printf("b: %d\n", b);
    printf("a += b: %d\n", a += b); // add AND assignment    a =
a + b
    printf("a: %d\n", a);
    printf("b: %d\n", b);
    printf("a -= b: %d\n", a -= b); // subtract AND
assignment a = a - b
    printf("a: %d\n", a);
    printf("b: %d\n", b);
    printf("a *= b: %d\n", a *= b); // multiply AND
assignment a = a * b
    printf("a: %d\n", a);
    printf("b: %d\n", b);
    printf("a /= b: %d\n", a /= b); // divide AND assignment
a = a / b
    printf("a: %d\n", a);
    printf("b: %d\n", b);
    printf("a %= b: %d\n", a %= b); // modulus AND assignment
a = a % b

```

```

        printf("a &= b: %d\n", a &= b);    // bitwise AND AND
assignment a = a & b

        printf("a |= b: %d\n", a |= b);    // bitwise OR AND
assignment a = a | b

        printf("a ^= b: %d\n", a ^= b);    // bitwise XOR AND
assignment a = a ^ b

        printf("a >>= b: %d\n", a >>= b); // right shift AND
assignment a = a >> b

        printf("a <<= b: %d\n", a <<= b); // left shift AND
assignment a = a << b

    return 0;
}

```

---

```

// C Program to demonstrate the use of Misc operators
#include <stdio.h>

int main()
{
    // integer variable
    int num = 10;
    int *add_of_num = &num;

    printf("sizeof(num) = %d bytes\n", sizeof(num)); // sizeof
operator

    printf("&num = %p\n", &num);                      // address
of operator

    printf("*add_of_num = %d\n", *add_of_num);         //
dereference operator

    printf("(10 < 5) ? 10 : 20 = %d\n", (10 < 5) ? 10 : 20);
// conditional ternary operator

```

```
    printf("(float)num = %f\n", (float)num);           //
```

type casting operator

```
    return 0;  
}
```

---

---

## Chapter: Conditional Instructions in C Programming

---

### 1. Introduction

In C programming, **conditional instructions** are used to **make decisions**. They allow a program to execute **different blocks of code based on whether a condition is true or false**.

Without conditional instructions, programs would execute statements strictly in sequence, making them incapable of handling real-world logic such as comparisons, choices, and validations.

---

### 2. What is a Condition?

A **condition** is an expression that evaluates to either:

- **True (non-zero)** or
- **False (zero)**

#### Example

`a > b`

`x == 0`

`marks >= 40`

These conditions are formed using **relational and logical operators**.

---

### 3. Types of Conditional Instructions in C

C provides the following conditional constructs:

1. `if` statement
2. `if-else` statement
3. Nested `if-else`
4. `else-if` ladder
5. Conditional (ternary) operator `?:`

## 6. switch statement

---

### 4. The if Statement

#### 4.1 Definition

The `if` statement executes a block of code **only when the given condition is true**.

---

#### 4.2 Syntax

```
if (condition)
{
    statement(s);
}
```

---

#### 4.3 Working

- Condition is evaluated
  - If condition is **true**, the block executes
  - If condition is **false**, the block is skipped
- 

#### 4.4 Example

```
int age = 20;

if (age >= 18)
{
    printf("Eligible to vote");
}
```

---

### 5. The if-else Statement

## 5.1 Definition

The if-else statement provides **two alternative paths of execution**.

---

## 5.2 Syntax

```
if (condition)
{
    statement(s);
}
else
{
    statement(s);
}
```

---

## 5.3 Example

```
int num = -5;

if (num > 0)
{
    printf("Positive number");
}
else
{
    printf("Negative number");
}
```

---

## 6. Nested if-else Statement

### 6.1 Definition



When an if or else block contains another if-else, it is called **nested if-else**.

---

## 6.2 Syntax

```
if (condition1)
{
    if (condition2)
    {
        statement;
    }
    else
    {
        statement;
    }
}
else
{
    statement;
}
```

---

## 6.3 Example

```
int a = 10, b = 20, c = 15;

if (a > b)
{
    if (a > c)
        printf("a is largest");
    else
```

```
        printf("c is largest");
    }
else
{
    if (b > c)
        printf("b is largest");
    else
        printf("c is largest");
}
```

---

## 7. The else-if Ladder

### 7.1 Definition

The else-if ladder is used to **check multiple conditions sequentially**.

---

### 7.2 Syntax

```
if (condition1)
{
    statement;
}
else if (condition2)
{
    statement;
}
else if (condition3)
{
    statement;
}
```

```
else
{
    statement;
}
```

---

### 7.3 Working

- Conditions are evaluated **top to bottom**
  - The **first true condition** executes
  - Remaining conditions are skipped
- 

### 7.4 Example

```
int marks = 72;

if (marks >= 90)
    printf("Grade A");
else if (marks >= 75)
    printf("Grade B");
else if (marks >= 60)
    printf("Grade C");
else if (marks >= 40)
    printf("Grade D");
else
    printf("Fail");
```

---

## 8. Conditional (Ternary) Operator

### 8.1 Definition

The conditional operator `?:` is a **compact alternative** to `if-else` for simple decisions.

---

## 8.2 Syntax

```
condition ? expression1 : expression2;
```

---

## 8.3 Example

```
int a = 10, b = 20;
```

```
int max;
```

```
max = (a > b) ? a : b;
```

```
printf("Maximum = %d", max);
```

---

## 8.4 Characteristics

- Shorter syntax
  - Returns a value
  - Used inside expressions
- 

# 9. The switch Statement

## 9.1 Definition

The switch statement is used when a variable is compared against **multiple constant values**.

---

## 9.2 Syntax

```
switch (expression)
```

```
{
```

```
    case constant1:
```

```
        statement;
```

```
        break;
```

```
    case constant2:
        statement;
        break;

    default:
        statement;
}
```

---

### 9.3 Rules of switch

1. Expression must be int, char, or enum
  2. Case values must be constants
  3. break prevents fall-through
  4. default is optional
- 

### 9.4 Example

```
int choice = 2;

switch (choice)
{
    case 1:
        printf("Option One");
        break;

    case 2:
        printf("Option Two");
        break;
}
```

```
    case 3:
        printf("Option Three");
        break;

    default:
        printf("Invalid Choice");
}
```

---

## 10. Fall-through in switch

If break is omitted, execution continues to the next case.

### Example

```
int x = 1;

switch (x)
{

    case 1:
        printf("One ");
    case 2:
        printf("Two ");
    default:
        printf("End");
}
```

Output:

One Two End

---

## 11. Comparison: if-else vs switch

| Feature     | if-else                | switch               |
|-------------|------------------------|----------------------|
| Conditions  | Any logical expression | Constant values only |
| Data type   | Any                    | int, char            |
| Readability | Less for many cases    | Better               |
| Flexibility | High                   | Limited              |

---

## 12. Common Logical Errors

1. Using = instead of ==
  2. Missing braces { }
  3. Forgetting break in switch
  4. Wrong condition order in else-if
  5. Assuming switch supports ranges
- 

## 13. Practical Example Program

```
#include <stdio.h>

int main()
{
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num == 0)
        printf("Zero");
```

```
        else if (num > 0)
            printf("Positive");
        else
            printf("Negative");

        return 0;
    }
```

---

## 14. Summary

- Conditional instructions control program flow
  - `if` executes code when condition is true
  - `if-else` provides two-way branching
  - Nested and ladder forms handle multiple decisions
  - `switch` is ideal for menu-driven programs
  - Correct structure avoids logical errors
- 

```
// if in C
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int age = 20;
```

```
    // If statement
```

```
    if (age >= 18)
```

```
    {
```



```
        printf("Eligible for vote");
    }
}
```

---

// if-else in C

```
#include <stdio.h>
```

```
int main()
{
    int age = 19;

    if (age >= 18)
    {
        printf("Eligible for vote");
    }
    else
    {
        printf("Not Eligible for vote");
    }
    return 0;
}
```

---

// Nested if-else in C

```
#include <stdio.h>
```

```
int main()
```

```

{
    int age = 15;

    if (age >= 18)
    {
        if (age >= 60)
            printf("Eligible to vote (Senior Citizen)\n");
        else
            printf("Eligible for vote\n");
    }
    else
    {
        printf("Not eligible to vote (Under 18)\n");
        if (age >= 13)
            printf("teenager\n");
        else
            printf("not a teenager\n");
    }

    return 0;
}

```

---

// if-else-if Ladder in C

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int i = 20;

// If else ladder with three conditions
if (i == 10)
    printf("Not Eligible");
else if (i == 15)
    printf("wait for three years");
else if (i == 20)
    printf("You can vote");
else
    printf("Not a valid age");

return 0;
}
```

---

// switch Statement in C

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // variable to be used in switch statement
```

```
    int day;
```

```
    printf("Enter the number of the day in a week: \n");
```

```
    scanf("%d", &day);
```

```
    // declaring switch cases
```

```
switch (day)
{
case (1):
    printf("this is Monday");
    break;
case (2):
    printf("this is Tuesday");
    break;
case (3):
    printf("this is Wednesday");
    break;
case (4):
    printf("this is Thursday");
    break;
case (5):
    printf("this is Friday");
    break;
case (6):
    printf("this is Saturday");
    break;
case (7):
    printf("this is Sunday");
    break;
}

return 0;
}
```

---

```
// conditional operator in C
/*
variable = Expression1 ? Expression2 : Expression3;
variable = (condition) ? Expression2 : Expression3;
(condition) ? (variable = Expression2) : (variable =
Expression3);
*/
```

```
// C program to find largest among two
// numbers using ternary operator
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int m = 5, n = 4;
```

```
    (m > n) ? printf("m is greater than n that is %d > %d",
                    m, n)
            : printf("n is greater than m that is %d > %d",
                    n, m);
```

```
    return 0;
```

```
}
```

---

```
#include <stdio.h>
```

```
// What will be the output of this program
```

```
// int a = 10;
```

```

// if (a = 11)
// printf("I am 11");
// else
// printf("I am not 11");

int main()
{
    int a = 10;
    if (a = 11) //if (a == 11) then output would be "I am not
11"
    {
        printf("I am 11");
    }
    else
    {
        printf("I am not 11");
    }
    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Write a program to determine whether a student has passed or
failed. To pass, a
// student requires a total of 40% and at least 33% in each
subject. Assume there
// are three subjects and take the marks as input from the
user.

```

```

int main()

```

```

{
    int marks1, marks2, marks3;
    printf(" enter marks1; \n");
    scanf("%d", &marks1);
    printf(" enter marks2;\n");
    scanf("%d", &marks2);
    printf(" enter marks3; \n");
    scanf("%d", &marks3);
    printf("the marks are %d %d and %d\n", marks1, marks2,
marks3);

    if (marks1 < 33 || marks2 < 33 || marks3 < 33)
    {
        printf("you have failed");
    }
    else if (((marks1 + marks2 + marks3) / 3) < 40)
    {
        printf(" you are failed due to less marks in individual
subject");
    }
    else
    {
        printf("you have passed");
    }
    return 0;
}

```

---

```

#include <stdio.h>

```

```
// Calculate income tax paid by an employee to the government
as per the slabs
// mentioned below:
// Income Slab          Tax
// 2.5 - 5.0L           5%
// 5.0L - 10.0L         20%
// Above 10.0L          30%

// Note that there is no tax below 2.5L. Take income amount as
an input from the user.
```

```
int main()
{
    int income;
    float tax = 0;

    printf("enter income : ");
    scanf("%d", &income);
    if (income <= 250000)
    {
        tax = 0;
    }
    else if ((income > 250000) && (income <= 500000))
    {
        tax = 0.05 * (income - 250000);
    }
    else if ((income > 500000) && (income <= 1000000))
    {
```



```

        tax = 0.05 * (500000 - 250000) + 0.2 * (income -
500000);
    }
    else
    {
        tax = 0.05 * (500000 - 250000) + 0.2 * (1000000 -
500000) + 0.3 * (income - 1000000);
    }
    printf("\nthe total tax to be paid is Rs %f", tax);

    printf("\nthe salary after tax is Rs %f", income - tax);

    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Write a program to find whether a year entered by the user
is a leap year or not.

```

```

// Take year as an input from the user.

```

```

int main()
{
    int year;
    printf("enter year: \n");
    scanf("%d", &year);
    if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
    {

```

```
        printf("this is a leap year");
    }
    else
    {
        printf("this is not a leap year");
    }
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program to determine whether a character entered by
the user is
```

```
// lowercase or not.
```

```
int main()
```

```
{
```

```
    char ch;
```

```
    printf("Enter a character: ");
```

```
    scanf("%c", &ch);
```

```
    if (ch >= 'a' && ch <= 'z')
```

```
    {
```

```
        printf("The character is lowercase.\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("The character is not lowercase.\n");
    }
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program to find greatest of four numbers
```

```
int main(){
    int a=6 , b=8 , c=24 , d=36;
    if(a>b && a>c && a>d){
        printf("A is the greatest of all");
    }
    else if(b>a && b>c && b>d){
        printf("B is the greatest of all");
    }
    else if(c>a && c>b && c>d){
        printf("C is the greatest of all");
    }
    else if(d>a && d>b && d>c){
        printf("D is the greatest of all");
    }
    return 0;
}
```

---

---

## Chapter: Loop Control Instructions in C Programming

---

### 1. Introduction

In programming, many tasks require executing the **same set of statements repeatedly**. Writing the same code again and again is inefficient and error-prone.

**Loop control instructions** allow a program to **execute a block of code repeatedly** as long as a specified condition remains true.

---

### 2. What is a Loop?

A **loop** is a control structure that:

1. Initializes a variable
  2. Tests a condition
  3. Executes a block of statements
  4. Updates the loop control variable
  5. Repeats the process
- 

### 3. Types of Loop Control Instructions in C

C provides **three looping constructs**:

1. while loop
2. do-while loop
3. for loop

Additionally, loop behaviour can be modified using:

- break
  - continue
-

## 4. The while Loop

### 4.1 Definition

The while loop is an **entry-controlled loop**.

The condition is checked **before** the loop body executes.

---

### 4.2 Syntax

```
while (condition)
{
    statements;
}
```

---

### 4.3 Working

- Condition is evaluated first
  - If true, loop body executes
  - If false, loop terminates
  - If condition is false initially, loop body will **not execute at all**
- 

### 4.4 Example

```
int i = 1;

while (i <= 5)
{
    printf("%d ", i);
    i++;
}
```

Output:

1 2 3 4 5

---

## 4.5 Infinite while Loop

```
while (1)
{
    printf("Infinite Loop");
}
```

---

## 5. The do-while Loop

### 5.1 Definition

The do-while loop is an **exit-controlled loop**.

The loop body executes **at least once**, regardless of the condition.

---

### 5.2 Syntax

```
do
{
    statements;
}
while (condition);
```

---

### 5.3 Working

- Loop body executes first
  - Condition is checked afterward
  - If true, loop repeats
- 

### 5.4 Example

```
int i = 1;
```

```
do
{
    printf("%d ", i);
    i++;
}
while (i <= 5);
```

---

## 5.5 Key Difference from while

```
int i = 10;

do
{
    printf("Executed once");
}
while (i < 5);
```

---

## 6. The for Loop

### 6.1 Definition

The for loop is a **compact and structured loop** that combines:

- Initialization
- Condition
- Update

in a single line.

---

### 6.2 Syntax

```
for (initialization; condition; update)
{
```

```
    statements;  
}
```

---

### 6.3 Example

```
for (int i = 1; i <= 5; i++)  
{  
    printf("%d ", i);  
}
```

---

### 6.4 Execution Flow

1. Initialization (once)
  2. Condition check
  3. Loop body execution
  4. Update
  5. Repeat
- 

### 6.5 Infinite for Loop

```
for (;;)   
{  
    printf("Infinite Loop");  
}
```

---

## 7. Nested Loops

### 7.1 Definition

A loop inside another loop is called a **nested loop**.

---

### 7.2 Example



```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 2; j++)
    {
        printf("i = %d j = %d\n", i, j);
    }
}
```

---

### 7.3 Pattern Printing Example

```
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= i; j++)
    {
        printf("* ");
    }
    printf("\n");
}
```

---

## 8. Loop Control Statements

---

### 8.1 break Statement

#### Definition

Terminates the loop immediately and transfers control outside the loop.

---

#### Example

```
for (int i = 1; i <= 10; i++)
```

```

{
    if (i == 5)
        break;
    printf("%d ", i);
}

```

Output:

1 2 3 4

---

## 8.2 continue Statement

### Definition

Skips the current iteration and continues with the next iteration.

---

### Example

```

for (int i = 1; i <= 5; i++)
{
    if (i == 3)
        continue;
    printf("%d ", i);
}

```

Output:

1 2 4 5

---

## 9. Comparison of Looping Constructs

| Feature           | while   | do-while | for     |
|-------------------|---------|----------|---------|
| Control type      | Entry   | Exit     | Entry   |
| Minimum execution | 0 times | 1 time   | 0 times |

| Feature  | while              | do-while      | for              |
|----------|--------------------|---------------|------------------|
| Best use | Unknown iterations | At least once | Known iterations |

---

## 10. Common Looping Errors

1. Missing loop update
  2. Infinite loops
  3. Off-by-one errors
  4. Misplaced break or continue
  5. Incorrect loop condition
- 

## 11. Example Programs

### 11.1 Sum of First N Natural Numbers

```
int n, sum = 0;

for (int i = 1; i <= n; i++)
{
    sum = sum + i;
}
```

---

### 11.2 Reverse a Number

```
int num, rev = 0;

while (num != 0)
{
    rev = rev * 10 + num % 10;
    num = num / 10;
}
```

---

## 12. Best Practices

- Prefer for loop when iterations are known
  - Use meaningful loop variable names
  - Avoid deeply nested loops when possible
  - Always test loop boundary conditions
- 

## 13. Summary

- Loops automate repetitive tasks
  - while checks condition first
  - do-while executes at least once
  - for offers compact syntax
  - break and continue control loop flow
  - Nested loops handle multi-level repetition
- 

```
// Multiplication Table from 1 to 5
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // Initialize outer loop counter
```

```
    int i = 1;
```

```
    // Outer while loop to print a multiplication
```

```
    // table for all numbers up to 5
```

```
while (i <= 5)
{

    // Initialize inner loop counter for each row
    int j = 1;

    // Inner while loop to print each value in table
    while (j <= 5)
    {
        printf("%d ", i * j);
        j++;
    }
    printf("\n");
    i++;
}
return 0;
}
```

---

```
// Print Table from 1 to 5 Using For Loop
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // Outer for loop to print a multiplication
```

```
    // table for all numbers upto 5
```

```
    for (int i = 1; i <= 5; i++)
```

```
{

    // Inner loop to print each value in table
    for (int j = 1; j <= 5; j++)
    {
        printf("%d ", i * j);
    }
    printf("\n");
}
return 0;
}
```

---

// Nested Loops

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // Outer loop runs 3 times
```

```
    for (int i = 0; i < 3; i++)
```

```
    {
```

```
        // Inner loop runs 2 times for each
```

```
        // outer loop iteration
```

```
        for (int j = 0; j < 2; j++)
```

```
        {
```

```
            printf("i = %d, j = %d\n", i, j);
```

```
        }
```

```
    }  
    return 0;  
}
```

---

// Nested for loop refers to any type of loop that is defined inside a 'for' loop.

```
#include <stdio.h>
```

```
int main()  
{
```

```
    // limit for i  
    int m = 4;
```

```
    // limit for j  
    int n = 5;
```

```
    // Outer loop  
    for (int i = 0; i < m; i++)  
    {  
        printf("i = %d: ", i);
```

```
        // InnerLoop  
        for (int j = 0; j < n; j++)  
        {  
            printf("%d ", i * n + j);  
        }  
        printf("\n");
```

```
    }

    return 0;
}
```

---

```
// Nested while Loops
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int end = 5;
```

```
    printf("Pattern Printing using Nested While loop");
```

```
    int i = 1;
```

```
    while (i <= end)
```

```
    {
```

```
        printf("\n");
```

```
        int j = 1;
```

```
        while (j <= i)
```

```
        {
```

```
            printf("%d ", j);
```

```
            j = j + 1;
```

```
        }
```

```
        i = i + 1;
```

```
    }
```



```
    return 0;
}
```

---

```
// Nested do-while Loops
```

```
#include <stdio.h>
```

```
int main()
{
    int i = 1;

    do
    {
        int j = 1;
        do
        {
            printf("%d ", j);
            j++;
        } while (j <= 3);

        printf("\n");
        i++;
    } while (i <= 2);

    return 0;
}
```

---

```
// Hybrid Nested Loops
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i = 1;
```

```
    while (i <= 2)
```

```
    {
```

```
        // Outer while loop
```

```
        for (int j = 1; j <= 2; j++)
```

```
        {
```

```
            // Inner for loop
```

```
            printf("i = %d, j = %d\n", i, j);
```

```
        }
```

```
        i++;
```

```
    }
```

```
    return 0;
```

```
}
```

---

```
// Break Inside Nested Loops
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i = 0;
```

```
    for (int i = 0; i < 5; i++)
```

```

{
    for (int j = 0; j < 3; j++)
    {

        // This inner loop will break when i==3
        if (i == 3)
        {
            break;
        }
        printf("* ");
    }
    printf("\n");
}
return 0;
}

```

---

// Continue Inside Nested Loops

```

#include <stdio.h>

```

```

int main()

```

```

{

    int i = 0;
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 3; j++)
        {

```

```
        // This inner loop will skip when j==2
        if (j == 2)
        {
            continue;
        }
        printf("%d ", j);
    }
    printf("\n");
}
return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program to print multiplication table of a given
number n
```

```
int main()
{
    int n;
    printf(" Enter the number to print its multiplication
table: ");
    scanf("%d", &n);
    for (int i = 1; i <= 10; i++)
    {
        printf("%d x %d = %d\n", n, i, n * i);
    }
    return 0;
}
```

```
}
```

---

```
#include <stdio.h>
```

```
// Write a program to print multiplication table of 10 in  
reversed order.
```

```
int main()  
{  
    int n;  
    printf(" Enter the number to print its multiplication table  
in reverse order: ");  
    scanf("%d", &n);  
    for (int i = 10; i ; i--)  
    {  
  
        printf("%d x %d = %d\n", n, i, n * i);  
    }  
    return 0;  
}
```

---

A do while loop is executed:

- a. At least once.
- b. At least twice.
- c. At most once.

ans -> a. At least once

---

What can be done using one type of loop can also be done using the other two

types of loops - true or false?

ans -> true

---

```
#include <stdio.h>
```

```
// Write a program to sum first ten natural numbers using while loop.
```

```
int main()
{
    int i = 1;
    int sum = 0;
    while (i <= 10)
    {
        sum += i;
        i++;
    }
    printf(" The sum of first 10 natural numbers is %d\n", sum);
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program to sum first ten natural numbers using 'do-while' loop.
```

```
int main()
{
```

```
int i = 1;
int sum = 0;
do
{
    sum += i;
    i++;
} while (i <= 10);
printf(" the sum of first 10 natural number is %d\n ",
sum);
return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program to sum first ten natural numbers using
'for' loop.
```

```
int main()
{
    int sum = 0;
    for (int i = 1; i <= 10; i++)
    {
        sum += i;
    }
    printf("the sum of first n natural numbers is %d\n", sum);

    return 0;
}
```

---

```

#include <stdio.h>

// Write a program to calculate the sum of the numbers
occurring in the
// multiplication table of 8. (consider 8 x 1 to 8 x 10).

int main()
{
    int sum = 0;
    for (int i = 1; i <= 10; i++)
    {
        sum += (8 * i);
    }

    printf("the sum of the numbers occuring in the
multiplication table of 8 is %d\n", sum);

    return 0;
}

```

---

```

#include <stdio.h>

// Write a program to calculate the factorial of a given number
using a for loop.

// 3! = 1 x 2 x 3 = 6
// 4! = 1 x 2 x 3 x 4 = 24
// 5! = 1 x 2 x 3 x 4 x 5 = 120
// n! = 1 x 2 x 3 x ..... (n-2) x (n-1) x n

```



```

int main()
{
    int n, fact = 1;
    printf(" enter the number to find its factorial: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++)
    {
        fact *= i;
    }
    printf(" the factorial is %d\n", fact);

    return 0;
}

```

---

```

#include <stdio.h>

```

// Write a program to calculate the factorial of a given number using while loop.

```

int main()
{
    int i = 1, n, fact = 1;
    printf(" enter the number to find its factorial: ");
    scanf("%d", &n);
    while (i <= n)
    {
        fact *= i;
        i++;
    }
}

```

```
    }  
    printf(" the factorial is %d\n", fact);  
  
    return 0;  
}
```

---

```
#include <stdio.h>
```

```
// Write a program to check whether a given number is prime or  
not using for loops.
```

```
int main()  
{  
    int n;  
    int prime = 0;  
    printf(" enter the number to check whether it is prime or  
not: ");  
    scanf("%d", &n);  
    if (n == 0 || n == 1)  
    {  
        prime = 1;  
    }  
    else  
    {  
  
        for (int i = 2; i < n; i++)  
        {  
            if (n % i == 0 && n != 2)  
            {
```

```

        prime = 1;
        break;
    }
}
if (prime)
{
    printf(" the number is not a prime\n");
}
else
{
    printf(" the number is a prime\n");
}
return 0;
}

```

---

```

#include <stdio.h>

```

```

// Write a program to check whether a given number is prime or
not using other types of loops

```

```

int main()
{
    int n, i = 2;
    int prime = 0;
    printf(" enter the number to check whether it is prime or
not: ");
    scanf("%d", &n);
    if (n == 0 || n == 1)

```

```

{
    prime = 1;
}
else
{
    while (i < n)
    {
        if (n % i == 0 && n != 2)
        {
            prime = 1;
            break;
        }
        i++;
    }
    /*do
    {
        if (n % i == 0 && n != 2)
        {
            prime = 1;
            break;
        }
        i++;
    } while (i < n);*/
}
if (prime)
{
    printf(" the number is not a prime\n");
}

```

```
else
{
    printf(" the number is a prime\n");
}
return 0;
}
```

---

---

## Chapter: Functions and Recursion in C Programming

---

### 1. Introduction

In C programming, large programs are divided into **smaller, manageable units** called **functions**.

Functions improve **modularity, readability, reusability, and debugging** of programs.

**Recursion** is a special technique where a function **calls itself** to solve a problem by breaking it into smaller subproblems.

---

### 2. What is a Function?

A **function** is a block of code that performs a specific task and can be **called multiple times** from different parts of a program.

#### Characteristics of a Function

- Has a name
  - May take input (parameters)
  - May return a value
  - Executes only when called
- 

### 3. Advantages of Using Functions

1. Code reusability
  2. Reduced program size
  3. Easier debugging and testing
  4. Better program structure
  5. Improved readability
-

## 4. Types of Functions in C

Functions in C are classified into:

1. **Library (Built-in) Functions**
  2. **User-defined Functions**
- 

## 5. Library Functions

Library functions are **predefined functions** provided by C libraries.

### Examples

```
printf(), scanf()      // stdio.h
strlen(), strcpy()     // string.h
sqrt(), pow()          // math.h
```

### Example

```
#include <stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

---

## 6. User-Defined Functions

User-defined functions are written by the programmer to perform specific tasks.

---

## 7. Components of a User-Defined Function

A function consists of three parts:

1. Function declaration (prototype)

2. Function definition
3. Function call

---

## 8. Function Declaration (Prototype)

### Purpose

- Informs the compiler about function name, return type, and parameters
- Enables type checking

### Syntax

```
return_type function_name(parameter_list);
```

### Example

```
int add(int, int);
```

---

## 9. Function Definition

### Syntax

```
return_type function_name(parameter_list)
{
    statements;
    return value;
}
```

### Example

```
int add(int a, int b)
{
    return a + b;
}
```

---

## 10. Function Call

A function is executed when it is called.



### Example

```
int sum = add(10, 20);
```

---

## 11. Categories of User-Defined Functions

Based on arguments and return value, functions are classified into **four types**.

---

### 11.1 No Arguments, No Return Value

```
void display()  
{  
    printf("Welcome to C Programming");  
}
```

Call:

```
display();
```

---

### 11.2 Arguments, No Return Value

```
void square(int x)  
{  
    printf("Square = %d", x * x);  
}
```

Call:

```
square(5);
```

---

### 11.3 No Arguments, Return Value

```
int getNumber()  
{  
    return 10;  
}
```

Call:

```
int n = getNumber();
```

---

#### 11.4 Arguments and Return Value

```
int multiply(int a, int b)
{
    return a * b;
}
```

---

### 12. Passing Arguments to Functions

---

#### 12.1 Call by Value

In call by value, **a copy of the actual value** is passed.

Changes inside the function **do not affect** the original variable.

##### Example

```
void change(int x)
{
    x = 20;
}

int main()
{
    int a = 10;
    change(a);
    printf("%d", a); // Output: 10
}
```

---

## 12.2 Call by Reference

In call by reference, **address of the variable** is passed.

Changes inside the function **affect the original variable**.

### Example

```
void change(int *x)
{
    *x = 20;
}

int main()
{
    int a = 10;
    change(&a);
    printf("%d", a); // Output: 20
}
```

---

## 13. Return Statement

The return statement:

- Sends a value back to the calling function
- Terminates function execution

### Example

```
int max(int a, int b)
{
    if (a > b)
        return a;
    return b;
}
```

---

## 14. Scope of Variables in Functions

| Variable Type | Scope                       |
|---------------|-----------------------------|
| Local         | Inside function             |
| Global        | Entire program              |
| Static        | Retains value between calls |

---

### Example: Static Variable

```
void counter()
{
    static int c = 0;
    c++;
    printf("%d\n", c);
}
```

---

## 15. Recursion

### 15.1 Definition

Recursion is a process in which a function **calls itself** directly or indirectly.

---

### 15.2 Requirements of Recursion

1. **Base condition** (termination condition)
  2. **Recursive call**
  3. Progress toward base condition
- 

## 16. Recursive Function Structure

```
return_type function()  
{  
    if (base_condition)  
        return value;  
    else  
        return function(smaller_problem);  
}
```

---

## 17. Example: Factorial Using Recursion

### Mathematical Definition

$$n! = n \times (n-1)!$$

$$0! = 1$$

### Program

```
int factorial(int n)  
{  
    if (n == 0 || n == 1)  
        return 1;  
    else  
        return factorial(n - 1) * n;  
}
```

---

## 18. Example: Fibonacci Series Using Recursion

```
int fibonacci(int n)  
{  
    if (n == 0)  
        return 0;  
    else if (n == 1 || n == 2)  
        return n - 1;
```

```
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}
```

---

## 19. Recursion vs Iteration

| Feature    | Recursion    | Iteration |
|------------|--------------|-----------|
| Memory     | More (stack) | Less      |
| Speed      | Slower       | Faster    |
| Code       | Shorter      | Longer    |
| Complexity | Higher       | Lower     |

---

## 20. When to Use Recursion

- ✓ Problems with natural repetition
  - ✓ Tree and graph traversal
  - ✓ Divide-and-conquer algorithms
  - ✗ Simple loops
  - ✗ Performance-critical code
- 

## 21. Common Errors in Recursion

1. Missing base condition
  2. Infinite recursion
  3. Stack overflow
  4. Incorrect return value
  5. Excessive recursive calls
- 

## 22. Example Program Combining Functions and Recursion

```
#include <stdio.h>

int factorial(int);

int factorial(int n)
{
    if (n == 0 || n == 1)
        return 1;
    else
        return factorial(n - 1) * n;
}

int main()
{
    int n = 5;
    printf("Factorial = %d", factorial(n));
    return 0;
}
```

---

## 23. Best Practices

- Use meaningful function names
- Keep functions small and focused
- Avoid unnecessary global variables
- Prefer iteration when recursion is not needed
- Always ensure base condition in recursion

---

## 24. Summary

- Functions divide programs into reusable blocks
  - User-defined functions improve modularity
  - Arguments and return values control data flow
  - Call by value and call by reference behave differently
  - Recursion solves problems using self-calling functions
  - Proper design avoids stack overflow and inefficiency
- 

```
// C Program to illustrate the use of user-defined function
```

```
#include <stdio.h>
```

```
// Function prototype
```

```
int sum(int, int);
```

```
// Function definition
```

```
int sum(int x, int y)
```

```
{
```

```
    int sum;
```

```
    sum = x + y;
```

```
    return sum;
```

```
}
```

```
// Driver code
```

```
int main()
```

```
{
```

```
    int x = 10, y = 11;
```

```
    // Function call
```

```
    int result = sum(x, y);
```



```
    printf("Sum of %d and %d = %d ", x, y, result);

    return 0;
}



---



#include <stdio.h>

// Function that takes parameters by value
// call by value
void func(int val)
{

    // Changing the value
    val = 123;
}

int main()
{
    int x = 1;

    // Passing x by value to func()
    func(x);
    printf("%d", x); // original value of x is not changed and
    remains 1

    return 0;
}



---



#include <stdio.h>
```

```

// Function that takes parameters by pointer
// call by reference using pointers
void func(int *val)
{

    // Changing the value
    *val = 123;
}

int main()
{
    int x = 1;

    // Passing address of x
    func(&x);
    printf("%d", x); // original value of x is changed to 123

    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Function definition
// factorial 5! = 1 * 2 * 3 * 4 * 5
// factorial 4! = 1 * 2 * 3 * 4
// factorial 3! = 1 * 2 * 3
// factorial 2! = 1 * 2

```

```
// factorial 1! = 1
// factorial 0! = 1 (base condition)
// factorial n! = 1 * 2 * 3 * ... * n-2 * n-1 * n
// factorial n-1! = 1 * 2 * 3 * ... * n-2 * n-1
// factorial n! = factorial (n-1)! * n (recursive condition)
```

```
int factorial(int);
```

```
int factorial(int n)      //recursive function
{
    if (n == 0 || n == 1)
        return 1;
    else
        return factorial(n - 1) * n;
}
```

```
int main()
{
    int a = 5;
    printf("Factorial of %d is %d\n",a , factorial(a));
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program using function to find average of three
numbers.
```

```
float calculate_average(int, int, int);
```

```
float calculate_average(int a, int b, int c)
{
    return (a + b + c) / 3;
}
```

```
int main()
{
    printf("Average of 3, 4, and 5 is: %f\n",
calculate_average(3, 4, 5));
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a function to convert Celsius temperature into
Fahrenheit.
```

```
float covert(float);
```

```
float covert(float celsius)
{
    return ((celsius * 9) / 5) + 32;
}
```

```
int main()
{
    printf("Temperature in Fahrenheit: %.2f\n", covert(37.0));
    return 0;
}
```

```
}
```

---

```
#include <stdio.h>
```

```
// Write a function to calculate force of attraction on a body  
of mass 'm' exerted by
```

```
// earth. Consider  $g = 9.8\text{m/s}^2$ 
```

```
float calculate_force(float);
```

```
float calculate_force(float force)
```

```
{  
    return force * 9.8;  
}
```

```
int main()
```

```
{  
    float mass;  
    printf("enter the value of mass: ");  
    scanf("%f", &mass);  
    printf("Force of attraction in Newtons: %.2f\n",  
calculate_force(mass));  
    return 0;  
}
```

---

```
#include <stdio.h>
```

```
// Write a program using recursion to calculate n th element of  
Fibonacci series.
```

```

// Fibonacci series = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
// fibonacci(n) = fibonacci(n-1) + fibonacci(n-2)

int fibonacci(int);

int fibonacci(int n)
{
    if (n == 0)
        return 0;
    else if (n == 1 || n == 2)
        return n - 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

int main()
{
    int n = 5;
    printf("%d th element of Fibonacci series is  %d\n ", n,
    fibonacci(n));

    return 0;
}

```

---

```

#include <stdio.h>

// What will the following line produce in a C program:
// int a = 4;

```

```
// printf("%d %d %d \n", a, ++a, a++)
```

```
int main()
```

```
{
```

```
    int a = 4;
```

```
    printf("%d %d %d \n", a, ++a, a++);
```

```
    /*
```

```
    evaluation order of compiler from right to left
```

```
    Step 1: a++ => returns 4, a becomes 5
```

```
    Step 2: ++a => a becomes 6, returns 6
```

```
    Step 3: a    => returns 6
```

```
    Output Explanation:
```

```
    a , ++a , a++
```

```
    a, ++a, 4  now a = 5
```

```
    a, 6, 4 now a = 6
```

```
    6, 6, 4
```

```
    but the output may vary depending on the compiler
```

```
    6 6 4
```

```
    4 5 5 is also possible
```

```
    */
```

```
    return 0;
```

```
}
```

---

```
#include <stdio.h>
```

```
// Write a recursive function to calculate the sum of first 'n'  
natural numbers.
```

```
// sum of n th natural numbers = 1 + 2 + 3 + 4 + 5 .... + n-1 + n
```

```
// sum(n) = sum(n-1) + n
```

```
float sum_natural(float);
```

```
float sum_natural(float n)
```

```
{
```

```
    if (n == 1)
```

```
        return 1;
```

```
    else
```

```
        return sum_natural(n - 1) + n;
```

```
}
```

```
int main()
```

```
{
```

```
    int n = 5;
```

```
    printf("the sum_natural of the first %d natural numbers is  
%.2f\n", n, sum_natural(n));
```

```
    return 0;
```

```
}
```

---

```
#include <stdio.h>
```

```
/*
```

```
Write a program using function to print the following pattern  
(first n lines)
```



```
*  
  
* * *  
  
* * * * *  
  
* * * * * * *
```

```
*/
```

```
int main()  
{  
    int n = 4;  
    for (int i = 0; i < n; i++)  
    {  
        // this loop runs from 0 to 2  
        // if i = 0 , print 1 star  *  
        // if i = 1 , print 3 stars * * *  
        // if i = 2 , print 5 stars * * * * *  
        // if i = 3 , print 7 stars * * * * * * *  
        // number of stars to be printed in each row = (2*i +  
1)        for (int j = 0; j < (2 * i + 1); j++)  
        {  
            printf("* ");  
        }  
        printf("\n"); // move to the next line after each row  
    }  
    return 0;  
}
```

---

---

## Chapter: Pointers in C Programming

---

### 1. Introduction

In C programming, a **pointer** is a variable that **stores the memory address of another variable**.

Pointers give C its power and flexibility, enabling **dynamic memory management, efficient function calls, array manipulation, and data structures** like linked lists, stacks, and trees.

Understanding pointers is essential for mastering C.

---

### 2. Memory Address and Address Operator

Every variable in a program is stored at a **unique memory location**.

#### Example

```
int x = 10;
```

```
printf("%p", &x);
```

- `&x` → Address of variable `x`
  - `%p` → Format specifier for address
- 

### 3. Definition of Pointer

A **pointer** is a variable that stores the **address of another variable**.

#### Syntax

```
data_type *pointer_name;
```

#### Example

```
int *p;
```

Here:

- `p` is a pointer to an integer

- \* indicates pointer variable
- 

#### 4. Pointer Initialization

A pointer must be initialized with the **address of a variable**.

##### Example

```
int x = 10;
```

```
int *p = &x;
```

- p stores address of x
  - \*p accesses the value stored at that address
- 

#### 5. Dereferencing a Pointer

Dereferencing means **accessing the value stored at the address** held by the pointer.

##### Example

```
int x = 10;
```

```
int *p = &x;
```

```
printf("%d", *p); // Output: 10
```

- p → address
  - \*p → value at that address
- 

#### 6. Pointer and Variable Relationship

```
int x = 5;
```

```
int *p = &x;
```

```
*p = 20;
```

```
printf("%d", x); // Output: 20
```

Changing \*p changes the value of x.

---

## 7. Size of Pointer

The size of a pointer depends on the **system architecture**, not on the data type.

```
printf("%d", sizeof(int *));
```

- 4 bytes (32-bit system)
- 8 bytes (64-bit system)

---

## 8. Types of Pointers

---

### 8.1 Integer Pointer

```
int a = 10;  
int *p = &a;
```

---

### 8.2 Character Pointer

```
char ch = 'A';  
char *cp = &ch;
```

---

### 8.3 Float Pointer

```
float f = 3.14;  
float *fp = &f;
```

---

## 9. Pointer Arithmetic

Pointers can be incremented or decremented.

### Rules

- Increment moves pointer by size of data type
- Depends on data type, not by 1 byte

### Example

```
int arr[] = {10, 20, 30};  
int *p = arr;  
  
p++;  
printf("%d", *p); // Output: 20
```

---

## 10. Pointers and Arrays

An array name itself is a **constant pointer** to the first element.

```
int arr[3] = {1, 2, 3};  
int *p = arr;  
  
Access using pointer  
for (int i = 0; i < 3; i++)  
{  
    printf("%d ", *(p + i));  
}
```

---

## 11. Array of Pointers

```
char *names[] = {"Ram", "Shyam", "Mohan"};
```

Each element stores address of a string.

---

## 12. Pointer to Pointer

A pointer that stores the address of another pointer.

### Example

```
int x = 10;  
int *p = &x;  
int **pp = &p;
```

```
printf("%d", **pp); // Output: 10
```

---

## 13. Pointers and Functions

---

### 13.1 Call by Value (Without Pointer)

```
void change(int x)
{
    x = 20;
}
```

---

### 13.2 Call by Reference (Using Pointer)

```
void change(int *x)
{
    *x = 20;
}
```

---

### Example

```
int a = 10;
change(&a);
printf("%d", a); // Output: 20
```

---

## 14. Pointer as Function Parameter

```
void printArray(int *arr, int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}
```

---

## 15. Returning Pointer from Function

```
int* getValue()  
{  
    static int x = 10;  
    return &x;  
}
```

⚠ Never return address of a local variable.

---

## 16. Null Pointer

A pointer that does not point to any valid memory location.

```
int *p = NULL;
```

Used to avoid garbage addresses.

---

## 17. Void Pointer

A pointer that can store address of any data type.

```
void *vp;  
int x = 10;  
vp = &x;
```

Typecasting required for dereferencing.

```
printf("%d", *(int *)vp); // Output: 10
```

---

## 18. Dangling Pointer

A pointer pointing to a memory location that has been freed.

```
int *p = (int *)malloc(sizeof(int));  
free(p);
```

After free, p becomes dangling.

---

## 19. Wild Pointer

A pointer that is declared but not initialized.

```
int *p; // wild pointer
```

---

## 20. Dynamic Memory Allocation

Using pointers with heap memory.

### Functions

- malloc()
  - calloc()
  - realloc()
  - free()
- 

### Example

```
int *p = (int *)malloc(5 * sizeof(int));
```

---

## 21. Common Pointer Errors

1. Dereferencing uninitialized pointer
  2. Memory leak (missing free)
  3. Returning local variable address
  4. Incorrect pointer arithmetic
  5. Using dangling pointers
- 

## 22. Example Program

```
#include <stdio.h>
```

```
int main()
```



```
{  
    int arr[] = {10, 20, 30};  
    int *p = arr;  
  
    for (int i = 0; i < 3; i++)  
    {  
        printf("%d ", *(p + i));  
    }  
  
    return 0;  
}
```

---

## 23. Best Practices

- Always initialize pointers
  - Use NULL when pointer is unused
  - Free dynamically allocated memory
  - Avoid excessive pointer arithmetic
  - Use meaningful pointer names
- 

## 24. Summary

- Pointers store memory addresses
  - Dereferencing accesses actual data
  - Arrays and pointers are closely related
  - Pointers enable call by reference
  - Dynamic memory relies on pointers
  - Proper usage avoids runtime errors
-

```

#include <stdio.h>

int main()
{
    int i = 72;

    int* j = &i; // j is a pointer to an integer and it stores
the address of i

    int k = *j; // k is assigned the value at the address
stored in j, which is the value of i

    // & is the address-of operator

    printf(" the address of i id %p\n", &i); //&i gives the
address of i

    printf(" the address of i id %p\n", j); // %p is address
format specifier for printing pointer addresses

    printf(" the address of j id %p\n", &j); //&j gives the
address of j

    printf(" the address of k id %p\n", &k); //&k gives the
address of k

    printf(" the address of i id %u\n", &i); //%u is used to
print unsigned int values

    printf(" the address of i id %u\n", j);
    printf(" the address of k is %u\n", &k);

    // * is the value-at-address operator (dereferencing
operator)

    // int *j means declare j as a pointer to an integer

    printf(" the value of i is %d\n", i);

    printf(" the value at address j is %d\n", *(&i)); // *(&i)
dereferences the address of i to get its value

```

```
    printf(" the value of address j is %d\n", *(&j)); // *(&j)
dereferences the address of j to get the value stored in j,
which is the address of i
```

```
    printf(" the value at address j is %d\n", *j);    // *j
dereferences j to get the value of i
```

```
    printf(" the value of k is %d\n", k);            // k
holds the value of i
```

```
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// This code will cause a compilation error
// because ptr is a constant pointer and cannot be reassigned
// to point to another variable (b in this case).
```

```
int main()
{
    int a = 90;
    int b = 50;

    // Creating a constant pointer
    int *const ptr = &a;

    // Trying to reassign it to b
    ptr = &b;

    return 0;
}
```

---

```
#include <stdio.h>
```

```
//use of function pointers
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
int main() {
```

```
    // Declare a function pointer that matches  
    // the signature of add() function  
    int (*fptr)(int, int);
```

```
    // Assign address of add()  
    fptr = &add;
```

```
    // Call the function via ptr  
    printf("%d", fptr(10, 5));
```

```
    return 0;  
}
```

---

```
#include <stdio.h>
```

```
int main()  
{
```

```

    char i = 'A';

    char* j = &i; // j is a pointer to a character and it
stores the address of i

    float k = 5.232;

    float* l = &k; // l is a pointer to a float and it stores
the address of k

    printf(" the address of i is %p\n", &i); //&i gives the
address of i

    printf(" the address of j is %p\n", j); //&j gives the
address of j

    printf(" the address of k is %p\n", &k); //&k gives the
address of k

    printf(" the address of l is %p\n", l); //&l gives the
address of l

    return 0;
}

```

---

```

#include <stdio.h>

```

```

//multiple levels of pointers (pointer to pointer)

```

```

int main()
{
    int var = 10;

    // Pointer to int
    int *ptr1 = &var;

```

```

    // Pointer to pointer (double pointer)
    int **ptr2 = &ptr1;

    // Accessing values using all three
    printf("var: %d\n", var);
    printf("*ptr1: %d\n", *ptr1);
    printf("**ptr2: %d", **ptr2);

    return 0;
}

```

---

```

#include <stdio.h>
// Demonstrates call by value in C

int sum(int, int);

int sum(int a, int b) // call by value function definition
{
    return a + b;
}

int main()
{
    int x = 4, y = 5;
    printf(" the sum is %d\n", sum(x, y)); // call by value
    function call
    printf(" the sum is %d\n", sum(4, 5));
    return 0;
}

```

---

```
#include <stdio.h>
```

```
// Demonstrates call by reference in C
```

```
int sum(int *, int *); // function prototype
```

```
// sum function should change the values of a and b to  
demonstrate call by reference
```

```
int sum(int *a, int *b) // call by reference function  
definition
```

```
{
```

```
    *a = 6;           // changing the value at the address  
pointed to by a
```

```
    *b = 7;           // changing the value at the address  
pointed to by b
```

```
    return *a + *b; // return the sum of the values at the  
addresses pointed to by a and b
```

```
}
```

```
int main()
```

```
{
```

```
    int x = 4, y = 5;
```

```
    //& operator is used to pass the address of variables x and  
y
```

```
    printf(" the sum is %d\n", sum(&x, &y)); // call by  
reference function call
```

```
    printf(" the value of x is %d\n", x);    // x should now be  
6
```

```
    printf(" the value of y is %d\n", y);    // y should now be
7
    // printf(" the sum is %d\n", sum(4, 5));
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Demonstrates swapping two integers using pointers in C
```

```
void swap(int *, int *); // function prototype
```

```
void swap(int *a, int *b) // swap function definition
{
    int temp; // temporary variable to hold value during swap
    temp = *a; // store the value at address a in temp
    *a = *b;   // store the value at address b in address a
    *b = temp; // store the value in temp in address b
}
```

```
int main()
{
    //&x and &y are the addresses of x and y respectively
    int x = 10, y = 20;
    printf("Before swap: x = %d, y = %d\n", x, y);
    swap(&x, &y); // call by reference
    printf("After swap: x = %d, y = %d\n", x, y);
    return 0;
}
```



---

```
#include <stdio.h>

// Demonstrates pointer to pointer in C

int main()
{
    int i = 6;    // normal integer variable
    int* j = &i;  // pointer to integer variable i
    int** k = &j; // pointer to pointer to integer

    printf(" the value of i is %d\n", i);    // direct
access to i
    printf(" the value of i is %d\n", *j);    // access to i
via pointer j
    printf(" the value of i is %d\n", **k);    // access to i
via pointer k
    printf(" the value of i is %d\n", *(&i));  // access to i
via its address
    printf(" the value of i is %d\n", **(&j)); // access to i
via pointer j's address
    printf(" the value of i is %d\n", ***(&k)); // access to i
via pointer k's address

    return 0;
}
```

---

```
#include <stdio.h>

// Write a program to print the address of a variable. Use this
address to get the
// value of the variable.
```

```

int main()
{
    int i = 2;
    int *ptr = &i;
    printf(" the value of i is %d\n", i);
    printf(" the address of i is %u\n", &i);
    printf(" the value of i using pointer is %d\n", *ptr);
    printf(" the address stored in pointer is %u\n", ptr);
    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Write a program having a variable 'i'. Print the address of
'i'. Pass this variable to
// a function and print its address. Are these addresses same?
Why?

```

```

int value_of_i(int *);

```

```

int value_of_i(int *ptr)
{
    printf(" the address of ptr is %u\n", ptr);
    printf(" the value of i using pointer is %d\n", *ptr);
    return *ptr;
}

```

```

int main()

```

```
{
    int i = 2;
    int *ptr = &i;
    printf(" the address of i is %u\n", &i);
    value_of_i(ptr);
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program to change the value of a variable to ten
times of its current value.
```

```
int ten_times(int *);
```

```
int ten_times(int *a)
{
    * a = (*a) * 10;
    printf(" ten times the value of i is %d\n", *a);
    return *a;
}
```

```
int main()
{
    int i = 10;
    printf(" the current value of i is %d\n", i);
    ten_times(&i);
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a function and pass the value by reference.
```

```
int refference_example(int *);
```

```
int refference_example(int *a)
```

```
{
```

```
    *a = *a + 10;
```

```
    printf(" the updated value of i inside the function is  
%d\n", *a);
```

```
    return *a;
```

```
}
```

```
int main()
```

```
{
```

```
    int i = 20;
```

```
    printf(" the current value of i is %d\n", i);
```

```
    refference_example(&i);
```

```
    return 0;
```

```
}
```

---

```
#include <stdio.h>
```

```
// Write a program using a function which calculates the sum  
and average of two
```

```
// numbers. Use pointers and print the values of sum and
average in main().
```

```
int sum(int *, int *);
```

```
float average(int *, int *);
```

```
int sum(int *x, int *y)
```

```
{
```

```
    int s = *x + *y;
```

```
    printf(" Sum of a and b is %d\n", s);
```

```
    return s;
```

```
}
```

```
float average(int *x, int *y)
```

```
{
```

```
    float avg = ((*x + *y) / 2.0);
```

```
    printf(" Average of a and b is %.2f\n", avg);
```

```
    return avg;
```

```
}
```

```
int main()
```

```
{
```

```
    int a = 10, b = 20;
```

```
    sum(&a, &b);
```

```
    average(&a, &b);
```

```
    return 0;
```

```
}
```

---

```
#include <stdio.h>
```

// Write a program to print the value of a variable i by using  
"pointer to pointer" type of variable.

```
int main()
{
    int i = 72;
    int *j = &i;
    int **k = &j;
    printf(" the value of i is %d\n", i);
    printf(" the value of i in pointer j is %d\n", *j);
    printf(" the value of i in pointer to pointer k is %d\n",
**k);

    return 0;
}
```

---

```
#include <stdio.h>
```

// Write a program to change the value of a variable to ten  
times of its current value  
// using call by value and verify that it does not change the  
value of the said variable.

```
int ten_times(int);
```

```
int ten_times(int a)
```

```
{
    printf(" ten times the value of i is %d\n", a * 10);
    return a;
}
```

```
}
```

```
int main()
```

```
{
```

```
    int i = 10;
```

```
    printf(" the current value of i is %d\n", i);
```

```
    ten_times(i);
```

```
    return 0;
```

```
}
```

---

---

## Chapter: Arrays in C Programming

---

### 1. Introduction

In C programming, an **array** is a collection of **elements of the same data type** stored in **contiguous memory locations**.

Arrays allow efficient storage and processing of large amounts of related data using a single variable name.

Without arrays, handling multiple values would require declaring many individual variables, which is inefficient and difficult to manage.

---

### 2. Definition of an Array

An **array** is a fixed-size data structure that stores **multiple values of the same data type** under a single name, accessed using an index.

---

### 3. Need for Arrays

Consider storing marks of 100 students.

✗ Without array:

```
int m1, m2, m3, ..., m100;
```

✓ With array:

```
int marks[100];
```

---

### 4. Declaration of Arrays

#### 4.1 Syntax

```
data_type array_name[size];
```

#### Example

```
int arr[5];
```

```
float prices[10];
```



```
char name[20];
```

---

## 5. Initialization of Arrays

---

### 5.1 Compile-Time Initialization

```
int arr[5] = {10, 20, 30, 40, 50};
```

---

### 5.2 Partial Initialization

```
int arr[5] = {1, 2};
```

Remaining elements are initialized to 0.

---

### 5.3 Size Omission

```
int arr[] = {2, 4, 6, 8};
```

Compiler automatically calculates size.

---

## 6. Accessing Array Elements

Array indexing starts from 0.

```
int arr[3] = {5, 10, 15};
```

```
printf("%d", arr[0]); // 5
```

```
printf("%d", arr[2]); // 15
```

---

## 7. Input and Output of Array Elements

### 7.1 Input Using Loop

```
int arr[5];
```

```
for (int i = 0; i < 5; i++)
```

```
{
    scanf("%d", &arr[i]);
}
```

---

## 7.2 Output Using Loop

```
for (int i = 0; i < 5; i++)
{
    printf("%d ", arr[i]);
}
```

---

## 8. Memory Representation of Arrays

Array elements are stored in **contiguous memory locations**.

```
int arr[3] = {10, 20, 30};
```

Memory layout:

```
arr[0]   arr[1]   arr[2]
```

Address calculation:

Address of arr[i] = Base + i × sizeof(data\_type)

---

## 9. Types of Arrays

---

### 9.1 One-Dimensional Array

#### Definition

A **one-dimensional array** stores elements in a linear sequence.

#### Example

```
int marks[5] = {70, 80, 65, 90, 75};
```

---

#### Program: Sum of Elements

```
int sum = 0;

for (int i = 0; i < 5; i++)
{
    sum = sum + marks[i];
}
```

---

## 9.2 Two-Dimensional Array

### Definition

A **two-dimensional array** is an array of arrays, often used to represent tables or matrices.

---

### Declaration

```
int matrix[3][3];
```

---

### Initialization

```
int matrix[2][2] = {{1, 2}, {3, 4}};
```

---

### Accessing Elements

```
printf("%d", matrix[1][0]); // 3
```

---

### Example: Matrix Addition

```
for (int i = 0; i < 2; i++)
{
    for (int j = 0; j < 2; j++)
    {
        sum[i][j] = a[i][j] + b[i][j];
    }
}
```

```
    }  
}
```

---

### 9.3 Multi-Dimensional Arrays

```
int arr[2][3][4];
```

Used in advanced applications like scientific computing.

---

## 10. Arrays and Functions

---

### 10.1 Passing Array to Function

Array name acts as a pointer to the first element.

```
void display(int arr[], int n)  
{  
    for (int i = 0; i < n; i++)  
        printf("%d ", arr[i]);  
}
```

---

### Function Call

```
display(arr, 5);
```

---

## 11. Arrays and Pointers

Array name represents the base address.

```
int arr[3] = {10, 20, 30};
```

```
int *p = arr;
```

Access using pointer:

```
printf("%d", *(p + 1)); // 20
```

---

## 12. Character Arrays (Strings)

Strings are character arrays terminated by '\0'.

```
char name[] = "Shamik";
```

---

### Input String

```
fgets(name, sizeof(name), stdin);
```

---

## 13. Array of Strings

```
char names[3][20] = {"Ram", "Shyam", "Mohan"};
```

---

## 14. Dynamic Arrays

Using dynamic memory allocation.

```
int *arr;  
arr = (int *)malloc(5 * sizeof(int));
```

---

## 15. Searching in Arrays

---

### 15.1 Linear Search

// implementation of Linear Search

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int linear_Search(int arr[], int size, int element)
```

```
{  
    for (int i = 0; i < size; i++)  
    {  
        if (arr[i] == element)  
        {
```

```

        return i;
    }
}
return -1;
}
int main()
{
    int arr[MAX] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
    int element;
    printf("Enter the element to be search : \n");
    scanf("%d", &element);
    int result = linear_Search(arr, MAX, element);
    if (result == -1)
    {
        printf("Element not found in the array.\n");
    }
    else
    {
        printf(" The Element %d found at index %d in the
array.\n", element, result);
    }
    return 0;
}

```

---

## 15.2 Binary Search

// implementation of Binary Search

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10
int binary_search(int arr[], int size, int element)
{
    int low, mid, high;
    low = 0;
    high = size - 1;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (arr[mid] == element)
        {
            return mid;
        }
        if (arr[mid] < element)
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }
    return -1;
}
int main()
{
```

```

int arr[MAX] = {2, 3, 6, 9, 56, 64, 74, 82, 86, 95};
int element;
printf("enter the element to be searched :\n");
scanf("%d", &element);
int result = binary_search(arr, MAX, element);
if (result == -1)
{
    printf(" the element is not found in the arrray");
}
else
{
    printf("the element %d is found in %d index of the
array", element, result);
}
return 0;
}

```

---

## 16. Sorting Arrays

---

### 16.1 Bubble Sort

```
#include <stdio.h>
```

```

void printArray(int *A, int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("\n%d", A[i]);
    }
}

```



```

        printf("\n");
    }
void bubbleSort(int *A, int n)
{
    int temp;
    for (int i = 0; i < n - 1; i++) // number of passes
    {
        for (int j = 0; j < n - 1 - i; j++) // comparisons in
each pass
        {
            if (A[j] > A[j + 1])
            {
                temp = A[j];
                A[j] = A[j + 1];
                A[j + 1] = temp;
            }
        }
    }
}

int main()
{
    int A[] = {12, 54, 65, 7, 23, 9};
    int n = 6;
    printArray(A, n); // print array before sorting
    bubbleSort(A, n); // function to sort the array
    printArray(A, n); // print array after sorting
    return 0;
}

```

---

## 16.2 Insertion Sort

```
#include <stdio.h>
```

```
void printArray(int *A, int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("\n%d", A[i]);
    }
    printf("\n");
}
```

```
void insertionSort(int *A, int n) // ascending order
{
    int key, j;
    for (int i = 1; i < n; i++) // loop for passes
    {
        key = A[i];
        j = i - 1;
        while (j >= 0 && A[j] > key)
        {
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = key;
    }
}
```

```

/*
void insertionSort(int *A, int n) //desending order
{
    int key, j;
    for (int i = 1; i <= n - 1; i++) // loop for passes
    {
        key = A[i];
        j = i - 1;
        while (j >= 0 && A[j] < key) // loop for each passes
        {
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = key;
    }
}
*/

```

```

int main()
{
    int A[] = {12, 54, 65, 7, 23, 9};
    int n = 6;
    printArray(A, n);    // print array before sorting
    insertionSort(A, n); // function to sort the array
    printArray(A, n);    // print array after sorting
    return 0;
}

```

---

### 16.3 Selection Sort

```
#include <stdio.h>
```

```
void printArray(int *A, int n)
```

```
{
    for (int i = 0; i < n; i++)
    {
        printf("\n%d", A[i]);
    }
    printf("\n");
}
```

```
void selectionSort(int *A, int n)
```

```
{
    int indexOfMin, temp;
    printf("Running Selection Sort....\n");
    for (int i = 0; i < n - 1; i++)
    {
        indexOfMin = i;
        for (int j = i + 1; j < n; j++)
        {
            if (A[j] < A[indexOfMin])
            {
                indexOfMin = j;
            }
        }
        temp = A[i]; // swap A[i] and A[indexOfMin]
        A[i] = A[indexOfMin];
        A[indexOfMin] = temp;
    }
}
```

```

    }
}
int main()
{
    int A[] = {3, 5, 2, 13, 12};
    int n = 5;
    printArray(A, n);    // print array before sorting
    selectionSort(A, n); // function to sort the array
    printArray(A, n);    // print array after sorting
    return 0;
}

```

---

## 16.4 Quick Sort

```

#include <stdio.h>

void Printarray(int *A, int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("\n%d", A[i]);
    }
    printf("\n");
}

int partition(int A[], int low, int high)
{
    int pivot = A[low];
    int i = low + 1;
    int j = high;
    int temp;

```

```

do
{
    while (A[i] <= pivot)
    {
        i++;
    }
    while (A[j] > pivot)
    {
        j--;
    }
    if (i < j)
    {
        temp = A[i];
        A[i] = A[j];
        A[j] = temp;
    }
} while (i < j);
temp = A[low]; // swap A[low] and A[j]
A[low] = A[j];
A[j] = temp;
return j;
}

void Quicksort(int A[], int low, int high)
{
    int partitionIndex; // index of pivot after partition
    if (low < high)
    {
        partitionIndex = partition(A, low, high);
    }
}

```

```

        Quicksort(A, low, partitionIndex - 1); // to sort left
subarray
        Quicksort(A, partitionIndex + 1, high); // to sort
right subarray
    }
}
int main()
{
    int A[] = {12, 54, 65, 7, 23, 9};
    int n = 6;
    Printarray(A, n);        // print array before sorting
    Quicksort(A, 0, n - 1); // function to sort the array
    Printarray(A, n);        // print array after sorting
    return 0;
}

```

---

## 16.5 Merge Sort

```
#include <stdio.h>
```

```

void Printarray(int *A, int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("\n%d", A[i]);
    }
    printf("\n");
}

```

```
void Merge(int A[], int low, int mid, int high)
```

```

{
    int i, j, k, B[100];
    i = low;
    j = mid + 1;
    k = low;

    while (i <= mid && j <= high)
    {
        if (A[i] < A[j])
        {
            B[k++] = A[i++];
        }
        else
        {
            B[k++] = A[j++];
        }
    }

    while (i <= mid)
    {
        B[k++] = A[i++];
    }

    while (j <= high)
    {
        B[k++] = A[j++];
    }
}

```



```

        for (int i = low; i <= high; i++)
        {
            A[i] = B[i];
        }
    }

```

```

void Mergesort(int A[], int low, int high)
{
    int mid;
    if (low < high)
    {
        mid = (low + high) / 2;
        Mergesort(A, low, mid);
        Mergesort(A, mid + 1, high);
        Merge(A, low, mid, high);
    }
}

```

```

int main()
{
    int A[] = {9, 14, 4, 8, 7, 5, 6};
    int n = 7;

    Printarray(A, n);        // print array before sorting
    Mergesort(A, 0, n - 1); // function to sort the array
    Printarray(A, n);        // print array after sorting

    return 0;
}

```

}

---

## 17. Common Errors with Arrays

1. Index out of bounds
2. Uninitialized array elements
3. Incorrect loop limits
4. Buffer overflow
5. Confusing arrays with pointers

---

## 18. Advantages of Arrays

- Efficient data storage
- Easy traversal using loops
- Better code organization
- Reduced memory wastage

---

## 19. Limitations of Arrays

- Fixed size
- Cannot grow dynamically (static arrays)
- Insertion and deletion are costly

---

## 20. Example Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[5], sum = 0;
```

```
    for (int i = 0; i < 5; i++)
        scanf("%d", &arr[i]);

    for (int i = 0; i < 5; i++)
        sum += arr[i];

    printf("Sum = %d", sum);
    return 0;
}
```

---

## 21. Best Practices

- Always validate array index
  - Use constants for array size
  - Prefer loops over hardcoding
  - Use dynamic arrays when size is unknown
- 

## 22. Summary

- Arrays store multiple values of same type
  - Stored in contiguous memory
  - Index starts from 0
  - Support multi-dimensional data
  - Strongly connected with pointers and functions
- 

```
#include <stdio.h>
```

```
// Update Array Elements
```

```
int main()
{
    int arr[5] = {2, 4, 8, 12, 16};
    // Update the first element of the array
    arr[0] = 1; // Updated from 2 to 1
    printf("%d", arr[0]);
    return 0;
}
```

---

```
#include <stdio.h>
// program to take input in an array and print it
int main()
{
    int marks[5];

    printf("enter marks of 5 students\n");

    // scanf("%d", &marks[0]);
    // scanf("%d", &marks[1]);
    // scanf("%d", &marks[2]);
    // scanf("%d", &marks[3]);
    // scanf("%d", &marks[4]);

    for (int i = 0; i < 5; i++)
    {
        scanf("%d", &marks[i]);
    }
}
```

```
    for (int i = 0; i < 5; i++)
    {
        printf("the value of marks at index %d is %d\n", i,
marks[i]);
    }
    // printf("Marks of student 1: %d\n", marks[0]);
    // printf("Marks of student 2: %d\n", marks[1]);
    // printf("Marks of student 3: %d\n", marks[2]);
    // printf("Marks of student 4: %d\n", marks[3]);
    // printf("Marks of student 5: %d\n", marks[4]);

    return 0;
}
```

---

```
#include <stdio.h>
// Array Traversal
// reverse order printing of array elements

int main()
{
    int arr[5] = {2, 4, 8, 12, 16};

    // Printing array elements in normal order
    printf("Printing Array Elements\n");
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
    // Printing array element in reverse order
    printf("Printing Array Elements in Reverse\n");
    for (int i = 4; i >= 0; i--)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Find Size of Array
```

```
int main()
{
    int arr[5] = {2, 4, 8, 12, 16};

    int size = sizeof(arr) / sizeof(arr[0]); // Total size of
    array divided by size of one element
    printf("%d", size);
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Find Size of Array using Pointer Arithmetic
```

```
int main()
{

    int arr[5] = {1, 2, 3, 4, 5};

    // Calculate size using pointer arithmetic
    int n = *(&arr + 1) - arr; // Points to the end of the
array and subtracts the start address

    printf("%d", n);
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Demonstrates pointer arithmetic
```

```
int main()
{
    // int a = 5;
    // int *ptr = &a;
    // printf("the address of a is %u\n", &a);
    // printf("the address of a is %u\n", ptr);
    // ptr++; // incrementing the pointer by 4 bytes (cause int
is 4 bytes)
    // printf("the address of a after incrementing pointer is
%u\n", ptr);
    // return 0;
```

```
// char a = 'B';
// char *ptr = &a;
// printf("the address of a is %u\n", &a);
// printf("the address of a is %u\n", ptr);
// ptr++; // incrementing the pointer by 1 byte (cause char
is 1 byte)
// printf("the address of a after incrementing pointer is
%u\n", ptr);
// return 0;
```

```
int arr[5] = {10, 20, 30, 40, 50};
int *p1 = arr;    // Points to arr[0]
int *p2 = &arr[3]; // Points to arr[3]
```

```
printf("Initial pointer p1 points to value: %d\n", *p1);
printf("Initial pointer p2 points to value: %d\n\n", *p2);
```

```
// 1. Addition of a number to a pointer
```

```
p1 = p1 + 2;
printf("After p1 = p1 + 2, p1 points to: %d\n", *p1);
```

```
// 2. Subtraction of a number from a pointer
```

```
p1 = p1 - 1;
printf("After p1 = p1 - 1, p1 points to: %d\n", *p1);
```

```
// 3. Subtraction of one pointer from another
```

```
int diff = p2 - p1;
printf("Difference between p2 and p1: %d elements\n",
diff);
```



```
// 4. Comparison of two pointers
if (p1 < p2)
{
    printf("p1 is pointing to an earlier element in the
array than p2\n");
}
else if (p1 > p2)
{
    printf("p1 is pointing to a later element in the array
than p2\n");
}
else
{
    printf("p1 and p2 point to the same element\n");
}
return 0;
}
```

---

```
#include <stdio.h>
```

```
// Accessing Array Elements using Pointers
```

```
int main()
```

```
{
```

```
    // Declare an array
```

```
    int arr[3] = {5, 10, 15};
```

```

    // Access first element using index
    printf("%d\n", arr[0]);

    // Access first element using pointer
    printf("%d\n", *arr);

    return 0;
}

```

---

```

#include <stdio.h>
// program to show how to access array elements using pointers
int main()
{
    int marks[] = {12, 324, 56, 66};

    int *ptr = &marks[0]; // pointer pointing to first element
of array
    // int *ptr = marks;    // same as int *ptr = &marks[0];

    /*for (int i = 0; i < 4; i++)
    {
        printf("the value of marks at index %d is %d\n", i,
marks[i]);
    }*/
    for (int i = 0; i < 4; i++)
    {
        // dereferencing pointer to get value at that address
        printf("the value of marks at index %d is %d\n", i,
*ptr);
    }
}

```

```
        ptr++; // incrementing pointer to point to next element
of array
    }
```

```
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Traverse Array using Pointer to First Element
```

```
int main()
{
    int arr[5] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]); // Calculate size of
the array
```

```
    // Defining the pointer to first element of array
```

```
    int *ptr = &arr[0]; // or simply int *ptr = arr;
```

```
    // Traversing array using pointer arithmetic
```

```
    for (int i = 0; i < n; i++) // or i < 5
```

```
        printf("%d ", ptr[i]);
```

```
    return 0;
```

```
}
```

---

```
#include <stdio.h>
```

```
// Demonstrates 2D arrays from user input and printing them in
matrix form
```

```

int main()
{
    int arr[3][2];

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            printf("enter value for arr[%d][%d]: ", i, j);
            scanf("%d", &arr[i][j]);
        }
    }

    // for (int i = 0; i < 3; i++)
    // {
    //     for (int j = 0; j < 2; j++)
    //     {
    //         printf("the value of arr[%d][%d] is %d\n", i, j,
arr[i][j]);
    //     }
    // }

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            printf("%d\t", arr[i][j]);
        }
        printf("\n");
    }
}

```

```

    }

    return 0;
}



---


#include <stdio.h>

// Traverse 2D Array using Pointer Notation

int main()
{
    int arr[3][3] = {{1, 2, 3}, {5, 6, 7}, {9, 10, 12}};

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)

            // (*(arr + 1) + j) is equivalent to arr[1][j]
            printf("%d ", (*(arr + i) + j)); // Accessing
            element at i th row and j th column using pointer notation
            printf("\n");
        }

    return 0;
}



---


#include <stdio.h>

// create a metrix of 3*3*3 using 3d array

```

```
int main()
{
    int arr[3][3][3];
    printf(" Enter 27 values for the 3x3x3 matrix:\n");

    // Input values into the 3D array
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            for (int k = 0; k < 3; k++)
            {
                scanf("%d", &arr[i][j][k]);
            }
        }
    }

    // Output the values from the 3D array
    printf(" The 3x3x3 matrix is:\n");
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            for (int k = 0; k < 3; k++)
            {
                printf("%d ", arr[i][j][k]);
            }
            printf("\n");
        }
    }
}
```

```

        }
        printf("\n");
    }

    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Traverse 3D Array using Pointer Notation

```

```

int main()
{
    int arr[2][3][2] = {{{5, 10}, {6, 11}, {7, 12}},
                        {{20, 30}, {21, 31}, {22, 32}}};

    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            for (int k = 0; k < 2; k++)

                // *((*(arr + i) + j) + k) is equivalent to
arr[i][j][k]

                printf("%d ", *((*(arr + i) + j) + k)); //
Accessing element at i th depth, j th row and k th column using
pointer notation

                printf("\n");
            }
        }
    }
}

```

```
        printf("\n");
    }

    return 0;
}



---


#include <stdio.h>

//2D array input from user

int main()
{
    int arr[3][3];
    printf("Enter elements for a 3x3 matrix:\n");
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            scanf("%d", &arr[i][j]); // Taking input for
            element at i th row and j th column
        }
    }

    printf("The entered 2D array is:\n");
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
```



```

        printf("%d ", arr[i][j]); // Accessing element at i
th row and j th column
    }
    printf("\n");
}

return 0;
}

```

---

```

#include <stdio.h>

```

```

//passing 2D array by function

```

```

void print2DArray(int arr[3][3], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            printf("%d ", arr[i][j]); // Accessing element at i
th row and j th column
        }
        printf("\n");
    }
}

```

```

int main()
{
    int arr[3][3];

```

```

printf("Enter elements for a 3x3 matrix:\n");
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        scanf("%d", &arr[i][j]); // Taking input for
element at i th row and j th column
    }
}
printf("The entered 2D array is:\n");
print2DArray(arr, 3, 3);
return 0;
}

```

---

```

#include <stdio.h>

```

```

// Passing 2D array to function using pointer

```

```

// Funtion that takes 2d array as parameter

```

```

void print2DArray(int (*arr)[3], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            printf("%d ", (*(arr + i) + j)); // Accessing
element at i th row and j th column
        printf("\n");
    }
}

```

```

int main()
{
    int arr[3][3];
    printf("Enter elements for a 3x3 matrix:\n");
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            scanf("%d", &arr[i][j]); // Taking input for
element at i th row and j th column
        }
    }
    print2DArray(arr, 3, 3); // Passing 2D array to function
using pointer

    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Initializing and printing 3D array

```

```

int main()
{

    int arr[2][3][2] = {{{1, 1}, {2, 3}, {4, 5}}, {{6, 7}, {8,
9}, {10, 11}}};

```

```

// Loop through the depth
for (int i = 0; i < 2; ++i)
{

    // Loop through the rows of each depth
    for (int j = 0; j < 3; ++j)
    {

        // Loop through the columns of each row
        for (int k = 0; k < 2; ++k)
        {
            printf("arr[%i][%i][%i] = %d    ", i, j, k,
arr[i][j][k]);
        }
        printf("\n");
    }
    printf("\n\n");
}
return 0;
}

```

---

```

#include <stdio.h>

```

```

// find maximum value in an array element

```

```

int main()

```

```

{

```

```

    // Initialize an array

```

```

    int arr[] = {23, 12, 45, 20, 90, 89, 95, 32, 65, 19};

```

```

    // Find the size of the array
    int n = sizeof(arr) / sizeof(arr[0]);

    // Intialize the variable which will denote the maximum
    element

    // Find the maximum value in the array and store it in max
    int max = arr[0]; // assuming first element is the maximum
    for (int i = 0; i < n; i++)
    {
        if (max < arr[i])
            max = arr[i];
    }

    // print the elements of the array
    printf("Array Elements: ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // print the maximum value
    printf("The maximum value of the array is: %d", max);

    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Function to calculate sum of array elements

```

```

int getSum(int arr[], int n)
{

    int sum = 0; // Initialize sum to 0
    for (int i = 0; i < n; i++)
    {
        sum = sum + arr[i]; // Add each element to sum
    }
    return sum;
}

//using recursion
/*
int getSum(int arr[], int n) // Recursive function to calculate
sum of array elements
{

    // Base case: No elements left
    if (n == 0)
        return 0;

    // Add current element and move to the rest of the array
    return arr[n - 1] + getSum(arr, n - 1); // Recursive call
}
*/

int main()
{

```

```
int arr[] = {1, 2, 3, 4, 5};
int n = sizeof(arr) / sizeof(arr[0]);

//int result = getSum(arr, n);

printf(" the sum of the array elements is: %d", getSum(arr,
n));
return 0;
}
```

---

```
#include <stdio.h>
```

```
// Reverse Array Elements by swap the elements
```

```
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void reverse(int arr[], int n)
{
    for (int i = 0; i < n / 2; i++)
    {
        int temp = arr[i];
        arr[i] = arr[n - i - 1];
        arr[n - i - 1] = temp;
    }
}
```

```

    }
    //{1,2,3,4,5,6} => {6,5,4,3,2,1}
    // n = 6
    //{    1    ,    2    ,    3    ,    4    ,    5    ,    6
}
    // index 0 , index 1 , index 2 , index 3 , index 4 , index
5
    //(index 0 , index 5) => {6,2,3,4,5,1}
    //(index 1 , index 4) => {6,5,3,4,2,1}
    //(index 2 , index 3) => {6,5,4,3,2,1} stop!
    // i from 0 to n/2
    // arr[i] = arr[n-i-1]
    // arr[i] = arr[n - 1 - i]
    // arr[0] = arr[5 - 1 - 0] = arr[4],
    // arr[1] = arr[5 - 1 - 1] = arr[3],
    // arr[2] = arr[5 - 1 - 2] = arr[2],
    // arr[3] = arr[5 - 1 - 3] = arr[1],
    // arr[4] = arr[5 - 1 - 4] = arr[0] ....
}

```

```

int main()
{
    int arr[] = {1, 2, 3, 4, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: ");
    printArray(arr, n);
    printf("Reversed array: ");
    reverse(arr, n);
    printArray(arr, n);
}

```



```
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Reverse Array Elements by for loop
```

```
void printArray(int arr[], int n)
```

```
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
void printReverse(int arr[], int n)
```

```
{
    for (int i = n - 1; i >= 0; i--)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
int main()
```

```
{
    int arr[] = {1, 2, 3, 4, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: ");
```

```
    printArray(arr, n);  
    printf("Reversed array: ");  
    printReverse(arr, n);  
    return 0;  
}
```

---

```
#include <stdio.h>
```

```
// Reverse Array Elements Using Temporary Array
```

```
void printArray(int arr[], int n)  
{  
    for (int i = 0; i < n; i++)  
    {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
}
```

```
void reverseArray(int arr[], int n)  
{  
    int temp[n];  
  
    // Store elements into temp in reverse order  
    for (int i = 0; i < n; i++)  
    {  
        temp[i] = arr[n - 1 - i];  
    }  
}
```

```

    // temp[i] = arr[n - 1 - i]
    // temp[0] = arr[5 - 1 - 0] = arr[4],
    // temp[1] = arr[5 - 1 - 1] = arr[3],
    // temp[2] = arr[5 - 1 - 2] = arr[2],
    // temp[3] = arr[5 - 1 - 3] = arr[1],
    // temp[4] = arr[5 - 1 - 4] = arr[0]

    // Copy reversed array back to original array
    for (int i = 0; i < n; i++)
    {
        arr[i] = temp[i];
    }

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

int main()
{
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    // Print the array as same as arr[]
    printf("Original array: ");
    printArray(arr, n);

    // Reverse the array arr[]
    printf("Reversed array: ");
    reverseArray(arr, n);
}

```

```
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Insert Element at Specific Position
```

```
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
void insert(int arr[], int *n, int pos, int val)
{
```

```
    // Shift elements to the right
    for (int i = *n; i > pos; i--)
        arr[i] = arr[i - 1];
```

```
    // Insert val at the specified position
    arr[pos] = val;
```

```
    // Increase the current size
```

```

        (*n)++;
    }

int main()
{
    int arr[] = {20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int pos, value;
    printf("The array is:\n");
    printArray(arr, n);

    printf("Enter the position (1 to %d) where you want to
insert: ", n + 1);
    scanf("%d", &pos);
    // Convert 1-based to 0-based
    pos--;

    printf("Enter the value to insert: ");
    scanf("%d", &value);

    insert(arr, &n, pos, value);

    printf("The array after insertion:\n");
    printArray(arr, n);

    return 0;
}

```

---

```
#include <stdio.h>
```

```
// Insert an Element at the End of an Array
```

```
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
void insertLast(int arr[], int *n, int val)
{

    // Insert val at last
    arr[*n] = val;

    // Increase the current size
    (*n)++;
}
```

```
int main()
{
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int value;
    printf("The array is:\n");
}
```

```

    printArray(arr, n);
    printf("Enter the value to insert at the end of the array:");
    scanf("%d", &value);

    // Insert the value at the end
    insertLast(arr, &n, value);
    printf("The array after insertion:\n");
    printArray(arr, n);

    return 0;
}

```

---

```

#include <stdio.h>

```

```

// replace Element at Specific Position in an Array

```

```

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

```

```

void replace(int arr[], int n, int pos, int val)
{
    if (pos < 0 || pos >= n)

```

```

    {
        printf("Invalid position!\n");
        return;
    }
    arr[pos] = val;
}

int main()
{
    int arr[] = {10, 20, 30, 60, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int pos, value;
    printf("The array is:\n");
    printArray(arr, n);

    printf("Enter the position (1 to %d) where you want to
replace: ", n + 1);
    scanf("%d", &pos);
    // Convert 1-based to 0-based
    pos--;

    printf("Enter the value to insert: ");
    scanf("%d", &value);

    replace(arr, n, pos, value);

    printf("The array after replacement:\n");
    printArray(arr, n);

```



```
    return 0;
}



---



#include <stdio.h>

// Delete the Given Element from an Array

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void delete(int arr[], int *n, int key)
{
    // Find the element
    int i = 0;
    while (arr[i] != key)
        i++;

    // Shifting the right side elements one
    // position towards left
    for (int j = i; j < *n - 1; j++)
    {
```

```

        arr[j] = arr[j + 1];
    }

    // Decrease the size
    (*n)--;
}

int main()
{
    int arr[] = {10, 20, 30, 35, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key;
    printf("The array is:\n");
    printArray(arr, n);

    printf("Enter the element which you want to delete: ");
    scanf("%d", &key);

    // Delete the key from array
    delete(arr, &n, key);

    printf("The array after insertion:\n");
    printArray(arr, n);

    return 0;
}

```

---

```

#include <stdio.h>

```

```
// Delete the Last Element from an Array
```

```
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
void deleteLast(int arr[], int *n)
{
    // Decrease the size
    (*n)--;
}
```

```
int main()
{
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("The array is:\n");
    printArray(arr, n);

    // Delete the last element
    deleteLast(arr, &n);
}
```

```

    printf("The array after insertion:\n");
    printArray(arr, n);

    return 0;
}

```

---

```

#include <stdio.h>

```

```

// Create an array of 10 numbers. Verify using pointer
arithmetic that (ptr+2) points
// to the third element where ptr is a pointer pointing to the
first element of the array.

```

```

int main()
{
    int arr[10] = {0 , 1, 2, 3, 4, 5, 6, 7, 8, 9};
    int * ptr = arr;
    printf("The address at index 3 is %u\n", (ptr + 2));
    printf("The value at index 3 is %d\n", *(ptr + 2));

    return 0;
}

```

---

If S[3] is a 1-D array of integers then \*(S+3) refers to the third element:

- (i) True.
- (ii) False.
- (iii) Depends.

ans -> (ii) False.

In C, array indexing is zero-based.

If:

```
int S[5] = {10, 20, 30, 40, 50};
```

Then:

ExpressionRefers to

S or &S[0]first element

\*(S + 0) S[0] → 10

\*(S + 1) S[1] → 20

\*(S + 2) S[2] → 30

\*(S + 3) S[3] → fourth element

So:

Third element → S[2] → \*(S + 2)

\*(S + 3) → fourth element

Key rule to remember (very exam-friendly)

\*(S + i) ≡ S[i]

Index i always counts from 0, not 1.

---

```
#include <stdio.h>
```

```
// Write a program to create an array of 10 integers and store  
multiplication table of 5 in it.
```

```
int main()  
{  
    int multi[10];  
    for (int i = 0; i < 10; i++)  
    {  
        multi[i] = 5 * (i + 1);  
    }  
}
```

```
    }  
    for (int i = 0; i < 10; i++)  
    {  
        printf("the value of 5 X %d is %d\n", i + 1, multi[i]);  
    }  
    printf("\n");  
    return 0;  
}
```

---

```
#include <stdio.h>
```

```
// Write a program to create an array of 10 integers and store  
multiplication table
```

```
// for a general input provided by the user using scanf.
```

```
int main()  
{  
    int n;  
    int multi[10];  
    printf(" enter the number to generate the table for: \n");  
    scanf("%d", &n);  
  
    for (int i = 0; i < 10; i++)  
    {  
        multi[i] = n * (i + 1);  
    }  
    for (int i = 0; i < 10; i++)  
    {
```

```
        printf("the value of %d X %d is %d\n", n, i + 1,
multi[i]);
    }
    printf("\n");
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program to create an array of 10 integers and store
multiplication table
```

```
// for a general input provided by the user using scanf.
```

```
int main()
{
    int n;
    int multi[10];
    printf(" enter the number to generate the table for: \n");
    scanf("%d", &n);

    for (int i = 0; i < 10; i++)
    {
        multi[i] = n * (i + 1);
    }
    for (int i = 0; i < 10; i++)
    {
        printf("the value of %d X %d is %d\n", n, i + 1,
multi[i]);
    }
}
```

```
    printf("\n");
    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Write a program containing a function which reverses the
array passed to it.
```

```
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
// swapping of two numbers with temp variable
// temp = a;
// a = b;
// b = temp;
```

```
//{1,2,3,4,5,6} => {6,5,4,3,2,1}
// n = 6
//{ 1 , 2 , 3 , 4 , 5 , 6 }
// index 0 , index 1 , index 2 , index 3 , index 4 , index 5
//(index 0 , index 5) => {6,2,3,4,5,1}
//(index 1 , index 4) => {6,5,3,4,2,1}
```



```

//(index 2 , index 3) => {6,5,4,3,2,1} stop!
// i from 0 to n/2
// arr[i] = arr[n-i-1]
// arr[i] = arr[n - 1 - i]
// arr[0] = arr[5 - 1 - 0] = arr[4],
// arr[1] = arr[5 - 1 - 1] = arr[3],
// arr[2] = arr[5 - 1 - 2] = arr[2],
// arr[3] = arr[5 - 1 - 3] = arr[1],
// arr[4] = arr[5 - 1 - 4] = arr[0] ....

```

```

void reverse(int arr[], int n)
{
    for (int i = 0; i < n / 2; i++) // i from 0 to n/2
    {
        int temp = arr[i];
        arr[i] = arr[n - i - 1];
        arr[n - i - 1] = temp;
    }
}

```

```

int main()
{
    int arr[] = {1, 2, 3, 4, 5, 6};
    printArray(arr, 6);
    reverse(arr, 6);
    printArray(arr, 6);
    return 0;
}

```

---

```
#include <stdio.h>
```

```
// Write a program containing functions which counts the number  
of positive
```

```
// integers in an array.
```

```
int countPositive(int arr[], int n)
```

```
{  
    int count = 0;  
    for (int i = 0; i < n; i++)  
    {  
        if (arr[i] > 0)  
        {  
            count++;  
        }  
    }  
    return count;  
}
```

```
int countEven(int arr[], int n)
```

```
{  
    int count = 0;  
    for (int i = 0; i < n; i++)  
    {  
        if (arr[i] % 2 == 0)  
        {  
            count++;  
        }  
    }  
}
```

```
    }  
    return count;  
}
```

```
int main()  
{  
    int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};  
  
    printf("The number of positive elements in the array is:  
%d\n", countPositive(arr, 10));  
    printf("The number of even elements in the array is: %d\n",  
countEven(arr, 10));  
    return 0;  
}
```

---

```
#include <stdio.h>
```

```
// Create an array of size 3 x 10 containing multiplication  
tables of the numbers 2,7  
// and 9 respectively.
```

```
int main()  
{  
    int arr[3][10];  
    int num[] = {2, 7, 9};  
    for (int i = 0; i < 3; i++)  
    {  
        for (int j = 0; j < 10; j++)  
        {
```

```

        arr[i][j] = num[i] * (j + 1);
    }
}
for (int i = 0; i < 3; i++)
{
    printf("Multiplication table of %d is:\n", num[i]);
    for (int j = 0; j < 10; j++)
    {
        printf("%d X %d = %d\n", num[i], j + 1, arr[i][j]);
    }
    printf("\n");
}

return 0;
}

```

---

```

#include <stdio.h>

```

```

// Create an array of size 3 x 10 containing multiplication
tables

```

```

// for a custom input given by the user of 3 numbers.

```

```

int main()
{
    int n1, n2, n3;
    printf("Enter three numbers to generate their
multiplication tables:\n");
    scanf("%d %d %d", &n1, &n2, &n3);
    int arr[3][10];
}

```

```
int num[] = {n1, n2, n3};
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 10; j++)
    {
        arr[i][j] = num[i] * (j + 1);
    }
}
for (int i = 0; i < 3; i++)
{
    printf("Multiplication table of %d is:\n", num[i]);
    for (int j = 0; j < 10; j++)
    {
        printf("%d X %d = %d\n", num[i], j + 1, arr[i][j]);
    }
    printf("\n");
}

return 0;
}
```

---

---

## Chapter: Strings in C Programming

---

### 1. Introduction to Strings

In C programming, a **string** is a sequence of characters stored in a **character array** and terminated by a special character called the **null character** ('\0').

#### Key idea

C does **not** have a built-in string data type. Strings are handled using arrays of characters.

#### Example

```
char name[] = "Shamik";
```

Memory representation:

```
S h a m i k \0
```

The null character marks the **end of the string**.

---

### 2. Declaration of Strings

Strings can be declared in multiple ways.

#### 2.1 Character Array Declaration

```
char str[10];
```

This creates a character array that can hold **9 characters + 1 null character**.

---

#### 2.2 String Initialization at Declaration

```
char city[] = "Kolkata";
```

The compiler automatically adds '\0'.

---

#### 2.3 Character-wise Initialization

```
char lang[] = {'C', 'P', 'r', 'o', 'g', '\0'};
```

You must explicitly add '\0' here.

---

### 3. Input and Output of Strings

#### 3.1 Using scanf()

```
char name[20];  
scanf("%s", name);
```

⚠ Limitation:

- Stops reading at **space**
  - Unsafe for long input
- 

#### 3.2 Using gets() (Deprecated and Unsafe)

```
gets(name);
```

🚫 Do not use in modern programs.

---

#### 3.3 Using fgets() (Recommended)

```
char name[50];  
fgets(name, sizeof(name), stdin);
```

- ✓ Safe
  - ✓ Reads spaces
  - ✓ Prevents buffer overflow
- 

#### 3.4 Output using printf() and puts()

```
printf("%s", name);  
puts(name);
```

---

### 4. String Length

Using strlen()

Defined in <string.h>

```
#include <string.h>
```

```
int len = strlen("Hello");
```

- Counts characters **excluding** '\0'

---

## 5. String Copy

Using **strcpy()**

```
char src[] = "C Programming";
```

```
char dest[20];
```

```
strcpy(dest, src);
```

⚠ Destination array must be large enough.

---

Using **strncpy()** (Safer)

```
strncpy(dest, src, sizeof(dest));
```

---

## 6. String Concatenation

Using **strcat()**

```
char first[20] = "Hello ";
```

```
char second[] = "World";
```

```
strcat(first, second);
```

Result:

Hello World

---

Using **strncat()**



```
strncat(first, second, 3);
```

Adds only first 3 characters of second.

---

## 7. String Comparison

Using `strcmp()`

```
int result = strcmp("abc", "abd");
```

Return values:

- `0` → equal
  - `< 0` → first is smaller
  - `> 0` → first is greater
- 

**Example**

```
if (strcmp(pass, "admin") == 0)
{
    printf("Access granted");
}
```

---

## 8. Accessing Individual Characters

Strings are arrays, so indexing works.

```
char word[] = "Computer";
```

```
printf("%c", word[0]); // C
```

```
printf("%c", word[3]); // p
```

---

## 9. Modifying Strings

Characters can be modified directly.

```
char text[] = "Hello";
```

```
text[0] = 'Y';
```

```
printf("%s", text); // Yello
```

⚠ String literals cannot be modified:

```
char *p = "Hello";
```

```
p[0] = 'Y'; // Undefined behaviour
```

---

## 10. String Traversal

### Using Loop

```
char str[] = "C Language";
```

```
int i = 0;
```

```
while (str[i] != '\0')
{
    printf("%c ", str[i]);
    i++;
}
```

---

## 11. Common String Functions

| Function | Purpose          |
|----------|------------------|
| strlen() | Length of string |
| strcpy() | Copy string      |
| strcat() | Concatenate      |
| strcmp() | Compare          |
| strchr() | Find character   |
| strstr() | Find substring   |

---

**Example: strchr()**

```
char str[] = "Programming";  
char *pos = strchr(str, 'g');  
  
printf("%s", pos); // gramming
```

---

**Example: strstr()**

```
char text[] = "Learn C Programming";  
char *sub = strstr(text, "C");  
  
printf("%s", sub); // C Programming
```

---

**12. Array of Strings**

Useful for storing multiple names.

```
char names[3][20] = {"Ram", "Shyam", "Mohan"};
```

Access:

```
printf("%s", names[1]); // Shyam
```

---

**13. Strings Using Pointers****Pointer to String Literal**

```
char *msg = "Hello World";
```

✓ Memory efficient

✗ Read-only

---

**Pointer with Dynamic Memory**

```
char *p;
```

```
p = (char *)malloc(20 * sizeof(char));
```

```
strcpy(p, "Dynamic String");
```

---

#### **14. Dynamic String Input**

```
char *str;
```

```
str = (char *)malloc(50);
```

```
fgets(str, 50, stdin);
```

Always free memory:

```
free(str);
```

---

#### **15. Passing Strings to Functions**

##### **Passing Character Array**

```
void display(char str[])
```

```
{
```

```
    printf("%s", str);
```

```
}
```

---

##### **Passing Pointer**

```
void show(char *str)
```

```
{
```

```
    printf("%s", str);
```

```
}
```

---

#### **16. String Reversal (Example Program)**

```
#include <stdio.h>
```

```

#include <string.h>

int main()
{
    char str[50];
    int i, len;

    fgets(str, 50, stdin);
    len = strlen(str) - 1;

    for (i = len; i >= 0; i--)
    {
        printf("%c", str[i]);
    }

    return 0;
}

```

---

standard string handling functions in C are defined in the <string.h> header file.

### 17. Searching Functions

| Function         | Description                                               |
|------------------|-----------------------------------------------------------|
| <b>strchr()</b>  | Finds the first occurrence of a character.                |
| <b>strrchr()</b> | Finds the last occurrence of a character.                 |
| <b>strstr()</b>  | Finds the first occurrence of a substring.                |
| <b>strspn()</b>  | Gets the length of a prefix substring matching a set.     |
| <b>strcspn()</b> | Gets the length of a prefix substring not matching a set. |
| <b>strpbrk()</b> | Finds the first occurrence of any character from a set.   |
| <b>strtok()</b>  | Splits a string into tokens based on delimiters.          |

## 18. Copying & Concatenation Functions

| Function         | Description                                                               |
|------------------|---------------------------------------------------------------------------|
| <b>strcpy()</b>  | Copies a source string to a destination string.                           |
| <b>strncpy()</b> | Copies up to $n$ characters from source to destination.                   |
| <b>strcat()</b>  | Appends / concatenate a source string to the end of a destination string. |
| <b>strncat()</b> | Appends / concatenate up to $n$ characters from source to destination.    |

## 19. Comparison & Length Functions

| Function         | Description                                                           |
|------------------|-----------------------------------------------------------------------|
| <b>strlen()</b>  | Calculates the exact length of a string (excluding <code>\0</code> ). |
| <b>strcmp()</b>  | Compares two strings alphabetically (case-sensitive).                 |
| <b>strncmp()</b> | Compares up to $n$ characters of two strings.                         |
| <b>strcoll()</b> | Compares two strings using the current locale rules.                  |
| <b>strxfrm()</b> | Transforms a string based on the current locale.                      |

## 20. Raw Memory Functions (Also in <string.h>)

| Function         | Description                                             |
|------------------|---------------------------------------------------------|
| <b>memset()</b>  | Fills a block of memory with a specific byte value.     |
| <b>memcpy()</b>  | Copies a block of memory from source to destination.    |
| <b>memmove()</b> | Copies memory block safely even if regions overlap.     |
| <b>memcmp()</b>  | Compares two blocks of memory byte by byte.             |
| <b>memchr()</b>  | Finds the first occurrence of a byte in a memory block. |

## 21. Error Handling Functions

| Function          | Description                                            |
|-------------------|--------------------------------------------------------|
| <b>strerror()</b> | Returns a pointer to the textual error message string. |

---

## 22. Common Mistakes with Strings

1. Forgetting `'\0'`
2. Buffer overflow
3. Using `gets()`

4. Modifying string literals
5. Using `scanf("%s")` for full sentences

---

## 18. Summary

- Strings in C are **character arrays**
- Always terminated by `'\0'`
- `<string.h>` provides powerful functions
- Prefer `fgets()` over `scanf()` or `gets()`
- Understand difference between **array strings** and **pointer strings**

---

```
#include <stdio.h>

// Strings in C are represented as arrays of characters
// terminated by a null character '\0'.

int main()
{
    // char str[] = {'a', 'b', 'c', '\0'};
    char str[] = "abc"; //same as doing str[] = {'a', 'b', 'c',
'\0'};
    for (int i = 0; i < 3; i++)
    {
        printf(" first character is %c \n", str[i]);
        // %c format specifier for characters
    }

    return 0;
}
```

---

```
#include <stdio.h>
```

```
// Strings in C are represented as arrays of
//characters terminated by a null character '\0'.
```

```
int main()
{
    // char str[] = {'a', 'b', 'c', '\0'};
    char str[] = "abc"; // same as doing str[] = {'a', 'b',
    'c', '\0'};

    /*
    for (int i = 0; i < 3; i++)
    {
        printf("%c", str[i]);
    }*/

    printf("%s", str); // %s format specifier for strings

    return 0;
}
```

---

```
#include <stdio.h>
#include <string.h> // Needed for strchr
// This program demonstrates how to read a string from user
input and remove the trailing newline character.
```

```
int main()
{
    char name[50];

    fgets(name, sizeof(name), stdin);
```



```
    // Remove the trailing newline character
    name[strcspn(name, "\n")] = 0;

    printf("Hello, %s!", name); // Now it prints on the same
line

    return 0;
}
```

---

```
#include <stdio.h>

// The gets() function is used to read a string from the
standard input (stdin)

// and store it in the specified character array. It reads
characters until a

// newline character is encountered or the end of the file
(EOF) is reached.

// The newline character is replaced with a null character
('\0') to terminate the string properly.

// However, it's important to note that gets() is considered
unsafe and should

// be avoided in modern C programming due to potential buffer
overflow vulnerabilities.

// Instead, it's recommended to use fgets() for safer input
handling.

int main()
{
    char str[30];

    gets(str); // The entered string is stored in str
```

```
printf("%s", str);

// puts(str); // Prints the string & places the cursor on
the next line

printf(" hi");
return 0;
}
```

---

```
#include <stdio.h>
#include <string.h>

// This program demonstrates the use of various string
functions in C,
// including strlen(), strcpy(), strcat(), and strcmp().
int main()
{
    char str[] = "shamik";

    // strlen() is used to count the number of characters in
the string excluding the null ('\0') characters.
    printf(" the length of the string is: %d\n", strlen(str));

    char target[30];

    // strcpy() is used to copy the content of second string
into first string passed to it.
    strcpy(target, str);
    printf(" the copied string is: %s", target);

    // strcat() is used to concatenate two strings
    char s1[56] = "Shamik";
    char s2[56] = "Majumder";
```

```

    strcat(s1, s2);
    printf(" the concatenated string is: %s\n", s1);

    // strcmp() is used to compare two strings
    int a = strcmp(s1, s2);
    int b = strcmp(s2, s1);
    printf(" the result of comparing s1 and s2 is: %d\n", a);
    printf(" the result of comparing s2 and s1 is: %d\n", b);

    return 0;
}

```

---

Which of the following is used to appropriately read a multi-word string.

1. gets()
2. puts()
3. printf()
4. scanf()

ans -> 1. gets()

gets() reads an entire line including spaces until a newline is encountered.

That makes it capable of reading multi-word strings like:

Hello World from C

Why the others are wrong

puts()

✗ Used to display a string, not read it.

printf()

✗ Used for output, not input.

scanf()

✗ With %s, it stops reading at the first space:

```
scanf("%s", str); // reads only "Hello"
```

Important real-world note (not usually tested)

Although gets() is the correct MCQ answer, it is unsafe and removed from modern C standards due to buffer overflow issues.

✓ In real programs, use:

```
fgets(str, sizeof(str), stdin);
```

---

```
#include <stdio.h>
```

```
// Write a program to take string as an input from the user  
using %c and %s confirm
```

```
// that the strings are equal.
```

```
int main()
```

```
{
```

```
    char str[7];
```

```
    // scanf("%s", str);
```

```
    for (int i = 0; i < 6; i++)
```

```
    {
```

```
        scanf("%c", &str[i]);
```

```
        fflush(stdin); //fflush(stdin) the input buffer to  
avoid reading the newline character
```

```
    }
```

```
    str[6] = '\0';
```

```

    printf("%s", str);

    return 0;
}

```

---

```

#include <stdio.h>

// chcek the length of a string without using strlen()
function.

// write your own version of strlen function from <string.h>

int mystrlen(char str[])
{
    int i = 0, count;
    char c = str[i];

    while (c != '\0')
    {
        c = str[i];
        i++;
    }

    count = i - 1;
    return count;
}

int main()

```

```
{  
    char str[] = "Shamik";  
  
    printf(" The length of the string is: %d", mystrlen(str));  
    return 0;  
}
```

---

```
#include <stdio.h>
```

```
// Write a function slice() to slice a string. It should change  
the original string such
```

```
// that it is now the sliced string. Take 'm' and 'n' as the  
start and ending position
```

```
// for slice.
```

```
// Function to slice a string in place
```

```
void slice(char str[], int m, int n)
```

```
{
```

```
    int i = 0;
```

```
    // Loop from the starting index 'm' to the ending index 'n'
```

```
    for (int j = m; j <= n && str[j] != '\0'; j++)
```

```
    {
```

```
        str[i] = str[j]; // Shift characters to the front
```

```
        i++;
```

```
    }
```

```
    // Add the null terminator at the end of the new sliced  
string
```

```
        str[i] = '\\0';
    }

int main()
{
    char str[] = "Shamik Majumder";

    // Call the function to modify the string
    slice(str, 1, 12);

    // Print the modified string
    printf("%s\\n", str);

    return 0;
}
```

---

```
#include <stdio.h>
```

```
// check the length of a string without using strlen()
function.

// copy a string without using strcpy() function.

// Write your own version of strlen and strcpy functions from
<string.h>
```

```
int mystrlen(char str[])
{
    int i = 0, count;
    char c = str[i];
```

```

    while (c != '\0')
    {
        c = str[i];
        i++;
    }

    count = i - 1;
    return count;
}

void mystrcpy(char target[], char str[])
{
    for (int i = 0; i < mystrlen(str); i++)
    {
        target[i] = str[i];
    }
    target[mystrlen(str)] = '\0';
}

int main()
{
    char str[] = "shamik";
    char target[30];
    printf(" The length of the string is: %d\n",
mystrlen(str));
    printf(" The copied string is: ");
    mystrcpy(target, str);
    printf("%s", target);
    return 0;
}

```



```
}
```

---

```
#include <stdio.h>
```

```
#include <string.h>
```

```
// Write a program to encrypt a string by adding 1 to the ASCII  
value of its characters.
```

```
int main()
```

```
{
```

```
    char str[] = "Shamik";
```

```
    for (int i = 0; i < strlen(str); i++)
```

```
    {
```

```
        str[i] = str[i] + 1; // Increment the ASCII value of  
each character by 1
```

```
    }
```

```
    printf("%s", str);
```

```
    return 0;
```

```
}
```

---

```
#include <stdio.h>
```

```
#include <string.h>
```

```
// Write a program to decrypt the string encrypted using  
encrypt function in problem 6
```

```
int main()
```

```
{
    char str[] = "Tibnjl";

    for (int i = 0; i < strlen(str); i++)
    {
        str[i] = str[i] - 1; // Decrement the ASCII value of
each character by 1 to decrypt
    }

    printf("%s", str);
    return 0;
}
```

---

```
#include <stdio.h>
```

```
#include <string.h>
```

```
// Write a program to count the occurrence of a given character
in a string.
```

```
int main()
```

```
{
    char str[] = "Shamik Majumder";
    char c = 'm'; // Character to count
    int count = 0;
    for (int i = 0; i < strlen(str); i++)
    {
        if (str[i] == c)
        {
            count++;
        }
    }
}
```

```

        }
    }

    printf("the number of occurrences of '%c' in the string is:
%d", c, count);

    return 0;
}

```

---

```

#include <stdio.h>
#include <string.h>

```

// Write a program to check whether a given character is present in a string or not.

```

int main()
{
    char str[] = "Shamik Majumder";
    char c = 'd'; // Character to check
    int contains = 0;
    for (int i = 0; i < strlen(str); i++)
    {
        if (str[i] == c)
        {
            contains = 1;
            break;
        }
    }
    if (contains)

```

```
    {
        printf("Yes..! the character is available in the
string");
    }
    else
    {
        printf("Sorry..! the character is not available in the
string ");
    }

    return 0;
}
```

---

---

## Chapter: Structures in C Programming

---

### 1. Introduction

In C programming, basic data types like int, float, and char are often insufficient to represent complex real-world entities.

For example, a **student** has a roll number, name, marks, and grade, all of different data types.

To handle such cases, C provides a powerful user-defined data type called a **structure**.

---

### 2. What is a Structure?

A **structure** is a user-defined data type that allows grouping **variables of different data types** under a single name.

Each variable inside a structure is called a **member**.

---

### 3. Why Use Structures?

Without structures:

```
int roll;  
char name[20];  
float marks;
```

With structures:

```
struct student  
{  
    int roll;  
    char name[20];  
    float marks;  
};
```

## Advantages

- Logical grouping of related data
  - Better data organization
  - Easier program maintenance
  - Foundation for advanced data structures
- 

## 4. Declaration of a Structure

### 4.1 Syntax

```
struct structure_name
{
    data_type member1;
    data_type member2;
    ...
};
```

---

### 4.2 Example

```
struct student
{
    int roll;
    char name[20];
    float marks;
};
```

⚠ The semicolon after the closing brace is mandatory.

---

## 5. Declaring Structure Variables

After defining a structure, variables can be declared.

### 5.1 Declaration After Structure Definition

```
struct student s1, s2;
```

---

## 5.2 Declaration Along with Structure

```
struct student  
{  
    int roll;  
    char name[20];  
    float marks;  
} s1, s2;
```

---

## 6. Accessing Structure Members

The **dot operator (.)** is used to access structure members.

### Example

```
s1.roll = 101;  
strcpy(s1.name, "Shamik");  
s1.marks = 85.5;
```

---

## 7. Input and Output of Structure Members

### 7.1 Input

```
scanf("%d", &s1.roll);  
scanf("%s", &s1.name);  
scanf("%f", &s1.marks);
```

---

### 7.2 Output

```
printf("%d %s %.2f", s1.roll, s1.name, s1.marks);
```

---

## 8. Initialization of Structure Variables

## 8.1 Initialization at Declaration

```
struct student s1 = {101, "Amit", 88.5};
```

---

## 8.2 Partial Initialization

```
struct student s2 = {102};
```

Uninitialized members get default values.

---

## 9. Array of Structures

An **array of structures** stores multiple records of the same type.

---

### 9.1 Declaration

```
struct student s[3];
```

---

### 9.2 Example

```
for (int i = 0; i < 3; i++)  
{  
    scanf("%d %s %f", &s[i].roll, &s[i].name, &s[i].marks);  
}
```

---

### 9.3 Displaying Records

```
for (int i = 0; i < 3; i++)  
{  
    printf("%d %s %.2f\n", s[i].roll, s[i].name, s[i].marks);  
}
```

---

## 10. Nested Structures

A structure inside another structure is called a **nested structure**.



---

### Example

```
struct date
{
    int day, month, year;
};
```

```
struct student
{
    int roll;
    char name[20];
    struct date dob;
};
```

---

### Accessing Nested Members

```
s1.dob.day = 12;
s1.dob.month = 12;
s1.dob.year = 2001;
```

---

## 11. Structures and Functions

---

### 11.1 Passing Structure to Function (Call by Value)

```
void display(struct student s)
{
    printf("%d %s %.2f", s.roll, s.name, s.marks);
}
```

Call:

```
display(s1);
```

---

## 11.2 Passing Structure by Reference

More efficient for large structures.

```
void display(struct student *s)
{
    printf("%d %s %.2f", s->roll, s->name, s->marks);
}
```

---

## 12. Pointer to Structure

A pointer that stores the address of a structure variable.

---

### Example

```
struct student s1;
struct student *ptr = &s1;
```

---

### Accessing Members using Arrow Operator (->)

Instead of writing `(*ptr).roll`, we can use arrow operator `->` to access structure properties as follows:

```
(*ptr).roll = 101;
(*ptr).marks = 90.0;
// here -> is known as the arrow operator.
ptr->roll = 101;
ptr->marks = 90.0;
```

---

## 13. Structure and Arrays

Structures can contain arrays as members.

---

### Example

```
struct student
{
    int roll;
    char name[30];
    int marks[5];
};
```

---

## 14. Dynamic Memory Allocation for Structures

Structures can be dynamically allocated using `malloc()`.

---

### Example

```
struct student *ptr;
ptr = (struct student *)malloc(sizeof(struct student));
```

---

### Accessing Members

```
ptr->roll = 201;
strcpy(ptr->name, "Ravi");
```

---

## 15. Typedef with Structure

The `typedef` keyword simplifies structure usage. `typedef` is more commonly used with structures.

---

### Example

```
typedef struct Employee
{
    int id;
```

```
    float salary;  
} Emp;  
Usage:  
Emp e1;
```

---

## 16. Comparison of Structure Variables

Direct comparison using == is **not allowed**.

✗ Invalid:

```
if (s1 == s2)
```

✓ Correct approach:

```
if (s1.roll == s2.roll)
```

---

## 17. Structure Assignment

Structure variables of the same type **can be assigned directly**.

```
s2 = s1;
```

All members are copied.

---

## 18. Difference Between Structure and Array

| Feature    | Structure    | Array       |
|------------|--------------|-------------|
| Data types | Different    | Same        |
| Access     | Dot operator | Index       |
| Use case   | Records      | Collections |

---

## 19. Difference Between Structure and Union

| Feature | Structure | Union |
|---------|-----------|-------|
|---------|-----------|-------|

| Feature | Structure                | Union               |
|---------|--------------------------|---------------------|
| Memory  | Separate for each member | Shared              |
| Size    | Sum of members           | Largest member      |
| Usage   | Multiple values at once  | One value at a time |

---

## 20. Common Errors with Structures

1. Missing semicolon after structure definition
  2. Using . instead of -> with pointers
  3. Incorrect memory allocation size
  4. Forgetting header files for string functions
  5. Passing large structures by value unnecessarily
- 

## 21. Example Program

```
#include <stdio.h>
#include <string.h>

struct student
{
    int roll;
    char name[20];
    float marks;
};

int main()
{
    struct student s1 = {101, "Shamik", 89.5};
```

```
printf("Roll: %d\n", s1.roll);  
printf("Name: %s\n", s1.name);  
printf("Marks: %.2f\n", s1.marks);  
  
return 0;  
}
```

---

## 22. Best Practices

- Use meaningful structure names
  - Prefer passing structures by reference
  - Use `typedef` for cleaner code
  - Initialize structure members properly
  - Group logically related data only
- 

## 23. Summary

- Structures group different data types
  - Members are accessed using `.` or `->`(Pointer Structure)
  - Arrays and pointers work seamlessly with structures
  - Structures model real-world entities
  - They form the base of advanced data structures
-

---

## Chapter: Unions in C Programming

---

### 1. Introduction

In C programming, when a program needs to store **different data types in the same memory location at different times**, a **union** is used.

A union is similar to a structure, but with a key difference: **all members of a union share the same memory location**.

This makes unions **memory-efficient** and useful in system-level programming.

---

### 2. What is a Union?

A **union** is a user-defined data type that allows storing **different data types in the same memory location**, but **only one member can contain a valid value at a time**.

---

### 3. Why Use Unions?

Consider storing data where only one value is needed at a time.

Example:

- Integer ID
- Floating salary
- Character grade

Using a structure wastes memory. Using a union saves memory.

---

### 4. Declaration of a Union

#### 4.1 Syntax

```
union union_name
```

```
{
    data_type member1;
    data_type member2;
    ...
};
```

---

## 4.2 Example

```
union data
{
    int i;
    float f;
    char c;
};
```

⚠ Semicolon after closing brace is mandatory.

---

## 5. Declaring Union Variables

### 5.1 After Union Definition

```
union data d1;
```

---

### 5.2 Along with Union Definition

```
union data
{
    int i;
    float f;
} d1, d2;
```

---

## 6. Accessing Union Members



Union members are accessed using the **dot (.) operator**, same as structures.

---

### Example

```
d1.i = 10;  
printf("%d", d1.i);
```

⚠ Accessing a member different from the last assigned one gives **undefined or garbage value**.

---

## 7. Memory Allocation in Union

The memory allocated to a union is equal to the **size of its largest member**.

---

### Example

```
union test  
{  
    int a;    // 4 bytes  
    char b;   // 1 byte  
    double c; // 8 bytes  
};
```

Memory allocated:

```
sizeof(union test) = 8 bytes
```

---

## 8. Union vs Structure (Memory View)

### Structure

```
struct sample  
{  
    int a;  
    float b;
```

```
    char c;  
};
```

Memory used:

4 + 4 + 1 = 9 bytes (approx, padding may apply)

---

## Union

```
union sample  
{  
    int a;  
    float b;  
    char c;  
};
```

Memory used:

4 bytes (largest member)

---

## 9. Initialization of Union

At the time of declaration, **only the first member** can be initialized.

---

### Example

```
union data d1 = {10};
```

This initializes i.

---

## 10. Assigning Values to Union Members

Only **one member** should be used at a time.

---

### Example

```
union data d1;
```

```
d1.i = 5;
printf("%d\n", d1.i);
```

```
d1.f = 3.14;
printf("%f\n", d1.f);
```

After assigning f, i becomes invalid.

---

## 11. Array of Unions

---

### Declaration

```
union data arr[3];
```

---

### Example

```
arr[0].i = 10;
arr[1].f = 2.5;
arr[2].c = 'A';
```

---

## 12. Union Inside Structure

Unions are often used inside structures to save memory.

---

### Example

```
struct employee
{
    int id;
    union
    {
```

```
        int hours;  
        float salary;  
    } pay;  
};
```

---

### Usage

```
e1.pay.salary = 25000.50;
```

---

## 13. Pointer to Union

---

### Example

```
union data d1;  
union data *p = &d1;
```

```
p->i = 10;  
printf("%d", p->i);
```

---

## 14. Union and Functions

---

### 14.1 Passing Union by Value

```
void display(union data d)  
{  
    printf("%d", d.i);  
}
```

---

### 14.2 Passing Union by Reference

```
void display(union data *d)
```

```
{  
    printf("%d", d->i);  
}
```

---

## 15. Typedef with Union

Simplifies union usage.

---

### Example

```
typedef union  
{  
    int i;  
    float f;  
} Data;
```

Usage:

```
Data d1;
```

---

## 16. Difference Between Union and Structure

| Feature       | Structure      | Union          |
|---------------|----------------|----------------|
| Memory        | Separate       | Shared         |
| Valid members | All at once    | One at a time  |
| Size          | Sum of members | Largest member |
| Use case      | Records        | Variant data   |

---

## 17. Difference Between Union and Enumeration

| Feature | Union | Enum |
|---------|-------|------|
|---------|-------|------|

| Feature     | Union          | Enum            |
|-------------|----------------|-----------------|
| Data stored | Values         | Constants       |
| Memory      | Yes            | Compile-time    |
| Usage       | Memory sharing | Named constants |

---

## 18. Common Errors with Unions

1. Accessing inactive union member
  2. Assuming union stores all values simultaneously
  3. Forgetting which member was last assigned
  4. Incorrect size assumptions
  5. Confusing union with structure
- 

## 19. Practical Example Program

```
#include <stdio.h>
```

```
union number  
{  
    int i;  
    float f;  
};
```

```
int main()  
{  
    union number n;  
  
    n.i = 10;
```

```
printf("Integer: %d\n", n.i);

n.f = 3.14;
printf("Float: %.2f\n", n.f);

return 0;
}
```

---

## 20. When to Use Unions

- ✓ Memory-constrained applications
  - ✓ System programming
  - ✓ Embedded systems
  - ✓ Interpreters and compilers
  - ✗ When all values are needed simultaneously
- 

## 21. Best Practices

- Track active union member manually
  - Combine union with enum for safety
  - Use meaningful union names
  - Prefer structures when memory is not critical
- 

## 22. Summary

- Union stores different data types in shared memory
- Only one member is valid at a time

- Memory allocated equals largest member size
  - Unions save memory but require careful handling
  - Widely used in low-level programming
-



---

## Chapter: Enumeration (enum) in C Programming

---

### 1. Introduction

In C programming, many situations require the use of **named constants** instead of raw numeric values.

For example, using numbers like 0, 1, 2 to represent days or states makes programs difficult to read and maintain.

To solve this problem, C provides a user-defined data type called **Enumeration**.

---

### 2. What is Enumeration?

An **enumeration (enum)** is a **user-defined data type** that consists of a set of **named integer constants**.

Each constant in an enumeration is called an **enumerator**.

---

### Simple Definition

Enumeration allows a variable to take one value from a predefined set of named constants.

---

### 3. Why Use Enumeration?

Using numeric constants:

```
int day = 3;
```

Using enumeration:

```
enum day {Mon, Tue, Wed, Thu, Fri};
```

### Advantages

- Improves code readability
- Reduces programming errors

- Makes programs self-documenting
  - Easier debugging and maintenance
- 

## 4. Syntax of Enumeration

### 4.1 General Syntax

```
enum enum_name
{
    constant1,
    constant2,
    constant3,
    ...
};
```

---

### 4.2 Example

```
enum day
{
    Mon,
    Tue,
    Wed,
    Thu,
    Fri
};
```

---

## 5. Enumeration Constants and Values

By default:

- First constant = 0
- Next constants increment by 1

### Example

```
enum day {Mon, Tue, Wed};
```

Internally:

```
Mon = 0
```

```
Tue = 1
```

```
Wed = 2
```

---

## 6. Declaring Enumeration Variables

### 6.1 Declaration After Enum Definition

```
enum day today;
```

---

### 6.2 Declaration Along with Enum Definition

```
enum day  
{  
    Mon, Tue, Wed, Thu, Fri  
} today;
```

---

### 6.3 Initialization

```
enum day today = Wed;
```

---

## 7. Accessing and Using Enum Variables

Enum variables can store **only the defined enumeration constants**.

---

### Example

```
enum day today = Mon;
```

```
if (today == Mon)
```

```
{  
    printf("Start of the week");  
}
```

---

## 8. Assigning Custom Values to Enum Constants

Enumeration constants can be assigned **explicit values**.

---

### Example

```
enum status  
{  
    SUCCESS = 1,  
    FAILURE = 0,  
    PENDING = -1  
};
```

---

### Mixed Assignment

```
enum number  
{  
    ONE = 1,  
    TWO,  
    THREE = 10,  
    FOUR  
};
```

Result:

ONE = 1

TWO = 2

THREE = 10

FOUR = 11

---

## 9. Enum and Integer Relationship

Enumeration constants are of type **int** internally.

---

### Example

```
enum color {Red, Green, Blue};

printf("%d", Red);    // Output: 0
```

---

### Assigning Integer to Enum (Allowed but Not Recommended)

```
enum color c;
c = 5;    // Logical error, compiler allows it
```

---

## 10. Enumeration and Switch Statement

Enums are commonly used with switch for clean decision-making.

---

### Example

```
enum choice {Add = 1, Sub, Mul, Div};

switch (choice)
{
    case Add:
        printf("Addition");
        break;
    case Sub:
        printf("Subtraction");
        break;
```

```
}
```

---

### 11. Enum vs #define

| Feature     | enum   | #define   |
|-------------|--------|-----------|
| Type safety | Yes    | No        |
| Debugging   | Easy   | Difficult |
| Scope       | Scoped | Global    |
| Readability | High   | Medium    |

---

### 12. Enum vs const Variables

| Feature         | enum          | const            |
|-----------------|---------------|------------------|
| Multiple values | Yes           | No               |
| Type            | int           | Any type         |
| Memory          | Usually none  | Memory allocated |
| Use case        | Fixed options | Fixed value      |

---

### 13. Typedef with Enumeration

typedef simplifies enum usage.

---

#### Example

```
typedef enum
{
    LOW,
    MEDIUM,
    HIGH
```

```
} Priority;
```

Usage:

```
Priority p = HIGH;
```

---

## 14. Enumeration in Functions

---

### Passing Enum to Function

```
void showDay(enum day d)
{
    if (d == Sun)
        printf("Sunday");
}
```

---

### Returning Enum from Function

```
enum status getStatus()
{
    return SUCCESS;
}
```

---

## 15. Enumeration and Arrays

Enums can be used as array indices.

---

### Example

```
enum week {Mon, Tue, Wed, Thu, Fri};
```

```
int workingHours[5] = {8, 8, 8, 8, 6};
```

```
printf("%d", workingHours[Mon]);
```

---

## 16. Anonymous Enumeration

An enumeration without a name.

---

### Example

```
enum
{
    OFF,
    ON
};
```

Used when enum variables are not needed.

---

## 17. Limitations of Enumeration

1. Enum values are integers only
  2. Compiler does not restrict invalid assignments
  3. No string representation by default
  4. Cannot store multiple values simultaneously
- 

## 18. Common Errors with Enumeration

1. Assuming enum is a string
  2. Using invalid integer values
  3. Forgetting enum name during variable declaration
  4. Confusing enum with #define
  5. Assuming enum size is always 4 bytes
-



## 19. Practical Example Program

```
#include <stdio.h>

enum day {Mon, Tue, Wed, Thu, Fri, Sat, Sun};

int main()
{
    enum day today = Fri;

    if (today == Fri)
        printf("Weekend is near!");

    return 0;
}
```

---

## 20. Best Practices

- Use enums instead of magic numbers
  - Use meaningful enum constant names
  - Combine enum with switch
  - Use typedef for cleaner syntax
  - Avoid assigning arbitrary integers to enum variables
- 

## 21. Summary

- Enumeration is a user-defined data type
- Stores named integer constants
- Improves readability and maintainability
- Commonly used with switch and states

- Safer alternative to numeric constants
-

---

## Chapter: File Handling in C Programming

---

### 1. Introduction

Until now, most programs work with data entered from the keyboard and displayed on the screen. Such data is **temporary** and is lost once the program terminates.

**File handling** allows a C program to:

- Store data permanently on secondary storage (disk)
- Retrieve data later
- Process large volumes of data efficiently

File handling is essential for applications like databases, payroll systems, student records, and log files.

---

### 2. What is a File?

A **file** is a collection of data stored permanently on a storage device under a specific name.

#### Characteristics of a File

- Stored on disk
- Has a name and path
- Can be opened, read, written, and closed
- Data persists after program execution

---

### 3. Types of Files in C

C supports **two types of files**:

1. **Text Files**
2. **Binary Files**

---

### 3.1 Text Files

- Store data in human-readable form
- Data is stored as characters
- Easy to view and edit

Example:

Roll: 101

Marks: 85

---

### 3.2 Binary Files

- Store data in binary (machine-readable) form
- Faster and more memory-efficient
- Not readable directly

Example:

01001010 10101100

---

## 4. File Pointer

In C, file operations are performed using a **file pointer**.

### Definition

A **file pointer** is a pointer of type FILE that points to a file structure.

---

### Declaration

```
FILE *fptr;
```

FILE is defined in:

```
#include <stdio.h>
```

---

## 5. Opening a File

To perform any operation, a file must be opened using the `fopen()` function.

---

### 5.1 `fopen()` Function

#### Syntax

```
fptr = fopen("filename", "mode");
```

---

#### Example

```
FILE *fptr;  
fptr = fopen("data.txt", "r");
```

---

### 5.2 File Opening Modes

| Mode | Meaning                   |
|------|---------------------------|
| "r"  | Open for Read             |
| "rb" | Open for Read in binary   |
| "w"  | Open for Write            |
| "wb" | Open for Write in binary  |
| "a"  | Open for Append           |
| "ab" | Open for Append in binary |
| "r+" | Open for Read and write   |
| "w+" | Open for Write and read   |
| "a+" | Open for Append and read  |

---

#### Binary Modes

"rb", "wb", "ab", "rb+", "wb+", "ab+",

---

### 5.3 Checking File Open Error

```
if (fptr == NULL)
{
    printf("File not found");
    return 0;
}
```

---

## 6. Closing a File

After file operations, the file must be closed using `fclose()`.

---

### Syntax

```
fclose(fptr);
```

---

### Importance

- Frees system resources
  - Ensures data is properly written
  - Prevents file corruption
- 

## 7. Reading from a File (Text Files)

---

### 7.1 Using `fgetc()`

Reads one character at a time from the file.

---

### Syntax

```
ch = fgetc(fptr);
```

---

### Example

```
char ch;
while (1)
{
    ch = fgetc(fp);
    printf("%c", ch);
    // when all the content of a file has been read
    break;
    if (ch == EOF)
    {
        break;
    }
}
```

---

## 7.2 Using fgets()

Reads a line of text from the file.

---

### Syntax

```
fgets(buffer, size, fp);
```

---

### Example

```
char line[100];
fgets(line, 100, fp);
printf("%s", line);
```

---

## 7.3 Using fscanf()

Reads formatted data.

---

**Example**

```
int id;  
float marks;  
  
fscanf(fp, "%d %f", &id, &marks);
```

---

**8. Writing to a File (Text Files)**

---

**8.1 Using fputc()**

Writes a single character.

---

**Example**

```
fputc('A', fp);
```

---

**8.2 Using fputs()**

Writes a string.

---

**Example**

```
fputs("Hello File", fp);
```

---

**8.3 Using fprintf()**

Writes formatted output.

---

**Example**

```
fprintf(fp, "Roll: %d Marks: %.2f", roll, marks);
```



---

## 9. Binary File Handling

Binary files store data exactly as it is in memory.

---

### 9.1 Writing Binary Data using `fwrite()`

---

#### Syntax

```
fwrite(&variable, sizeof(variable), 1, fptr);
```

---

#### Example

```
struct student s = {101, "Amit", 85.5};  
fwrite(&s, sizeof(s), 1, fptr);
```

---

### 9.2 Reading Binary Data using `fread()`

---

#### Syntax

```
fread(&variable, sizeof(variable), 1, fptr);
```

---

#### Example

```
struct student s;  
fread(&s, sizeof(s), 1, fptr);
```

---

## 10. File Positioning Functions

Used to move the file pointer within a file.

---

### 10.1 `ftell()`

Returns the current position.

```
long pos = ftell(fptr);
```

---

## 10.2 fseek()

Moves file pointer to a specific position.

---

### Syntax

```
fseek(fptr, offset, origin);
```

---

### Origin Values

- SEEK\_SET
  - SEEK\_CUR
  - SEEK\_END
- 

### Example

```
fseek(fptr, 0, SEEK_SET);
```

---

## 10.3 rewind()

Moves file pointer to the beginning.

```
rewind(fptr);
```

---

## 11. End of File (EOF)

EOF indicates the end of file.

---

### Example

```
while (!feof(fptr))  
{
```

```
    ch = fgetc(fptr);  
    if (ch != EOF)  
        putchar(ch);  
}
```

---

## 12. File Handling with Structures

---

### Example: Writing Structure to File

```
struct student  
{  
    int roll;  
    char name[20];  
    float marks;  
};
```

```
FILE *fptr = fopen("stud.dat", "wb");  
fwrite(&s, sizeof(s), 1, fptr);
```

---

### Reading Structure

```
fread(&s, sizeof(s), 1, fptr);
```

---

## 13. Command Line File Handling (Brief)

Files can be accessed using command-line arguments.

```
int main(int argc, char *argv[])
```

---

## 14. Common Errors in File Handling

1. Forgetting to close file

2. Using wrong file mode
  3. Not checking NULL file pointer
  4. Reading from write-only file
  5. Writing to read-only file
- 

## 15. Example Program: Copy File Content

```
#include <stdio.h>

int main()
{
    FILE *fptr1, *fptr2;
    char ch;

    fptr1 = fopen("source.txt", "r");
    fptr2 = fopen("dest.txt", "w");

    if (fptr1 == NULL || fptr2 == NULL)
    {
        printf("File error");
        return 1;
    }

    while ((ch = fgetc(fptr1)) != EOF)
    {
        fputc(ch, fptr2);
    }

    fclose(fptr1);
```

```
    fclose(fp2);  
  
    return 0;  
}
```

---

## 16. Advantages of File Handling

- Permanent data storage
  - Efficient handling of large data
  - Supports structured data storage
  - Enables real-world applications
- 

## 17. Limitations

- Slower than memory access
  - Requires careful error handling
  - File corruption possible if mishandled
- 

## 18. Best Practices

- Always check file open status
  - Close files properly
  - Use binary files for structured data
  - Use text files for readability
  - Handle EOF correctly
- 

## 19. Summary

- Files provide permanent storage
- C supports text and binary files

- File pointer controls file access
  - `fopen()` and `fclose()` manage file lifecycle
  - `fread()` and `fwrite()` handle binary data
  - File positioning functions allow random access
  - File handling is essential for real-world programs
-

---

## Chapter: Dynamic Memory Allocation in C Programming

---

### 1. Introduction

In many programs, the amount of memory required cannot be determined at compile time.

For example, when the number of students, records, or data items depends on user input, **static memory allocation** becomes insufficient.

To handle such situations, C provides **Dynamic Memory Allocation (DMA)**, which allows memory to be allocated and deallocated **during program execution**.

Dynamic memory is allocated from a special memory area called the **heap**.

---

### 2. Static vs Dynamic Memory Allocation

#### 2.1 Static Memory Allocation

- Memory allocated at compile time
- Fixed size
- Cannot be resized

```
int arr[10];
```

---

#### 2.2 Dynamic Memory Allocation

- Memory allocated at runtime
- Size decided during execution
- Can be resized or freed

```
int *arr = (int *)malloc(10 * sizeof(int));
```

---

### Comparison Table

| Feature         | Static       | Dynamic  |
|-----------------|--------------|----------|
| Allocation time | Compile time | Runtime  |
| Size            | Fixed        | Variable |
| Memory area     | Stack        | Heap     |
| Flexibility     | Low          | High     |

---

### 3. Heap Memory

The **heap** is a large pool of memory used for dynamic allocation.

#### Characteristics

- Managed manually by programmer
  - Slower than stack access
  - Persists until explicitly freed
  - Shared across functions
- 

### 4. Header File for DMA

All dynamic memory functions are declared in:

```
#include <stdlib.h>
```

---

### 5. Dynamic Memory Allocation Functions

C provides **four standard functions** for dynamic memory management:

1. malloc()
  2. calloc()
  3. realloc()
  4. free()
-



## 6. malloc() – Memory Allocation

### 6.1 Definition

malloc() allocates a **single block of memory** of specified size and returns a pointer to the first byte.

---

### 6.2 Syntax

```
ptr = (data_type *)malloc(size_in_bytes);
```

---

### 6.3 Example

```
int *p;  
p = (int *)malloc(5 * sizeof(int));
```

---

### 6.4 Key Points

- Memory is **uninitialized**
- Returns NULL if allocation fails
- Casting is optional in C (but common in textbooks)

#### 1. Using malloc

You must pass the total calculated byte size. The memory will contain unpredictable random numbers (**garbage**).

```
// Allocates space for 3 integers (e.g., 3 * 4 bytes =  
12 bytes)  
  
// malloc – one argument: total bytes  
  
int *p = (int*) malloc(3 * sizeof(int));  
  
// memory: [? ? ?]  
  
// ptr[0] through ptr[2] contain random garbage values  
until assigned  
  
free(ptr); // always required for both  
  
ptr = NULL; // good practice after freeing
```

---

## 6.5 Checking Allocation Failure

```
if (p == NULL)
{
    printf("Memory allocation failed");
    exit(1);
}
```

---

## 7. calloc() – Contiguous Allocation

### 7.1 Definition

calloc() allocates memory for **multiple elements** and initializes all bytes to **zero**.

---

### 7.2 Syntax

```
ptr = (data_type *)calloc(n, size);
```

---

### 7.3 Example

```
int *p;
p = (int *)calloc(5, sizeof(int));
```

---

### 7.4 Difference Between malloc() and calloc()

| Feature        | malloc         | calloc          |
|----------------|----------------|-----------------|
| Initialization | Garbage values | Zero            |
| Arguments      | One            | Two             |
| Speed          | Faster         | Slightly slower |

---

## 8. Accessing Dynamically Allocated Memory

Dynamic memory is accessed using **pointers and indexing**.

## 2. Using `calloc`

You pass the count and size separately. The memory is guaranteed to be clean and filled with **zeros**.

```
// Allocates space for 3 integers and clears them
// calloc - two arguments: count, size of each
int *p = (int*) calloc(3, sizeof(int));
// memory: [0 0 0]
// ptr[0] through ptr[2] are all automatically 0
free(ptr); // always required for both
ptr = NULL; // good practice after freeing
```

---

### Example

```
for (int i = 0; i < 5; i++)
{
    p[i] = i * 10;
}
```

---

| Feature               | <b>malloc (Memory Allocation)</b>                                       | <b>calloc (Contiguous Allocation)</b>                                                                     |
|-----------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Initialization</b> | Leaves memory <b>uninitialised</b> (contains garbage values).           | Initializes all allocated memory bits to <b>zero</b> .                                                    |
| <b>Arguments</b>      | Takes <b>1 argument</b> : total number of bytes.<br><br>1 (total bytes) | Takes <b>2 arguments</b> : number of elements and size of each element.<br><br>2 (count + per-item bytes) |
| <b>Speed</b>          | <b>Faster</b> , because it does not clear the memory.                   | <b>Slower</b> , because it takes time to clear the memory to zero.                                        |

|                                 |                                                           |                                                          |
|---------------------------------|-----------------------------------------------------------|----------------------------------------------------------|
| <b>Syntax</b>                   | <code>ptr = malloc(total_size);</code>                    | <code>ptr = calloc(num_elements, element_size);</code>   |
| <b>Memory after allocation:</b> | <code>ptr = malloc(3 * sizeof(int))</code><br>[?] [?] [?] | <code>ptr = calloc(3, sizeof(int))</code><br>[0] [0] [0] |

---

## 9. free() – Deallocating Memory

### 9.1 Definition

`free()` releases dynamically allocated memory back to the heap.

---

### 9.2 Syntax

`free(ptr);`

---

### 9.3 Example

`free(p);`

`p = NULL;`

---

### Importance

- Prevents memory leaks
- Frees unused memory
- Improves program efficiency

---

## 10. realloc() – Reallocation of Memory

### 10.1 Definition

`realloc()` changes the size of previously allocated memory.

---

### 10.2 Syntax

```
ptr = (data_type *)realloc(ptr, new_size);
```

---

### 10.3 Example

```
p = (int *)realloc(p, 10 * sizeof(int));
```

---

### 10.4 Behaviour

- Preserves old data
  - May move memory to new location
  - Returns NULL if reallocation fails
- 

## 11. Example: Dynamic Array Using malloc()

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int *arr = (int *)malloc(n * sizeof(int));
```

```
    if (arr == NULL)
```

```
    {
```

```
        printf("Memory allocation failed");
```

```
        return 1;
```

```
    }
```

```
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    free(arr);
    return 0;
}
```

---

## 12. Dynamic Memory Allocation for Structures

### Example

```
struct student
{
    int roll;
    float marks;
};
```

```
struct student *s;
s = (struct student *)malloc(sizeof(struct student));
```

---

### Accessing Members

```
s->roll = 101;
s->marks = 85.5;
```

---

## 13. Dynamic Memory Allocation for Arrays of Structures

```
struct student *s;
s = (struct student *)malloc(n * sizeof(struct student));
```

---

## 14. Common Memory Problems

---

### 14.1 Memory Leak

Occurs when allocated memory is not freed.

```
p = malloc(100);  
// missing free(p)
```

---

### 14.2 Dangling Pointer

Pointer refers to freed memory.

```
free(p);  
// p still points to freed memory
```

---

### 14.3 Wild Pointer

Uninitialized pointer.

```
int *p; // wild pointer
```

---

### 14.4 Double Free Error

```
free(p);  
free(p); // error
```

---

## 15. Best Practices in DMA

- Always check return value of allocation functions
- Free memory after use
- Set pointer to NULL after free
- Avoid unnecessary reallocations
- Use meaningful pointer names

---

## 16. Dynamic vs Static Arrays

| Feature     | Static Array | Dynamic Array |
|-------------|--------------|---------------|
| Size        | Fixed        | Variable      |
| Allocation  | Compile time | Runtime       |
| Memory      | Stack        | Heap          |
| Flexibility | Low          | High          |

---

## 17. Practical Example: Sum Using Dynamic Memory

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, sum = 0;
    printf("Enter size: ");
    scanf("%d", &n);

    int *arr = (int *)calloc(n, sizeof(int));

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
        sum += arr[i];
    }
}
```



```
    printf("Sum = %d", sum);

    free(arr);
    return 0;
}
```

---

## **18. Advantages of Dynamic Memory Allocation**

1. Efficient memory usage
  2. Supports variable data size
  3. Essential for complex data structures
  4. Enables runtime flexibility
  5. Reduces memory wastage
- 

## **19. Limitations of Dynamic Memory Allocation**

1. Slower than static allocation
  2. Programmer must manage memory
  3. Risk of memory leaks
  4. Debugging is harder
- 

## **20. Summary**

- Dynamic memory is allocated at runtime from heap
- `malloc()` allocates uninitialized memory
- `calloc()` allocates zero-initialized memory
- `realloc()` resizes allocated memory
- `free()` releases memory
- Proper memory management is crucial

- DMA is essential for advanced C programming
-

---

## Chapter: Error Handling in C Programming

---

### 1. Introduction

In real-world programs, errors are unavoidable. They may occur due to:

- Invalid user input
- File access failure
- Memory allocation failure
- Arithmetic errors
- Logical mistakes in code

**Error handling** is the process of **detecting, reporting, and managing errors** so that a program behaves safely and predictably instead of crashing or producing incorrect results.

Unlike some modern languages, **C does not provide built-in exception handling**. Instead, error handling in C is achieved through **return values, error codes, global variables, and library functions**.

---

### 2. Types of Errors in C

Errors in C programs can be broadly classified into **four categories**:

1. Compile-time errors
2. Run-time errors
3. Logical errors
4. System and library errors

---

### 3. Compile-Time Errors

#### 3.1 Definition

Compile-time errors are detected by the **compiler** during program compilation.

The program does not compile until these errors are fixed.

---

### 3.2 Common Causes

- Syntax errors
  - Missing semicolons
  - Undeclared variables
  - Type mismatch
- 

### 3.3 Example

```
#include <stdio.h>
```

```
int main()  
{  
    int a = 10  
    printf("%d", a);  
    return 0;  
}
```

**Error:** Missing semicolon after 10

---

### 3.4 Handling Compile-Time Errors

Compile-time errors are handled by:

- Carefully reading compiler error messages
  - Fixing syntax and declaration mistakes
  - Using warnings (-Wall flag)
-

## 4. Run-Time Errors

### 4.1 Definition

Run-time errors occur **while the program is executing**.

The program compiles successfully but fails during execution.

---

### 4.2 Common Causes

- Division by zero
  - Invalid memory access
  - File not found
  - Memory allocation failure
- 

### 4.3 Example: Division by Zero

```
int a = 10, b = 0;  
int c = a / b;    // Run-time error
```

---

### 4.4 Handling Run-Time Errors

Run-time errors are handled using:

- Conditional checks
  - Validation of inputs
  - Return values of functions
- 

## 5. Logical Errors

### 5.1 Definition

Logical errors occur when a program **runs successfully but produces incorrect output**.

---

### 5.2 Example

```
int a = 10, b = 20;
if (a > b)
    printf("b is greater");
The condition is logically incorrect.
```

---

### 5.3 Handling Logical Errors

- Proper testing
  - Debugging
  - Code review
  - Step-by-step tracing
- 

## 6. Error Handling Using Return Values

Many C library functions indicate errors through **return values**.

---

### 6.1 Example: File Handling

```
FILE *fp = fopen("data.txt", "r");

if (fp == NULL)
{
    printf("File cannot be opened");
}
```

---

### 6.2 Example: Memory Allocation

```
int *p = (int *)malloc(10 * sizeof(int));

if (p == NULL)
{
```

```
    printf("Memory allocation failed");  
    return 1;  
}
```

---

## 7. Error Handling Using `errno`

### 7.1 What is `errno`?

`errno` is a **global integer variable** defined in the header file:

```
#include <errno.h>
```

It stores an **error code** set by system calls and library functions when an error occurs.

---

### 7.2 How `errno` Works

- Initially set to 0
  - Modified only when an error occurs
  - Must be checked immediately after a failing function call
- 

### 7.3 Example

```
#include <stdio.h>
```

```
#include <errno.h>
```

```
FILE *fp = fopen("nofile.txt", "r");
```

```
if (fp == NULL)  
{  
    printf("Error code: %d\n", errno);  
}
```

---

## 8. Using perror() for Error Messages

### 8.1 Definition

perror() prints a **descriptive error message** corresponding to the current value of errno.

---

#### Syntax

```
perror("custom message");
```

---

#### Example

```
#include <stdio.h>
```

```
#include <errno.h>
```

```
FILE *fp = fopen("nofile.txt", "r");
```

```
if (fp == NULL)
```

```
{
```

```
    perror("File error");
```

```
}
```

Output example:

File error: No such file or directory

---

## 9. Using strerror() Function

### 9.1 Definition

strerror() converts an error code into a **human-readable string**.

---

#### Syntax

```
char *strerror(int errnum);
```



---

### Example

```
#include <stdio.h>
#include <string.h>
#include <errno.h>

printf("Error: %s", strerror(errno));
```

---

## 10. Error Handling in File Operations

---

### 10.1 File Open Error

```
FILE *fp = fopen("test.txt", "r");

if (fp == NULL)
{
    perror("Error opening file");
    return 1;
}
```

---

### 10.2 File Read Error

```
if (fscanf(fp, "%d", &x) == EOF)
{
    printf("Read error or end of file");
}
```

---

## 11. Error Handling in Dynamic Memory Allocation

---

### Example

```
int *arr = (int *)calloc(n, sizeof(int));

if (arr == NULL)
{
    perror("Memory error");
    exit(1);
}
```

---

## 12. Error Handling Using Exit Codes

### 12.1 exit() Function

The `exit()` function terminates the program and returns a status code to the operating system.

---

### Syntax

```
exit(status);
```

---

### Example

```
#include <stdlib.h>

if (fp == NULL)
{
    printf("Fatal error");
    exit(EXIT_FAILURE);
}
```

---

### Common Exit Codes

`EXIT_SUCCESS`

EXIT\_FAILURE

---

### 13. User-Defined Error Handling

Programs can define and handle their own errors using conditions.

---

#### Example

```
int age;
scanf("%d", &age);

if (age < 0)
{
    printf("Invalid age entered");
}
```

---

### 14. Defensive Programming in C

#### Definition

Defensive programming is a technique where the programmer **anticipates errors and handles them safely**.

---

#### Techniques

- Input validation
  - Boundary checks
  - Null pointer checks
  - Resource cleanup
- 

#### Example

```
if (ptr != NULL)
```

```
{  
    free(ptr);  
}
```

---

## 15. Assertions (assert.h)

### 15.1 Definition

Assertions are used to **detect programming errors during debugging**.

---

#### Syntax

```
#include <assert.h>  
assert(condition);
```

---

#### Example

```
int x = 5;  
assert(x > 0);
```

If the condition fails, the program terminates.

---

## 16. Error Handling in Functions

Functions should communicate errors using:

- Return values
  - Output parameters
  - Error codes
- 

#### Example

```
int divide(int a, int b, int *result)  
{  
    if (b == 0)
```

```
        return -1;

    *result = a / b;
    return 0;
}
```

---

## 17. Common Error Handling Mistakes

1. Ignoring return values
  2. Not checking NULL pointers
  3. Overwriting errno before checking
  4. Poor error messages
  5. Memory leaks on error paths
- 

## 18. Best Practices for Error Handling in C

- Always validate inputs
  - Check return values of library functions
  - Use `perror()` for system errors
  - Free resources before exiting
  - Separate error-handling code clearly
- 

## 19. Example Program: Robust File Handling

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fp = fopen("data.txt", "r");
```

```
if (fp == NULL)
{
    perror("File open failed");
    exit(EXIT_FAILURE);
}

printf("File opened successfully");

fclose(fp);
return EXIT_SUCCESS;
}
```

---

## 20. Summary

- C does not have built-in exception handling
  - Errors are handled using return values and conditions
  - `errno` stores system error codes
  - `perror()` and `strerror()` provide readable messages
  - Defensive programming improves reliability
  - Proper error handling prevents crashes and data loss
  - Essential for writing robust, real-world C programs
-

---

## Chapter: Exception Handling in C Programming

---

### 1. Introduction

An **exception** is an abnormal or unexpected event that occurs during the execution of a program and disrupts the normal flow of instructions.

Examples include division by zero, file access failure, memory allocation failure, and invalid input.

Modern programming languages such as C++ and Java provide **built-in exception handling mechanisms** (try, catch, throw).

However, **the C programming language does not support built-in exception handling.**

Instead, C handles exceptional situations using:

- Return values
- Error codes
- `errno`
- `setjmp()` and `longjmp()`
- Signal handling

This chapter explains **how exception handling is conceptually understood in C and how similar behaviour is implemented.**

---

### 2. What is Exception Handling?

**Exception handling** is a mechanism to:

1. Detect an abnormal condition
2. Transfer control to a special handler
3. Recover gracefully or terminate safely

#### Example of an Exception Situation

```
int a = 10, b = 0;
```

```
int c = a / b;    // Exception: division by zero
```

---

### 3. Does C Support Exception Handling?

✗ No, C does not support exception handling directly.

C does **not** have:

- try
- catch
- throw

✓ Instead, C relies on **manual and library-based mechanisms** to handle exceptional conditions.

---

### 4. Need for Exception Handling in C

Even without native support, exception handling is necessary in C because:

- Programs interact with external resources (files, memory, devices)
- Runtime failures must be handled safely
- Crashes lead to data loss and undefined behaviour

Therefore, C programmers simulate exception handling using alternative techniques.

---

### 5. Techniques Used for Exception Handling in C

C supports exception-like behaviour using the following methods:

1. Error codes and return values
  2. Global variable `errno`
  3. `perror()` and `strerror()`
  4. `setjmp()` and `longjmp()`
  5. Signal handling (`signal()` function)
-



## 6. Exception Handling Using Return Values

Many C functions return **special values** to indicate errors.

---

### Example: Division Function

```
int divide(int a, int b, int *result)
{
    if (b == 0)
        return -1;    // error code

    *result = a / b;
    return 0;        // success
}
```

---

### Usage

```
int res;
if (divide(10, 0, &res) != 0)
{
    printf("Error: Division by zero");
}
```

✓ Simple

✓ Widely used

✗ Requires discipline from programmer

---

## 7. Exception Handling Using errno

---

### 7.1 What is errno?

errno is a **global variable** set by system calls and library functions when an error occurs.

Header file:

```
#include <errno.h>
```

---

### Example

```
#include <stdio.h>
```

```
#include <errno.h>
```

```
FILE *fp = fopen("missing.txt", "r");
```

```
if (fp == NULL)
{
    printf("Error code: %d\n", errno);
}
```

---

## 7.2 Limitations of `errno`

- Must be checked immediately
  - Not reset automatically
  - Cannot distinguish multiple errors easily
- 

## 8. Using `perror()` and `strerror()`

---

### 8.1 `perror()`

Prints a human-readable error message related to `errno`.

```
perror("File Error");
```

---

### Example

```
FILE *fp = fopen("data.txt", "r");
```

```
if (fp == NULL)
{
    perror("File open failed");
}
```

---

## 8.2 strerror()

Returns error message as a string.

```
#include <string.h>
```

```
printf("%s", strerror(errno));
```

---

## 9. Exception Handling Using setjmp() and longjmp()

This is the **closest mechanism to try-catch in C**.

---

### 9.1 setjmp() and longjmp() Overview

Header file:

```
#include <setjmp.h>
```

- setjmp() saves the current execution state
  - longjmp() jumps back to that state
- 

### Conceptual Comparison

| C++   | C                    |
|-------|----------------------|
| try   | setjmp               |
| throw | longjmp              |
| catch | conditional handling |

---

## 9.2 Syntax

```
jmp_buf env;
```

```
setjmp(env);
```

```
longjmp(env, value);
```

---

## 9.3 Example: Simulated Exception Handling

```
#include <stdio.h>
```

```
#include <setjmp.h>
```

```
jmp_buf env;
```

```
void divide(int a, int b)
```

```
{
```

```
    if (b == 0)
```

```
        longjmp(env, 1);
```

```
    printf("Result = %d\n", a / b);
```

```
}
```

```
int main()
```

```
{
```

```
    if (setjmp(env) == 0)
```

```
    {
```

```
        divide(10, 0);
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("Exception: Division by zero");
    }
    return 0;
}
```

---

### Explanation

- `setjmp()` sets recovery point
  - `longjmp()` transfers control on error
  - Acts like exception handling
- 

### Limitations

- Hard to debug
  - Skips normal stack cleanup
  - Not recommended for routine error handling
- 

## 10. Exception Handling Using Signal Handling

---

### 10.1 What is a Signal?

A **signal** is a software interrupt sent to a program to indicate an exceptional condition.

Examples:

- Division by zero
  - Illegal memory access
  - Program termination
- 

### 10.2 `signal()` Function

Header file:

```
#include <signal.h>
```

---

### Syntax

```
signal(signal_type, handler_function);
```

---

### Example: Handling Division by Zero

```
#include <stdio.h>
```

```
#include <signal.h>
```

```
#include <stdlib.h>
```

```
void handler(int sig)
```

```
{
```

```
    printf("Exception caught: Division by zero\n");
```

```
    exit(1);
```

```
}
```

```
int main()
```

```
{
```

```
    signal(SIGFPE, handler);
```

```
    int a = 10, b = 0;
```

```
    printf("%d", a / b);
```

```
    return 0;
```

```
}
```

---

## 11. Comparison: Error Handling vs Exception Handling in C

| Feature          | Error Handling | Exception-like Handling |
|------------------|----------------|-------------------------|
| Built-in support | Yes            | No                      |
| Control transfer | Manual         | Automatic               |
| Complexity       | Low            | High                    |
| Safety           | High           | Risky                   |
| Usage            | Preferred      | Rare                    |

---

## 12. Why C Does Not Support Native Exceptions

1. Simplicity of language design
  2. Performance considerations
  3. Predictable control flow
  4. Manual control preferred in system programming
- 

## 13. Best Practices for Exception-Like Handling in C

- Prefer return values and error codes
  - Use `errno` for system-level errors
  - Use `setjmp/longjmp` only when unavoidable
  - Handle signals carefully
  - Always release resources on failure
- 

## 14. Common Mistakes

1. Ignoring error return values
2. Overusing `setjmp()` and `longjmp()`
3. Not freeing memory on error
4. Assuming C supports try-catch

## 5. Confusing exception handling with debugging

---

### 15. Example: Robust Function with Error Handling

```
int safe_divide(int a, int b, int *result)
{
    if (b == 0)
        return -1;

    *result = a / b;
    return 0;
}
```

---

### 16. Summary

- C does **not** support built-in exception handling
  - Exceptional situations are handled manually
  - Error codes and return values are primary tools
  - `errno`, `perror()`, and `strerror()` provide diagnostics
  - `setjmp()` and `longjmp()` simulate exceptions
  - Signal handling manages critical runtime errors
  - Proper handling improves program reliability and safety
- 

## Using `goto` for Exception Handling in C Programming

---

### 1. Introduction

The C programming language does **not provide built-in exception handling** mechanisms such as `try`, `catch`, and `throw`.



However, C programmers still need a way to **handle abnormal conditions**, perform **error recovery**, and ensure **proper cleanup of resources**.

One commonly discussed (and sometimes controversial) approach is using the **goto statement** to simulate **exception-like control flow**, especially in **low-level and system programming**.

---

## 2. What is the goto Statement?

The goto statement is an **unconditional jump statement** that transfers program control to a labelled statement within the same function.

### Syntax

```
goto label;
```

```
/* ... */
```

```
label:  
    statements;
```

---

## 3. Why Use goto for Exception Handling?

Although goto is generally discouraged, it is used for **error handling** in C because:

- C lacks structured exception handling
- Deeply nested if statements reduce readability
- Multiple resources may need cleanup on error
- goto allows a **single error-handling block**

### Key Idea

goto is not used for normal control flow, but for **centralized error handling and cleanup**, similar to a catch block.

---

#### 4. Concept of Exception Handling Using goto

In this approach:

1. Errors are detected using conditions
2. Control jumps to an error-handling label
3. Cleanup code executes
4. Function exits safely

##### Logical Structure

Normal code

↓

Error detected → goto error\_handler

↓

Cleanup

↓

Exit

---

#### 5. Basic Example: Using goto for Error Handling

##### Example: Division with Error Handling

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10, b = 0, result;
```

```
    if (b == 0)
```

```
        goto error;
```

```
    result = a / b;
```

```

    printf("Result = %d\n", result);
    return 0;

error:
    printf("Exception: Division by zero\n");
    return 1;
}

```

### Explanation

- Error condition detected
- `goto error` transfers control
- Error message printed
- Program exits safely

---

## 6. Using `goto` for Resource Cleanup (Most Common Use)

This is the **most accepted and practical use** of `goto`.

---

### Problem Without `goto` (Nested ifs)

```

if (open_file())
{
    if (allocate_memory())
    {
        if (read_data())
        {
            // success
        }
    }
}

```

This quickly becomes unreadable.

---

### **Solution Using goto**

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fp = NULL;
    int *arr = NULL;

    fp = fopen("data.txt", "r");
    if (fp == NULL)
        goto cleanup;

    arr = (int *)malloc(10 * sizeof(int));
    if (arr == NULL)
        goto cleanup;

    printf("Resources allocated successfully\n");

cleanup:
    if (arr != NULL)
        free(arr);
    if (fp != NULL)
        fclose(fp);

    return 0;
```

```
}
```

---

### Why This Works Well

- Single exit point
- All cleanup in one place
- No deeply nested logic
- Easy to maintain

This pattern is widely used in **kernel and embedded C code**.

---

### 7. goto vs try-catch (Conceptual Comparison)

| Concept          | try-catch (C++) | goto (C)             |
|------------------|-----------------|----------------------|
| Built-in         | Yes             | No                   |
| Structured       | Yes             | No                   |
| Cleanup handling | Automatic       | Manual               |
| Control jump     | Controlled      | Unconditional        |
| Safety           | High            | Programmer-dependent |

---

### 8. Multiple Error Labels Using goto

Different error situations can jump to different labels.

```
if (fp == NULL)
    goto file_error;
```

```
if (arr == NULL)
    goto memory_error;
```

```
file_error:
    printf("File error\n");
    return 1;
```

```
memory_error:
    printf("Memory error\n");
    fclose(fp);
    return 1;
```

---

## 9. Rules for Using **goto** Safely

To avoid chaos:

1. Use **goto only for error handling**
2. Jump **forward only**, never backward
3. Keep labels at the end of functions
4. Never replace loops or conditionals
5. Use meaningful label names (cleanup, error)

---

## 10. Common Mistakes When Using **goto**

1. Creating spaghetti code
2. Jumping across variable initialization
3. Using **goto** for normal logic
4. Forgetting resource cleanup
5. Multiple uncontrolled jumps

---

## 11. When **goto** is Acceptable

- ✓ Resource cleanup
- ✓ Error recovery paths

- ✓ Low-level system programming
  - ✓ Embedded systems
  - ✗ Business logic
  - ✗ Loop replacement
  - ✗ Large application flow control
- 

## 12. Exam-Oriented Answer (Short Form)

In C, `goto` is sometimes used for exception-like handling by transferring control to a common error-handling or cleanup section. Although C does not support exceptions, `goto` helps avoid deeply nested conditionals and ensures proper release of resources. Its use should be limited to error handling only.

---

## 13. Complete Example: Exception-Like Handling Using `goto`

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fp = NULL;
    int *data = NULL;

    fp = fopen("input.txt", "r");
    if (!fp)
        goto error;

    data = (int *)malloc(100 * sizeof(int));
    if (!data)
        goto error;
```

```
printf("Program executed successfully\n");
```

```
error:
```

```
    if (data)
        free(data);
    if (fp)
        fclose(fp);
```

```
    return 0;
```

```
}
```

---

#### **14. Advantages of Using `goto` for Exception Handling**

- Centralized error handling
  - Cleaner code structure
  - Efficient resource cleanup
  - No extra function calls
- 

#### **15. Disadvantages**

- No type safety
  - Manual cleanup required
  - Can reduce readability if misused
  - Not structured exception handling
- 

#### **16. Summary**

- C has no built-in exception handling
- `goto` can simulate exception-like behaviour
- Mainly used for error handling and cleanup



- Avoids deeply nested conditions
  - Must be used carefully and sparingly
  - Accepted practice in system-level C programming
-

---

## Chapter: Preprocessor Directives in C Programming

---

### 1. Introduction

Before a C program is compiled, it passes through a special phase called **preprocessing**.

During this phase, instructions known as **preprocessor directives** are executed.

Preprocessor directives:

- Are handled by the **C preprocessor**
- Begin with the symbol **#**
- Are executed **before compilation**
- Do not end with a semicolon

They are mainly used for:

- Including header files
- Defining constants and macros
- Conditional compilation
- File control

---

### 2. What is a Preprocessor?

The **preprocessor** is a program that processes the source code **before it is compiled**.

#### Functions of the Preprocessor

- Expands macros
- Includes header files
- Removes comments
- Handles conditional compilation

---

### 3. Syntax of Preprocessor Directives

`#directive_name optional_arguments`

Example:

`#include <stdio.h>`

---

### 4. Types of Preprocessor Directives

Preprocessor directives in C are classified into:

1. File inclusion directives
  2. Macro definition directives
  3. Conditional compilation directives
  4. Other special directives
- 

### 5. File Inclusion Directives (`#include`)

#### 5.1 Purpose

The `#include` directive is used to **include the contents of another file** into the current program.

---

#### 5.2 Syntax

**System header files**

`#include <stdio.h>`

**User-defined header files**

`#include "myfile.h"`

---

#### 5.3 Example

`#include <stdio.h>`

```
int main()
{
    printf("Hello World");
    return 0;
}
```

---

## 5.4 Difference Between < > and " "

| < >                         | " "                              |
|-----------------------------|----------------------------------|
| Searches system directories | Searches current directory first |
| Used for standard libraries | Used for user-defined files      |

---

## 6. Macro Definition Directives (#define)

---

### 6.1 Object-like Macros

Used to define **symbolic constants**.

---

#### Syntax

```
#define NAME value
```

---

#### Example

```
#define PI 3.14159
```

```
float area = PI * r * r;
```

- ✓ No memory allocated
  - ✓ Faster than variables
-

## 6.2 Function-like Macros

Macros that look like functions.

---

### Syntax

```
#define MACRO_NAME(parameters) expression
```

---

### Example

```
#define SQUARE(x) (x * x)
```

```
int result = SQUARE(5);
```

---

### Important Note

Always use parentheses to avoid errors.

✗ Wrong:

```
#define SQUARE(x) x * x
```

✓ Correct:

```
#define SQUARE(x) (x * x)
```

---

## 7. Difference Between Macro and Function

| Feature       | Macro             | Function   |
|---------------|-------------------|------------|
| Execution     | Preprocessor time | Runtime    |
| Speed         | Faster            | Slower     |
| Type checking | No                | Yes        |
| Memory        | No overhead       | Uses stack |

---

## 8. Undefining a Macro (#undef)

### Purpose

Removes a previously defined macro.

---

### Syntax

```
#undef MACRO_NAME
```

---

### Example

```
#define MAX 100
```

```
#undef MAX
```

After this, MAX is no longer available.

---

## 9. Conditional Compilation Directives

Conditional compilation allows **selective compilation of code**.

---

### 9.1 #if, #elif, #else, #endif

---

### Syntax

```
#if condition
```

```
    statements
```

```
#elif condition
```

```
    statements
```

```
#else
```

```
    statements
```

```
#endif
```

---

### Example

```
#define VALUE 10

#if VALUE > 10
    printf("Greater");
#else
    printf("Smaller or Equal");
#endif
```

---

## 9.2 #ifdef Directive

Checks if a macro is defined.

---

### Syntax

```
#ifdef MACRO
```

---

### Example

```
#define DEBUG

#ifdef DEBUG
    printf("Debug mode enabled");
#endif
```

---

## 9.3 #ifndef Directive

Checks if a macro is **not defined**.

---

### Example

```
#ifndef MAX
#define MAX 50
```

```
#endif
```

---

## 10. Header Guard (Include Guard)

Used to prevent **multiple inclusion of header files**.

---

### Example

```
#ifndef MYHEADER_H  
#define MYHEADER_H
```

```
// declarations
```

```
#endif
```

This avoids redefinition errors.

---

## 11. Special Preprocessor Directives

---

### 11.1 #line Directive

Changes line number and file name for error messages.

---

### Example

```
#line 100 "test.c"
```

---

### 11.2 #error Directive

Forces the compiler to display an error message.

---

### Example

```
#error "This is a custom error message"
```



---

### 11.3 #pragma Directive

Provides special instructions to the compiler.

---

#### Example

```
#pragma once
```

Used to avoid multiple header inclusion.

---

## 12. Predefined Macros

C provides several predefined macros.

---

### Common Predefined Macros

| Macro                 | Description           |
|-----------------------|-----------------------|
| <code>__FILE__</code> | Current file name     |
| <code>__LINE__</code> | Current line number   |
| <code>__DATE__</code> | Compilation date      |
| <code>__TIME__</code> | Compilation time      |
| <code>__STDC__</code> | Standard C compliance |

---

#### Example

```
printf("File: %s Line: %d", __FILE__, __LINE__);
```

---

## 13. Macros with Multiple Statements

Use backslash (\) for multi-line macros.

---

### **Example**

```
#define PRINT(x) \  
    printf("Value = %d\n", x);
```

---

### **14. Advantages of Preprocessor Directives**

1. Improves code readability
  2. Reduces code duplication
  3. Enables conditional compilation
  4. Increases program flexibility
  5. Saves execution time
- 

### **15. Limitations of Preprocessor Directives**

1. No type checking
  2. Debugging becomes difficult
  3. Can introduce subtle bugs
  4. Overuse reduces code clarity
- 

### **16. Common Errors with Preprocessor Directives**

1. Missing parentheses in macros
  2. Multiple header inclusion
  3. Using macros instead of functions unnecessarily
  4. Forgetting `#endif`
  5. Redefining macros accidentally
- 

### **17. Practical Example Program**

```
#include <stdio.h>
```

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

```
int main()  
{  
    int x = 10, y = 20;  
    printf("Max = %d", MAX(x, y));  
    return 0;  
}
```

---

## 18. Best Practices

- Use macros for constants, not logic
  - Prefer `const` over macros when possible
  - Always use header guards
  - Keep macros simple and readable
  - Avoid complex nested macros
- 

## 19. Summary

- Preprocessor directives are executed before compilation
  - They start with `#` and have no semicolon
  - `#include` is used for file inclusion
  - `#define` creates macros and constants
  - Conditional compilation controls code inclusion
  - Predefined macros provide compilation information
  - Proper use improves portability and maintainability
-

---

## Chapter: Command Line Arguments in C Programming

---

### 1. Introduction

Normally, a C program takes input from the keyboard during execution using functions like `scanf()` or `fgets()`.

However, in many real-world applications, it is useful to pass input **at the time the program is executed**, rather than during runtime interaction.

**Command Line Arguments** provide this facility. They allow values to be passed to a C program **from the command prompt** when the program starts executing.

---

### 2. What are Command Line Arguments?

**Command line arguments** are values supplied to a program **when it is invoked from the command line**.

These arguments are:

- Passed by the operating system
  - Received by the `main()` function
  - Stored as strings
- 

### Example (Program Execution)

`myprogram 10 20`

Here:

- `myprogram` → program name
  - `10, 20` → command line arguments
- 

### 3. `main()` Function with Command Line Arguments

To receive command line arguments, the `main()` function must be defined with parameters.

---

### 3.1 Syntax

```
int main(int argc, char *argv[])
```

OR

```
int main(int argc, char **argv)
```

Both are equivalent.

---

### 3.2 Meaning of Parameters

#### **argc (Argument Count)**

- Stores the **total number of command line arguments**
- Includes the program name
- Type: int

#### **argv (Argument Vector)**

- An array of strings (`char *`)
  - Stores each argument as a string
  - `argv[0]` always contains the **program name**
- 

## 4. Understanding argc and argv

### Example Command

```
sum.exe 5 7
```

Values received:

```
argc = 3
```

```
argv[0] = "sum.exe"
```

```
argv[1] = "5"
```

```
argv[2] = "7"
```

---

## Diagrammatic View

argv

|

+++> argv[0] --> program name

+++> argv[1] --> first argument

+++> argv[2] --> second argument

---

## 5. Simple Program Using Command Line Arguments

### Example: Display All Arguments

```
#include <stdio.h>
```

```
int main(int argc, char *argv[])
{
    int i;

    for (i = 0; i < argc; i++)
    {
        printf("argv[%d] = %s\n", i, argv[i]);
    }

    return 0;
}
```

---

## 6. Why Command Line Arguments Are Strings

All command line arguments are received as **strings**, even if they represent numbers.

Example:

prog 10

Inside the program:

```
argv[1] = "10"
```

To use numeric values, **conversion is required**.

---

## 7. Converting Command Line Arguments

---

### 7.1 Using atoi()

Converts string to integer.

```
#include <stdlib.h>
```

```
int num = atoi(argv[1]);
```

---

#### Example

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    int a = atoi(argv[1]);
```

```
    int b = atoi(argv[2]);
```

```
    printf("Sum = %d", a + b);
```

```
    return 0;
```

```
}
```

---

### 7.2 Using atof()

Converts string to float.

```
float x = atof(argv[1]);
```

---

### 7.3 Using `strtol()` (Safer)

```
long int n = strtol(argv[1], NULL, 10);
```

Preferred for error checking.

---

## 8. Validating Command Line Arguments

Programs must check whether the required number of arguments is provided.

---

### Example

```
if (argc != 3)
{
    printf("Usage: program num1 num2");
    return 1;
}
```

---

## 9. Example: Addition Using Command Line Arguments

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        printf("Usage: add num1 num2");
        return 1;
    }
}
```



```

    }

    int a = atoi(argv[1]);
    int b = atoi(argv[2]);

    printf("Sum = %d", a + b);
    return 0;
}

```

Execution:

add 10 20

Output:

Sum = 30

---

## 10. Command Line Arguments and Arrays

argv itself is an **array of character pointers**.

```
char *argv[];
```

Each element:

- Points to a string
- Ends with NULL after the last argument

---

## 11. argc vs argv (Summary Table)

| Feature               | argc           | argv            |
|-----------------------|----------------|-----------------|
| Type                  | int            | char **         |
| Meaning               | Argument count | Argument values |
| Includes program name | Yes            | Yes             |
| Indexing              | No             | Yes             |

---

## 12. Passing File Names as Command Line Arguments

Command line arguments are often used to pass file names.

---

### Example

```
copy source.txt dest.txt
```

---

### Program

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        printf("Usage: copy source destination");
        return 1;
    }

    printf("Source: %s\n", argv[1]);
    printf("Destination: %s\n", argv[2]);

    return 0;
}
```

---

## 13. Advantages of Command Line Arguments

1. No interactive input required
2. Faster program execution

3. Useful for batch processing
4. Ideal for automation and scripting
5. Widely used in system utilities

---

#### 14. Limitations of Command Line Arguments

1. Limited number of arguments
2. All inputs are strings
3. No built-in validation
4. Not user-friendly for beginners

---

#### 15. Common Errors with Command Line Arguments

1. Forgetting to check `argc`
2. Accessing `argv` out of bounds
3. Assuming numeric type without conversion
4. Ignoring invalid input
5. Confusing `argv[0]` with first data argument

---

#### 16. Difference Between Command Line Input and Runtime Input

| Feature   | Command Line                  | Runtime Input                               |
|-----------|-------------------------------|---------------------------------------------|
| Timing    | Before execution              | During execution                            |
| Functions | <code>main(argc, argv)</code> | <code>scanf()</code> , <code>fgets()</code> |
| Usage     | Automation                    | User interaction                            |
| Data type | Strings                       | Typed input                                 |

---

#### 17. Practical Example: Find Maximum Number

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Provide numbers as arguments");
        return 1;
    }

    int max = atoi(argv[1]);

    for (int i = 2; i < argc; i++)
    {
        int val = atoi(argv[i]);
        if (val > max)
            max = val;
    }

    printf("Maximum = %d", max);
    return 0;
}
```

---

## 18. Best Practices

- Always validate argc
- Use `strtol()` for safe conversions
- Display proper usage messages

- Avoid hardcoding argument positions
  - Use meaningful command formats
- 

## **19. Summary**

- Command line arguments pass data at program startup
  - Received using `argc` and `argv`
  - All arguments are strings
  - Conversion is required for numeric operations
  - Widely used in system programs and utilities
  - Essential for automation and file-based programs
-

| Precedence | Operator            | Description                       | Associativity |
|------------|---------------------|-----------------------------------|---------------|
| 1          | ()                  | Parentheses (function call)       | Left-to-Right |
|            | []                  | Array Subscript (Square Brackets) |               |
|            | .                   | Dot Operator                      |               |
|            | ->                  | Structure Pointer Operator        |               |
|            | expr++ ,<br>expr--  | Postfix increment, decrement      |               |
| 2          | ++expr / --<br>expr | Prefix increment, decrement       | Right-to-Left |
|            | + / -               | Unary plus, minus                 |               |
|            | ! , ~               | Logical NOT, Bitwise complement   |               |
|            | (type)              | Cast Operator                     |               |
|            | *                   | Dereference Operator              |               |

| Precedence | Operator                   | Description                                       | Associativity        |
|------------|----------------------------|---------------------------------------------------|----------------------|
|            | <b>&amp;</b>               | Address of Operator                               |                      |
|            | <b>sizeof</b>              | Determine size in bytes                           |                      |
| 3          | <b>*,/,%</b>               | Multiplication, division, modulus                 | <b>Left-to-Right</b> |
| 4          | <b>+/-</b>                 | Addition, subtraction                             | <b>Left-to-Right</b> |
| 5          | <b>&lt;&lt; , &gt;&gt;</b> | Bitwise shift left, Bitwise shift right           | <b>Left-to-Right</b> |
| 6          | <b>&lt; , &lt;=</b>        | Relational less than, less than or equal to       | <b>Left-to-Right</b> |
|            | <b>&gt; , &gt;=</b>        | Relational greater than, greater than or equal to |                      |
| 7          | <b>== , !=</b>             | Relational is equal to, is not equal to           | <b>Left-to-Right</b> |
| 8          | <b>&amp;</b>               | Bitwise AND                                       | <b>Left-to-Right</b> |
| 9          | <b>^</b>                   | Bitwise exclusive OR                              | <b>Left-to-Right</b> |
| 10         | <b> </b>                   | Bitwise inclusive OR                              | <b>Left-to-Right</b> |

| Precedence | Operator | Description                                | Associativity |
|------------|----------|--------------------------------------------|---------------|
| 11         | &&       | Logical AND                                | Left-to-Right |
| 12         |          | Logical OR                                 | Left-to-Right |
| 13         | ?:       | Ternary conditional                        | Right-to-Left |
| 14         | =        | Assignment                                 | Right-to-Left |
|            | += , -=  | Addition, subtraction assignment           |               |
|            | *= , /=  | Multiplication, division assignment        |               |
|            | %= , &=  | Modulus, bitwise AND assignment            |               |
|            | ^= ,  =  | Bitwise exclusive, inclusive OR assignment |               |
|            | <<=, >>= | Bitwise shift left, right assignment       |               |
| 15         | ,        | comma (expression separator)               | Left-to-Right |

---



---

## Integer Types (Typical 64-bit)

| Type           | Size (Bytes) | Range                                           |
|----------------|--------------|-------------------------------------------------|
| char           | 1            | -128 to 127                                     |
| unsigned char  | 1            | 0 to 255                                        |
| short          | 2            | -32,768 to 32,767                               |
| unsigned short | 2            | 0 to 65,535                                     |
| int            | 4            | -2,147,483,648 to 2,147,483,647                 |
| unsigned int   | 4            | 0 to 4,294,967,295                              |
| long           | 8            | $-9.22 \times 10^{18}$ to $9.22 \times 10^{18}$ |
| unsigned long  | 8            | 0 to $1.84 \times 10^{19}$                      |
| long long      | 8            | Same as long                                    |

## Floating-Point Types

| Type        | Size (Bytes) | Range (Approx.)        | Precision          |
|-------------|--------------|------------------------|--------------------|
| float       | 4            | 1.2E-38 to 3.4E+38     | ~6 decimal places  |
| double      | 8            | 2.3E-308 to 1.7E+308   | ~15 decimal places |
| long double | 10 or 16     | 3.4E-4932 to 1.1E+4932 | ~19 decimal places |

---

## Common Format Specifiers

| Specifier       | Data Type    | Description                             |
|-----------------|--------------|-----------------------------------------|
| <b>%d or %i</b> | int          | Signed decimal integer                  |
| <b>%u</b>       | unsigned int | Unsigned decimal integer                |
| <b>%f</b>       | float        | Decimal floating point                  |
| <b>%lf</b>      | double       | Double-precision floating point         |
| <b>%c</b>       | char         | Single character                        |
| <b>%s</b>       | char[]       | String of characters                    |
| <b>%p</b>       | void *       | Memory address (pointer) in hexadecimal |
| <b>%%</b>       | None         | Prints a literal '%' character          |

# Integers & Modifiers

Modifiers are used for different sizes and bases of integer types.

- **Short:** %hi (signed) and %hu (unsigned).
- **Long:** %ld (signed) and %lu (unsigned).
- **Long Long:** %lld (signed) and %llu (unsigned).
- **Octal:** %o (unsigned octal).
- **Hexadecimal:** %x (lowercase) and %X (uppercase).

# Floating-Point Variations

Beyond standard decimal notation, C supports scientific and flexible formats.

- **Scientific:** %e (lowercase) or %E (uppercase).
- **Shorter Format:** %g or %G (automatically chooses between %f and %e based on value).
- **Long Double:** %Lf.

| Decimal | Characters | Category |
|---------|------------|----------|
| 48      | 0          | Digit    |
| 49      | 1          | Digit    |
| 50      | 2          | Digit    |
| 51      | 3          | Digit    |
| 52      | 4          | Digit    |
| 53      | 5          | Digit    |
| 54      | 6          | Digit    |
| 55      | 7          | Digit    |
| 56      | 8          | Digit    |
| 57      | 9          | Digit    |

| Decimal | Characters | Category  |
|---------|------------|-----------|
| 65      | A          | Uppercase |
| 66      | B          | Uppercase |
| 67      | C          | Uppercase |
| 68      | D          | Uppercase |
| 69      | E          | Uppercase |
| 70      | F          | Uppercase |
| 71      | G          | Uppercase |
| 72      | H          | Uppercase |
| 73      | I          | Uppercase |
| 74      | J          | Uppercase |
| 75      | K          | Uppercase |
| 76      | L          | Uppercase |
| 77      | M          | Uppercase |
| 78      | N          | Uppercase |
| 79      | O          | Uppercase |
| 80      | P          | Uppercase |
| 81      | Q          | Uppercase |
| 82      | R          | Uppercase |
| 83      | S          | Uppercase |
| 84      | T          | Uppercase |
| 85      | U          | Uppercase |
| 86      | V          | Uppercase |
| 87      | W          | Uppercase |

|    |   |           |
|----|---|-----------|
| 88 | X | Uppercase |
| 89 | Y | Uppercase |
| 90 | Z | Uppercase |

| Decimal | Characters | Category  |
|---------|------------|-----------|
| 97      | a          | Lowercase |
| 98      | b          | Lowercase |
| 99      | c          | Lowercase |
| 100     | d          | Lowercase |
| 101     | e          | Lowercase |
| 102     | f          | Lowercase |
| 103     | g          | Lowercase |
| 104     | h          | Lowercase |
| 105     | i          | Lowercase |
| 106     | j          | Lowercase |
| 107     | k          | Lowercase |
| 108     | l          | Lowercase |
| 109     | m          | Lowercase |
| 110     | n          | Lowercase |
| 111     | o          | Lowercase |
| 112     | p          | Lowercase |
| 113     | q          | Lowercase |
| 114     | r          | Lowercase |
| 115     | s          | Lowercase |
| 116     | t          | Lowercase |

|            |          |           |
|------------|----------|-----------|
| <b>117</b> | <b>u</b> | Lowercase |
| <b>118</b> | <b>v</b> | Lowercase |
| <b>119</b> | <b>w</b> | Lowercase |
| <b>120</b> | <b>x</b> | Lowercase |
| <b>121</b> | <b>y</b> | Lowercase |
| <b>122</b> | <b>z</b> | Lowercase |

| <b>Decimal</b> | <b>Characters</b> | <b>Category</b> |
|----------------|-------------------|-----------------|
| <b>32</b>      | <b>␣</b>          | Space           |
| <b>33</b>      | <b>!</b>          | Symbol          |
| <b>34</b>      | <b>"</b>          | Symbol          |
| <b>35</b>      | <b>#</b>          | Symbol          |
| <b>36</b>      | <b>\$</b>         | Symbol          |
| <b>37</b>      | <b>%</b>          | Symbol          |
| <b>38</b>      | <b>&amp;</b>      | Symbol          |
| <b>39</b>      | <b>'</b>          | Symbol          |
| <b>40</b>      | <b>(</b>          | Symbol          |
| <b>41</b>      | <b>)</b>          | Symbol          |
| <b>42</b>      | <b>*</b>          | Symbol          |
| <b>43</b>      | <b>+</b>          | Symbol          |
| <b>44</b>      | <b>,</b>          | Symbol          |
| <b>45</b>      | <b>-</b>          | Symbol          |
| <b>46</b>      | <b>.</b>          | Symbol          |
| <b>47</b>      | <b>/</b>          | Symbol          |
| <b>58</b>      | <b>:</b>          | Symbol          |

|     |   |        |
|-----|---|--------|
| 59  | ; | Symbol |
| 60  | < | Symbol |
| 61  | = | Symbol |
| 62  | > | Symbol |
| 63  | ? | Symbol |
| 64  | @ | Symbol |
| 91  | [ | Symbol |
| 92  | \ | Symbol |
| 93  | ] | Symbol |
| 94  | ^ | Symbol |
| 95  | _ | Symbol |
| 96  | ` | Symbol |
| 123 | { | Symbol |
| 124 |   | Symbol |
| 125 | } | Symbol |
| 126 | ~ | Symbol |

---

1. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int a = 2;
    if (a >> 1)
        printf("%d", a);
}
```

- a. 0
- b. 1
- c. 2 ✓
- d. No Output

The output of the code is 2.

- **Variable initialization:** a is set to 2 (binary 10).
  - **Right shift operation:** a >> 1 shifts the bits of 2 one position to the right.
  - **Result:** Binary 10 becomes 01 (decimal 1).
  - **Condition check:** In C, any non-zero value is **true**. Since 1 is true, the if statement executes.
  - **Printing:** The original value of a (which is still 2) is printed.
- 

## 2. What will be the output of the following C code?

```
#include <stdio.h>

void main(){
    int a = 5, b = -7, c = 0, d;
    d = ++a && ++b || ++c;
    printf("%d %d %d %d", a, b, c, d);
}
```

- a. 6 -6 0 0
- b. 6 -6 1 1
- c. -6 -6 0 1
- d. 6 -6 0 1 ✓

The correct answer is **d. 6 -6 0 1**.

---

### Why this happens

The result depends on **short-circuit evaluation** in C logical operators:

- **Step 1: ++a**
  - a increments from 5 to 6.

- 6 is non-zero (True).
  - **Step 2: ++b**
    - Because the left side of the && was true, ++b must be evaluated.
    - b increments from -7 to -6.
    - -6 is non-zero (True).
  - **Step 3: The && result**
    - (True && True) evaluates to **True (1)**.
  - **Step 4: Short-circuit the ||**
    - The left side of the || is already True.
    - In C, if the first part of an OR is true, the second part is **never executed**.
    - Therefore, ++c is **skipped**. c remains 0.
  - **Step 5: Final assignment**
    - The entire expression is True, so d is assigned 1.
- 

3. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    printf("%d ", 1);
    goto l1;
    printf("%d ", 2);
l1:goto l2;
    printf("%d ", 3);
l2:printf("%d ", 4);
}
```



- a. 1 4 ✓
- b. Compile time error
- c. 1 2 4
- d. 1 3 4

- `printf("%d ", 1);` The program starts here and prints **1** to the console.
  - `goto l1;` The execution immediately jumps to the label `l1:`. This means the line `printf("%d ", 2);` is **skipped entirely**.
  - `l1: goto l2;` As soon as the program arrives at the `l1` label, it encounters another instruction to jump to `l2:`.
  - **Skipped Code** Because of the second jump, the line `printf("%d ", 3);` is also **skipped**.
  - `l2: printf("%d ", 4);` The program arrives at the `l2` label and executes the final print statement, outputting **4**.
- 

#### 4. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int i = 0, j = 0;
while (i < 2) {
    l1 : i++;
    while (j < 3) {
        printf("Loop");
        goto l1;
    }
}

a. Loop Loop
b. Compilation error
c. Loop Loop Loop Loop
```

#### d. Infinite Loop ✓

- ⊗ **Initialization:** The variables are set to  $i = 0$  and  $j = 0$ .
  - ⊗ **Outer Loop:** The condition  $i < 2$  is checked. It is true ( $0 < 2$ ), so the program enters the loop.
  - ⊗ **The Label (l1):** The program hits the label l1: and increments  $i$  to 1.
  - ⊗ **Inner Loop:** The condition  $j < 3$  is checked. It is true ( $0 < 3$ ).
    - It prints "**Loop**".
    - It hits goto l1;.
  - ⊗ **The Jump:** The goto sends the program back to the l1: label *inside* the outer loop but *outside* the inner loop.
  - ⊗ **The Fatal Flaw:**
    - When it jumps back to l1:, it increments  $i$  again ( $i$  becomes 2).
    - It then enters the while ( $j < 3$ ) loop again.
    - **Critically,  $j$  is never incremented.** Since  $j$  remains 0, the condition  $j < 3$  is always true.
- 

5. C99 standard guarantees uniqueness of \_\_\_\_\_ characters for internal names.

- a. 31
- b. 63 ✓
- c. 12
- d. 14

#### Why the answer is 63

In C, compilers have limits on how many characters they "look at" to distinguish one variable name from another. If you have two variables with names longer than the limit, and they are identical up to that limit, the compiler might treat them as the same variable.

The C99 standard increased these minimum requirements significantly compared to the older C89/C90 standards:

- **Internal Identifiers:** The standard guarantees that the first **63 characters** are significant. This means you can have very long, descriptive variable names without worrying about name collisions, provided they differ within those first 63 characters.
  - **External Identifiers:** For names that need to be seen by the linker (like global functions or global variables), the guarantee is much lower—only **31 characters**. This is because the linker is often a separate program with its own older limitations.
- 

## 6. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int x = 1, y = 2;
    int z = x & y == 2;
    printf("%d", z);
}
```

- a. 0
- b. 1 ✓
- c. Compile time error
- d. Undefined behaviour

## The Order of Operations

In C, the **Equality operator (==)** has higher precedence than the **Bitwise AND operator (&)**. This means the compiler evaluates the expression as if it were written with these parentheses: `int z = x & (y == 2);`

Here is the step-by-step execution:

1. **Evaluate the Equality (`y == 2`):**
  - Since `y` is initialized to 2, the expression `2 == 2` is **True**.
  - In C, a "True" result from a logical comparison is represented by the integer **1**.
2. **Evaluate the Bitwise AND (`x & 1`):**

- The value of x is 1.
  - In binary, 1 is ...0001.
  - The operation is 0001 & 0001.
  - The result is 1.
3. **Assignment:**
- The result 1 is assigned to z.
4. **Output:**
- `printf("%d", z);` prints 1.

7. What will be the output of the following C code?

```
#include <stdio.h>

int *m()
{
    int *p = 5;
    return p;
}

void main()
{
    int *k = m();
    printf("%d", k);
}
```

- a. 5 ✓
- b. Junk value
- c. 0
- d. Error

### The Logic Breakdown

1. **Function m():**

- Inside this function, a pointer `int *p` is declared.
- The line `int *p = 5;` is the "trick." Instead of making `p` point to a variable that holds the number 5, you are telling the pointer to point directly to **memory address 5**.
- The function then returns this address (5).

2. **Function `main()`:**

- The returned address (5) is caught by `int *k`. So, `k` now stores the address **5**.

3. **The `printf` statement:**

- The line is `printf("%d", k);`.
- Crucially, there is **no asterisk (\*)** before `k`. If it were `*k`, the program would try to look inside memory address 5, likely causing a crash (Segmentation Fault) because address 5 is restricted.
- Since it just asks to print `k` (the value stored in the pointer itself), it simply prints the address it holds: **5**.

---

8. What will be the output of the following C code?

```
#include <stdio.h>

enum m{    JAN, FEB, MAR};

enum m foo();

int main()
{
    enum m i = foo();
    printf("%d", i);
}

int foo()
{
    return JAN;
}
```

- a. Compile time error ✓
- b. 0
- c. Depends on the compiler
- d. Depends on the standard

**The Conflict: Conflicting Types**

The error occurs because the function `foo()` is declared twice with **different return types** in the same scope:

1. **First Declaration:** `enum m foo();` This tells the compiler that `foo` is a function returning an `enum m`.
2. **Second Declaration (Definition):** `int foo() { ... }` Further down, the function is defined as returning an `int`.

In C, an `enum` and an `int` are often compatible in practice, but the **compiler requires the prototypes to match exactly**. Because the signature changed from `enum m` to `int`, the compiler will throw a "conflicting types" or "redefinition" error.

---

9. Which data type is most suitable for storing a number 65000 in a 32-bit system?

- a. signed short
- b. unsigned short ✓
- c. long
- d. int

#### Why unsigned short is the best choice

To find the "most suitable" type, we look for the one that fits the number while occupying the least number of bytes.

1. **Size of a short:** On most 32-bit systems, a short is **16 bits** (2 bytes).
  2. **The Range Calculation:**
    - A 16-bit space can hold  $2^{16} = 65,536$  distinct values.
    - **signed short:** Splits this range into positive and negative numbers (approximately **-32,768 to 32,767**). Since 65,000 is greater than 32,767, it **cannot** fit here.
    - **unsigned short:** Uses the entire range for positive numbers (**0 to 65,535**). Since 65,000 falls within this range, it fits perfectly.
- 

10. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    int i = 0;
```

```

        if (i == 0)
        {
            printf("Hello");
            break;
        }
    }

```

- a. Hello is printed infinite times
- b. Hello
- c. Varies
- d. Compile time error ✓

### The Problem: **break** is out of place

In C programming, the `break` statement has a very specific "jurisdiction." It can only be used inside:

1. **Loops** (`for`, `while`, `do-while`) to exit the loop prematurely.
2. **Switch statements** to prevent "fall-through" to the next case.

### Why it fails here

In your code snippet, the `break` statement is placed inside an **`if` statement**.

- An `if` statement is a conditional, not a loop.
- Since there is no enclosing loop or `switch` block around that `if` statement, the compiler doesn't know what it is supposed to "break" out of.

### The Error Message

If you were to run this through a compiler like GCC, you would see an error message similar to:

```
error: break statement not within loop or switch
```

### 11. What will be the output of the following C code?

```

#include <stdio.h>

void main()

```

```

{
    double b = 3 && 5 & 4 % 3;
    printf("%lf", b);
}

```

- a. 3.000000
- b. 4.000000
- c. 5.000000
- d. 1.000000 ✓

### Step 1: Order of Operations

In C, operators are not just read left-to-right; they follow a specific priority (precedence). Here is the order for the operators in your code:

1. **Modulo (%)**: Highest priority here.
2. **Bitwise AND (&)**: Medium priority.
3. **Logical AND (&&)**: Lowest priority here.

### Step 2: Step-by-Step Calculation

1. **Handle the Modulo (%)**: First, we calculate  $4 \% 3$  (4 divided by 3 has a remainder of 1).
  - *Expression becomes:*  $3 \&\& 5 \& 1$
2. **Handle the Bitwise AND (&)**: Next, we calculate  $5 \& 1$ . To do this, we look at their binary representations:
  - 5 in binary is 101
  - 1 in binary is 001
  - $101 \& 001$  results in 001, which is 1.
  - *Expression becomes:*  $3 \&\& 1$
3. **Handle the Logical AND (&&)**: Finally, we evaluate  $3 \&\& 1$ .
  - In C, any non-zero value is treated as **True**.
  - $\text{True} \&\& \text{True}$  results in **1** (True).

---

### 12. What will be the output of the following C code?

```

#include <stdio.h>

int main()
{

```



```

float f = 1;
switch (f)
{
    case 1.0: printf("yes");
        break;
    default: printf("default");
}
}

```

- a. yes
- b. yes default
- c. Undefined behaviour
- d. Compile time error ✓

### The Rule

In C, the expression inside a `switch` statement **must evaluate to an integer type** (such as `int`, `char`, or `enum`).

### Why this code fails

1. **Floating Point Variable:** You have declared `float f = 1;`.
2. **The Switch Constraint:** When you write `switch (f)`, the compiler checks the data type of `f`. Since `f` is a `float`, the compiler immediately rejects it.
3. **Case Labels:** Similarly, the case labels (like `case 1.0:`) must also be **integer constant expressions**. Floating-point constants are not allowed.

### Why does C have this restriction?

Floating-point numbers (like `float` and `double`) are often imprecise due to how they are stored in memory. For example, a value intended to be `1.0` might actually be stored as `0.99999999`.

If the `switch` statement allowed floats, comparing these "almost-accurate" values would lead to unpredictable behavior. To prevent this, the C standard requires the reliability of **exact integer matches**.

13. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int x = 2, y = 0;
    int z = (y++) ? 2 : y == 1 && x;
    printf("%d", z);
    return 0;
}
```

- a. 0
- b. 1 ✓
- c. 2
- d. Undefined behaviour

## 1. The Ternary Operator Structure

The line `int z = (y++) ? 2 : y == 1 && x;` uses the ternary operator condition ? value\_if\_true : value\_if\_false.

## 2. Evaluating the Condition: (y++)

- In C, the **post-increment (y++)** means "use the current value of y first, then increment it."
- The current value of y is **0**.
- Since 0 is considered **False** in C, the condition evaluates to False.
- **Crucial Step:** Immediately after the condition is checked, y is incremented. So, y now becomes **1**.

## 3. Evaluating the "False" Expression: y == 1 && x

Because the condition was False, the program moves to the expression after the colon: `y == 1 && x`.

- **Part A (y == 1):** Since y was incremented in the previous step, `1 == 1` is **True** (represented as 1 in C).
- **Part B (&& x):** The logical AND (&&) now checks the value of x.
  - x is **2**, which is non-zero, so it is considered **True**.
- **Result:** True && True results in **1**.

---

14. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    printf("%d ", 1);
    11:12: printf("%d ", 2);
    printf("%d", 3);
}
```

- a. Compilation error
- b. 1 2 3 ✓
- c. 1 2
- d. 1 3

1. What are 11: and 12: ?

In C, any identifier followed by a colon (like 11: or label\_name:) is a **label**.

- Labels are typically used as targets for `goto` statements.
- They tell the compiler: "This specific spot in the code has this name."

2. Can you have multiple labels together?

**Yes.** You can "stack" as many labels as you want before a statement. In your code:

```
11:12: printf("%d ", 2);
```

Both 11 and 12 point to the exact same line of code (the second `printf`). If you had a `goto 11;` or a `goto 12;` anywhere else in the program, it would jump straight to that second `printf`.

3. Do labels affect normal execution?

**No.** Labels are completely ignored during the normal top-to-bottom execution of a program.

- The program starts at `main()`.
- It executes `printf("%d ", 1);` → Prints **1**.

- It sees the labels `l1:` and `l2:`, ignores them, and executes `printf("%d ", 2);` → Prints **2**.
  - It executes `printf("%d", 3);` → Prints **3**.
- 

15. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    {
        int x = 8;
    }

    printf("%d", x);
}
```

- a. 8
- b. 0
- c. Undefined
- d. Compile time error ✓

### The Concept of "Block Scope"

In C, whenever you use curly braces `{ }`, you are creating a new **block**. Any variable declared inside that block is "local" to that block and only exists until the closing brace `}`.

1. **Inside the inner block:** You declare `int x = 8;`. The variable `x` is born here.
2. **At the closing brace `}`:** The variable `x` is destroyed. It effectively "dies" and the memory it used is freed up.
3. **The `printf` line:** When you try to print `x` outside of those braces, the compiler looks for a variable named `x` in the current scope. Since `x` was already destroyed, the compiler thinks `x` hasn't been defined at all.

### The Error

Because the compiler cannot find a definition for `x` that is valid at the line where `printf` is called, it will throw an error like:

error: 'x' undeclared (first use in this function)

### How to make it work

To make the code print 8, you would need to declare x in a scope that "encompasses" the `printf` call:

```
#include <stdio.h>

void main() {
    int x; // Declared in the outer block
    {
        x = 8; // Assigned in the inner block
    }
    printf("%d", x); // Works! Prints 8
}
```

---

### 16. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    int x = 97;
    int y = sizeof(x++);
    printf("x is %d", x);

}
```

- a. x is 97 ✓
- b. x is 98
- c. x is 99
- d. Run time error

### The Secret of `sizeof`

The key to understanding this is knowing that **`sizeof` is a compile-time operator**, not a run-time operator.

When the compiler sees `sizeof (x++)`, it does the following:

1. It looks at the **type** of the expression inside the parentheses.
2. Since `x` is an `int`, the expression `x++` will result in an `int`.
3. The compiler replaces the entire `sizeof(x++)` with the size of an integer (usually 4) right then and there.

### Why `x` doesn't change

Because `sizeof` is resolved during compilation, the expression `x++` is **never actually executed** when the program runs. The "side effect" of the increment operator (`++`) is ignored because the CPU never sees that instruction; it only sees the constant value (like 4) that the compiler put in its place.

### Step-by-Step Execution

1. `int x = 97;` — `x` is initialized to 97.
2. `int y = sizeof(x++);` — The compiler determines the size of an `int`. It puts that value into `y`. **The `++` is ignored.**
3. `printf("x is %d", x);` — Since `x++` never actually happened, `x` is still 97.

---

### 17. What will be the output of the following C code?

```
#include <stdio.h>

static int x;

void main()
{
    int x;
    printf("x is %d", x);
}
```

- a. 0
- b. Junkvalue ✓
- c. Run time error
- d. Nothing

### 1. Variable Shadowing

In this code, you have two variables named `x`:

- **The Global (Static) x:** Declared outside `main()` as `static int x;`. In C, static global variables are automatically initialized to **0**.
- **The Local x:** Declared inside `main()` as `int x;`.

When you have a local variable with the same name as a global variable, the local one "shadows" (hides) the global one. Inside `main()`, any mention of `x` refers strictly to the **local** version.

## 2. Junk Value (Undefined Behavior)

The local `int x;` is an **automatic variable**. Unlike static or global variables, automatic variables are not initialized by the compiler.

- When the program allocates space for the local `x` on the stack, it doesn't clear out whatever was in that memory location before.
- Therefore, `x` contains whatever "leftover" data was already in that memory slot. This is commonly called a **Junk Value**.

## 3. Why the Answer isn't 0

If the `printf` had happened inside a function where the local `x` wasn't defined, it would have accessed the static global `x` and printed 0. But because the local declaration `int x;` exists, the global 0 is ignored.

---

## 18. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int i = 0, j = 0;
    while (i < 2)
    {
        i++;
        while (j < 3)
        {
            printf("loop");
        }
    }
}
```

```

        goto l1;
    }

}

}

```

- a. loop loop ✓
- b. Compile time error
- c. loop loop loop loop
- d. Infinite loop

### Step-by-Step Execution Trace

1. **Initialization:** `i = 0, j = 0.`
2. **Outer Loop Starts:** `while (i < 2)` is true.
  - `i` becomes **1**.
  - **Inner Loop Starts:** `while (j < 3)` is true (since `j` is 0).
  - **Print:** "loop" is printed for the **1st time**.
  - **The Jump:** `goto l1;` triggers. The program jumps back to the label `l1`, which is right at the start of the outer `while` loop.
3. **Back at the Outer Loop:** `while (i < 2)` is checked again. Since `i` is 1, it is still true.
  - `i` becomes **2**.
  - **Inner Loop Starts:** `while (j < 3)` is checked. **Crucial Point:** Note that `j` was never incremented; it is still **0**. So the condition is true.
  - **Print:** "loop" is printed for the **2nd time**.
  - **The Jump:** `goto l1;` triggers again. The program jumps back to `l1`.
4. **Final Check:** The program checks the outer loop condition `while (i < 2)`.
  - Since `i` is now **2**, the condition is **False**.
  - The outer loop terminates, and the program ends.

19. What will be the output of the following C code?

```

#include <stdio.h>

# define max    void m(){printf("hi");}

void main()
{

```



```

        max;

        m();
    }

```

- a. Run time error
- b. hi hi
- c. Nothing
- d. hi ✓

## 1. The Macro Definition

The line `#define max void m(){printf("hi");}` tells the preprocessor:

*"Every time you see the word max, replace it with the entire string void m(){printf("hi");}."*

## 2. Preprocessor Expansion

Before the compiler looks at the logic, it performs the substitution. The code inside `main()` changes from this:

```

void main()
{
    max;
    m();
}

```

To this:

```

C
void main()
{
    void m(){printf("hi");}; // 'max' was replaced here
    m();                     // This calls the function
defined above
}

```

## 3. Nested Functions

In standard C (ANSI/ISO), defining a function inside another function (like `m()` inside `main()`) is technically not allowed. However, many compilers (like **GCC**) support **nested functions** as an extension.

- **max** ;: This line becomes the definition of function `m()` .
- **m()** ;: This line then calls that newly defined function.
- **The Result**: The program executes `printf("hi")` once and prints **hi**.

20. How many times `i` value is checked in the following C code?

```
#include <stdio.h>

int main()
{
    int i = 0;
    do {
        i++;
        printf("in while loop");
    }

    while (i < 3);
}
```

- a. 2
- b. 3 ✓
- c. 4
- d. 1

**Trace Table**

Initially, `i = 0`.

| Iteration | Action inside loop           | Value of <code>i</code> after <code>i++</code> | Condition Check       | Result          |
|-----------|------------------------------|------------------------------------------------|-----------------------|-----------------|
| 1st       | <code>printf</code> executes | 1                                              | <code>1 &lt; 3</code> | True (Check #1) |
| 2nd       | <code>printf</code>          | 2                                              | <code>2 &lt; 3</code> | True (Check     |

| Iteration | Action inside loop | Value of i after i++ | Condition Check | Result                     |
|-----------|--------------------|----------------------|-----------------|----------------------------|
|           | executes           |                      |                 | #2)                        |
| 3rd       | printf executes    | 3                    | 3 < 3           | <b>False</b><br>(Check #3) |

21. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int x = 2, y = 1;
    x *= x + y;
    printf("%d", x);
    return 0;
}
```

- a. 5
- b. 6 ✓
- c. Undefined behaviour
- d. Compile time error

### The Logic Breakdown

Here is the step-by-step math for the expression:

- **Original values:**  $x = 2, y = 1$
- **Expression:**  $x *= (x + y)$
- **Step 1 (Addition):** First, the right-hand side is evaluated:  $2 + 1 = 3$
- **Step 2 (Multiplication):** Then, the current value of  $x$  (which is 2) is multiplied by that result:  $2 * 3$ .
- **Result:**  $x$  becomes **6**.

---

22. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int i = 0, j = 0;
    while (i < 5, j < 10)
    {
        i++;
        j++;
    }

    printf("%d, %d", i, j);
}
```

- a. 5, 5
- b. 5, 10
- c. 10, 10 ✓
- d. Syntax error

### 1. The Comma Operator Rule

In C, when multiple expressions are separated by a comma, they are all evaluated from left to right, but **only the result of the rightmost expression is used** for the final value.

In the expression `while (i < 5, j < 10)`:

1. `i < 5` is evaluated.
2. `j < 10` is evaluated.
3. The loop **only cares about the result of `j < 10`** to decide whether to keep running.

### 2. Execution Trace

Since `j < 10` is the "deciding" condition, the loop will continue as long as `j` is less than 10, regardless of what happens to `i`.

| Iteration | i<br>value | j<br>value | i < 5<br>(Discarded) | j < 10<br>(Decisive) | Result   |
|-----------|------------|------------|----------------------|----------------------|----------|
| 1st       | 0 → 1      | 0 → 1      | True                 | True                 | Continue |
| 5th       | 4 → 5      | 4 → 5      | True                 | True                 | Continue |
| 6th       | 5 → 6      | 5 → 6      | False                | True                 | Continue |
| 10th      | 9 → 10     | 9 → 10     | False                | True                 | Continue |
| 11th      | 10         | 10         | False                | False                | Stop     |

---

23. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    static int x;
    if (x++ < 2)        main();
}
```

- a. Infinite calls to main
- b. Run time error
- c. Varies
- d. main is called twice ✓

### 1. What is a static variable?

In C, a static variable is initialized only **once** and it retains its value even after the function finishes. In this code, static int x; is automatically initialized to **0**. It does not reset to 0 every time main() is called.

### 2. The Execution Trace

Here is exactly what happens step-by-step:

- **First Call (main starts):**
  - x is 0.
  - The if condition checks  $x++ < 2$ .
  - Because it is a **post-increment** ( $x++$ ), the check uses the *current* value (0) and then increments it.
  - **Check:** Is  $0 < 2$ ? **Yes.** \* x becomes 1.
  - Since the condition is true, main() is called again.
- **Second Call (Recursion):**
  - x is now 1 (remembered from the first call).
  - **Check:** Is  $1 < 2$ ? **Yes.**
  - x becomes 2.
  - Since the condition is true, main() is called a third time.
- **Third Call (Recursion):**
  - x is now 2.
  - **Check:** Is  $2 < 2$ ? **No.**
  - x becomes 3, but the if condition fails.
  - The third call finishes and "returns" to the second call.

### 3. Conclusion

The question asks how many times main is called.

1. The OS calls main (Initial).
2. The code calls main (First recursion).
3. The code calls main (Second recursion).

---

24. What is the output of the following program ?

```
#include<stdio.h>

enum colors{BLACK,BLUE,CYAN};

void main()
{
    printf("%d..%d..%d",BLACK,BLUE,CYAN);
}
```

- a. BLACK,BLUE,CYAN
- b. 0..0..0
- c. 0..1..2 ✓
- d. No output

Why the answer is 0..1..2

In C programming, an `enum` (enumeration) is a user-defined data type that consists of integral constants. Here is the breakdown of what happens in that specific code:

- **Default Assignment:** By default, the first identifier in an `enum` is assigned the value `0`, the second is `1`, the third is `2`, and so on.
- **The Logic:**
  - `BLACK` is the first element, so it becomes `0`.
  - `BLUE` is the second element, so it becomes `1`.
  - `CYAN` is the third element, so it becomes `2`.
- **The `printf` Function:** The format string `"%d. . %d. . %d"` tells the program to print these constant integer values separated by double dots.

| Scenario        | Example                             | Result                                                    |
|-----------------|-------------------------------------|-----------------------------------------------------------|
| Manual Reset    | <code>enum {A=5, B, C};</code>      | A=5, B=6, C=7 (Values increment from the last set number) |
| Specific Values | <code>enum {A=1, B=10, C=3};</code> | A=1, B=10, C=3 (You can assign any integer)               |
| Direct Usage    | <code>printf("%d", BLUE);</code>    | Prints the integer value, not the string "BLUE"           |

25. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    char *str = "hello, world
";

    printf("%d", strlen(str));
    return 0;
}
```

a. 11

- b. 12
- c. 13 ✓
- d. compilation error

### The Breakdown

The code defines a string: `char *str = "hello, world\n";`

To find the length, we count every character inside the double quotes:

1. **h** (1)
2. **e** (2)
3. **l** (3)
4. **l** (4)
5. **o** (5)
6. **,** (6)
7. **[space]** (7)
8. **w** (8)
9. **o** (9)
10. **r** (10)
11. **l** (11)
12. **d** (12)
13. **\n** (13) — **Crucial Point:** The newline character `\n` is an escape sequence. Even though it is written with two symbols (a backslash and an 'n'), it represents a **single character** in memory (the ASCII newline).

The total count is **13**.

---

---

---

---

26. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int x = 2, y = 2;
    x /= x / y;
    printf("%d", x);
    return 0;
}
```



- a. 2 ✓
- b. 1
- c. 0.5
- d. Undefined behaviour

### Step-by-Step Execution

To solve this, you need to follow the C order of operations.

#### 1. Variable Initialization

- $x = 2$
- $y = 2$

#### 2. Expanding the Shorthand Operator

The expression  $x /= \text{expression}$  is equivalent to  $x = x / (\text{expression})$ .

So,  $x /= x / y$  becomes:

$$x = x / (x / y)$$

#### 3. Inner Calculation (Right side of the /=)

First, we calculate the part to the right of the assignment:

- $x / y \rightarrow 2 / 2$
- Result = 1

#### 4. Final Assignment

Now, substitute that result back into the expanded equation:

- $x = x / 1$
- $x = 2 / 1$
- $x = 2$

The final value of x is **2**.

---

---

---

27. What will be the output of the following C code?

```
#include <stdio.h>

int x = 5;

void main()
```

```

{
    int x = 3;
    printf("%d", x);
    {
        int x = 4;
    }

    printf("%d", x);
}

```

- a. 3 3 ✓
- b. 3 4
- c. 3 5
- d. Run time error

### The Breakdown of the Code

The core concept here is that C uses **block scope**. A variable defined inside a set of curly braces { } is only "visible" within those braces.

#### 1. Global Scope

- `int x = 5;` is declared outside `main()`. This is a global variable.

#### 2. Local Scope (Function level)

- Inside `void main()`, we declare `int x = 3;`.
- This **shadows** (hides) the global `x`. Any mention of `x` in the main body now refers to this 3.
- `printf("%d", x);` prints 3.

#### 3. Block Scope (The inner braces)

- Inside the inner { }, we declare `int x = 4;`.
- This creates a **new** variable `x` that exists *only* inside those braces. It shadows the previous `x = 3`.
- However, nothing is printed while this `x = 4` is active. Once the code hits the closing brace }, this version of `x` is destroyed.

#### 4. Returning to Local Scope

- After the inner block ends, the program reaches the final `printf("%d", x);`.
- Since the inner `x = 4` no longer exists, the program looks back at the function-level scope where `x` is still **3**.
- It prints **3** again

---

---

---

---

28. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    int k = 4;
    int *const p = &k;
    int r = 3;
    p = &r;
    printf("%d", p);
}
```

- a) Address of k
- b) Address of r
- c) Compile time error ✓
- d) Address of k + address of r

**The Declaration:** `int *const p`

Since the pointer `p` is declared to be constant, trying to assign it with a new value results in an error.

When you write `int *const p = &k;`, you are creating a **constant pointer**.

- **What it means:** The address stored in `p` is locked. Once `p` is pointed at `k`, it is legally obligated to point at `k` for the rest of its life.
- **The Violation:** In your code, the line `p = &r;` attempts to change that stored address to point to `r` instead.

- **The Result:** The compiler sees you trying to modify a `const` variable and stops the process immediately with an error (typically something like assignment of read-only variable 'p').
- 
- 
- 

29. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    int x = 4;
    int *p = &x;
    int *k = p++;
    int r = p - k;
    printf("%d", r);
}
```

- a. 4
- b. 8
- c. 1 ✓
- d. Run time error

### Step-by-Step Breakdown

1. `int *p = &x;`

The pointer `p` stores the memory address of `x`. Let's assume for this example that the address of `x` is **1000**.

2. `int *k = p++;`

This is a **post-increment** operation.

- First, the current value of `p` (**1000**) is assigned to `k`.
- Then, `p` is incremented. Since `p` is an `int` pointer, it moves to the next integer location. If an `int` is 4 bytes, `p` now becomes **1004**.

3. `int r = p - k;`

- `int r = p - k;`: You are subtracting address 1000 from address 1004.
- In C, when you subtract two pointers of the same type, the result is not the number of bytes.
- The result is the number of elements between the two addresses.
- Formula:  $(\text{Address1} - \text{Address2}) / \text{sizeof}(\text{type})$
- Calculation:  $(1004 - 1000) / 4 = 1$

---

---

---

---

30. What will be the output of the following C code? (Assuming that we have entered the value 1 in the standard input)

```
#include <stdio.h>

void main()
{
    double ch;
    printf("enter a value between 1 to 2:");
    scanf("%lf", &ch);
    switch (ch)
    {
        case 1:          printf("1");
                        break;
        case 2:          printf("2");
                        break;
    }
}
```

- a. Compile time error ✓
- b. 1
- c. 2
- d. Varies

#### Quick Rules for Switch Statements:

1. **Expression Type:** Must be `int`, `char`, or an `enum`.

2. **Case Labels:** Must be **constants** (you cannot use a variable like `case x:`).
3. **No Floats:** `float` and `double` will always trigger a compiler error if used as the switch variable.

```
double ch;  
...  
switch (ch) // Error happens here
```

### Why does this fail?

- **Integer Requirement:** The C standard requires the controlling expression of a `switch` statement to have an **integral type** (such as `int`, `char`, `short`, `long`, or `enum`).
  - **Floating-Point Precision:** Floating-point numbers (`float` and `double`) are stored in a way that involves tiny precision errors. For example, a value you think is `1.0` might actually be stored as `0.9999999999999999`.
  - **Exact Matching:** Because `switch` cases look for an **exact match**, using floating-point numbers would be unreliable and "fuzzy." Therefore, the C language designers simply prohibited them in `switch` blocks to avoid logical bugs.
- 
- 
- 
- 

### 31. What will be the output of the following C code?

```
#include <stdio.h>  
  
int main()  
{  
    int a = -1, b = 4, c = 1, d;  
    d = ++a && ++b || ++c;  
    printf("%d, %d, %d, %d", a, b, c, d);  
    return 0;  
}
```

- a. 0, 4, 2, 1 ✓
- b. 0, 5, 2, 1
- c. -1, 4, 1, 1
- d. 0, 5, 1, 0

## The Step-by-Step Execution

- **Step A: ++a**
    - a starts at -1.
    - ++a (pre-increment) changes a to **0**.
    - In C, 0 is considered **False**.
  - **Step B: The Short-Circuit of &&**
    - Since the left side of the && (++a) resulted in 0 (False), the entire && expression is guaranteed to be false.
    - **Crucial Point:** Because of short-circuiting, the compiler **skips** ++b entirely. b remains **4**.
  - **Step C: Evaluating the ||**
    - Now the expression looks like: d = (0) || ++c;.
    - Since the left side of the || is 0 (False), the compiler **must** evaluate the right side to determine the final result.
    - ++c increments c from 1 to **2**.
    - 2 is non-zero, so it is considered **True**.
  - **Step D: Assigning d**
    - The logical expression (False || True) results in **1** (the standard integer value for "True" in C).
    - d is assigned **1**.
- 
- 
- 

## 32. Which keyword can be used for coming out of recursion?

- a. break
- b. return ✓
- c. exit
- d. both break and return

## The Role of return in Recursion

In C programming, a recursive function calls itself multiple times, creating a "stack" of function calls. To stop this process and prevent an infinite loop, we use a **Base Case**.

When the base case is met, the return statement:

- **Stops** the current instance of the function.
- **Sends** a value (or just control) back to the previous function call in the stack.
- **Unwinds** the recursion until it reaches the original caller.

## Why break is Incorrect

Many students mistakenly choose `break`, but here is why it doesn't work for recursion:

- **Scope:** The `break` keyword is strictly used to exit **loops** (`for`, `while`, `do-while`) or `switch` statements.
- **Function Level:** `break` cannot exit a function. If you put a `break` inside a recursive function without a loop, the compiler will throw an error.

### Why `exit` is Overkill

- **System Level:** The `exit()` function terminates the **entire program** immediately.
  - **Result:** While it technically "comes out" of the recursion, it also closes your application, which is usually not the goal of a specific algorithm.
- 
- 
- 

33. What will be the output of the following C code? (Assuming that we have entered the value 2 in the standard input)

```
#include <stdio.h>

void main()
{
    int ch;
    printf("enter a value between 1 to 2:");
    scanf("%d", &ch);
    switch (ch)
    {
        case 1:printf("1");
                break;
                printf("hi");
        default:printf("2");
    }
}
```



- a. 1
- b. hi 2
- c. Run time error
- d. 2 ✓

### The Input Path

When you enter the value **2**, the variable `ch` becomes **2**. The program then looks for `case 2` inside the `switch` block.

### Matching the `default` Case

Since there is no `case 2` explicitly defined in the code, the `switch` automatically jumps to the **`default:`** label.

- The code under `default` is `printf("2");`.
- Therefore, the number **2** is printed to the console.

### Why doesn't it print "hi" or "1"?

- **Case 1 is skipped:** Since `ch` is **2**, the program completely ignores everything inside `case 1`.
- **The "Unreachable" Code:** You might notice `printf("hi");` sitting after a `break;` inside `case 1`. Even if you had entered **1**, the program would print "1" and then hit the `break`, which immediately exits the `switch`. The `printf("hi");` would **never** execute under any circumstances because it is placed after the exit command.

---

---

---

---

34. What will be the output of the following C code?

```
#include <stdio.h>

int const print()
{
    printf("jecacracker.com");
    return 0;
}

void main()
{
```

```
    print();  
}
```

- a. Error because function name cannot be preceded by `const`
- b. jecacracker.com ✓
- c. jecacracker.com is printed infinite times
- d. Blank screen, no output

### Understanding `int const print()`

In C, placing `const` before a function name (like `int const print()` or `const int print()`) modifies the **return value**, not the function itself.

- It tells the compiler that the integer returned by this function should be treated as a constant.
- **Is it legal?** Yes. While it is rarely useful for a simple `int` (since integers are returned by value anyway), it is perfectly valid syntax. It does **not** cause a compilation error.

### Why the output is "jecacracker.com"

When `main()` calls `print()`, the following happens:

1. The program enters the `print()` function.
2. The `printf("jecacracker.com");` statement executes, sending that text to the standard output.
3. The function returns 0.
4. The program finishes.

### Analysis of the Options

- **Option 1 (Error):** Incorrect. C allows `const` in the return type.
  - **Option 2 (jecacracker.com):** **Correct.** The function executes normally and prints the string.
  - **Option 3 (Infinite times):** Incorrect. There is no loop or recursive call back to `print()` without a base case.
  - **Option 4 (Blank screen):** Incorrect. The `printf` statement is guaranteed to run when the function is called.
-

35. Which of the following format identifier can never be used for the variable `var`?

```
#include <stdio.h>

int main()
{
    char *var = "Advanced Training in C by
jecacracker.com";
}
```

- a. `%f` ✓
- b. `%d`
- c. `%c`
- d. `%s`

#### Why `%f` can NEVER be used

The format specifier `%f` is strictly reserved for **floating-point numbers** (`float` or `double`).

- When `printf` sees `%f`, it expects a 4 or 8-byte numeric value in a specific IEEE 754 format.
- If you pass a character pointer to `%f`, the program will try to interpret a memory address as a decimal number, which results in **Undefined Behavior** (usually a crash or garbage values).

#### Why the others CAN be used

While some may look strange, they are technically "legal" in C under the right context:

1. **`%s` (String):** This is the standard way to use `var`. It tells the program to start at the pointer's address and print every character until it hits the null terminator (`\0`).
2. **`%d` (Integer):** Since a pointer is just a memory address (a number), using `%d` will print the **numerical value of the memory address** where the string is stored. While not common, it is valid C.
3. **`%c` (Character):** `%c` can be used to print the indexed position., you can use `%c` if you dereference the pointer or use an index (e.g., `var[0]`) to print a single character from the string.

---

36. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    for (int i = 0; i < 1; i++)
        printf("In for loop");
}
```

- a. Compile time error
- b. In for loop
- c. Depends on the standard compiler implements ✓
- d. Run time error

| Standard     | Rule for for(int i...)              | Result               |
|--------------|-------------------------------------|----------------------|
| C89 / ANSI C | Not allowed inside the loop header. | Compile Error        |
| C99          | Allowed.                            | Prints "In for loop" |
| C11 / C17    | Allowed.                            | Prints "In for loop" |

---

37. Which of the following is not a valid variable name declaration?

- a. float PI = 3.14;
- b. double PI = 3.14;
- c. int PI = 3.14;
- d. #define PI 3.14 ✓

#define PI 3.14 is a macro preprocessor, it is a textual substitution.

| Option    | Syntax                | Type                 | Is it a Variable Declaration?                                              |
|-----------|-----------------------|----------------------|----------------------------------------------------------------------------|
| a, b, & c | type name<br>= value; | Variable Declaration | Yes. These allocate memory and associate a name with a specific data type. |
| d         | #define<br>PI 3.14    | Macro Preprocessor   | No. This is a directive for the compiler's preprocessor.                   |

38. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int a = 10;
    if (a == a--)
        printf("TRUE 1");
    a = 10;
    if (a == --a)
        printf("TRUE 2");
}
```

- a. TRUE 1
- b. TRUE 2 ✓
- c. TRUE 1 TRUE 2
- d. Compiler Dependent

**Case 1: if (a == a--)**

1. a starts at 10.
2. The right side post decrement (a--) evaluates first: It uses the current value (10) for the comparison, then decrements a to 9 as a side effect.

3. The left side (a) evaluates next, which is now 9.
4. The condition becomes `9 == 10`, which is **False**. Nothing prints.

Then, `a = 10;` → **a is reassign to 10.**

**Case 2: `if (a == --a)`**

1. a is now 10.
2. The right side pre decrement (`--a`) evaluates first: It immediately decrements a to 9 and uses 9 for the comparison.
3. The left side (a) evaluates next, which is now 9.
4. The condition becomes `9 == 9`, which is **True**. It prints **TRUE 2**.

---

**39. What will be the output of the following C code?**

```
#include <stdio.h>

int main()
{
    int x = 3, y = 2;
    int z = x /= y %= 2;
    printf("%d", z);
}
```

- a. 1
- b. Compile time error
- c. Floating point exception ✓
- d. Segmentation fault

## **1. Understanding Operator Associativity**

The expression in question is:

```
int z = x /= y %= 2;
```

In C, assignment operators like `/=` and `%=` have **right-to-left associativity**. This means the compiler starts evaluating from the right side and moves to the left.

## **2. The Step-by-Step Evaluation**

Let's follow the math based on the initial values  $x = 3$  and  $y = 2$ :

- **Step 1 ( $y \mathrel{/=}$  2):** This is equivalent to  $y = y \ \% \ 2$ 
  - Since 2 divided by 2 has a remainder of 0,  $y$  becomes 0.
- **Step 2 ( $x \mathrel{/=}$   $y$ ):** Now the expression looks like  $x = x \ / \ y$ .
  - Since  $y$  is now 0, the computer attempts to perform  $3 \ / \ 0$ .

### 3. Why "Floating Point Exception"?

In many operating systems (like Linux/Unix), a **division by zero** error is reported as a Floating point exception (SIGFPE), even if you are working with integers.

---

40. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int a = -1, b = 4, c = 1, d;
    d = ++a && ++b || ++c;
    printf("%d, %d, %d, %d", a, b, c, d);
    return 0;
}
```

- a. 0, 4, 2, 1 ✓
- b. 0, 5, 2, 1
- c. -1, 4, 1, 1
- d. 0, 5, 1, 0

#### Step-by-Step Execution

The expression is:  $d = ++a \ \&\& \ ++b \ || \ ++c;$

##### 1. Evaluate $++a$

- $a$  starts at -1.
- $++a$  (pre-increment) makes  $a = 0$ .
- In C, 0 is treated as **False**.

## 2. Evaluate ++a && ++b

- Since the left side (++a) is 0 (False), the **short-circuit** happens.
- The compiler **skips** ++b entirely.
- **Result:** b stays 4. The entire && block results in 0.

## 3. Evaluate (0) || ++c

- Now we have 0 || ++c. Since the left side is 0, the || operator **must** check the right side to see if the whole expression can still be true.
- ++c is executed. c moves from 1 to 2.
- Since 2 is non-zero (True), the final value of d becomes 1 (True).

---

## 41. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    int k = 4;
    int *const p = &k;
    int r = 3;
    p = &r;
    printf("%d", p);
}
```

- a. Address of k
- b. Address of r
- c. Compile time error ✓
- d. Address of k + address of r

Since the pointer p is declared to be constant, trying to assign it with a new value results in an error.

| Declaration | Can change value (*p = 10)? | Can change address (p = &r)? |
|-------------|-----------------------------|------------------------------|
|             |                             |                              |



| Declaration                     | Can change value (*p = 10)? | Can change address (p = &r)? |
|---------------------------------|-----------------------------|------------------------------|
| <code>int *p</code>             | Yes                         | Yes                          |
| <code>const int *p</code>       | No                          | Yes                          |
| <code>int *const p</code>       | Yes                         | No (Current Case)            |
| <code>const int *const p</code> | No                          | No                           |

42. What will be the output of the following C code (after linking to source file having definition of ary1)?

```
#include <stdio.h>

int main()
{
    extern ary1[];
    printf("%d", ary1[0]);
}
```

- a. Value of ary1[0];
- b. Compile time error due to multiple definition
- c. Compile time error because size of array is not provided
- d. Compile time error because datatype of array is not provided ✓

### The Problem: Missing Data Type

In C, every variable or array must be declared with a **data type** (like `int`, `char`, `float`, etc.) so the compiler knows how much memory to expect and how to interpret the data bits.

Look closely at the declaration inside `main()`: `extern ary1[];`

- **extern:** This tells the compiler that `ary1` is defined in another file or elsewhere in the program. This part is fine.
  - **ary1 []:** This tells the compiler that `ary1` is an array. This part is also fine for an external declaration (you don't need the size).
  - **The Missing Piece:** There is no **type** keyword. The compiler doesn't know if this is an array of integers, characters, or something else
- 

43. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    register int x = 0;
    if (x < 2)
    {
        x++;
        main();
    }
}
```

- a. Segmentation fault ✓
- b. main is called twice
- c. main is called once
- d. main is called thrice

### 1. The Trap: Local Variables vs. Recursion

At first glance, it looks like the `if (x < 2)` condition should stop the code after `x` increments a few times. However, there is a fundamental issue with how variables work in functions:

- `x` is a **local variable** (specifically a `register` variable, which is just a hint to store it in a CPU register for speed).
- Every time `main()` calls itself (`main();`), a **new instance** of the function is created on the **Stack memory**.

- Each new instance of `main()` gets its **own brand new copy** of `x`, initialized to 0.

## 2. The Infinite Loop Logic

1. **First Call:** `x = 0`. Since `0 < 2`, it increments `x` to 1 and calls `main()`.
2. **Second Call:** This is a *new* `main`. It initializes its own `x` to 0. Since `0 < 2`, it increments `x` to 1 and calls `main()`.
3. **Third Call:** Again, a *new* `main` initializes its own `x` to 0...

Because the `x` in the new function call doesn't "know" about the `x` in the previous function call, the condition `x < 2` will **always be true** for every new level of recursion.

## 3. Why "Segmentation Fault"?

As `main()` keeps calling itself, the computer allocates more and more memory on the **Stack** to keep track of these function calls.

- Eventually, the program runs out of Stack memory.
- This is known as a **Stack Overflow**.
- In a Linux/Unix environment (often used for these types of exams), a Stack Overflow is reported as a **Segmentation fault** because the program tried to access memory addresses it wasn't allowed to (outside the bounds of the stack).

## 44. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    if (~0 == 1)        printf("yes");
    else                printf("no");
}
```

- a. yes
- b. no ✓
- c. compile time error
- d. undefined

## 1. The Bitwise NOT Operator (~)

The `~` operator performs a **Bitwise NOT**, which flips every bit in a number:

- 0 becomes 1
- 1 becomes 0

## 2. How `~0` is Evaluated

In C, the integer 0 is represented as a series of zeros in binary (usually 32 bits).

- **Binary of 0:** 00000000 00000000 00000000 00000000
- **Applying `~0`:** Every bit is flipped: 11111111 11111111 11111111 11111111

In a **Two's Complement** system (which almost all modern computers use), a value with all bits set to 1 represents the number **-1**.

## 3. The Comparison

The code evaluates the expression: `if (~0 == 1)`

Since `~0` is actually **-1**, the condition becomes:

`if (-1 == 1)`

- This is **False**.
- Because the condition is false, the program skips the if block and executes the else block.
- Result: It prints **"no"**.

---

45. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    signed char chr;
    chr = 128;
    printf("%d", chr);
    return 0;
}
```

- a. 128
- b. -128 ✓
- c. Depends on the compiler
- d. None of the mentioned

### The Range of a `signed char`

In C, a `char` typically occupies **1 byte (8 bits)**.

- An unsigned `char` ranges from **0 to 255**.
- A signed `char` uses **Two's Complement** representation, giving it a range of **-128 to 127**.

### The Overflow Logic

When you assign 128 to a `signed char`, you are providing a value that is exactly one step outside its maximum positive limit (127).

To see what happens, look at the binary representation of 128:

- **128 in binary:** 10000000

In a **signed** 8-bit system, the leftmost bit (the Most Significant Bit) acts as the **sign bit**.

- If the sign bit is 0, the number is positive.
- If the sign bit is 1, the number is negative.

### Binary Interpretation

Because `chr` is declared as `signed`, the computer interprets that binary 10000000 differently:

- The bit 1 at the start signals a negative number.
- In Two's Complement, the pattern 10000000 specifically represents the lowest possible value, which is **-128**.

| Value      | Binary (8-bit)  | Signed Interpretation |
|------------|-----------------|-----------------------|
| 127        | 01111111        | 127                   |
| <b>128</b> | <b>10000000</b> | <b>-128</b>           |

| Value | Binary (8-bit) | Signed Interpretation |
|-------|----------------|-----------------------|
| 129   | 10000001       | -127                  |

---

46. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    int i = 0, j = 0;
    for (i = 0; i < 5; i++)
    {
        for (j = 0; j < 4; j++)
        {
            if (i > 1)
                break;
        }

        printf("Hi ");
    }
}
```

- a. Hi is printed 5 times ✓
- b. Hi is printed 9 times
- c. Hi is printed 7 times
- d. Hi is printed 4 times

#### The Structure of the Loops

- **Outer Loop (i):** Runs from 0 to 4 (5 iterations total).
- **Inner Loop (j):** Runs from 0 to 3 (intended to run 4 times per outer loop).
- **The `printf`:** Notice that the `printf("Hi ");` is **inside the outer loop** but **outside the inner loop**.

#### How the `break` Works

The `break` statement only exits the **innermost** loop that contains it.

- When `i` is 0 or 1, the condition `if (i > 1)` is **false**. The inner loop runs all 4 times, finishes naturally, and then `printf` executes.
- When `i` reaches 2, 3, or 4, the condition `if (i > 1)` becomes **true**. The `break` triggers immediately.
- **Crucially:** This `break` only kills the `j` loop. It does **not** skip the rest of the `i` loop.

| Outer Iteration (i) | Inner Loop (j)            | Condition (i > 1) | Action          | Result      |
|---------------------|---------------------------|-------------------|-----------------|-------------|
| 0                   | Runs 4 times              | False             | Finishes j loop | Prints "Hi" |
| 1                   | Runs 4 times              | False             | Finishes j loop | Prints "Hi" |
| 2                   | <b>Breaks immediately</b> | <b>True</b>       | Exits j loop    | Prints "Hi" |
| 3                   | <b>Breaks immediately</b> | <b>True</b>       | Exits j loop    | Prints "Hi" |
| 4                   | <b>Breaks immediately</b> | <b>True</b>       | Exits j loop    | Prints "Hi" |

#### 47. What is the scope of a function?

- Whole source file in which it is defined
- From the point of declaration to the end of the file in which it is defined
- Any source file in a program
- From the point of declaration to the end of the file being compiled ✓

| Concept | Definition |
|---------|------------|
|---------|------------|

| Concept              | Definition                                                                                              |
|----------------------|---------------------------------------------------------------------------------------------------------|
| Scope                | The region of the program where a name (like a function name) is "visible" to the compiler.             |
| Point of Declaration | The line where the compiler first learns the function's name and return type.                           |
| Translation Unit     | The .c file plus all the header files included in it; this is what "the file being compiled" refers to. |

48. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int i = 0, j = 0;
    while (l1: i < 2)      {
        i++;
        while (j < 3)      {
            printf("loop");
            goto l1;
        }
    }
}
```

- a. loop loop
- b. Compile time error ✓
- c. loop loop loop loop
- d. Infinite loop



## The Error: Syntax Violation

In C, a label (like `ll:`) must be followed by a **statement**. The code attempts to place the label inside the parentheses of the `while` condition: `while (ll: i < 2)`

The compiler expects an expression inside those parentheses, but a label is not an expression. This breaks the fundamental syntax rules of the C language.

---

49. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    char a = 'A';
    char b = 'B';
    int c = a + b % 3 - 3 * 2;
    printf("%d", c);
}
```

- a. 65
- b. 58
- c. 64
- d. 59 ✓

### 1. Identify the ASCII Values

In C, when you perform math on `char` variables, the compiler uses their integer ASCII values:

- `'A' = 65`
- `'B' = 66`

### 2. The Equation Breakdown

The expression is:

```
int c = a + b % 3 - 3 * 2;
```

Substituting the values: `int c = 65 + 66 % 3 - 3 * 2;`

### 3. Step-by-Step Order of Operations

In C, **Multiplication (\*) and Modulo (%)** have higher precedence than Addition (+) and Subtraction (-). They are evaluated from left to right.

1. **Modulo first:**  $66 \% 3$ 
  - 66 divided by 3 is exactly 22 with a remainder of **0**.
  - Equation becomes:  $65 + 0 - 3 * 2$
2. **Multiplication next:**  $3 * 2$ 
  - Result is **6**.
  - Equation becomes:  $65 + 0 - 6$
3. **Addition and Subtraction:** (Left to right)
  - $65 + 0 = 65$
  - $65 - 6 = 59$

---

50. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    const int p;
    p = 4;
    printf("p is %d", p);
    return 0;
}
```

- a. p is 4
- b. Compile time error ✓
- c. Run time error
- d. p is followed by a garbage value

Since the constant variable has to be declared and defined at the same time, not doing it results in an error.

#### The Rule of `const`

When you declare a variable as `const`, you are telling the compiler that its value **cannot be changed** after it has been initialized.

In the provided code:

1. `const int p;` — You declare a constant integer. Since it isn't initialized here, it contains a "garbage value."
2. `p = 4;` — You are attempting to **assign** a value to a constant variable.

In C, a `const` variable must be initialized **at the time of declaration**. Once that line of code is passed, any further attempt to modify `p` using an assignment operator (`=`) is a violation of the language rules, leading to an error like:

```
error: assignment of read-only variable 'p'
```

```
#include <stdio.h>
int main() {
    const int p = 4; // Declaration and Initialization
    printf("p is %d", p);
    return 0;
}
```

---

### 51. Which of following is not accepted in C?

- a. `static a = 10;` //static as
- b. `static int func (int);` //parameter as static
- c. `static static int a;` //a static variable prefixed with static ✓
- d. all of the mentioned

### Why "static static int a;" fails

In C, you can only use **one** storage class specifier (like `static`, `auto`, `register`, or `extern`) for a single variable declaration.

- **Redundancy Error:** Repeating the keyword `static` is a syntax error because the compiler expects a type (like `int`) or another qualifier, not a duplicate specifier.
- **Conflict Rule:** Even if you used two different ones (e.g., `static register int a;`), it would still fail because a variable cannot have two different storage "behaviors" at once.

```
static a = 10;
```

This is allowed because of **Implicit Int**.

- In older C standards (C89/90), if you provide a storage class but omit the data type, the compiler assumes the type is `int`.

- While modern compilers (C99 and later) will give you a warning, it is technically "accepted" by the language grammar.

**static int func (int);**

This is a perfectly valid **function prototype**.

- When used with a function, `static` limits the **scope** of that function to the file in which it is defined (Internal Linkage). This is a common practice to "hide" functions from other parts of a project.

**52. What will be the output of the following C code?**

```
#include <stdio.h>

int main()
{
    int x = 2, y = 0;
    int z = y && (y |= 10);
    printf("%d", z);
    return 0;
}
```

- a. 1
- b. 0 ✓
- c. Undefined behaviour due to order of evaluation
- d. 2

| Step        | Action     | Value of y | Value of z |
|-------------|------------|------------|------------|
| Start       | y = 0      | 0          | Undefined  |
| Left of &&  | Evaluate y | 0 (False)  | Undefined  |
| Right of && | (y  = 10)  | Skipped!   | Undefined  |

| Step  | Action        | Value of y | Value of z |
|-------|---------------|------------|------------|
| Final | Assign result | 0          | 0          |

---

53. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    #define const int
    const max = 32;
    printf("%d", max);
}
```

- a. Run time error
- b. 32 ✓
- c. int
- d. const

### 1. The Preprocessor Step

The preprocessor is a text-substitution tool that runs **before** the actual compilation. When it sees `#define const int`, it tells the compiler: *"Every time you see the word `const`, replace it with the word `int`."*

### 2. The Code Substitution

Before the compiler actually reads the logic, the code is transformed like this:

- **Original line:** `const max = 32;`
- **After substitution:** `int max = 32;`

By the time the compiler starts its work, the keyword `const` (which normally makes a variable read-only) has been completely replaced by the data type `int`.

### 3. Why it works

- **Valid Syntax:** After the substitution, the line becomes a standard integer declaration (`int max = 32;`). This is perfectly valid C code.
  - **The `printf` Function:** The `printf("%d", max);` line simply prints the value stored in the integer variable `max`, which is **32**.
- 

54. What will be the output of the following C code?

```
#include <stdio.h>

#define foo(x, y) #x #y    int main()
{
    printf("%s", foo(k, 1));
    return 0;
}
```

- a. `kl` ✓
- b. `k 1`
- c. `xy`
- d. Compile time error

### The Mechanism: Stringification

In C, when you put a `#` before a parameter in a macro definition, the preprocessor converts that parameter into a **string literal** (wrapped in double quotes).

- **The Macro:** `#define foo(x, y) #x #y`
- **The Call:** `foo(k, 1)`

### Step-by-Step Execution:

1. The preprocessor replaces `x` with `k` and `y` with `1`.
2. Because of the `#` operators, `k` becomes `"k"` and `1` becomes `"1"`.
3. The macro expansion results in: `"k" "1"`

In C, when two or more string literals are placed next to each other with only whitespace between them, the compiler **automatically concatenates** them into a single string.

- `"k" + "1"` becomes `"k1"`.

Therefore, the `printf("%s", "k1");` statement outputs **kl**.

---

55. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int x = 1;
    short int i = 2;
    float f = 3;
    if (sizeof((x == 2) ? f : i) == sizeof(float))
printf("float");
    else if (sizeof((x == 2) ? f : i) == sizeof(short
int))
        printf("short int");
}
```

- a. float ✓
- b. short int
- c. Undefined behaviour
- d. Compile time error

### 1. The Ternary Operator Rule

The ternary operator (`condition ? expression1 : expression2`) has a very specific rule regarding data types: the entire expression must have a **single, consistent type**, determined at **compile-time**.

The compiler looks at both possible outcomes (`f`, which is a `float`, and `i`, which is a `short int`) and determines a "common" type that can accommodate both.

### 2. Usual Arithmetic Conversions

In C, when you have two different numeric types in an expression, the compiler performs **Type Promotion**:

- A `float` has a higher "rank" than a `short int`.
- Therefore, the compiler promotes the `short int` to a `float` so that the overall expression has a predictable type.

- The type of the expression `((x == 2) ? f : i)` is determined to be **float** before the program even runs.

### 3.The **sizeof** Behavior

The `sizeof` operator is evaluated at **compile-time**. It does not care whether the condition `x == 2` is true or false at runtime. It only looks at the **final type** of the expression.

- `sizeof(float)` is typically **4 bytes**.
- `sizeof(short int)` is typically **2 bytes**.

Because the ternary expression was promoted to a float, `sizeof` returns the size of a float.

### Breakdown of the Logic

1. **Code:** `(x == 2) ? f : i`
2. **Types involved:** float (f) and short int (i).
3. **Resulting type:** float (due to promotion).
4. **sizeof evaluation:** `sizeof(float)`.
5. **Comparison:** `sizeof(float) == sizeof(float)` is **True**.
6. **Output:** float

56. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int x = 2, y = 2;
    int z = x ^ y & 1;
    printf("%d", z);
}
```

- a. 1
- b. 2 ✓
- c. 0
- d. 1 or 2



## 1. Operator Precedence

In C, the **Bitwise AND (&)** operator has higher precedence than the **Bitwise XOR (^)** operator. This means the expression is evaluated as if it were written like this:  $z = x \wedge (y \& 1);$

## 2. Step-by-Step Evaluation

Given:  $x = 2, y = 2$

### Step 1: Evaluate $(y \& 1)$

- Binary representation of 2 is 10
- Binary representation of 1 is 01
- $10 \& 01 = 00$  (which is **0** in decimal)

### Step 2: Evaluate $x \wedge 0$

- Binary representation of  $x$  (2) is 10
- Binary representation of 0 is 00
- $10 \wedge 00 = 10$  (which is **2** in decimal)

**Final Result:**  $z = 2$

---

## 57. Which of the following is a correct format for declaration of function?

- a. `return-type function-name(argument type);` ✓
- b. `return-type function-name(argument type){}`
- c. `return-type (argument type)function-name;`
- d. all of the mentioned

## Function Declaration vs. Definition

- **Declaration (Prototype):** This tells the compiler about the function's name, return type, and parameters before the function is actually used. It **must** end with a semicolon (;).
    - **Format:** `return-type function-name(parameter_list);`
    - **Example:** `int add(int a, int b);`
  - **Definition:** This contains the actual body (the code) of the function. It uses curly braces {} and **does not** have a semicolon after the parentheses.
    - **Format:** `return-type function-name(parameter_list)`  
`{ // code }`
-

58. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    const int i = 10;
    int *ptr = &i;
    *ptr = 20;
    printf("%d", i);
    return 0;
}
```

- a. Compile time error
- b. Compile time warning and printf displays 20 ✓
- c. Undefined behaviour
- d. 10

**Changing const variable through non-constant pointers invokes compiler warning.**

---

59. What will be the output of the following C code on a 64 bit machine?

```
#include <stdio.h>

union Sti
{
    int nu;
    char m;
};

int main()
{
    union Sti s;
    printf("%d", sizeof(s));
}
```

```
        return 0;
    }
```

- a. 8
- b. 5
- c. 9
- d. 4 ✓

### 1. The Core Difference: Union vs. Struct

In C, the way memory is allocated for a custom data type depends entirely on whether you use the keyword `struct` or `union`:

- **Struct:** Allocates enough memory for **all** members combined (plus potential padding). If this were a struct, it would take 4 bytes for the `int` + 1 byte for the `char` = 5 bytes (likely padded to 8).
- **Union:** Allocates memory only for the **largest single member**. All members share the same memory address. You can only store one member's value at a time.

### 2. Sizing the Members

To find the size of `union St1`, we look at the size of its individual parts on a 64-bit machine:

- `int nu;`: Typically occupies **4 bytes**.
- `char m;`: Typically occupies **1 byte**.

### 3. Calculating `sizeof(s)`

Since a union only allocates enough space for its largest member to ensure it can hold any of its defined ty

Size of Union =  $\max(\text{size of member}_1, \text{size of member}_2, \dots)$

Size of Union =  $\max(4 \text{ bytes}, 1 \text{ byte}) = 4 \text{ bytes}$

---

60. What will be the output of the following C code?

```
#include <stdio.h>
```

```
int main()
{
    short i;
    for (i = 1; i >= 0; i++)
        printf("%d", i);
}
```

- a. The control won't fall into the for loop
- b. Numbers will be displayed until the signed limit of short and throw a runtime error
- c. Numbers will be displayed until the signed limit of short and program will successfully terminate ✓
- d. This program will get into an infinite loop and keep printing numbers with no errors

- **Initialization:** The variable `i` is a `short` (typically 16-bit) and starts at **1**.
- **Condition:** The loop continues as long as `i >= 0`.
- **Increment:** `i` increases by 1 in each iteration.

### Why it Terminates (The "Signed Limit")

A `short` is a signed integer. On most modern systems, a 16-bit signed `short` has a range from **-32,768** to **32,767**.

- The loop will print numbers from 1 up to **32,767**.
- When `i` is 32,767 and the `i++` command executes, **signed integer overflow** occurs.
- In the standard [two's complement](#) representation used by almost all CPUs, adding 1 to the maximum positive value (32,767) causes the bits to "wrap around" to the most negative value: **-32,768**.

### The Turning Point

Once `i` becomes **-32,768**, the loop condition `i >= 0` is checked:

- Is `-32,768 >= 0`? **No**.
  - The condition fails, and the loop terminates successfully.
-

61. What is the output of the following program ?

```
#include<stdio.h>

void main()
{
    char xy=0;
    for(;xy>0;xy++);
    printf("%d",xy);
}
```

- a. 2
- b. 0 ✓
- c. Compiler error
- d. 1

```
char xy = 0;           // Initialize xy with 0
for(; xy > 0; xy++);    // Loop condition: is xy greater
                        // than 0?
printf("%d", xy);      // Print the value of xy
```

**Why is the output 0?**

1. **Initialization:** The variable `xy` is assigned the value 0.
2. **The Condition Check:** In a `for` loop, the condition (`xy > 0`) is checked **before** the first iteration.
3. **The Comparison:** The program asks: "Is `0 > 0`?"
  - This evaluates to **False**.
4. **Loop Termination:** Since the condition is false immediately, the body of the loop (and the `xy++` increment) is **never executed**.
5. **The Result:** Control passes directly to the `printf` statement. Since `xy` was never modified, it remains 0.
6. If `xy` had been initialized to 1, the loop would have started. It would continue to increment until the variable "overflowed" (since `char` has a limited range, usually -128 to 127).

---

62. Which of the following will occur if we call the `free()` function on a NULL pointer?

- a. Compilation Error
- b. Runtime Error
- c. Undefined Behaviour

d. The program will execute normally ✓

In C, the `free()` function is designed to be "safe" when handling null pointers. According to the C standard (specifically **ISO/IEC 9899**):

"If `ptr` is a null pointer, no action occurs."

This means the function performs a built-in check:

1. **Check:** Is the pointer `NULL`?
  2. **Action:** If yes, do nothing and return immediately.
  3. **Result:** The program continues to the next line of code without crashing.
- 

**63. Which of the following is not a valid C variable name?**

- a. `int number;`
- b. `float rate;`
- c. `int variable count; ✓`
- d. `int $main;`

In C, **spaces are not allowed** within a variable name.

When the compiler sees `int variable count;`, it interprets it as follows:

1. `int`: The data type.
2. `variable`: The name of the variable.
3. `count`: An unexpected extra word.

Because of that space, the compiler doesn't know what to do with the word `count` and will throw a **syntax error**.

## C Variable Naming Rules

To be valid, a variable name must follow these criteria:

- **Characters:** It can only contain letters (a-z, A-Z), digits (0-9), and underscores (`_`).
- **First Character:** It **must** start with a letter or an underscore. It cannot start with a number.
- **No Spaces:** You cannot use spaces.
- **No Special Symbols:** You cannot use symbols like `@`, `#`, `%`, or `$` (though some compilers like GCC allow `$` as an extension, it is not standard C).
- **Keywords:** You cannot use reserved keywords (like `int`, `while`, or `return`) as names.

`int $main;` **Technically Invalid in Standard C.** However, many modern compilers (like GCC) allow the \$ sign, which is why it is often considered "more valid" than a name with a space, but in a strict exam context, it is usually frowned upon. The option with the space is *always* the primary error.

---

64. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    enum {ORANGE = 5, MANGO, BANANA = 4, PEACH};
    printf("PEACH = %d", PEACH);
}
```

- a. PEACH = 3
- b. PEACH = 4
- c. PEACH = 5 ✓
- d. PEACH = 6

### The Logic of Enumeration Values

When you define an `enum`, the values follow these two rules:

1. **Explicit Assignment:** If a name is assigned a specific value (e.g., `ORANGE = 5`), it takes that value.
2. **Implicit Increment:** If a name is NOT assigned a value, it automatically takes the value of the previous element **plus 1**.

### Step-by-Step Calculation

Let's trace the values in the declaration `enum {ORANGE = 5, MANGO, BANANA = 4, PEACH};`:

- **ORANGE = 5:** This is explicitly set to 5.
- **MANGO:** No value is assigned, so it takes `ORANGE + 1`. Thus, **MANGO = 6**.
- **BANANA = 4:** This is explicitly set to 4. (Note: Enums don't have to be in increasing order).

- **PEACH**: No value is assigned, so it takes the value of the previous element (BANANA) plus 1. Thus, **PEACH = 5**.

| Enum Constant | Value | Reason              |
|---------------|-------|---------------------|
| ORANGE        | 5     | Explicitly assigned |
| MANGO         | 6     | ORANGE + 1          |
| BANANA        | 4     | Explicitly assigned |
| PEACH         | 5     | BANANA + 1          |

65. What will be the output of the program?

```
#include <stdio.h>

int main() {
    int i = 2;
    int j = i + (1,2,3,4,5);
    printf("%d",j);
    return 0;
}
```

- 7 ✓
- 2
- 3
- None of these

### How the Comma Operator Works

The comma operator ( , ) has the lowest precedence of all C operators. It works by:

1. Evaluating each expression from **left to right**.
2. Discarding the values of all expressions except the **last one**.
3. Returning the value of the **final expression** as the result of the entire group.



## Step-by-Step Execution

Let's break down the logic the compiler follows:

- **The Parentheses (1, 2, 3, 4, 5):**
    - The compiler evaluates 1, then 2, then 3, then 4, and finally 5.
    - It "throws away" the results of 1 through 4.
    - The entire parenthetical expression evaluates to 5.
  - **The Addition:**
    - Now the code effectively becomes `int j = i + 5;`.
    - Since `i` was initialized to 2, the calculation is `2 + 5`.
  - **The Result:**
    - `j` becomes 7.
- 

66. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int *p = NULL;
    for (foo(); p; p = 0)
        printf("In for loop");
    printf("After loop");
}
```

- a. In for loop after loop
- b. Compile time error ✓
- c. Infinite loop
- d. Depends on the value of NULL

### 1. Undefined Function `foo()`

The most immediate reason for the compile-time error is the call to `foo()` inside the `for` loop initialization: `for (foo(); p; p = 0)`

- In C, every function must be declared or defined before it is used.
- The compiler searches for a prototype (declaration) for `foo()`. Since it isn't defined anywhere in the snippet and there is no header file containing it, the

compiler throws an error (typically "implicit declaration of function" or "undefined reference").

## 2. Logic Breakdown (If `foo` were defined)

Even if we assume `foo()` was defined elsewhere, the loop would behave as follows based on the pointer logic:

- **Initialization:** `foo()` is executed once.
- **Condition Check (`p`):** The pointer `p` was initialized to `NULL`. In C, `NULL` evaluates to **false**.
- **Result:** Because the condition `p` is false from the very start, the body of the loop (`printf("In for loop");`) would **never execute**.
- **After Loop:** The program would jump straight to `printf("After loop");`.

## 3. Pointer Assignment Issue

There is a secondary point of interest in the "increment" section of the loop: `p = 0`.

- While `p` is a pointer (`int *p`), assigning `0` to a pointer is technically legal in C (it sets the pointer to `NULL`).
- However, if you intended to change an integer value, this code wouldn't do it because `p` is an address, not the value itself.

---

## 67. What will be the output of the following C code?

```
#include <stdio.h>

void foo();

int main()
{
    void foo();
    foo();
    return 0;
}

void foo()
{
```

```

        printf("2 ");
    }

```

- a. Compile time error
- b. 2 ✓
- c. Depends on the compiler
- d. Depends on the standard

## 1. Global vs. Local Declaration

In the code, `void foo();` appears in two places:

- **Globally:** Before `main()`. This tells the compiler that a function named `foo` exists and returns nothing.
- **Locally:** Inside `main()`. This is also a valid C practice. Redeclaring a function locally within a block is redundant if it's already declared globally, but it is **not an error**. It simply reinforces to the compiler that `foo` is available within the scope of `main`.

## 2. Function Call and Definition

- When `foo();` is called inside `main`, the compiler looks for its definition.
- The definition is provided at the bottom:

```

void foo() {
    printf("2 ");
}

```

- Since the function is defined correctly, the program executes the `printf` statement and prints **2**.

| Element                              | Purpose                                                 | Valid?                                |
|--------------------------------------|---------------------------------------------------------|---------------------------------------|
| <code>void foo();</code><br>(Global) | Tells the compiler <code>foo</code> exists.             | ✓ Yes                                 |
| <code>void foo();</code><br>(Local)  | Re-declares <code>foo</code> inside <code>main</code> . | ✓ Yes (Redundant no action but legal) |
| <code>foo();</code>                  | Calls the function.                                     | ✓ Yes                                 |

| Element                    | Purpose             | Valid?                                                                                  |
|----------------------------|---------------------|-----------------------------------------------------------------------------------------|
| <code>printf("2 ");</code> | Executes the logic. |  Yes |

---

68. What will be the output of the following C code?

```
#include <stdio.h>

void m();

void n()
{
    m();
}

void main()
{
    void m()
    {
        printf("hi");
    }
}

a. hi
b. Compile time error ✓
c. Nothing
d. Varies
```

### 1. The Core Issue: Nested Functions

In C, you cannot define a function inside the body of another function.

- Look at the `main()` function in the snippet. Inside its curly braces `{ ... }`, there is a definition for `void m()`.

- While some compilers (like GCC) have "extensions" that might allow this, it is **not part of the standard C language**. Most standard-compliant compilers will treat the definition of `m()` inside `main()` as a syntax error.

## 2. Scope and Linkage

Even if a compiler allowed the nested definition, there is a logical "disconnect" in this code:

- **The Call:** Inside function `n()`, there is a call to `m()`.
- **The Prototype:** At the top, `void m();` tells function `n()` that a function named `m` exists *somewhere* globally.
- **The Reality:** The actual implementation of `m()` is hidden inside `main()`. Functions defined inside a block (if allowed) are local to that block. Function `n()` cannot "see" or reach inside `main()` to find the definition of `m()`.

## 3. Missing Global Definition

Because `m()` is defined locally within `main`, it does not satisfy the global declaration `void m();` made at the top of the file. When the compiler (or linker) looks for the version of `m()` that `n()` is trying to call, it finds nothing in the global scope, leading to an "undefined reference" or "static declaration" error.

```
#include <stdio.h>

void m(); // Prototype

void n() {
    m(); // Now it can find the global m
}

// Move this OUTSIDE of main
void m() {
    printf("hi");
}

void main() {
    n(); // Call n, which calls m
}
```

| Feature | Status | Reason                                    |
|---------|--------|-------------------------------------------|
| Global  | ✓      | Tells the compiler <code>m</code> exists. |

| Feature            | Status                                                                                  | Reason                                                               |
|--------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Prototype          | Valid                                                                                   |                                                                      |
| Function n()       |  Valid | Correctly calls m.                                                   |
| Nested m() in main |  Error | Standard C does not allow defining functions inside other functions. |
| Visibility         |  Error | n() cannot access a function defined locally inside main().          |

69. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int ThisIsVariableName = 12;
    int ThisIsVariablename = 14;
    printf("%d", ThisIsVariablename);
    return 0;
}
```

- a. The program will print 12
- b. The program will print 14 ✓
- c. The program will have a runtime error
- d. The program will cause a compile-time error due to redeclaration

Variable names **ThisIsVariablename** and **ThisIsVariableName** are both distinct as C is case sensitive.

---

70. What will be the output of the following C function?

```
#include <stdio.h>

void reverse(int i);

int main()
{
    reverse(1);
}

void reverse(int i)
{
    if (i > 5)        return ;
    printf("%d ", i);
    return reverse((i++, i));
}
```

- a. 1 2 3 4 5 ✓
- b. Segmentation fault
- c. Compilation error
- d. 5 4 3 2 1

### 1. Understanding the **reverse** Function

The function uses **recursion**, meaning it calls itself. There are three key parts to this function's logic:

- **The Base Case:** `if (i > 5) return ;` — This stops the recursion once `i` reaches 6.
- **The Action:** `printf("%d ", i);` — This prints the current value of `i` followed by a space.
- **The Recursive Call:** `return reverse((i++, i));` — This is where it gets interesting due to the **comma operator**.

### 2. The Comma Operator Explained

In C, the expression `(i++, i)` uses the comma operator. It works like this:

1. The left expression `(i++)` is evaluated first. If `i` was 1, it prepares to increment to 2.
2. There is a **sequence point** at the comma, meaning the increment of `i` is completed before moving on.
3. The right expression `(i)` is then evaluated.
4. The value of the **entire expression** is the value of the rightmost part.

**Example:** If `i = 1`, then `(i++, i)` evaluates to **2**. Therefore, `reverse(1)` calls `reverse(2)`.

| Call Level | Value of i | i > 5? | Output      | Next Call                                   |
|------------|------------|--------|-------------|---------------------------------------------|
| Initial    | 1          | No     | 1           | <code>reverse((1++, 2)) → reverse(2)</code> |
| Recurse 1  | 2          | No     | 2           | <code>reverse((2++, 3)) → reverse(3)</code> |
| Recurse 2  | 3          | No     | 3           | <code>reverse((3++, 4)) → reverse(4)</code> |
| Recurse 3  | 4          | No     | 4           | <code>reverse((4++, 5)) → reverse(5)</code> |
| Recurse 4  | 5          | No     | 5           | <code>reverse((5++, 6)) → reverse(6)</code> |
| Recurse 5  | 6          | Yes    | <i>None</i> | <b>Returns (Stops)</b>                      |

---

71. How many times while loop condition is tested in the following C code snippets, if `i` is initialized to 0 in both the cases?

case1 ->

```
while (i < n)
{
    i++;
}
```



-----

case2 ->

do

{

    i++;

}while (i <= n);

a. n, n

b. n, n+1

c. n+1, n

d. n+1, n+1 ✓

| Feature     | Case 1 (while)      | Case 2 (do-while)     |
|-------------|---------------------|-----------------------|
| Initial i   | 0                   | 0                     |
| Condition   | i < n               | i <= n                |
| True Tests  | n times (0 to n-1)  | n times (1 to n)      |
| False Tests | 1 time (when i = n) | 1 time (when i = n+1) |
| Total Tests | n + 1               | n + 1                 |

---

72. What will be the output of the following C code?

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int x = 3, i = 0;
```

```
    do {
```

```
        x = x++;
```

```
        i++;
```

```

        }while (i != 3);
    printf("%d", x);
}

```

- a. Undefined behaviour
- b. Output will be 3 ✓
- c. Output will be 6
- d. Output will be 5

**The Core Logic:  $x = x++$ ;**

In the expression  $x = x++$ ;, there are three distinct steps happening due to the rules of **post-increment**:

1. **Value Retrieval:** The current value of  $x$  (which is **3**) is set aside to be used for the assignment.
2. **Increment:** The variable  $x$  is incremented in memory (it becomes **4**).
3. **Assignment:** The value set aside in step 1 (**3**) is now assigned back to  $x$ . This overwrites the incremented value, bringing  $x$  back to **3**.

| Iteration | Start<br>$x$ | Operation $x = x++$                          | Start<br>$i$ | Operation<br>$i++$ | Loop<br>Condition ( $i \neq 3$ ) |
|-----------|--------------|----------------------------------------------|--------------|--------------------|----------------------------------|
| 1st       | 3            | $x$ becomes 4,<br>then assigned<br>back to 3 | 0            | 1                  | $1 \neq 3$<br>(True)             |
| 2nd       | 3            | $x$ becomes 4,<br>then assigned<br>back to 3 | 1            | 2                  | $2 \neq 3$<br>(True)             |
| 3rd       | 3            | $x$ becomes 4,<br>then assigned<br>back to 3 | 2            | 3                  | $3 \neq 3$<br>(False)            |

---

73. What will be the output of the following C code?

```
#include <stdio.h>
```

```

int main()
{
    int x = 0;
    if (x == 1)
    if (x >= 0)
        printf("true");
    else
        printf("false");
}

```

- a. true
- b. false
- c. Depends on the compiler
- d. No print statement ✓

### The Logic Breakdown

In your code, the indentation is actually a bit misleading. Here is how the compiler sees the structure:

1. **Outer If:** `if (x == 1)`
  - Since `x` is initialized to 0, the condition `(0 == 1)` is **false**.
2. **Inner If & Else:** \* Because the first condition is false, the compiler skips the **entire block** that follows it.
  - In C, an `else` always attaches to the **nearest** unsigned `if` above it. This means the `else` belongs to `if (x >= 0)`, not the outer `if (x == 1)`.

```

int x = 0;
if (x == 1) { // This condition is FALSE
    // Everything inside this block is ignored
    if (x >= 0)
        printf("true");
    else
        printf("false");
}
// Result: Nothing happens, so nothing is printed.

```

---

74. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    auto i = 10;
    const auto int *p = &i;
    printf("%d", i);
}
```

- a. 10 ✓
- b. Compile time error
- c. Depends on the standard
- d. Depends on the compiler

### 1. The **auto** Keyword

In standard C (specifically C89/C90 and C99), `auto` is a storage class specifier that tells the compiler a variable has a "local" lifetime.

- Since all variables declared inside a function are `auto` by default, writing `auto i = 10;` is exactly the same as writing `int i = 10;` (though in older standards, the type `int` was often assumed if omitted).

### 2. The Pointer Declaration

The line `const auto int *p = &i;` looks complex, but it follows standard C rules:

- **const**: This means the value being pointed to cannot be modified through this pointer (`p`).
- **auto**: Again, this just specifies the storage class.
- **int \*p**: This declares `p` as a pointer to an integer.
- **&i**: This assigns the address of the variable `i` to the pointer `p`.

### 3. The **printf** Statement

The code executes `printf("%d", i);`.

- It doesn't actually use the pointer `p` for the output.

- It simply looks at the value stored in the variable `i`, which is **10**, and prints it to the console.

---

**75. Which of the following cannot be static in C?**

- a. Variables
- b. Functions
- c. Structures
- d. None of the mentioned ✓

| Element    | Effect of static                                                        |
|------------|-------------------------------------------------------------------------|
| Variables  | Extends lifetime (if local) or limits scope to the file (if global).    |
| Functions  | Limits the function's visibility to the file it is defined in.          |
| Structures | Acts like a static variable; retains data and limits scope to the file. |

---

**76. What will be the output of the following C code?**

```
#include <stdio.h>
int *i;    int main()
{
    if (i == NULL)        printf("true");
    return 0;
}
```

- a. true ✓
- b. true only if NULL value is 0
- c. Compile time error

d. Nothing

## 1. Global Variable Initialization

In the code, `int *i;` is declared **outside** of the `main()` function. In C, global variables (those with static storage duration) are automatically initialized to zero if no value is explicitly provided.

- For an integer, this means it becomes 0.
- For a pointer, this means it becomes NULL.

## 2. The Comparison

Inside `main()`, the code checks `if (i == NULL)`.

- Since `i` was automatically initialized to NULL as a global pointer, this condition evaluates to **true**.
- Therefore, the program executes `printf("true");`.

---

77. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    int i = -9;
    printf("%d %d %d", i++, ++i, ++i);
}
```

- a. Compile error
- b. -6 -7 -8
- c. -7 -7 -8
- d. -7 -6 -6 ✓

Arguments are often evaluated **right to left** (common on x86), and **all side-effects apply before values are passed**:

| Step | Operation       | i value after | Value passed |
|------|-----------------|---------------|--------------|
| 1st  | ++i (rightmost) | -9 → -8       | -8           |
| 2nd  | ++i (middle)    | -8 → -7       | -7           |

|            |                             |                      |                                       |
|------------|-----------------------------|----------------------|---------------------------------------|
| <b>3rd</b> | <code>i++</code> (leftmost) | <code>-7 → -6</code> | <code>-7</code> (pre-increment value) |
|------------|-----------------------------|----------------------|---------------------------------------|

Since `printf` **prints left to right**:

```
i++ → -7 // post precedence is higher than pre
++i → -6 ← (i is now -6 after all side effects)
++i → -6

Output: -7 -6 -6
```

---

**78. What will be the output of the following C code?**

```
#include <stdio.h>

int main()
{
    int i = 5;
    printf("%d %d %d", i, i++, ++i);
    return 0;
}
```

- a) 5 5 7
- b) 5 5 6
- c) 7 6 7 ✓
- d) 7 6 6

Arguments evaluated **right to left** (common x86 behavior):

| Step       | Operation                    | <code>i</code> after | Value passed         |
|------------|------------------------------|----------------------|----------------------|
| <b>1st</b> | <code>++i</code> (rightmost) | <code>5 → 6</code>   | <b>6</b>             |
| <b>2nd</b> | <code>i++</code> (middle)    | <code>6 → 7</code>   | <b>6</b> (old value) |
| <b>3rd</b> | <code>i</code> (leftmost)    | stays <b>7</b>       | <b>7</b>             |

Since `printf` **prints left to right**:

```
i → 7

i++ → 6 // post precedence is higher than pre
```

`++i → 7 ← (all side effects already applied, i is 7)`

Output: 7 6 7

---

79. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{

    int x = 5;

    int y = ++x + x++;

    printf("%d %d", y,x);
}
```

- a) 12 7
- b) 11 6
- c) 13 7 ✓
- d) Undefined behaviour

Many compilers **apply all side effects first**, then compute the expression:

| Step | Operation                  | Effect on <b>x</b>                                | Value Used                |
|------|----------------------------|---------------------------------------------------|---------------------------|
| 1    | <code>++x</code> (pre)     | <b>5</b> → <b>6</b> but <b>6</b> doesn't not used | N/A                       |
| 2    | <code>x++</code> (post)    | <b>6</b> is used then <b>6</b> → <b>7</b>         | <b>6</b> (pre-post value) |
| 3    | <code>++x</code> (pre)     | <b>7</b> is used in pre operation                 | <b>7</b>                  |
| 4    | <code>y = ++x + x++</code> | —                                                 | <b>7 + 6 = 13</b>         |

`y = 7 + 6 = 13`

`x = 7` (two increments from 5)

Output: 13 7

---

80. What will be the output of the following C code?



```
#include <stdio.h>

int main()

{
    int x = 5;
    int y = x++ + ++x;
    printf("%d %d", y,x);
}
```

- a) 12 7 ✓
- b) 11 6
- c) 13 7
- d) Undefined behaviour

Many compilers **apply all side effects first**, then compute the expression:

| Step     | Operation     | Effect on <b>x</b>                           | Value Used        |
|----------|---------------|----------------------------------------------|-------------------|
| <b>1</b> | x++ (post)    | <b>5</b> is used then increment <b>5 → 6</b> | <b>5</b>          |
| <b>2</b> | ++x (pre)     | <b>6 → 7</b>                                 | <b>7</b>          |
| <b>3</b> | y = x++ + ++x | —                                            | <b>5 + 7 = 12</b> |

- $y = 5 + 7 = 12$
- $x = 7$  (two increments from 5)
- Output: 12 7

81. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int x = 3;

    int y = x++ * ++x;

    printf("%d", y);
}
```

```

    return 0;
}

```

a) 12  
 b) 15 ✓  
 c) 16  
 d) Undefined behaviour

Many compilers **apply all side effects first**, then compute the expression:

| Step | Operation       | Effect on $x$                              | Value Used   |
|------|-----------------|--------------------------------------------|--------------|
| 1    | $x++$ (post)    | 3 is used then increment $3 \rightarrow 4$ | 3            |
| 2    | $++x$ (pre)     | $4 \rightarrow 5$                          | 5            |
| 3    | $y = x++ + ++x$ | —                                          | $3 * 5 = 15$ |

- $y = 3 * 5 = 15$
- Output: 15

82. What will be the output of the following C code?

```

#include <stdio.h>

int main()
{
    int a = 5;

    printf("%d", a+++a);
    return 0;
}

```

- a) 10  
 b) 11 ✓  
 c) Compiler error  
 d) Undefined behaviour
- 

When you write  $a+++a = (a++) + a$ , you are doing two things in a single expression without a sequence point between them:

1. **Modifying** the variable `a`.
2. **Accessing** the same variable `a` to calculate the sum.

| Step | Operation               | Effect on <code>a</code>                            | Value Used        |
|------|-------------------------|-----------------------------------------------------|-------------------|
| 1    | <code>a++</code> (post) | <b>5</b> is used then increment <b>5</b> → <b>6</b> | <b>5</b>          |
| 2    | <code>a</code>          | The value is incremented to <b>6</b>                | <b>6</b>          |
| 3    | <code>a+++a</code>      | <code>a+++a = (a++) + a</code>                      | <b>5 + 6 = 11</b> |

- `a+++a = (a++) + a`
  - `a+++a = 5 + 6`
  - Output: 11
- 

83. What will be the output of the following C code?

```
#include <stdio.h>
```

```
int main()
{
```

```
int i = 0;
```

```
for(; i < 3;)
```

```
printf("%d", i++);
return 0;
```

```
}
```

- a) 0 1 2 ✓
- b) 1 2 3
- c) Compiler error
- d) Undefined behaviour

### 1. The Structure of the `for` loop

A standard C `for` loop follows this syntax: `for (initialization; condition; increment/decrement)`. In your code, the **initialization** and **increment** sections are left blank:

- `int i = 0;` (Initialization happens outside the loop).
- `; i < 3;` (The condition is checked before each iteration).
- `)` (The increment section is empty because the increment happens inside the `printf`).

## 2. Post-Increment Execution

The expression `i++` is a **post-increment** operator. This means:

1. The **current value** of `i` is passed to the `printf` function first.
2. The value of `i` is **incremented by 1** only *after* the current value has been used.

| Iteration | Condition ( <code>i &lt; 3</code> ) | <code>printf</code> value | Value of <code>i</code> after <code>i++</code> |
|-----------|-------------------------------------|---------------------------|------------------------------------------------|
| 1st       | <code>0 &lt; 3</code> (True)        | Prints <b>0</b>           | <code>i</code> becomes 1                       |
| 2nd       | <code>1 &lt; 3</code> (True)        | Prints <b>1</b>           | <code>i</code> becomes 2                       |
| 3rd       | <code>2 &lt; 3</code> (True)        | Prints <b>2</b>           | <code>i</code> becomes 3                       |
| 4th       | <code>3 &lt; 3</code> (False)       | <i>Loop terminates</i>    | <code>i</code> remains 3                       |

84. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    printf("%d", *(arr + *(arr + 1) / 10));
    return 0;
}

a) 10
b) 20
c) 30 ✓
d) 40
```

### 1. Evaluate the Inner Expression: `*(arr + 1)`

- `arr` is the base address of the array (pointing to the first element, 10).
- `arr + 1` points to the second element in the array.
- `*(arr + 1)` dereferences that pointer, giving us the value **20**.

### 2. Perform the Division

The expression now looks like this: `*(arr + 20 / 10)`.

- `20 / 10 = 2`.
- The expression simplifies to: `*(arr + 2)`.

### 3. Evaluate the Outer Expression: `*(arr + 2)`

- `arr + 2` points to the third index (the 3rd element) of the array.
- Let's look at the array mapping:
  - `arr[0]` is 10
  - `arr[1]` is 20
  - **`arr[2]` is 30**
  - **`arr[3]` is 40**
  - `arr[4]` is 50

Wait—let's re-examine that index carefully!

- `arr + 0` → 10
- `arr + 1` → 20
- `arr + 2` → 30
- `arr + 3` → 40

Wait, looking at the math again:

1. `*(arr + 1)` is **20**.
2. `20 / 10` is **2**.
3. `*(arr + 2)` is the value at index 2.
4. Index 0 is 10, Index 1 is 20, **Index 2 is 30**.

---

85. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    char c = 255;
    c = c + 10;
    printf("%d", c);
    return 0;
}
```

a) 256

- b) 9 ✓
- c) -245
- d) -1

- **Variable Initialization:** The line `char c = 255;` is the key. In most modern C compilers, a `char` is **signed** by default and has a range from **-128 to 127**.

- **Overflow during Assignment:** When you assign 255 to a signed `char`, it wraps around because 255 in binary is 11111111. In two's complement representation, 11111111 for a signed 8-bit integer is actually **-1**.

- **The Calculation:**

- Initially, `c = -1`.
- The operation `c = c + 10;` becomes `-1 + 10`.
- The result is **9**.

- **Printing:** The `printf("%d", c);` statement prints the integer value of `c`, which is **9**.

| Step | Operation                     | Value of c (Decimal) |
|------|-------------------------------|----------------------|
| 1    | <code>char c = 255;</code>    | -1 (due to overflow) |
| 2    | <code>c = c + 10;</code>      | 9                    |
| 3    | <code>printf("%d", c);</code> | <b>9</b>             |

86. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int a = 2;
    int b = 0;
    int y = (b == 0) ? a : (a > b) ? (b = 1): a;
    printf("%d", y);
}
```

- a. Compile time error
- b. 1
- c. 2 ✓
- d. 0

- **Evaluate the first condition:** The code checks `(b == 0)`.
  - Since `b` was initialized to 0, this condition is **true**.
- **Short-circuiting:** In a ternary operation, once the condition is met, the corresponding expression is executed, and the rest of the statement is **ignored**.
  - Because `(b == 0)` is true, the compiler immediately picks the first option: `a`.
  - The second part of the statement—the nested ternary `(a > b) ? (b = 1) : a`—is **never evaluated**.
- **Final Result:**
  - `y` becomes equal to the value of `a`.
  - Since `int a = 2`, the value of `y` is **2**.

### 87. When compiler accepts the request to use the variable as a register?

- a. It is stored in CPU ✓
- b. It is stored in cache memory
- c. It is stored in main memory
- d. It is stored in secondary memory

### The `register` Keyword in C

In C programming, the `register` storage class is a hint to the compiler that a specific variable will be accessed frequently.

- **The Request:** By using `register int x;`, you are suggesting to the compiler that it should store this variable in a **CPU register** instead of the RAM (main memory).
- **The Goal:** Accessing data from a CPU register is significantly faster than fetching it from RAM or even cache memory, which optimizes the program's execution speed.

### 1. Automatic (`auto`)

This is the default storage class for all local variables declared inside a function or block.

- **Storage Location: Main Memory (Stack)**
- **Default Value:** Garbage value (unpredictable).
- **Lifetime:** Deleted when the function/block is exited.

## 2. Register (**register**)

As seen in your previous question, this is used for variables that need very fast access.

- **Storage Location: CPU Register \* Default Value:** Garbage value.
- **Lifetime:** Deleted when the function/block is exited.
- **Note:** You cannot get the memory address (using `&`) of a register variable because it doesn't reside in RAM.

## 3. Static (**static**)

Static variables preserve their value even after they go out of scope.

- **Storage Location: Main Memory (Data Segment)**
- **Default Value:** Zero (0).
- **Lifetime:** Persists until the end of the entire program execution.

## 4. External (**extern**)

This is used to give a reference to a global variable that is visible to all program files.

- **Storage Location: Main Memory (Data Segment)**
- **Default Value:** Zero (0).
- **Lifetime:** Persists until the end of the program.

| Storage Class   | Storage Location   | Default Value | Scope        | Lifetime       |
|-----------------|--------------------|---------------|--------------|----------------|
| <b>auto</b>     | RAM (Stack)        | Garbage       | Local        | End of block   |
| <b>register</b> | CPU                | Garbage       | Local        | End of block   |
| <b>static</b>   | RAM (Data Segment) | Zero          | Local/Global | End of program |



| Storage Class | Storage Location   | Default Value | Scope  | Lifetime       |
|---------------|--------------------|---------------|--------|----------------|
| <b>extern</b> | RAM (Data Segment) | Zero          | Global | End of program |

---

### 88. What is the scope of an external variable?

- Whole source file in which it is defined
- From the point of declaration to the end of the file in which it is defined
- Any source file in a program
- From the point of declaration to the end of the file being compiled ✓

- **Visibility (Scope):** While a global variable exists for the entire program's duration, the compiler only "sees" it in a specific file from the line where it is declared down to the very end of that file. If you try to use it *above* its declaration in the same file, the compiler will throw an error.

- **Linkage vs. Scope:** This is where many students get confused.

- **Linkage** is global (it can be accessed by other files using the `extern` keyword).
- **Scope** (specifically "File Scope") is limited to the file where it is declared, starting from the declaration point.

```
void func1() {
    printf("%d", x); // ERROR: 'x' is not yet declared
in this scope
}
```

```
int x = 10; // Declaration point
```

```
void func2() {
    printf("%d", x); // SUCCESS: 'x' is visible from
here to the end of the file
}
```

## 1. Local Variables (Block Scope)

These are variables declared inside a function or a block { . . . }.

- **Scope:** Limited to the block in which they are declared. They cannot be accessed outside those curly braces.
- **Storage:** Usually the **Stack**.
- **Example:**

```
void myFunction() {  
    int x = 5; // Local scope starts  
} // Local scope ends here
```

## 2. Global Variables (File Scope)

These are variables declared outside of all functions.

- **Scope:** From the **point of declaration to the end of the file** being compiled.
- **Storage: Data Segment.**
- **Note:** As seen in your mock test, while they are "global," they are only visible to the compiler in the current file *after* they have been defined.

## 3. Formal Parameters (Function Prototype Scope)

These are the variables listed in the function header (the arguments).

- **Scope:** Limited to the entire function body.
- **Example:** In `void add(int a, int b)`, the variables `a` and `b` have scope throughout the `add` function.

## 4. Static Variables

Static variables have a unique relationship between scope and lifetime.

- **Internal Static:** Declared inside a function. It has **Local Scope** (only visible inside that function) but a **Global Lifetime** (it isn't destroyed when the function ends).
- **External Static:** Declared outside functions with the `static` keyword. It has **File Scope** but cannot be accessed by other files even with the `extern` keyword (it provides privacy/encapsulation).

| Variable Type | Scope               | Lifetime       | Initial Value |
|---------------|---------------------|----------------|---------------|
| Local (auto)  | Within the block {} | While block is | Garbage       |

| Variable Type          | Scope                         | Lifetime              | Initial Value |
|------------------------|-------------------------------|-----------------------|---------------|
|                        |                               | active                |               |
| <b>Global (extern)</b> | Point of decl. to end of file | Program duration      | 0             |
| <b>Static (Local)</b>  | Within the block {}           | Program duration      | 0             |
| <b>Static (Global)</b> | Only the current file         | Program duration      | 0             |
| <b>Register</b>        | Within the block {}           | While block is active | Garbage       |

---

89.What will be the output of the following C code?

```
#include <stdio.h>
void main()
{
    int a = -5;
    int k = (a++, ++a);
    printf("%d", k);
}
```

- a. -4
- b. -5
- c. 4
- d. -3 ✓

## 1. The Comma Operator (,)

In C, the comma operator evaluates each expression from left to right and **returns the value of the rightmost expression**.

- In the statement `k = (expression1, expression2)`, the value of `expression2` is what ultimately gets assigned to `k`.

## 2. Execution Step-by-Step

- **Initial State:** `a = -5`
- **Step 1 (`a++`):** The left side of the comma is evaluated. `a++` is a post-increment.
  - The value used in the expression is `-5`, but immediately after, `a` is incremented.
  - `a` becomes `-4`.
- **Step 2 (`++a`):** The right side of the comma is evaluated. `++a` is a pre-increment.
  - Since `a` is already `-4`, the pre-increment increases it first.
  - `a` becomes `-3`.
  - The value of this expression (`-3`) is returned because it is the rightmost part of the comma operator.
- **Assignment:** The value returned by the comma operator (`-3`) is assigned to `k`.

## Final Result

The `printf` outputs the value of `k`, which is `-3`.

---

90. Which of the following data type will throw an error on modulus operation(`%`)?

- a. `char`
- b. `short`
- c. `int`
- d. `float` ✓

| Data Type         | Can use %? | Reason                       |
|-------------------|------------|------------------------------|
| <code>int</code>  | Yes        | Standard integer division.   |
| <code>char</code> | Yes        | Treated as an 8-bit integer. |

| Data Type      | Can use %? | Reason                        |
|----------------|------------|-------------------------------|
| short          | Yes        | Treated as a 16-bit integer.  |
| float / double | No         | Requires <math.h> and fmod(). |

91. What will be the output of the following C code on a 32-bit machine?

```
#include <stdio.h>
int main()
{
    int x = 10000;
    double y = 56;
    int *p = &x;
    double *q = &y;
    printf("p and q are %d and %d", sizeof(p),
sizeof(q));
    return 0;
}
```

- a. p and q are 4 and 4 ✓
- b. p and q are 4 and 8
- c. compiler error
- d. p and q are 2 and 8

| Variable | Type   | Size (32-bit) | Size (64-bit) |
|----------|--------|---------------|---------------|
| x        | int    | 4 bytes       | 4 bytes       |
| y        | double | 8 bytes       | 8 bytes       |

| Variable | Type              | Size (32-bit) | Size (64-bit) |
|----------|-------------------|---------------|---------------|
| p        | int* (Pointer)    | 4 bytes       | 8 bytes       |
| q        | double* (Pointer) | 4 bytes       | 8 bytes       |

92. Comment on the following statement.

```
n = 1;
```

```
printf("%d, %dn", 3*n, n++);
```

- a. Output will be 3, 2
- b. Output will be 3, 1
- c. Output will be 6, 1 ✓
- d. Output is compiler dependent ✓

| Step | Operation  | Effect on n                                | Value Used |
|------|------------|--------------------------------------------|------------|
| 1    | n++ (post) | 1 is used then increment $1 \rightarrow 2$ | 1          |
| 2    | $3*n$      | $3 \times 2 \rightarrow 6$                 | 2          |

```
printf("%d, %dn", 3*n, n++);
```

```

|    | → 1
|
| → 6

```

compiler-dependent due to the order in which the arguments are evaluated. In C, the order of evaluation of function arguments is unspecified, meaning that the compiler can choose to evaluate them from left to right, right to left, or in any other order. This can lead to different results depending on how the compiler decides to evaluate the expressions.

93. Where in C the order of precedence of operators do not exist?

- a) Within conditional statements, if, else

- b) Within while, do-while
- c) Within a macro definition
- d) None of the mentioned ✓

| Context                | Does Precedence Apply? | Reason                                                                                       |
|------------------------|------------------------|----------------------------------------------------------------------------------------------|
| Conditional Statements | Yes                    | Expressions in if/else are standard C expressions.                                           |
| Loops                  | Yes                    | Expressions in while/for follow standard math/logic rules.                                   |
| Macros                 | Yes                    | Once expanded, the resulting expression is parsed by the compiler using standard precedence. |

---

94. Variable name resolution (number of significant characters for the uniqueness of variable) depends on \_\_\_\_\_

- a. Compiler and linker implementations ✓
- b. Assemblers and loaders implementations
- c. C language
- d. None of the mentioned

It depends on the standard to which compiler and linkers are adhering.

---

95. What will be the output of the following C code?

```
#include <stdio.h>

void foo(auto int i);    int main()
{
    foo(10);
}
```

```
void foo(auto int i)    {
    printf("%d", i );
}
```

- a. 10
- b. Compile time error ✓
- c. Depends on the standard
- d. None of the mentioned

### The Problem: Illegal Use of `auto`

In C, the `auto` keyword is a **storage class specifier** that can only be used for variables declared within a block (local variables). When you see the line:

```
void foo(auto int i);
```

You are attempting to use `auto` for a **function parameter**. According to the C standard (C89, C99, and C11):

- Function parameters have their own implicit storage duration.
- The only storage class specifier allowed for function parameters is `register`.
- Using `auto` or `static` in a function parameter list is **syntactically illegal**.

96. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    unsigned int a = 10;
    a = ~a;
    printf("%d", a);
}
```

- a. -9
- b. -10



- c. -11 ✓
- d. 10

## 1. The Binary Representation

The variable `a` is an unsigned int with a value of 10. In a 32-bit system, its binary form looks like this:

```
00000000 00000000 00000000 00001010
```

## 2. The Bitwise NOT Operation (~)

The operator `~` flips every single bit (0 becomes 1, and 1 becomes 0). When you execute `a = ~a;`, the bits become:

```
11111111 11111111 11111111 11110101
```

## 3. The `printf` Interpretation

This is the "trick" part of the question. Even though `a` is declared as an unsigned int, the `printf` function uses the `%d` format specifier.

- `%u` would treat the number as unsigned (resulting in a very large positive number, 4,294,967,285).
- `%d` tells `C` to treat that bit pattern as a **signed decimal integer** using **Two's Complement** notation.

## 4. Calculating Two's Complement

To find the decimal value of a signed binary number starting with 1:

1. **Invert the bits:** 00000000 00000000 00000000 00001010 (which is 10).
2. **Add 1:**  $10 + 1 = 11$
3. **Apply the negative sign:** -11.

---

97. Relational operators cannot be used on \_\_\_\_\_

- a. structure ✓
- b. long
- c. strings
- d. float

| Data Type                    | Can use == or <? | Reason                                                                      |
|------------------------------|------------------|-----------------------------------------------------------------------------|
| Primitive (int, float, long) | Yes              | Built-in support for numerical comparison.                                  |
| Structure                    | No               | Compiler cannot handle padding and multiple members automatically.          |
| String (char)*               | Technically Yes  | Compares memory addresses, not the text itself (usually not what you want). |

98. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    static int x = 3;
    x++;
    if (x <= 5)    {
        printf("hi");
        main();
    }
}

a. Run time error
b. hi
c. Infinite hi
d. hi hi ✓
```

### 1. The Power of static

In C, a `static` variable is initialized only **once** and persists across function calls. Unlike a regular local variable (which would be recreated and reset to 3 every time `main()` is called), the variable `x` remembers its previous value.

## 2. Execution Trace

Let's follow the code step-by-step:

- **Initial Call (Level 1):**
  - `static int x = 3;` (Initialized to 3).
  - `x++;` → `x` is now **4**.
  - `if (4 <= 5)` is **True**.
  - **Action:** Prints "**hi**" and calls `main()` again.
- **Recursive Call (Level 2):**
  - `static int x = 3;` is **skipped** (it was already initialized).
  - `x++;` → `x` is now **5**.
  - `if (5 <= 5)` is **True**.
  - **Action:** Prints "**hi**" and calls `main()` again.
- **Recursive Call (Level 3):**
  - `x++;` → `x` is now **6**.
  - `if (6 <= 5)` is **False**.
  - **Action:** The function returns without printing anything.

| Keyword                   | Result                   | Reason                                                                                         |
|---------------------------|--------------------------|------------------------------------------------------------------------------------------------|
| <b>static</b>             | <b>hi hi</b>             | Retains value across calls; eventually hits 6.                                                 |
| <b>auto</b>               | <b>Infinite hi</b>       | Resets to 3 every call; never hits 6.                                                          |
| <b>register</b>           | <b>Infinite hi</b>       | Same as auto; just requests CPU register storage.                                              |
| <b>extern (with init)</b> | <b>Compilation Error</b> | you cannot initialize a variable inside a function while declaring it as <code>extern</code> . |

---

99. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    switch (printf("Do"))    {
        case 1:              printf("First");
            break;
        case 2:              printf("Second");
            break;
        default:             printf("Default");
    }
```

```

        break;
    }

}

```

- a. Do
- b. DoFirst
- c. DoSecond ✓
- d. DoDefault

### 1. The `printf()` Return Value

In C, the `printf()` function doesn't just display text; it also **returns an integer** representing the total number of characters it successfully printed.

- In the expression `switch (printf("Do"))`, the string "Do" consists of **2 characters** ('D' and 'o').
- Therefore, `printf("Do")` prints Do to the screen and then returns the value **2**.

### 2. The `switch` Logic

The `switch` statement now evaluates the return value, which is **2**:

- **Step 1:** The program executes `printf("Do")`. The output is now: Do.
- **Step 2:** The `switch` receives the value **2**.
- **Step 3:** It looks for case `2:`.
- **Step 4:** It executes the code inside case `2:`, which is `printf("Second");`. The output becomes: DoSecond.
- **Step 5:** The `break;` statement exits the `switch` block.

---

### 100. What is short int in C programming?

- a. The basic data type of C
- b. Qualifier
- c. Short is the qualifier and int is the basic data type ✓
- d. None

| Component | Role | Examples |
|-----------|------|----------|
|-----------|------|----------|

| Component          | Role                                | Examples                             |
|--------------------|-------------------------------------|--------------------------------------|
| Basic Data Type    | Defines the core nature of data.    | int, char, float, double             |
| Qualifier/Modifier | Alters size or range.               | short, long, signed, unsigned        |
| Combined Result    | A specific type for specific needs. | unsigned int, long double, short int |

101. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int a = 0, i = 0, b;
    for (i = 0; i < 5; i++)
    {
        a++;
        if (i == 3)            break;
    }

    printf("%d", a);    }

a. 1
b. 2
c. 3
d. 4 ✓
```

| Iteration | Value of i | Action inside loop | Value of a | Condition (i == 3) | Result         |
|-----------|------------|--------------------|------------|--------------------|----------------|
| 1st       | 0          | a++                | 1          | 0 == 3<br>(False)  | Loop continues |
| 2nd       | 1          | a++                | 2          | 1 == 3<br>(False)  | Loop continues |
| 3rd       | 2          | a++                | 3          | 2 == 3<br>(False)  | Loop continues |
| 4th       | 3          | a++                | 4          | 3 == 3<br>(True)   | Break triggers |

102. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int i = 0;
    while (i = 0)          printf("True");
    printf("False");
}
```

- a. True (infinite time)
- b. True (1 time) False
- c. False ✓
- d. Compiler dependent

**The Core Issue: = vs ==**

In C programming, these two symbols perform very different tasks:

- **== (Comparison):** Checks if two values are equal (returns 1 for true, 0 for false).

- **= (Assignment):** Sets the variable on the left to the value on the right.

### Step-by-Step Execution Trace

1. **Initialization:** `int i = 0;` is declared.
2. **The Loop Condition:** The program evaluates the expression inside `while` (`i = 0`).
  - Instead of checking if `i` is equal to 0, the program **assigns** the value 0 to `i`.
  - In C, the result of an assignment expression is the value being assigned. Therefore, the expression (`i = 0`) evaluates to **0**.
3. **Boolean Evaluation:** In C, any non-zero value is "True," and **0 is "False."**
  - Since the condition evaluates to 0, the `while` loop treats this as **False** immediately.
4. **Skipping the Loop:** Because the condition is false, the code inside the loop (`printf("True");`) is skipped entirely.
5. **Final Print:** The program moves to the next line of code: `printf("False");`.

**Final Output:** False

| Code Snippet                | Logic                                                             | Result                                                      |
|-----------------------------|-------------------------------------------------------------------|-------------------------------------------------------------|
| <code>while (i == 0)</code> | Checks if <code>i</code> is 0. Since it is, the loop starts.      | <b>Infinite "True"</b> (since <code>i</code> never changes) |
| <code>while (i = 5)</code>  | Assigns 5 to <code>i</code> . Since 5 is non-zero, it is "True."  | <b>Infinite "True"</b>                                      |
| <code>while (i = 0)</code>  | Assigns 0 to <code>i</code> . Since 0 is "False," the loop fails. | <b>"False"</b>                                              |

103. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int x = 2, y = 0, l;
    int z;
```

```

    z = y = 1, l = x && y;
    printf("%d", l);
    return 0;
}

```

- a. 0
- b. 1 ✓
- c. Undefined behaviour due to order of evaluation can be different
- d. Compilation error

**The Expression: `z = y = 1, l = x && y;`**

This line contains two main parts separated by a **comma operator (,)**. The comma operator has the lowest precedence in C, meaning it acts as a separator that ensures expressions are evaluated from **left to right**.

**Step A: Left side of the comma (`z = y = 1`)**

- The assignment operator (=) works from right to left.
- First, 1 is assigned to y. Now, **y = 1**.
- Then, the value of y (which is 1) is assigned to z. Now, **z = 1**.

**Step B: Right side of the comma (`l = x && y`)**

- Now the program evaluates the logical AND operation: `x && y`.
- In C, any non-zero value is treated as **True**.
- x is **2** (True) and y is now **1** (True).
- `True && True` results in **1** (the integer representation of "True" in C).
- This result (**1**) is assigned to l.

**104. Functions in C are always \_\_\_\_\_**

- a. Internal
- b. External ✓
- c. Both Internal and External
- d. External and Internal are not valid terms for functions

**Functions are External:**



- **Global Visibility:** By default, any function you define in one file can be called from another file as long as the other file has the function's declaration (prototype).
- **No Nesting:** Unlike variables, which can be **internal** (local to a block or function), functions cannot be confined inside a local scope.
- **Linkage:** Functions have "external linkage" by default. This means the linker can see the function name and connect it to calls made from different translation units (different .c files).

### What about "Internal" Functions?

While the question states functions are "always external," C does have a way to limit a function's scope to a single file using the `static` keyword:

```
static void myPrivateFunction() {
    // This function has "internal linkage"
    // It can only be seen and used within this
    specific file.
}
```

---

105. What will be the output of the following C code?

```
#include <stdio.h>

register int x;

void main()
{
    printf("%d", x);
}
```

- a. Varies
- b. 0
- c. Junk value
- d. Compile time error ✓

| Storage Class | Scope | Lifetime       | Initial Value |
|---------------|-------|----------------|---------------|
| Auto          | Local | Function Block | Junk          |

| Storage Class | Scope        | Lifetime       | Initial Value |
|---------------|--------------|----------------|---------------|
| Register      | Local Only   | Function Block | Junk          |
| Static        | Local/Global | Program End    | 0             |
| Extern        | Global       | Program End    | 0             |

---

106. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int i = 0;
    char c = 'a';
    while (i < 2)
    {
        i++;
        switch (c)
        {
            case 'a':printf("%c", c);
                    break;
                    break;
        }
    }
    printf("after loop");
}
```

a. a after loop

- b. a a after loop ✓
- c. compilation error
- d. after loop

## 1. Variable Initialization

- `i` is set to 0.
- `c` is set to `'a'`.

## 2. The Loop Execution

The condition `while (i < 2)` will allow the loop to run twice (when `i` is 0 and 1).

- **First Iteration (`i = 0`):**
  - `i++` increments `i` to 1.
  - The `switch (c)` looks at `'a'`.
  - It matches case `'a'`, executes `printf("%c", c)`, and prints `"a"`.
  - **The `break`; exits the switch block, but *not* the loop. (The second `break`; is "dead code" and is never reached, but it does not cause an error).**
- **Second Iteration (`i = 1`):**
  - The condition `1 < 2` is true.
  - `i++` increments `i` to 2.
  - The `switch (c)` matches case `'a'` again and prints the second `"a"`.
  - The `break`; exits the switch block.

## 3. Loop Termination

- The condition `while (i < 2)` is checked again. Since `i` is now **2**, the condition is **false**, and the loop terminates.

## 4. Final Print

- The statement `printf("after loop");` executes, printing **"after loop"**

107. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
```

```

        int a = 1;
        switch (a)
        {
            case a:printf("Case A ");
            default:printf("Default");
        }
    }

```

- a. Case A
- b. Default
- c. Case A Default
- d. Compile time error ✓

### The Rule of Constant Expressions

In C, the labels in a `switch` statement (the parts after the word `case`) must be **constant expressions**.

- **What is a constant?** Something like 1, 5, or 'A'. These are fixed values known at the moment the code is compiled.
- **What is a variable?** Even though you initialized `int a = 1;`, the compiler still treats `a` as a **variable**. Its value could theoretically change during program execution, so the compiler does not allow it to be used as a `case` label.

### Why your code failed

In your snippet:

```

int a = 1;
switch (a) {
    case a: // Error occurs here!
        printf("Case A ");
        ...
}

```

The compiler sees `case a:` and says: *"Wait, a is a variable, not a fixed constant. I can't build the jump table for this switch."*

---

108. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    register int x = 5;
    m();
    printf("x is %d", x);
}
```

```
void m()
{
    x++;
}
```

- a. 6
- b. 5
- c. Junk value
- d. Compile time error ✓

## 1. Scope and Visibility

In C, variables have "scope," which determines where they can be accessed.

- The variable `x` is declared inside `main()`. This makes it a **local variable**.
- Local variables are only visible within the block `{ }` where they are defined.
- The function `m()` is outside of `main()`. It has no idea that a variable named `x` exists inside `main()`.

## 2. The "Undeclared Identifier" Error

When the compiler reaches the line `x++;` inside the function `m()`, it looks for a definition of `x`.

- It checks if `x` is a global variable (it isn't).
- It checks if `x` is defined within `m()` (it isn't).
- Because it can't find `x`, the compiler throws an error: `"error: 'x' undeclared (first use in this function)"`.

### 3. The `register` Keyword

While the `register` keyword is used here, it is actually a "red herring" (a distraction).

- The `register` keyword suggests to the compiler that the variable should be stored in a CPU register for faster access.
  - However, even if `x` were a standard `int`, the code would still fail for the same reason: **`m()` cannot see variables local to `main()`.**
- 

109. Consider the following C function

```
void swap (int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```

In order to exchange the values of two variables `x` and `y`.

- a. call `swap (x, y)`
- b. call `swap (&x, &y)`
- c. `swap (x,y)` cannot be used as it does not return any value
- d. `swap (x,y)` cannot be used as the parameters are passed by value ✓

### Call by Value

In C, when you pass variables to a function normally—like `swap (x, y)`—the language uses a mechanism called **Pass by Value** (or Call by Value).

- **What actually happens:** The function does **not** get the original variables `x` and `y`. Instead, it creates brand-new, temporary copies of their values (`a` and `b`) inside its own isolated memory space.
- **The result:** The function successfully swaps the values of `a` and `b` inside its own curly braces. But the moment the function finishes executing, those

temporary copies are destroyed. The original variables x and y back in your main program remain completely untouched.

### Pass by Reference

To make a swap function work properly in C, you have to pass the **memory addresses** of the variables using pointers (\*). Here is what the correct code would look like:

```
// The function must accept pointers (addresses)
void swap(int *a, int *b)
{
    int temp;
    temp = *a; // Go to the address 'a' points to, and
               // grab the value
    *a = *b;    // Assign the value at address 'b' to the
               // address 'a'
    *b = temp;  // Assign the temp value to the address
               // 'b'
}

// How you would call it in main:
// correct_swap(&x, &y);
```

---

#### 110. What is the first argument in command line arguments?

- a. The number of command-line arguments the program was invoked with
- b. A pointer to an array of character strings that contain the arguments
- c. Name of the program itself ✓
- d. None of the mentioned

### Understanding main(int argc, char \*argv[])

When you write a C program that accepts command-line inputs, you define your main function like this:

```
C
int main(int argc, char *argv[]) {
    // Code goes here
}
```

To understand why the first argument is the program's name, you have to look at what these two parameters actually track:

- **argc (Argument Count):** An integer that stores the *total number* of arguments passed into the program, including the program name itself.
- **argv (Argument Vector):** An array of strings (character pointers) representing each individual argument.

### The Index Breakdown (**argv[0]**)

In C, arrays are zero-indexed, meaning the very first item is located at index 0.

By standard convention in operating systems (like Linux, macOS, and Windows), when you run a compiled program from the terminal, the system automatically treats the command you typed to run it as the very first argument.

Therefore:

- **argv[0]** always points to the string containing the name of the program (or the path used to invoke it).
- **argv[1]** points to the *first actual user-provided argument*.
- **argv[2]** points to the second user-provided argument, and so on.

### A Concrete Example

Imagine you have compiled a program named `my_program`, and you run it in your terminal like this:

```
Bash
./my_program hello world
```

Here is exactly how the C runtime populates your variables:

- **argc** will equal 3 (because there are three elements total).
- **argv[0]** will be `./my_program` (the name/path of the executable).
- **argv[1]** will be `hello` (the first user argument).
- **argv[2]** will be `world` (the second user argument).

---

111. What will be the output of the following C code?

```
#include <stdio.h>

void main()
{
    int x = 1, y = 0, z = 5;
    int a = x && y || z++;
```



```
        printf("%d", z);  
    }
```

- a. 6 ✓
- b. 5
- c. 0
- d. Varies

## 1. Operator Precedence

In C, the logical AND (&&) operator has higher precedence than the logical OR (||) operator. This means the expression is grouped and evaluated like this:

```
C  
int a = (x && y) || z++;
```

## 2. Evaluating the Left Side: (x && y)

- x is 1 (which is true).
- y is 0 (which is false).
- 1 && 0 evaluates to 0 (false).

## 3. Evaluating the Right Side: || z++

Because the left side of the || operator evaluated to 0 (false), the compiler **must** evaluate the right side to determine the final result of the || operation. This is where **short-circuit evaluation** comes into play (if the left side had been true, it wouldn't have looked at the right side at all).

- The expression now looks at z++.
- z++ is a **post-increment** operation.
- In the expression itself, the current value of z (5) is used to calculate a.
- However, immediately after that line is executed, z is incremented by 1 in memory.

## 4. The Final Value of z

Because z++ was successfully executed, z changes from 5 to 6.

When `printf("%d", z);` runs, it prints the updated value of z, which is 6.

---

## 112. What is the size of an int data type?

- a. 4 Bytes

- b. 8 Bytes
- c. depends on system/compiler ✓
- d. can not be determined

| Environment / Data Model | Size of char | Size of short | Size of int | Size of long | Size of a Pointer |
|--------------------------|--------------|---------------|-------------|--------------|-------------------|
| 16-bit systems           | 1 Byte       | 2 Bytes       | 2 Bytes     | 4 Bytes      | 2 Bytes           |
| 32-bit systems           | 1 Byte       | 2 Bytes       | 4 Bytes     | 4 Bytes      | 4 Bytes           |
| 64-bit Linux / macOS     | 1 Byte       | 2 Bytes       | 4 Bytes     | 8 Bytes      | 8 Bytes           |
| 64-bit Windows           | 1 Byte       | 2 Bytes       | 4 Bytes     | 4 Bytes      | 8 Bytes           |

113. What is the meaning of the following C statement?

```
printf("%10s", state);
```

- a. 10 spaces before the string state is printed
- b. Print empty spaces if the string state is less than 10 characters ✓
- c. Print the last 10 characters of the string
- d. None of the mentioned

When you write `%10s`, you are telling the compiler: *"I want this string to occupy a space of **at least 10 characters** on the screen."*

Here is how the system handles different string lengths:

- **If the string is shorter than 10 characters:** The compiler automatically pads the output with **leading empty spaces** on the left until the total width equals 10.
- **If the string is exactly 10 characters or longer:** The field width is ignored, and the entire string prints normally without cutting anything off.

Option 1 ("*10 spaces before the string state is printed*") is a very common misconception.

If `state` holds the word `"Texas"` (5 characters), `%10s` does **not** print 10 spaces followed by `"Texas"` (which would be 15 characters total). Instead, it calculates:

**Total Width (10) - String Length (5) = 5 Empty Spaces**  
**Visual Example:**

If `state = "Texas"`, the output will look like this (where underscores represent empty spaces):

```
_ _ _ _ _ T e x a s
```

---

**114. While swapping 2 numbers what precautions to be taken care?**

```
b = (b / a);
```

```
a = a * b;
```

```
b = a / b;
```

- a. Data type should be either of short, int and long
- b. Data type should be either of float and double ✓
- c. All data types are accepted except for (char \*)
- d. This code doesn't swap 2 numbers

**Why Must the Data Type Be float or double?**

The restriction comes down to how **integer division** works in C programming.

If you use integer types (**short, int, long**), any fractional part resulting from a division is completely truncated (chopped off), which breaks the logic completely.

The quiz presents a clever snippet designed to swap two variables, `a` and `b`, using multiplication and division instead of a temporary third variable:

```
1. b = (b / a);  
2. a = a * b;  
3. b = a / b;
```

To understand how this works mathematically, let's trace it with two floating-point numbers, say `a = 2.0` and `b = 5.0`:

- **Step 1:** `b = (b / a);`
  - `b = 5.0 / 2.0 = 2.5`
- **Step 2:** `a = a * b;`
  - `a = 2.0 x 2.5 = 5.0` } (Notice that `a` has now successfully taken the original value of `b`)
- **Step 3:** `b = a / b;`
  - `b = 5.0 / 2.5 = 2.0` (Now `b` has successfully taken the original value of `a`)

The values are successfully swapped: a becomes 5.0 and b becomes 2.0.

---

**115. Functions can return enumeration constants in C?**

- a. true ✓
- b. false
- c. depends on the compiler
- d. depends on the standard

In C, an enumeration (`enum`) is essentially a user-defined data type consisting of integer constants. Under the hood, the C compiler treats enumeration constants as **integers** (`int`).

Because a function in C can return an integer, it can seamlessly return an `enum` type or its associated constants.

---

**116. What does the following declaration indicate?**

`int x: 8;`

- a. x stores a value of 8
- b. x is an 8-bit integer ✓
- c. Both A and B
- d. None of the above

**What is a Bit-Field?**

A bit-field is a mechanism inside a C **struct** (structure) or **union** that allows you to specify exactly how many **bits** of memory a variable should occupy. Instead of allocating the standard size for an integer (which is typically 4 bytes or 32 bits on modern systems), the colon (:) followed by a number tells the compiler to limit the size of that variable.

**Breaking Down the Declaration:**

- **int**: Specifies the base data type (determining whether the value is signed or unsigned).
  - **x**: The name of the structure member.
  - **: 8**: This is the bit-field width. It explicitly tells the computer to allocate exactly **8 bits** (1 byte) for x.
-

117. Which of the following are C preprocessors?

- a. `#ifdef`
- b. `#define`
- c. `#endif`
- d. `#for`

**#ifdef** : This is a valid conditional compilation directive. It checks if a macro identifier is currently defined.

**#define** : This is a valid macro substitution directive. It defines macro constants or substitution strings.

**#endif** : This is a valid conditional compilation directive. It closes the block opened by an `#if`, `#ifdef`, or `#ifndef` directive.

---

118. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int x = -2;
    if (!0 == 1)        printf("yes");
    else                printf("no");
}
```

- a. yes ✓
- b. no
- c. run time error
- d. undefined

In C, the logical NOT operator (!) has a higher precedence than the equality operator (==). This means !0 is evaluated first, before comparing anything to 1.

## 2. Evaluating !0

- In C, 0 represents **false**, and any non-zero value represents **true**.
- The logical NOT operator reverses this logic.
- Therefore, !0 (NOT false) evaluates to **true**, which is represented as the integer 1.

## 3. Evaluating the Comparison (1 == 1)

Now, substitute the result back into the condition:

- The expression becomes `1 == 1`.
- Since 1 is indeed equal to 1, this statement is **true** (evaluates to 1).

```
if (!0 == 1)
=> if (1 == 1)    // Because !0 becomes 1
=> if (1)         // Because 1 == 1 is true
```

---

**119. What will be the output of the following C code?**

```
#include<stdio.h>

main()
{
    typedef int a;
    a b=2, c=8, d;
    d=(b*2)/2+8;
    printf("%d",d);
}
```

- a. 10 ✓
- b. 16
- c. 8
- d. Error

### 1. The **typedef** Statement

```
typedef int a;
```

The **typedef** keyword creates an alias for an existing data type. In this line, **a** is defined as a synonym for **int**.

### 2. Variable Declaration

```
a b=2, c=8, d;
```

Since **a** is just another name for **int**, this line is exactly equivalent to writing:

```
int b=2, c=8, d;
```

- **b** is initialized to 2
- **c** is initialized to 8 (though it is never actually used in the calculation)
- **d** is declared to store the final result

### 3. The Expression Evaluation

```
d = (b * 2) / 2 + 8;
```

According to C operator precedence and associativity rules, the expression evaluates like this:

1. **Parentheses first (b \* 2):**

2 x 2 = 4

2. **Division next / 2:**

4 / 2 = 2

3. **Addition last + 8:**

2 + 8 = 10

So, d is assigned the value 10.

#### 4. The Output

```
printf("%d", d);
```

This prints the integer value of d, which prints 10 to the screen.

---

120. String operation such as `strcat(s, t)`, `strcmp(s, t)`, `strcpy(s, t)` and `strlen(s)` heavily rely upon.

- a. Presence of NULL character ✓
- b. Presence of new-line character
- c. Presence of any escape sequence
- d. None of the mentioned

In C programming, a string is not a built-in, distinct data type; instead, it is defined conventionally as a one-dimensional array of characters that terminates with a **NULL character (\0)**. Standard library functions like `strcat()`, `strcmp()`, `strcpy()`, and `strlen()` do not take an explicit string size argument. Instead, they sequentially scan through memory character-by-character until they encounter this specific null terminator (`\0`), which tells the function that the string has ended.

- `strlen(s)`: Counts characters individually and stops tracking precisely when it hits the `\0` marker.
- `strcpy(s, t)`: Copies characters from the source pointer to the destination until it reads and applies the trailing `\0`. [

- `strcat(s, t)`: Finds the end of the first string by searching for its `\0`, overrides it with the start of the second string, and appends a new `\0` at the final termination point.
- `strcmp(s, t)`: Compares characters step-by-step and safely breaks out of the loop if a `\0` is found in either string, indicating a mismatched length or end-of-string scenario.

If a string lacks this terminating null character, these operations will continue scanning past the allocated memory buffer, causing dangerous **buffer overflows** or **undefined behavior**.

---

121. which of the following statements are correct corresponding to the definition of integer?

- a. `signed int i;`
- b. `signed i;`
- c. `long i;`
- d. `long long i;`

- `signed int i;`: This is the most explicit form of declaration. It directly states that the variable `i` is a signed whole integer data type.
  - `signed i;`: In C/C++, when a type qualifier like `signed`, `unsigned`, `short`, or `long` is used by itself without an explicit data type, the compiler **implicitly defaults the underlying type to `int`**. Therefore, `signed i;` is automatically treated exactly like `signed int i;`.
  - `long i;`: Similar to the rule above, `long` is a shorthand specifier that the compiler automatically expands to mean `long int i;`. It defines a larger integer type.
  - `long long i;`: Introduced in the C99 and C++11 standards, this shorthand specifier automatically expands to `long long int i;`, creating a highly extended integer storage variable (typically 8 bytes or 64 bits).
- 

122. the value returned by `f(1)` is

```
int f(int n)
{
    static int i=1;
    if(n>=5)
        return n;
```



```

    n=n+i;
    i++;
    return f(n);
}

```

- a. 5
- b. 6
- c. 7 ✓
- d. 1

#### Call 1: $f(1)$

- $n = 1$
- `static int i = 1;` (Initialized to 1)
- **Check:** Is  $n \geq 5$ ? ( $1 \geq 5$  is False) . Skip the `if` block.
- $n = n + i \rightarrow n = 1 + 1 = 2$
- $i++ \rightarrow i$  becomes 2
- **Return statement:** `return f(2);` (Calls the function again with  $n = 2$ )

#### Call 2: $f(2)$

- $n = 2$
- (The line `static int i = 1;` is skipped because `i` already exists with a value of 2) .
- **Check:** Is  $n \geq 5$ ? ( $2 \geq 5$  is False) . Skip the `if` block.
- $n = n + i \rightarrow n = 2 + 2 = 4$
- $i++ \rightarrow i$  becomes 3
- **Return statement:** `return f(4);` (Calls the function again with  $n = 4$ )

#### Call 3: $f(4)$

- $n = 4$ , and  $i$  is currently 3.
- **Check:** Is  $n \geq 5$ ? ( $4 \geq 5$  is False) . Skip the `if` block.
- $n = n + i \rightarrow n = 4 + 3 = 7$
- $i++ \rightarrow i$  becomes 4
- **Return statement:** `return f(7);` (Calls the function again with  $n = 7$ )

#### Call 4: $f(7)$

- $n = 7$

- **Check:** Is `n >= 5`? (`7 >= 5` is **True**).
  - The code executes `return n;`, which returns 7.
- 

123. `#include <stdio.h>`

```
int main()
{
    float c = 5.0;
    printf ("Temperature in Fahrenheit is %.2f", (9/5)*c + 32);
    return 0;
}
```

- a. Temperature in Fahrenheit is 41.00
- b. Temperature in Fahrenheit is 37.00 ✓
- c. Temperature in Fahrenheit is 0.00
- d. Compiler Error

- Since both 9 and 5 are integers, `9 / 5` evaluates to 1 (not 1.8).
  - **Evaluate the parenthesis:** `9 / 5` becomes 1.
  - **Multiply by c:** `1 * c` → `1 * 5.0` becomes 5.0 (the integer 1 is automatically promoted to a float here).
  - **Add 32:** `5.0 + 32` becomes 37.0.
  - **Print formatting (%.2f) :** It prints with two decimal places, resulting in 37.00.
- 

124. `#include<stdio.h>`

```
int main()
{
    float x = 0.1;
    if ( x == 0.1 )
        printf("IF");
    else if (x == 0.1f)
        printf("ELSE IF");
    else
```

```

        printf("ELSE");
    }

```

- a. IF
- b. ELSE IF ✓
- c. ELSE
- d. Runtime Error

## 1. Floating-Point Binary Representation

Computers store numbers in binary (base 2). While a fraction like 0.1 looks perfectly clean in our normal decimal system (base 10), it **cannot be represented exactly** as a terminating fraction in binary.

Just like  $1/3$  becomes a repeating, infinite decimal in base 10 (0.3333...), 0.1 becomes a repeating, infinite fraction in binary:

(0.000110011001100110011...)₂

Because a computer has limited memory, it has to round this infinite number off at some point. Where it rounds off depends entirely on the data type.

## 2. float vs. double in C

C has two primary types for floating-point numbers:

- **float (Single Precision):** Typically uses 32 bits of memory. It rounds off the infinite binary sequence early, giving about 7 digits of decimal precision.
- **double (Double Precision):** Typically uses 64 bits of memory. It keeps going much further before rounding off, giving about 15 digits of decimal precision.

Because a double has more bits, it is a **much more precise approximation** of 0.1 than a float. Therefore:

Value in float ≠ Value in double

### Step A: The Variable Declaration

```
float x = 0.1;
```

By default, any fractional literal in C (like 0.1) is treated as a **double**.

However, because you are assigning it to a `float x`, the compiler forces that highly precise `double` approximation to be truncated and rounded down into a 32-bit `float` approximation.

### Step B: The First `if` Condition

```
if ( x == 0.1 )
```

- `x` is a `float`.
- `0.1` is a literal, meaning C treats it as a `double`.
- When comparing a `float` and a `double`, C automatically promotes the `float` to a `double` by padding it with zeros.

Because the original `float` version already threw away precision when it was rounded to 32 bits, padding it with zeros won't magically bring back the lost precision. It is being compared to a fresh, highly accurate 64-bit `double` representation of `0.1`.

Since a less precise `0.1` does not equal a more precise `0.1`, the condition evaluates to **false**, and the code skips `printf("IF");`.

### Step C: The `else if` Condition

```
else if (x == 0.1f)
```

- The `f` suffix tells the compiler: *"Treat this literal `0.1` explicitly as a 32-bit float, not a double."*
- Now, you are comparing the 32-bit `float x` directly with a 32-bit `float` literal `0.1f`.

Both sides have been rounded off at the exact same binary bit. They are perfectly identical, so this condition evaluates to **true**.

| Expression             | Type Left                                                   | Type Right          | Do they match?                               |
|------------------------|-------------------------------------------------------------|---------------------|----------------------------------------------|
| <code>x == 0.1</code>  | <code>float</code><br>(promoted to<br><code>double</code> ) | <code>double</code> | <b>No</b> (Different precision<br>rounding)  |
| <code>x == 0.1f</code> | <code>float</code>                                          | <code>float</code>  | <b>Yes</b> (Identical precision<br>rounding) |

---

125. What is the return-type of the function `sqrt()`?

- a. `int`
- b. `float`
- c. `double` ✓
- d. depends on the data type of the parameter

### The Standard Prototype in C (`<math.h>`)

In the original C standard (C89/C90), function overloading did not exist. Therefore, every math function needed a single, definitive signature. The library designers chose **`double`** as the default type for floating-point calculations to ensure maximum precision by default.

The standard prototype declared in `<math.h>` is:

```
double sqrt(double x);
```

Because of this:

- No matter what type of number you pass into `sqrt()` (an `int`, a `float`, or a `double`), the compiler automatically promotes or converts that argument into a `double`.
  - The function processes the calculation with 64-bit double precision and **returns a `double` value.**
- 

126. What will be the data type returned for the following C function?

```
#include <stdio.h>

int func()
{
    return (double)(char)5.0;
}
```

- a) `char`
- b) `int` ✓
- c) `double`
- d) multiple type-casting in return is illegal

### 1. Inside-Out Evaluation of the Return Expression

When the compiler looks at `return (double) (char) 5.0;`, it evaluates the expression from right to left (inside out):

1. `5.0` starts as a `double` literal.
2. `(char) 5.0` explicitly type-casts that `double` value into a `char`. The value becomes the integer 5 stored in a 1-byte character type.
3. `(double) (char) 5.0` then takes that `char` and type-casts it *back* into a `double`.

At this exact moment, the value of the expression evaluates to the floating-point number `5.0` with a data type of `double`.

## 2. The Final Step: Function Definition Wins

Even though the expression inside the parentheses evaluates to a `double`, it has to be handed back through the function's door.

Look at how the function is defined:

```
int func()
```

The prefix `int` is a strict contract with the compiler. It means: *"No matter what happens inside this function, whatever is returned must leave as an integer."*

Before the value actually exits the function, an **implicit conversion (coercion)** takes place. The compiler automatically converts the `double` value (`5.0`) to match the function's declared return type (`int`).

Therefore, the `5.0` is truncated into the integer 5, and the final data type returned by `func()` is an `int`.

---

127. What will `fopen` will return, if there is any error while opening a file?

- a. Nothing
- b. EOF
- c. NULL ✓
- d. Depends on compiler

### The `fopen()` Function Signature

The prototype for `fopen()` in the `<stdio.h>` library looks like this:

```
FILE *fopen(const char *filename, const char *mode);
```

Notice that the return type is a **FILE \*** (a pointer to a FILE structure).

- **On Success:** It returns a valid memory address pointing to an object that controls the opened file stream.
- **On Failure:** It cannot return a valid memory address, so it returns **NULL** (a special macro defined in C that represents a pointer pointing to nothing, typically equal to 0).

---

128. Which of the following declaration is not supported by C?

- a. `String str;` ✓
- b. `char *str;`
- c. `float str = 3e2;`
- d. Both `String str;` and `float str = 3e2;`

**String str;** This declaration is **not supported** because C does not have a built-in object or primitive data type named `String`. It is an invalid statement that will result in a compile-time error (though it is valid in other languages like Java).

**float str = 3e2;** This is also a **valid** declaration and initialization in C. It creates a floating-point variable and assigns it a value using **scientific notation** ( $3 \times 10^2$ ), which equals `300.0`)

---

129. Which of the following is true about structs in C?

- a. no data hiding
- b. functions are allowed in structure
- c. constructor are not allowed in the structure
- d. can not have static members

**1. no data hiding:** C structures do not support access modifiers like `private` or `protected`. All members are public by default and can be accessed by any part of the program that has the structure definition.

**3. constructor are not allowed in the structure:** C is not an object-oriented language and does not have language-level constructors or destructors. Memory for structures must be initialized manually or through helper functions.

**4. can not have static members:** You cannot declare a member as `static` inside a C structure. C requires structure elements to be placed contiguously in memory, which is incompatible with how `static` variables are stored.

---

130. The output will be

```
#include <stdio.h>

int main()
{
    int i = 3;
    printf("%d", (++i)++);
    return 0;
}
```

a. 4

b. 5

c. compile time error

d. l value required

**Step A:** The inner expression `++i` executes. It increments the value of `i` to 4.

**Step B:** In the C language standard, the result of a prefix increment operator `++i` is a **value (an r-value)**, not the variable/memory location itself. It simply returns the temporary value 4.

**Step C:** The outer postfix operator then tries to evaluate. The expression effectively simplifies to: `(4)++;`

### Why the Compiler Fails

Because 4 is a literal constant (an r-value) and not a variable with a modifiable memory address, you cannot increment it. The compiler throws an error because it has nowhere to store the newly incremented value.

### The Error Message

Depending on your compiler, you will see an error message similar to this:

**error:** lvalue required as left operand of assignment

*or*

**error:** lvalue required as increment operand

**131. Comment on the behaviour of the following C code?**

```
#include <stdio.h>
```



```
int main()
{
    int i = 2;
    i = i++ + i;
    printf("%d", i);
}
```

- a. = operator is not a sequence point ✓
- b. ++ operator may return value with or without side effects
- c. it can be evaluated as (i++)+i or i+(++i)
- d. = operator is a sequence point

The code `i = i++ + i;` invokes **undefined behavior** in C because it modifies the variable `i` and references it again in the same expression without an intervening sequence point.

- **Sequence Points:** In C, a sequence point is a specific moment in time during a program's execution where all previous side effects (like variable increments) are guaranteed to be complete, and no subsequent side effects have happened yet.
  - **The Assignment Operator (=):** The assignment operator does **not** act as a sequence point. Because there is no sequence point between the post-increment `i++` and the assignment to `i`, the compiler is free to order the operations in multiple ways. This leads to unpredictable and compiler-dependent results.
- 

132. What will be the output of the following C code?

```
#include <stdio.h>

void f();

int main()
{
    #define max 10

    f();

    return 0;
}
```

```
void f()
{
    printf("%d", max * 10);
}
```

- a. 100 ✓
- b. Compile time error since #define cannot be inside functions
- c. Compile time error since max is not visible in f()
- d. Undefined behaviour

### 1. #define is Scope-Agnostic (Global to the Preprocessor)

In C, compiler keywords like `int`, `void`, or `char` respect block scopes (like inside `{ }`). However, `#define` is a **preprocessor directive**, not a compiler statement.

- The preprocessor runs *before* the actual compilation starts.
- The C language allows you to place **preprocessor directives anywhere inside a source file, including inside the body of a function.**
- **It does not understand functions or local scopes.**
- Once the preprocessor sees `#define max 10`, it blindly substitutes the word `max` with `10` everywhere in the file *below* that line, regardless of whether it was written inside `main()` or outside.

### 2. The Order of Execution vs. Order of Text

Even though `max` is defined inside `main()`, what matters to the preprocessor is the **top-to-bottom text order** of the file.

Because `void f()` is written *below* `main()`, the preprocessor has already read and processed `#define max 10`. When it reaches `f()`, it successfully replaces `max` with `10`.

```
#include <stdio.h>
void f();
int main()
{
    // The #define line is stripped out by the
    preprocessor
    f();
    return 0;
}
```

```
void f()
{
    printf("%d", 10 * 10); // 'max' was replaced with
    '10'
}
```

Since  $10 * 10$  is mathematically 100, `printf` prints **100**.

---

**133. What will be the output of the following C code?**

```
#include <stdio.h>
```

```
int main()
```

```
{
    int i = 97, *p = &i;
    foo(&i);
    printf("%d ", *p);
}
```

```
void foo(int *p)
```

```
{
    int j = 2;
    p = &j;
    printf("%d ", *p);
}
```

- a. 2 2
- b. 2 97 ✓
- c. 97 97
- d. compile time error

**In `main()`:**

```
int i = 97, *p = &i;
```

- `i = 97`, and `p` points to `i`

**`foo(&i)` is called — passing the *address* of `i`:**

```
void foo(int *p)    // local copy of the pointer
{
    int j = 2;

    p = &j;          // local p now points to j
    printf("%d ", *p); // prints 2
}
```

- `foo` receives a **copy** of the pointer (**pass by value**)
- Inside `foo`, `p = &j` only changes the **local copy** of `p`
- `*p` is now `j = 2`, so it prints **2**

**Back in `main()` :**

```
printf("%d ", *p); // p still points to i
```

- The `p` in `main` was **never affected** — it still points to `i`
- `*p = 97`, so it prints **97**

**Key Concept: Pass by Value**



Even though `foo` receives a pointer, the **pointer itself is passed by value**.  
 Reassigning `p` inside `foo` has **no effect** on `main`'s `p`.

To actually change `main`'s pointer from inside a function, you'd need a **pointer to a pointer** (`int **p`).

**Output: 2 97**

**134. What will be the output of the following C code?**

```
#include <stdio.h>

int main()
{
```

```

    int i = 97;
    foo(&i);
    printf("%d ", i);
}
void foo(int *p)
{
    int j = 2;
    *p = j;
    printf("%d ", *p);
}

```

- a) 2 2 ✓
- b) 2 97
- c) 97 97
- d) compile time error

**In main() :**

```

int i = 97;
foo(&i);           // address of i passed to foo

```

**Inside foo() :**

```

void foo(int *p)    // p points to i in main
{
    int j = 2;
    *p = j;         // writes 2 into i's memory location
    printf("%d ", *p); // *p == i == 2, prints 2
}

```

- p still points to i
- \*p = j **dereferences** p and overwrites i's value with 2
- i in main is now **permanently changed to 2**

**Back in main() :**

```
printf("%d ", i);    // i was modified by foo!
```

- i is now 2, so prints 2

### Memory Picture

Before foo:                      After \*p = j:

p → [i = 97]              p → [i = 2] ← value overwritten!

vs

previous version:

Before p = &j:                      After p = &j:

p → [i = 97]                      p → [j = 2] ← i untouched!

### The Golden Rule

| Operation         | What it does                            | Affects original?                                                                     |
|-------------------|-----------------------------------------|---------------------------------------------------------------------------------------|
| <b>p = &amp;j</b> | Changes <b>where p points</b>           |  No  |
| <b>*p = j</b>     | Changes <b>the value at p's address</b> |  Yes |

Output: 2 2

---

135. What will be the output of the following C code?

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i = 97, *p = &i;
```

```
    foo(&p);
```

```
    printf("%d ", *p);
```

```
}
```

```
void foo(int **p)
```

```
{
```

```
    int j = 2;
```

```

    *p = &j;
    printf("%d ", **p);
}

```

- a) 2 2 ✓
- b) 2 97
- c) 97 97
- d) compile time error

This code correctly uses a **pointer-to-pointer** (**int \*\***) to modify the original pointer in main.

**In main() :**

```

int i = 97, *p = &i;
foo(&p);          // passing address of pointer p → int**

```

- i = 97
- p points to i
- &p is the address of the pointer itself (int \*\*)

**Inside foo() :**

```

void foo(int **p) // p is int**, points to main's pointer
{
    int j = 2;
    *p = &j; // dereferences int** → changes main's
    pointer to point to j
    printf("%d ", **p); // **p = *(&j) = 2 → prints 2
}

```

- \*p = &j → **dereferences** the double pointer and makes **main's p** point to j
- \*\*p → dereference twice → gets value of j = 2
- Prints 2

**Back in main() :**

```
printf("%d ", *p); // main's p now points to j (modified by foo)
```

- foo changed main's p to point to j (value = 2)
- So \*p = 2, prints 2

### Memory Picture — Step by Step

**Before foo() :**

main's p  [i = 97]

&p (int\*\*) passed to foo

**Inside foo() after \*p = &j:**

main's p  [j = 2] ← redirected!

[i = 97] ← now orphaned, no longer pointed to

**Back in main() :**

\*p = 2

### Why Not 2 97?

Unlike earlier versions where p in main was **unchanged**, here foo receives int\*\* and uses \*p = &j to **actually redirect main's pointer** to j = 2.

So both prints see 2.

**Output: 2 2**

---

**136. What will be the output of the following C code?**

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i = 97, *p = &i;
```

```
    foo(&p);
```



```

        printf("%d ", i);
    }
    void foo(int **p)
    {
        int j = 2;
        *p = &j;
        printf("%d ", **p);
    }

```

a) 2 2

b) 2 97 ✓

c) 97 97

d) compile time error

**In main() :**

```
int i = 97, *p = &i;
```

```
foo(&p);
```

- i = 97
- p points to i
- &p (int\*\*) is passed to foo

**Inside foo() :**

```
void foo(int **p)
```

```
{
```

```
    int j = 2;
```

```
    *p = &j;          // main's p now points to j, NOT i
```

```
    printf("%d ", **p); // **p = j = 2 → prints 2
```

```
}
```

- \*p = &j redirects **main's pointer p** to point to j
- But i itself is **never touched** — its value stays 97
- Prints 2

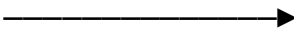
**Back in main() :**

```
printf("%d ", i);    // i was never modified!
```

- foo only changed **where p points**, not the value of i
- i is still 97 → prints **97**

### Memory Picture

Before foo:

p  [i = 97]

After \*p = &j inside foo:

p  [j = 2]

[i = 97] ← untouched, still 97

\*p = &j changes **where p points**. It has **no effect on i's value** – i remains 97.

**Output: 2 97**

---

**137. What will be the output of the following C code?**

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i = 97, *p = &i;
```

```
    foo(&p);
```

```
    printf("%d ", i);
```

```
}
```

```
void foo(int **p)
```

```
{
```

```
    int j = 2;
```

```
    **p = j;
```

```
    printf("%d ", **p);
```

}

- a) 2 2 ✓
- b) 2 97
- c) 97 97
- d) compile time error

**In main() :**

```
int i = 97, *p = &i;
```

```
foo(&p); // passes int** to foo
```

- i = 97
- p points to i
- &p is int\*\*, passed to foo

**Inside foo() :**

```
void foo(int **p)
```

```
{
```

```
    int j = 2;
```

```
    **p = j; // dereference twice → writes 2 into i  
directly!
```

```
    printf("%d ", **p); // **p = i = 2 → prints 2
```

```
}
```

Dereferencing chain:

p → holds address of main's p (int\*\*)

\*p → holds address of i (int\*)

\*\*p → holds value of i = 97 → now = 2 (int)

- \*\*p = j writes 2 directly into i's memory
- i in main is now **permanently changed to 2**
- Prints 2

**Back in main() :**

```
printf("%d ", i);    // i was modified by foo!
```

- i is now 2 → prints 2

## Memory Picture

Before foo:

&p → p → [i = 97]

After \*\*p = j:

&p → p → [i = 2] ← value overwritten!

**Output: 2 2**

## The Complete int\*\* Operations Summary

```
void foo(int **p)
```

```
{
```

```
    int j = 2;
```

```
    p = &j;    // ❌ Compile error: int* assigned to  
int**
```

```
    *p = &j;    // ✅ Redirects main's pointer – i  
untouched
```

```
    *p = j;    // ❌ Compile error: int assigned to int*
```

```
    **p = j;    // ✅ Writes into i directly – i changed ←  
THIS VERSION
```

```
}
```

## Full Version Tracker

| Version | foo (address) | foo (parameter) | Operation<br>in foo | printf<br>in main | Output |
|---------|---------------|-----------------|---------------------|-------------------|--------|
| v1      | &p            | int**           | *p = &j             | *p                | 2 2    |
| v2      | &p            | int**           | *p = &j             | i                 | 2 97   |

|           |    |       |         |    |             |
|-----------|----|-------|---------|----|-------------|
| <b>v4</b> | &p | int** | **p = j | i  | <b>2 2</b>  |
| <b>v3</b> | &p | int*  | *p = &j | *p | <b>2 97</b> |
| <b>v5</b> | &i | int*  | *p = j  | i  | <b>2 2</b>  |

---

138. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    char *str = "hello, world";
    char *str1 = "hello, world";
    if (strcmp(str, str1))
        printf("equal");
    else
        printf("unequal");
}
```

- a. unequal ✓
- b. equal
- c. compile time error
- d. depends on compiler

### 1. How `strcmp()` Works

The `strcmp(string1, string2)` function compares two strings character by character:

- If the strings are **identical**, it returns **0**.
- If the strings are different, it returns a non-zero value (either a positive or negative integer depending on which character has a higher ASCII value).

In this code, both `str` and `str1` point to the exact same literal text: "hello, world". Because they are perfectly identical, **`strcmp(str, str1)` returns 0**.

### 2. The `if` Statement Evaluation

In C, conditional statements treat integers as boolean values:

- **0** is treated as **False**.
- **Any non-zero value** is treated as **True**.

Substitute the result of your function back into the condition:

```
if (strcmp(str, str1)) // Becomes: if (0)
```

Because the condition evaluates to 0 (False), the code bypasses the `if` block entirely and jumps straight to the `else` block.

### 3. The Execution Flow

```
if (0) {
    printf("equal");      // Skipped because 0 is False
} else {
    printf("unequal");    // Executed!
}
```

### Final Output

**unequal**

---

### 139. What will be the output of the following C code?

```
#include <stdio.h>

enum colors{RED, BROWN, ORANGE};

void main()
{
    printf("%ld..%f..%d", RED, BROWN, ORANGE);
}
```

- a) Compiler Error
- b) 0..0.000000..1 ✓
- c) 0..1.000000..2
- d) Runtime Error

- **Enumeration Constants:** By default, C enumerations assign sequential integer values starting from 0. Therefore, `RED = 0`, `BROWN = 1`, and `ORANGE = 2`.
- **Format Mismatch and Undefined Behavior:** The code passes three integer variables (`int`) to a `printf` function but uses mismatched format specifiers.

- `%ld` Expects a long int. It successfully reads the value of `RED(0)`, printing `0`
  - `%f` explicitly expects a floating-point number (`double`) but receives `BROWN(1)`, an integer. In standard calling conventions (like `x86/x64`), integer arguments and floating-point arguments are passed via entirely different hardware registers. Because no floating-point value was explicitly passed, `%f` pulls an uninitialized or zeroed state from the floating-point register, outputting `0.000000`.
  - `%f`: Expects a `double` (which takes 8 bytes of data). Instead, it tries to read the integer value of `BROWN(1)`. Because a `double` uses a completely different binary representation (`IEEE 754` standard) and size than an integer, interpreting the integer bits as a floating-point number results in `0.000000`.
  - `%d` then expects the next integer. Since the previous `%f` did not consume any integer registers, `%d` successfully grabs the next available integer register containing `BROWN` (which is `1`), outputting `1`.
  - `%d`: Expects an integer. Due to the shift in how bytes were consumed by the previous `%f` specifier, it reads the remaining available data, which happens to evaluate to `1` (the value originally passed for `BROWN`)
  - This specific outcome depends heavily on target architecture calling conventions, which results in `0..0.000000..1` in standard test environments like GCC on `x86` platforms.
- 

140. Comment on the following 2 arrays with respect to P and Q.

```
int *a1[8];
```

```
int *(a2[8]);
```

P. Array of pointers

Q. Pointer to an array

- a. `a1` is P and `a2` is Q
- b. `a1` is P and `a2` is P ✓
- c. `a1` is Q and `a2` is Q
- d. `a1` is Q and `a2` is P
- `int *a1[8];`: According to operator precedence rules in C, the array subscript operator `[]` has higher precedence than the pointer dereference operator `*`. Therefore, `a1` binds to `[8]` first, making it an

array of 8 elements. The `int *` tells us that each element is a pointer to an integer.

- `int *(a2[8]);` The parentheses around `a2[8]` do not change the natural precedence. They simply reinforce that `a2` is an array of 8 elements first, and the `*` outside applies to those elements. It is functionally identical to the first declaration.

To declare a **Pointer to an array** (Option Q), we must explicitly use parentheses to force the `*` operator to bind to the variable name before the `[]` operator:

```
int (*a3)[8]; // a3 is a pointer to an array of 8
integers
```

---

```
141. #include <stdio.h>
```

```
    int main()
    {
        int var; /*Suppose address of var is 2000 */
        void *ptr = &var;
        *ptr = 5;
        printf("var=%d and *ptr=%d",var,*ptr);
        return 0;
    }
```

what will be the output?

- a. It will print `var=5` and `*ptr=2000`
- b. It will print `var=5` and `*ptr=5`
- c. It will print `var=5` and `*ptr=random number`
- d. compilation error ✓

A `void *` is a generic pointer that holds a memory address, but it has **no associated data type**. Because the compiler does not know how many bytes of memory a `void` data type occupies (unlike an `int` which is 4 bytes, or a `char` which is 1 byte), it cannot safely read from or write to that memory.

The compiler will generate errors on both lines where `*ptr` is used:

- `*ptr = 5;` **Error: "invalid use of void expression" or "dereferencing void \* pointer".**
- `printf(..., *ptr);` **Error: You cannot pass a dereferenced void expression to a function.**



To fix this, you must **typecast** the `void` pointer into an integer pointer (`int *`) before dereferencing it, so the compiler knows it is dealing with an integer:

```
#include <stdio.h>
int main()
{
    int var;
    void *ptr = &var;

    *(int *)ptr = 5; // Cast ptr to (int *) before
dereferencing

    printf("var=%d and *ptr=%d", var, *(int *)ptr);

    //Output: var=5 and *ptr=5

    return 0;
}
```

---

142. What will be the output of the following C code?

```
#include <stdio.h>
int main()
{
    printf("before continue ");
    continue;
    printf("after continue");
}
```

- a. Before continue after continue
- b. Before continue
- c. After continue
- d. Compile time error ✓

### The Core Issue: Misplaced `continue`

In C, the `continue` statement has a very specific and strict purpose: **it can only be used inside the body of a loop** (such as a `for`, `while`, or `do-while` loop).

When a loop encounters a `continue` statement, it immediately skips the rest of the code inside the current iteration and jumps straight to the next loop evaluation.

```
#include <stdio.h>

int main()
{
    printf("before continue ");
    continue; // <-- ERROR!
    printf("after continue");
}
```

Because there is no loop wrapped around it, the compiler doesn't know what it is supposed to "continue" into. As a result, the compiler throws an error that usually looks something like this:

**error: continue statement not within a loop**

---

**143. Which of the following is not a pointer declaration?**

- a. `char a[10];`
- b. `char a[] = {1, 2, 3, 4};`
- c. `char *str;`
- d. `char a; ✓`

- **`char *str;` (Explicit Pointer):** This is the most standard, explicit way to declare a pointer. The asterisk (\*) explicitly tells the compiler that `str` is a pointer variable meant to store the memory address of a `char` data type.

- **1 & 2. `char a[10];` and `char a[] = {1, 2, 3, 4};` (Implicit Pointers / Arrays):** In C and C++, arrays and pointers are highly interconnected.

- When you declare an array like `char a[10];` or `char a[]`, the identifier `a` acts as a **constant pointer** to the very first element of that array (`&a[0]`).
  - Because the array name decays into a pointer when passed to functions or used in expressions, these declarations inherently involve pointer mechanics under the hood.
- 

**144. what is the output of the following program ?**

```
#include<stdio.h>

void main()
{
```

```

int *j;
{
    int i = 1000;
    j = &i;
}
printf("%d", *j);
}

```

- a. 1000 ✓
- b. 0
- c. Garbage value
- d. Compiler error

---

145. An unrestricted use of goto statement is harmful because\_\_\_\_\_

- a. It is difficult to verify program ✓
- b. Memory require Is increased
- c. It increases execution time of the program.
- d. Compiler generates longer machine code.

An **unrestricted goto statement** allows a program to jump to any arbitrary location in the code. This creates complex, tangled execution paths known as "spaghetti code." It makes it incredibly difficult for developers to track the flow of logic, mathematically verify correctness, debug errors, or maintain the software.

---

146. What is the output of the following program ?

```

#include <stdio.h>

void main()
{
    int z=50;
    printf("%d", z++++++z);
}

```

- a) 53
- b) Compiler error
- c) 52
- d) lvalue required

During the lexical analysis phase, a C compiler reads characters from left to right and groups them into the longest possible valid tokens. This is known as the **Maximal Munch Rule** (or Greedy Parsing).

The expression `z++++++z` is parsed token-by-token by the compiler like this:

1. It sees `z` → Variable token.
2. It sees `++` → Postfix increment operator (leaving `++++z`).
3. It sees `++` → Another increment operator (leaving `++z`).
4. It sees `++` → Another increment operator (leaving `z`).
5. It sees `z` → Variable token.

Therefore, the compiler structurally groups the expression as:

`((z++)++) ++z`

#### The Root Cause of the Error

- `z++` is a postfix operation. It increments the variable `z` but evaluates to the *original value* of `z` (an **rvalue**, or temporary literal value like 50).
- The outer `++` operator then immediately attempts to increment that result: `(50)++`.
- In C, the increment/decrement operators (`++` or `--`) can only modify an **lvalue** (a persistent memory location, like a variable). They cannot be applied directly to a temporary value or literal constant.

Because the compiler detects an attempt to perform an increment operation on a temporary expression instead of a variable, it halts compilation and throws an **"lvalue required as increment operand"** error.

---

147. What will be the output of the following C code?

```
#include <stdio.h>

int main()
{
    int a = 4, n, i, result = 0;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        result += a;
}
```

- a. Addition of a and n
- b. Subtraction of a and n
- c. Multiplication of a and n ✓
- d. Division of a and n

- **Initialization:**

- `int a = 4, n, i, result = 0;`
- a is initialized to 4.
- result starts at 0 (the identity element for addition).
- n is the user input, and i is the loop counter.

- **Taking Input:**

- `scanf("%d", &n);`
- The program waits for the user to enter an integer. Let's assume the user enters 3 (so  $n = 3$ ).

- **The Loop Execution:**

- `for (i = 0; i < n; i++)`
- `result += a;`
- The loop runs exactly n times (from  $i = 0$  up to  $i = n - 1$ ). Inside the loop, `result += a` (which means `result = result + a`) executes on every iteration:

| Iteration | i value | Condition ( $i < 3$ ) | Action ( <code>result += 4</code> ) | New result value |
|-----------|---------|-----------------------|-------------------------------------|------------------|
| Start     | —       | —                     | —                                   | 0                |
| 1st       | 0       | $0 < 3$<br>(True)     | $0 + 4$                             | 4                |
| 2nd       | 1       | $1 < 3$<br>(True)     | $4 + 4$                             | 8                |
| 3rd       | 2       | $2 < 3$<br>(True)     | $8 + 4$                             | 12               |
| End       | 3       | $3 < 3$<br>(False)    | Loop terminates                     | 12               |

---

148. What do the following declaration signify?

`char **argv;`

- a. argv is a pointer to pointer.
- b. argv is a pointer to a char pointer.

- c. `argv` is a function pointer.
- d. `argv` is a member of function pointer.

- **Option b is correct (Most Specific):** The declaration `char **argv;` means `argv` stores the memory address of a `char*` (a pointer to a character). This is the standard way to represent an array of strings in C, commonly used for command-line arguments in the `main` function.
- **Option a is also correct (General):** Generically, any variable declared with two asterisks (`**`) is a pointer to a pointer. It holds the address of another pointer variable.

149. Which of the following function is used to find the first occurrence of a given string?

- a. `strchr()`
- b. `strrchr()`
- c. `strstr()` ✓
- d. `strnset()`

**`strstr()` is correct:** This standard C library function finds the first occurrence of a entire **substring** within another string. It returns a pointer to the beginning of the located substring, or `NULL` if it is not found.

standard string handling functions in C are defined in the `<string.h>` header file.

### Searching Functions

| Function               | Description                                               |
|------------------------|-----------------------------------------------------------|
| <code>strchr()</code>  | Finds the first occurrence of a character.                |
| <code>strrchr()</code> | Finds the last occurrence of a character.                 |
| <code>strstr()</code>  | Finds the first occurrence of a substring.                |
| <code>strspn()</code>  | Gets the length of a prefix substring matching a set.     |
| <code>strcspn()</code> | Gets the length of a prefix substring not matching a set. |
| <code>strpbrk()</code> | Finds the first occurrence of any character from a set.   |
| <code>strtok()</code>  | Splits a string into tokens based on delimiters.          |

### Copying & Concatenation Functions

| Function               | Description                                                                 |
|------------------------|-----------------------------------------------------------------------------|
| <code>strcpy()</code>  | Copies a source string to a destination string.                             |
| <code>strncpy()</code> | Copies up to <i>n</i> characters from source to destination.                |
| <code>strcat()</code>  | Appends / concatenate a source string to the end of a destination string.   |
| <code>strncat()</code> | Appends / concatenate up to <i>n</i> characters from source to destination. |

### Comparison & Length Functions

| Function         | Description                                             |
|------------------|---------------------------------------------------------|
| <b>strlen()</b>  | Calculates the exact length of a string (excluding \0). |
| <b>strcmp()</b>  | Compares two strings alphabetically (case-sensitive).   |
| <b>strncmp()</b> | Compares up to <i>n</i> characters of two strings.      |
| <b>strcoll()</b> | Compares two strings using the current locale rules.    |
| <b>strxfrm()</b> | Transforms a string based on the current locale.        |

#### Raw Memory Functions (Also in <string.h>)

| Function         | Description                                             |
|------------------|---------------------------------------------------------|
| <b>memset()</b>  | Fills a block of memory with a specific byte value.     |
| <b>memcpy()</b>  | Copies a block of memory from source to destination.    |
| <b>memmove()</b> | Copies memory block safely even if regions overlap.     |
| <b>memcmp()</b>  | Compares two blocks of memory byte by byte.             |
| <b>memchr()</b>  | Finds the first occurrence of a byte in a memory block. |

#### Error Handling Functions

| Function          | Description                                            |
|-------------------|--------------------------------------------------------|
| <b>strerror()</b> | Returns a pointer to the textual error message string. |

150. The value of *j* at the end of the execution of the following C program.

```
int incr (int i)
{
    static int count = 0;
    count = count + i;
    return (count);
}

main ()
{
    int i,j;
    for (i = 0; i <=4; i++)
        j = incr(i);
}
```

- a. 10 ✓
- b. 4
- c. 6
- d. 7

**The static Keyword**

The crucial element in this program is `static int count = 0;`.

- Unlike standard local variables, a **static local variable** is initialized **only once** (when the program first loads).
- It retains its previous value between successive function calls. It is not destroyed when the function exits.

### Execution Trace

The `main` function runs a `for` loop where `i` goes from 0 up to 4 (0, 1, 2, 3, 4) . Let's trace each iteration:

| Iteration | Value of <code>i</code> | Inside <code>incr(i)</code> ( <code>count = count + i</code> )                    | Value Returned to <code>j</code> |
|-----------|-------------------------|-----------------------------------------------------------------------------------|----------------------------------|
| 1st       | <code>i = 0</code>      | <code>count</code> is initialized to 0.<br><br><code>count = 0 + 0 → 0</code>     | <code>j = 0</code>               |
| 2nd       | <code>i = 1</code>      | <code>count</code> retains its value (0) .<br><br><code>count = 0 + 1 → 1</code>  | <code>j = 1</code>               |
| 3rd       | <code>i = 2</code>      | <code>count</code> retains its value (1) .<br><br><code>count = 1 + 2 → 3</code>  | <code>j = 3</code>               |
| 4th       | <code>i = 3</code>      | <code>count</code> retains its value (3) .<br><br><code>count = 3 + 3 → 6</code>  | <code>j = 6</code>               |
| 5th       | <code>i = 4</code>      | <code>count</code> retains its value (6) .<br><br><code>count = 6 + 4 → 10</code> | <code>j = 10</code>              |

---

151. What does the following fragment of C-program print?

```
char c[] = "Jecacracker";
```



```
char *p = c;
printf("%s", p + p[1] - p[3]) ;
```

- a. Jecacracker
- b. Jeca
- c. crack
- d. cracker ✓

## 1. Understanding the Variables

- `char c[] = "Jecacracker";`

This creates a character array stored in memory. Let's look at how the characters are indexed and positioned:

- `c[0] = 'J'`
- `c[1] = 'e'`
- `c[2] = 'c'`
- `c[3] = 'a'`
- ...and so on.

- `char *p = c;`

This creates a pointer `p` that points to the very first element of the array (`c[0]`). Because `p` points to the start of the string, `p[1]` is equivalent to `c[1]` ('e'), and `p[3]` is equivalent to `c[3]` ('a').

## 2. Breaking Down the Pointer Arithmetic

The expression inside the `printf` statement is:

`p + p[1] - p[3]`

When you perform arithmetic using character values like `p[1]` and `p[3]`, C uses their underlying **ASCII values**:

- `p[1]` corresponds to the character 'e'. The ASCII value of 'e' is **101**.
- `p[3]` corresponds to the character 'a'. The ASCII value of 'a' is **97**.

Now substitute these values back into the expression:

Expression = `p + 101 - 97`

Expression = `p + 4`

### 3. Final Result

The expression simplifies to  $p + 4$ .

In pointer arithmetic, adding 4 to a character pointer shifts its position forward by **4 bytes** (4 characters).

- $p + 0$  points to 'J' (index 0)
- $p + 1$  points to 'e' (index 1)
- $p + 2$  points to 'c' (index 2)
- $p + 3$  points to 'a' (index 3)
- **$p + 4$  points to 'c' (index 4)** → This is where the word "cracker" starts.

Since `printf("%s", ...)` prints the string starting from the memory address it is given until it hits a null terminator (`\0`), it starts printing from index 4 and outputs: **"cracker"**.

---

**152. Consider the following program in C language:**

```
#include <stdio.h>

main()
{
    int i;
    int *pi = &i;
    scanf("%d", pi);
    printf("%d\n", i+5);
}
```

**Which one of the following statements is TRUE?**

- Compilation fails.
- Execution results in a run-time error.
- On execution, the value printed is 5 more than the integer value entered. ✓
- On execution, the value printed is 5 more than the address of variable i.

## 1. Understanding the Pointer Initialization

```
int i;  
int *pi = &i;
```

- An integer variable `i` is declared. At this point, it contains a garbage value because it hasn't been initialized.
- A pointer to an integer, `pi`, is declared and initialized to hold the **memory address** of `i` (`&i`). This means `pi` is now pointing directly to the variable `i`.

## 2. The `scanf` Function Call

```
scanf("%d", pi);
```

- The `scanf` function expects a memory address where it should store the user's input.
- Normally, you see it written as `scanf("%d", &i);` to pass the address of `i`.
- Since `pi` already stores the address of `i` (`pi == &i`), passing `pi` directly is perfectly correct and valid syntax.
- Whatever integer value the user types in will be directly stored inside the variable `i`.

## 3. The `printf` Statement

```
printf("%d\n", i + 5);
```

- This line takes the integer value now stored inside `i`, adds 5 to it, and prints the result.

## Conclusion

Because the code is syntactically correct, it compiles and runs without any issues (ruling out statements 1 and 2). It performs arithmetic on the *value* stored in `i`, not its address (ruling out statement 4).

Therefore, "**On execution, the value printed is 5 more than the integer value entered**" is the true statement.

---

153. Consider the function `func` shown below:

```
int func(int num)  
{
```

```

    int count = 0;
    while (num)
    {
        count++;
        num >>= 1;
    }
    return (count);
}

```

The value returned by `func(435)` is

- a. 9 ✓
- b. 10
- c. 11
- d. 8

## 1. Understanding the Operations

- **`num >>= 1` (Right Shift Operator):** This operator shifts the bits of the binary representation of `num` to the right by 1 position. In integer arithmetic, right-shifting a number by 1 is exactly equivalent to **dividing the number by 2 and discarding the remainder** (integer division).
- **`while (num)` :** In C, any non-zero value is treated as **true**, and 0 is treated as **false**. The loop will continue running and incrementing `count` as long as `num` is greater than 0.

Essentially, this function counts **how many times a number can be divided by 2 before it becomes 0** (which corresponds to the number of bits needed to represent the number in binary).

## 2. Tracing the Execution with `num = 435`

Let's track the values of `num` and `count` through each iteration of the loop:

- **Initial State:** `num = 435`, `count = 0`

| Iteration      | Loop Condition<br>(num) / (num != 0) | count++          | num >>= 1 (Integer<br>Division by 2) |
|----------------|--------------------------------------|------------------|--------------------------------------|
| Start          | 435 (True)                           |                  |                                      |
| Iteration<br>1 | 435 is non-zero                      | count = 1        | 435 / 2 = 217                        |
| Iteration<br>2 | 217 is non-zero                      | count = 2        | 217 / 2 = 108                        |
| Iteration<br>3 | 108 is non-zero                      | count = 3        | 108 / 2 = 54                         |
| Iteration<br>4 | 54 is non-zero                       | count = 4        | 54 / 2 = 27                          |
| Iteration<br>5 | 27 is non-zero                       | count = 5        | 27 / 2 = 13                          |
| Iteration<br>6 | 13 is non-zero                       | count = 6        | 13 / 2 = 6                           |
| Iteration<br>7 | 6 is non-zero                        | count = 7        | 6 / 2 = 3                            |
| Iteration<br>8 | 3 is non-zero                        | count = 8        | 3 / 2 = 1                            |
| Iteration<br>9 | 1 is non-zero                        | <b>count = 9</b> | 1 / 2 = 0                            |

- **End of Loop:** On the next check, num is 0 (False) . The loop terminates.

---

154. Which of the following are correct preprocessor directives in C?

- a. #ifdef
- b. #if
- c. #elif
- d. #undef

- **#ifdef (If Defined):** Used for conditional compilation. It checks if a specific macro has been defined using `#define` earlier in the code. If it has, the compiler processes the following block of code.
  - **#if (If):** Starts a conditional compilation block. It evaluates whether a compile-time constant expression or macro expression is true (non-zero).
  - **#elif (Else If):** Acts exactly like an `else if` block within a conditional compilation chain. It provides an alternative condition if the preceding `#if` or `#elif` checks evaluate to false.
  - **#undef (Undefine):** Used to cancel or remove an existing macro definition. After an `#undef` directive is processed, the targeted macro is no longer recognized by the compiler.
- 

155. Why reference is not same as a pointer?

- a. A reference can never be null.
- b. A reference once established cannot be changed.
- c. Reference doesn't need an explicit dereferencing mechanism.
- d. None of the above

- **A reference can never be null:** Pointers can be explicitly set to `NULL` or `nullptr` to indicate they are not pointing to any valid memory location. A reference, however, is strictly an alias for an existing object and **must** be initialized to a valid variable when declared; it cannot be null in a well-formed program.
  - **A reference once established cannot be changed:** Pointers are distinct variables that store memory addresses; you can reassign a pointer to target a completely different variable at any time. A reference is permanently bound to its initial target upon declaration and cannot be "reseated" to refer to another object.
  - **Reference doesn't need an explicit dereferencing mechanism:** To access or modify the value stored at a pointer's address, you must use the explicit dereference operator (`*ptr`). References are handled by the compiler as direct aliases, meaning you interact with them using standard variable syntax without any extra operators.
- 

156. What will be the output of the following C code?

```
#include <stdio.h>

int main() {
    if (printf("%d", printf(""))) {
        printf("We are Happy");
    } else if (printf("1")) {
        printf("We are Sad");
    }
    return 0;
}
```

- a. 0We are Happy
- b. 1We are Happy ✓
- c. 1We are Sad
- d. compile time error

### Step-by-Step Explanation

To understand why this is the output, we need to look at how `printf()` works in C. The `printf()` function does two things:

1. It **prints** the formatted string to the console.
2. It **returns** the total number of characters it successfully printed.

Let's break down the evaluation of the first `if` statement line by line:

#### Step 1: The Innermost `printf` Executes

The condition starts evaluating from the inside out:

```
printf(" ")
```

- **Action:** It prints the character  `)` to the screen.
- **Return Value:** Since it printed exactly **1** character, this function call returns the integer value **1**.

#### Step 2: The Outermost `printf` Executes

Now substitute that return value back into the outer `printf`:

```
printf("%d", 1)
```

- **Action:** It prints the integer 1 directly to the screen (right next to the ) printed in step 1).
- **Return Value:** It printed exactly 1 character (the digit '1'), so this outer function also returns the integer value 1.

### Step 3: Evaluating the `if` Condition

The condition inside the `if` statement has now completely evaluated to its final return value:

```
if (1)
```

- In C, any non-zero value is treated as **true**. Since 1 is true, the program enters the `if` block.

### Step 4: Inside the `if` Block

Because the condition was true, the code inside the block executes:

```
printf("We are Happy");
```

- **Action:** It prints `We are Happy` to the screen.
- Because the `if` block executed, the `else if` block is entirely skipped.

### Final Console Output

Combining everything sequentially as it hits the console:

1. ) (From the inner `printf`)
2. 1 (From the outer `printf`)
3. `We are Happy` (From inside the `if` block)

**Output:** )1We are Happy

**157. What will be the output of the program?**

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    unsigned int i = 65535; /* Assume 2 byte integer*/
```

```
    while(i++ != 0)
```



```

        printf("%d",++i);
    printf("
");
    return 0;
}

```

- a. Infinite loop ✓
- b. 0 1 2 ... 65535
- c. 0 1 2 ... 32767 - 32766 -32765 -1 0
- d. No output

## 1. The Key Assumptions

- **unsigned int (2 bytes):** A 2-byte unsigned integer has a range of 0 to 65535.
- **Overflow Behavior:** If you add 1 to 65535, it wraps around (overflows) back to 0. If you add 1 to 0, it becomes 1.

### Iteration 1:

- **Initial value:** `i = 65535`.
- **The while condition (`i++ != 0`):** \* Because it is a **post-increment** (`i++`), the expression evaluates the *current* value of `i` first.
  - It checks: Is `65535 != 0`? **Yes (True)**.
  - Right after this check, `i` increments. Because of the 2-byte overflow, `65535 + 1` makes `i = 0`.
- **Inside the loop (`printf("%d", ++i) ;`):**
  - Here, we use a **pre-increment** (`++i`). This means `i` is incremented *before* it is printed.
  - `i` goes from 0 to 1.
  - The program prints: **1**

### Iteration 2:

- **Current value:** `i = 1`.
- **The while condition (`i++ != 0`):**
  - Post-increment check: Is `1 != 0`? **Yes (True)**.
  - `i` increments from 1 to 2.
- **Inside the loop (`printf("%d", ++i) ;`):**
  - Pre-increment: `i` increments from 2 to 3.
  - The program prints: **3**

### Iteration 3:

- **Current value:** `i = 3`.
- **The `while` condition:** Is `3 != 0`? **Yes (True)**. Then `i` becomes 4.
- **Inside the loop:** `i` becomes 5 and prints **5**.

Notice the pattern of the value of `i` at the start of the `while` condition check:

- Iteration 1: `i = 65535` (Odd)
- Iteration 2: `i = 1` (Odd)
- Iteration 3: `i = 3` (Odd)
- Iteration 4: `i = 5` (Odd)

Because `i` increments twice in every cycle (once in the `while` condition and once inside the `printf`), **`i` will only ever hold odd numbers** when it arrives at the `while` condition check.

As the loop continues, `i` will eventually climb up to 65533, then 65535.

When it hits 65535 again:

1. The condition checks `65535 != 0` (True).
2. `i` overflows to 0.
3. The `printf` pre-increments it to 1 and prints it.
4. The loop starts over with `i = 1`.

Because `i` jumps over 0 during the condition check (skipping from 65535 straight to 1), **the condition `i++ != 0` evaluates to `False`** and the loop never terminates. Thus, it is an **infinite loop**.

---

158. What is the output of the following programme?

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
    int *ptr;
    ptr = (int*)calloc(3,sizeof(int));
    ptr[2] = 30;
```

```
printf("%d", *ptr);
}
```

- a) 30
- b) 0 ✓
- c) calloc
- d) Error

- **Memory Allocation:** The function `calloc(3, sizeof(int))` allocates an array of 3 integers in memory.
- **Automatic Initialization:** Unlike `malloc`, `calloc` automatically initializes all allocated memory slots to **zero**. Therefore, the array starts as `[0, 0, 0]`.  
// Allocates 3 integers, ALL initialized to 0  
// Memory: [0]    [0]    [0]

```
      ↑    ↑    ↑
ptr[0] ptr[1] ptr[2]
```

- **Value Assignment:** The line `ptr[2] = 30;` changes the *third* element of the array.

The array becomes `[0,        0,        30]`.

```
      ↑    ↑    ↑
ptr[0] ptr[1] ptr[2]
```

- **Dereferencing:** The expression `*ptr` is identical to `ptr[0]`. It looks at the *first* element of the array, which is still 0.

## 1. Using malloc

You must pass the total calculated byte size. The memory will contain unpredictable random numbers (**garbage**).

```
// Allocates space for 3 integers (e.g., 3 * 4 bytes = 12 bytes)
```

```
// malloc – one argument: total bytes
```

```
int *p = (int*) malloc(3 * sizeof(int));
```

```
// memory: [? ? ?]
```

```
// ptr[0] through ptr[2] contain random garbage values until assigned
```

```
free(ptr); // always required for both
```

```
ptr = NULL; // good practice after freeing
```

## 2. Using calloc

You pass the count and size separately. The memory is guaranteed to be clean and filled with **zeros**.

```
// Allocates space for 3 integers and clears them
// calloc – two arguments: count, size of each
int *p = (int*) calloc(3, sizeof(int));
// memory: [0 0 0]
// ptr[0] through ptr[2] are all automatically 0
free(ptr); // always required for both
ptr = NULL; // good practice after freeing
```

| Feature                  | malloc (Memory Allocation)                                              | calloc (Contiguous Allocation)                                                                            |
|--------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Initialization           | Leaves memory <b>uninitialised</b> (contains garbage values).           | Initializes all allocated memory bits to <b>zero</b> .                                                    |
| Arguments                | Takes <b>1 argument</b> : total number of bytes.<br><br>1 (total bytes) | Takes <b>2 arguments</b> : number of elements and size of each element.<br><br>2 (count + per-item bytes) |
| Speed                    | <b>Faster</b> , because it does not clear the memory.                   | <b>Slower</b> , because it takes time to clear the memory to zero.                                        |
| Syntax                   | ptr = malloc(total_size);                                               | ptr = calloc(num_elements, element_size);                                                                 |
| Memory after allocation: | ptr = malloc(3 * sizeof(int))<br>[?] [?] [?]                            | ptr = calloc(3, sizeof(int))<br>[0] [0] [0]                                                               |

---

159. What is the output of the following programme?

```
#include<stdio.h>
void main()
{
```

```
int **ptr;
int temp = 65;
ptr[0] = &temp;
printf("%d", ptr[0][0]);
}
```

- a. Compile Error
- b. 65
- c. Segmentation fault ✓
- d. Runtime Error

```
int **ptr;
// ptr is declared but NOT initialized
// it holds a garbage/random memory address
ptr[0] = &temp;
// ptr[0] means *(ptr+0) = *ptr
// This tries to WRITE to wherever ptr points
// - which is a random/invalid memory location
// → CRASH (Segmentation fault)
```

**The root cause:** `ptr` is an uninitialized pointer. It was never allocated memory (no `malloc` or assignment like `ptr = &someArray`). Dereferencing it is **undefined behavior**, and on most systems causes a **segmentation fault at runtime**.

```
int *arr[1];      // array of 1 int-pointer
int **ptr = arr; // ptr now points to valid memory
int temp = 65;
ptr[0] = &temp;
printf("%d", ptr[0][0]); // safely prints 65
```

160. What is the advantage of `#define` over `const`?

- a. Data type is flexible ✓
- b. Can have a pointer
- c. Reduction in the size of the program

d. None of the mentioned

| Feature      | #define                               | const                |
|--------------|---------------------------------------|----------------------|
| Type         | No data type — pure text substitution | Strongly typed       |
| Processed by | Preprocessor (before compilation)     | Compiler             |
| Memory       | No memory allocated                   | Occupies memory      |
| Scope        | No scope rules                        | Follows scope rules  |
| Debugging    | Harder to debug                       | Easier to debug      |
| Pointer      | ✗ Cannot have a pointer               | ✓ Can have a pointer |

#define performs **textual/macro substitution** — it has **no specific data type**, making it flexible:

```
#define VALUE 10          // could act as int
#define VALUE 10.5        // could act as float
#define MSG "Hello"       // could act as string
```

The preprocessor simply **replaces the token** — no type checking is enforced, giving it type flexibility.

Whereas **const** **must** declare a type:

```
const int x = 10;         // strictly an int
const float y = 10.5;     // strictly a float
```

---

**161. For a given integer, which of the following operators can be used to “set” and “reset” a particular bit respectively?**

- a. | and &
- b. && and ||
- c. & and |
- d. || and &&

**Setting** a bit means forcing a specific bit to 1, and **resetting** a bit means forcing it to 0.

**Setting a Bit — using | (bitwise OR)**

**OR** with a mask that has 1 at the target position:

```
0101 1010    (original)
| 0000 0100    (mask: (1 << 2), set bit 2)
```

-----

0101 1110     bit 2 is now 1  
`n = n | (1 << bit_position);    // set bit`

### Resetting a Bit — using & (bitwise AND)

**AND** with a mask that has 0 at the target position (i.e., complement of the bit mask):

0101 1110    (original)  
& 1111 1011    (mask: ~(1 << 2), reset bit 2)  
-----  
0101 1010     bit 2 is now 0  
`n = n & ~(1 << bit_position);    // reset bit`

---

**162.** What is the value of the variable 'x' after executing the following code:

```
int a = 10;  
int *p = &a;  
int *q = p;  
int b = 20;  
p = &b;  
*q = 30;  
int x = a + b;
```

- a. 30
- b. 40
- c. 50 ✓
- d. 60

### 1. Variable Initialization

```
int a = 10;
```

- An integer variable a is allocated in memory and initialized with the value **10**.

## 2. Pointer Declarations

```
int *p = &a;  
int *q = p;
```

- `int *p = &a;` creates a pointer `p` that stores the memory address of `a` (i.e., `p` points to `a`).
- `int *q = p;` creates another pointer `q` and copies the address stored in `p` into `q`. Now, **both `p` and `q` point to `a`.**

## 3. Another Variable Setup

```
int b = 20;
```

- An integer variable `b` is allocated in memory and initialized with the value **20**.

## 4. Changing Pointer Assignment

```
p = &b;
```

- The pointer `p` is updated to store the address of `b`.
- **Crucial Note:** This action *only* changes where `p` points. It does **not** affect pointer `q`. Pointer `q` is still pointing directly to `a`.

## 5. Dereferencing and Modification

```
*q = 30;
```

- The `*` operator dereferences `q`, meaning "*the value at the address stored in `q`*".
- Since `q` still points to `a`, this line changes the value of `a` to **30**.

## 6. Final Calculation

```
int x = a + b;
```

- At this stage, let's look at the current values of our variables:
  - `a` was modified via `q`, so its value is now **30**.
  - `b` was never modified, so its value remains **20**.
- Therefore, `x = 30 + 20`, which equals **50**.

---

### 163. Which of the following problem causes an exception?

- a. Missing semicolon in statement in `main()`.
- b. A problem in calling function.
- c. A syntax error.



d. A run-time error. ✓

**A run-time error** occurs while the program is actively executing (running) rather than during compilation. When the system encounters an illegal operation at runtime—such as trying to divide an integer by zero, or accessing an array index that doesn't exist—it disrupts the normal flow and signals this issue by **throwing an exception**.

**Exception** handling mechanisms (like `try-catch` blocks) are specifically designed to trap and handle these runtime anomalies gracefully.

---

**164. Consider the following variable declarations and definitions in C**

**Choose the correct statement w.r.t. above variables.**

- a. `int var_9 = 1;`
- b. `int _ = 3;`
- c. `int 9_var = 2;`
- d. `int 1_ = 3;`

In the C programming language, identifier naming rules dictate that a variable name **must begin with a letter (a-z, A-Z) or an underscore (\_)**. It can then be followed by any combination of letters, underscores, or digits (0-9).

- **1. `int var_9 = 1;`**: Valid. It starts with a letter (v) and contains letters, an underscore, and a digit.
  - **2. `int _ = 3;`**: Valid. A single underscore (`_`) is a completely legal identifier name in C because the rules explicitly state a variable can start with and consist entirely of underscores.
- 

**165. In the context of the following `printf()` in C, pick the best statement.**

- i) `printf("%d",8);`
- ii) `printf("%d",090);`
- iii) `printf("%d",00200);`
- iv) `printf("%d",0007000);`

- a. i) would compile. And it will print 8.
- b. ii) would compile. and will print 72
- c. iii) would compile successfully and will print 128
- d. iv) would compile successfully and will print 3584

**In C:**

|                                           |   |         |           |
|-------------------------------------------|---|---------|-----------|
| Leading 0                                 | → | OCTAL   | (base 8)  |
| Leading 0x                                | → | HEX     | (base 16) |
| No prefix                                 | → | DECIMAL | (base 10) |
| Valid octal digits: 0,1,2,3,4,5,6,7 ONLY  |   |         |           |
| 8 and 9 are INVALID in octal!             |   |         |           |
| Valid decimal digits: 0,1,2,3,4,5,6,7,8,9 |   |         |           |
| more than 9 are INVALID in decimal!       |   |         |           |
| Valid Hexa digits: 0,1,2,3,4,5,6,7,8,9    |   |         |           |
| a-→10,b-→11,c-→12,d-→13,e-→14,f-→15       |   |         |           |
| more than 9 are INVALID in Hexa!          |   |         |           |

i) `printf("%d", 8);` ✓

8 is a decimal literal

%d prints decimal → prints 8 ✓

ii) `printf("%d", 090);`

090 starts with 0 → treated as OCTAL

But valid octal digits are only: 0, 1, 2, 3, 4, 5, 6, 7

digit '9' is INVALID in octal! ✗

**Result: COMPILATION ERROR** ⚡

"invalid digit '9' in octal constant"

iii) `printf("%d", 00200);` ✓

00200 starts with 0 → treated as OCTAL

Digits: 0, 0, 2, 0, 0 → all valid octal digits ✓

Converting 00200 (octal) to decimal:

$$\begin{aligned}
&= 0 \times 8^4 + 0 \times 8^3 + 2 \times 8^2 + 0 \times 8^1 + 0 \times 8^0 \\
&= 0 + 0 + 2 \times 64 + 0 + 0 \\
&= 0 + 0 + 128 + 0 + 0 \\
&= 128
\end{aligned}$$

**iv) `printf("%d", 0007000);`** ✓

0007000 starts with 0 → treated as OCTAL

Digits: 0, 0, 0, 7, 0, 0, 0 → all valid octal digits ✓

Converting 0007000 (octal) to decimal:

|           |   |   |   |   |   |   |   |
|-----------|---|---|---|---|---|---|---|
| Position: | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| Digit:    | 0 | 0 | 0 | 7 | 0 | 0 | 0 |

$$\begin{aligned}
&= 0 \times 8^6 + 0 \times 8^5 + 0 \times 8^4 + 7 \times 8^3 + 0 \times 8^2 + 0 \times 8^1 + 0 \times 8^0 \\
&= 0 + 0 + 0 + 7 \times 512 + 0 + 0 + 0 \\
&= 3584
\end{aligned}$$

**166. Which of the following statements mentioning the name of the array begins DOES NOT yield the base address?**

- a. When array name is used with the sizeof operator.
- b. When array name is operand of the & operator.
- c. When array name is passed to scanf() function.
- d. When array name is passed to printf() function.

- **Statement 1: When used with the sizeof operator**

**What it yields:** It evaluates to the **total size in bytes of the entire array block** (e.g., `sizeof(int) * array_size`), rather than the memory address of the first element.

- **Statement 2: When used as an operand of the & (address-of) operator**

**What it yields:** It yields a **pointer to the entire array** (type `data_type (*) [size]`). While its numerical memory value is identical to the base address, its pointer arithmetic type is entirely different (incrementing `&arr + 1` skips the entire array size rather than moving to the next individual element).

- **Why Statements 3 and 4 Do Yield the Base Address**

- **3. Passed to `scanf()` / 4. Passed to `printf()`:** In both cases, passing the raw array name (e.g., `scanf("%s", arr)` or `printf("%p", arr)`) causes the array name to immediately decay into a standard pointer pointing directly to the **base address** (the very first element, `&arr[0]`)
- 

167. What will be the output of the program ?

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    static char *s[] = {"black", "white", "pink", "violet"};
```

```
    char **ptr[] = {s+3, s+2, s+1, s}, ***p;
```

```
    p = ptr;
```

```
    ++p;
```

```
    printf("%s", **p+1);
```

```
    return 0;
```

```
}
```

- a. ink ✓
- b. ack
- c. ite
- d. let

## 1. The Memory Layout

Let's look at how the variables are initialized in memory:

```
static char *s[]
```

This is an array of character pointers (strings).

- `s[0]` points to "black"
- `s[1]` points to "white"
- `s[2]` points to "pink"
- `s[3]` points to "violet"

```
char ptr[]
```

This is an array of pointers that point to elements of `s`. Notice that the order is reversed:

- `ptr[0] = s + 3` (points to `s[3]`, which is "violet")
- `ptr[1] = s + 2` (points to `s[2]`, which is "pink")
- `ptr[2] = s + 1` (points to `s[1]`, which is "white")
- `ptr[3] = s` (points to `s[0]`, which is "black")

```
char ***p
```

`p` is a triple pointer. The line `p = ptr;` makes `p` point to the first element of the `ptr` array (`ptr[0]`).

## 2. Step-by-Step Execution

### Step 1: `++p;`

Because `p` initially points to `ptr[0]`, incrementing `p` makes it point to the next element, `ptr[1]`.

### Step 2: Evaluating `p + 1` inside `printf`

We need to follow the operator precedence rules here. Dereferencing (`*`) has higher precedence than addition (`+`). So, `p + 1` is evaluated as `(p) + 1`.

1. `p` points to `ptr[1]`.
2. `*p` gets the value stored at `ptr[1]`, which is `s + 2`.
3. `p` dereferences `s + 2`, which gives us the value stored at `s[2]`.
  - `s[2]` is the string pointer pointing to "pink".
4. `p + 1` adds 1 to this string pointer. In C, adding 1 to a character pointer moves it to the next character in the string.
  - `p` points to the start of "pink" ('p').
  - `p + 1` shifts the pointer by 1 character, pointing to "ink"

---

168. What is the output of the C code?

```
# include <stdio.h>

# define scanf "%s WBJECA "

int main()
{
    printf(scanf, scanf);
```

```

    getchar();
    return 0;
}

a. WBJECA
b. WBJECA WBJECA
c. %s WBJECA WBJECA ✓
d. error!

```

## 1. The Preprocessor Step (Text Replacement)

Before the compiler even looks at the logic of your code, the C preprocessor handles `#define` directives. It performs a **blind, literal text replacement** wherever it sees the macro token.

Your macro definition is:

```
#define scanf "%s WBJECA "
```

This means *every* occurrence of the word `scanf` in your code (that isn't inside a literal string) gets replaced with `"%s WBJECA "`.

## 2. The Substitution

Let's look at the `printf` statement inside `main()`:

```
printf(scanf, scanf);
```

After the preprocessor replaces both instances of `scanf`, the line literally becomes:

```
printf("%s WBJECA ", "%s WBJECA ");
```

## 3. How `printf` Executes It

Now, let's look at how `printf(format_string, argument)` works here:

1. **The Format String:** The first argument is `"%s WBJECA "`.
2. **The Format Specifier:** `printf` reads this string from left to right. The first thing it encounters is the `%s` specifier, which tells it: *"Expect a string argument and print it here."*
3. **The Argument:** The second argument provided is `"%s WBJECA "`.

So, `printf` substitutes the `%s` with the text `"%s WBJECA "`, and then continues printing the rest of the original format string (`WBJECA`).

```
printf("%s WBJECA ", "%s WBJECA ");
```

└──────────────────┘

### Step-by-Step Output Generation:

- **Step 1:** Handle the %s --> Prints the argument: %s WBJECA
- **Step 2:** Print the remainder of the format string --> Prints: WBJECA

### Combined Output:

%s WBJECA WBJECA

---

169. What is the output of the C code?

```
#include<stdio.h>

int main()
{
    int n;
    for(n = 7; n!=0; n--)
        printf("n = %d", n--);
    getchar();
    return 0;
}
```

- a. 7 6 5 4 3 2 1
- b. 6 4 2
- c. 7 5 3 2 1
- d. Infinite Loop ✓

### Iteration 1

- **Initialization:** n is set to 7.
- **Condition Check:** Is  $n \neq 0$ ? Yes ( $7 \neq 0$ ), so the loop body executes.
- **Loop Body (printf):** \*printf prints the current value of n (which is 7).

- Right after printing, the **post-decrement** inside `printf` runs: `n` drops from 7 to 6.
- Loop Iteration Step (`n--`)** : \* The for loop's own `n--` executes. `n` drops from 6 to 5.

#### Iteration 2

- Condition Check:** Is `n != 0`? Yes (`5 != 0`)
- Loop Body (`printf`):** \* `printf` prints the current value of `n` (which is 5).
  - Post-decrement** happens: `n` drops from 5 to 4.
- Loop Iteration Step (`n--`)** : \* The loop's `n--` executes. `n` drops from 4 to 3.

#### Iteration 3

- Condition Check:** Is `n != 0`? Yes (`3 != 0`)
- Loop Body (`printf`):** \* `printf` prints 3.
  - Post-decrement** happens: `n` drops from 3 to 2.
- Loop Iteration Step (`n--`)** : \* The loop's `n--` executes. `n` drops from 2 to 1.

#### Iteration 4 (The Critical Turning Point)

- Condition Check:** Is `n != 0`? Yes (`1 != 0`)
- Loop Body (`printf`):** \* `printf` prints 1.
  - Post-decrement** happens: `n` drops from 1 to 0.
- Loop Iteration Step (`n--`)** : \* The loop's `n--` executes. `n` drops from 0 to -1.

In **Iteration 5**, the condition check asks: Is `n != 0`?

Since `n` is now -1, the answer is **Yes** (`-1 != 0`)

Because both decrements skip right past 0 (going from 1 to 0 to -1), `n` will keep becoming more and more negative (-3, -5, -7, etc.). It will



completely miss the termination condition `n == 0`, causing the program to run into an **Infinite Loop**.

---

170. In C language "FILE" is a \_\_\_\_\_ .

- a. Pointer
- b. Structure ✓
- c. Data Type
- d. None of the above

In C, FILE is defined as a **structure** in `<stdio.h>`. It holds all the information needed to control a stream, such as:

```
typedef struct {
    int    _fd;           // file descriptor
    int    _flags;        // read/write/error flags
    char *_buf;           // pointer to buffer
    int    _bufsiz;       // buffer size
    int    _cnt;          // characters remaining in buffer
    // ... (implementation-specific fields)
} FILE;
```

So FILE is a **struct** that has been typedef'd, giving it the appearance of a simple type name.

| Option    | Why incorrect                                                                   |
|-----------|---------------------------------------------------------------------------------|
| Pointer   | FILE * is a pointer <i>to</i> a FILE structure — the pointer is not FILE itself |
| Data Type | It behaves like one due to typedef, but underneath it is a structure            |

#### Common confusion:

We always use `FILE *fp` (a pointer to FILE), which leads many to think FILE itself is a pointer — but the **pointer** is the `*`, not FILE.

```
FILE *fp = fopen("file.txt", "r"); // fp is a pointer to
a FILE structure
```

---

171. What is the output of the C code?

```
#include <stdio.h>
```

```
int main()
```

```

{
    char *str="A%%B";
    printf("A%%B ");
    printf("%s\n",str);
    return 0;
}

```

- a. A%%B A%%B
- b. A%B A%%B
- c. A A%%B
- d. error!

### 1. The String Initialization

```
char *str = "A%%B";
```

Here, `str` is just a regular string pointer pointing to a sequence of characters. In a normal C string literal, the `%` character has no special meaning; it's just a normal character like `'A'` or `'B'`.

- Therefore, the memory actually stores the literal characters: `A`, `%`, `%`, `B`, `\0`.

### 2. The First `printf` Statement

```
printf("A%%B ");
```

When you pass a string directly as the **first argument** of `printf`, it treats it as a **format string**.

In a `printf` format string, the **% sign is a special escape character** used to introduce format specifiers (like `%d` or `%s`). If you want to print a single literal `%` sign, the C standard requires you to double it (`%%`).

- `printf` reads `A` → prints `A`.
- **`printf` reads `%%` → parses it as a single literal `%` and prints `%`.**
- `printf` reads `B` → prints `B`.

**Output so far: A%B**

### 3. The Second `printf` Statement

```
printf("%s\n", str);
```

Here, the format string is just "%s\n".

- The %s tells printf: "Grab the string argument provided next and print its raw characters exactly as they are."
- The argument provided is str, which contains the literal characters A%B.

Since printf is just dumping the raw contents of str into the %s slot without parsing str's internal characters for format specifiers, it prints all the characters exactly as they sit in memory.

- **Output for this step: A%B**

---

**172. Which of the following typecasting is accepted by C?**

- a. Widening Conversions
- b. Narrowing Conversions
- c. Widening & Narrowing Conversions ✓
- d. None of the above

**1. Widening Conversions (Implicit):** This occurs automatically when a smaller data type is converted into a larger data type (e.g., converting an `int` to a `float` or a `double`). Because the destination type has a larger memory capacity, this conversion is completely safe and happens without any data loss.

**2. Narrowing Conversions (Explicit):** This happens when a larger data type is deliberately converted into a smaller data type (e.g., casting a `float` to an `int`). It usually requires manual intervention using a casting operator, like `(int)my_float`, because it truncates values and can result in data loss.

---

**173. What is the output of the C code?**

```
#include <stdio.h>

int main()
{
    int arr[5] = {1,2,3,4,5};
    int p, q, r;
    p = ++arr[1];
    q = arr[1]++;
```

```

    r = arr[p++];
    printf("%d, %d, %d", p, q, r);
    return 0;
}

```

- a. 4, 3, 4 ✓
- b. 3, 4, 5
- c. 3, 4, 4
- d. 4, 4, 5

```
int arr[5] = {1, 2, 3, 4, 5};
```

Remember that array indexing in C starts at 0:

- arr[0] = 1
- arr[1] = 2
- arr[2] = 3
- arr[3] = 4
- arr[4] = 5

### Step-by-Step Execution

**Step 1: `p = ++arr[1];`**

- This uses a **pre-increment** on arr[1].
- arr[1] is currently 2. Pre-increment means *"increment the value first, then use it."*
- arr[1] becomes 3.
- arr[1] = 2 → 3
- This new value (3) is assigned to p.
- **Current State:** p = 3, arr[1] = 3

**Step 2: `q = arr[1]++;`**

- This uses a **post-increment** on arr[1].
- arr[1] is currently 3 (from the previous step). Post-increment means *"use the current value first, then increment it."*
- The current value 3 is assigned to q.
- Then, arr[1] is incremented from 3 to 4.
- arr[1] = 3 → 4
- **Current State:** q = 3, arr[1] = 4

**Step 3: `r = arr[p++];`**

- This uses a **post-increment** on `p` inside the array index bracket.
- `p` is currently 3 (from Step 1). Because it is a post-increment, we use the current value of `p` to index the array *first*.
- So, it evaluates to: `r = arr[3];`
- Looking at our array, `arr[3] = 4`. So `r` becomes 4.
- *After* evaluating the array line, `p` is incremented by 1 (moving from 3 to 4).
- `p = 3 → 4`
- **Current State:** `p = 4, q = 3, r = 4`

### The final `printf`

```
printf("%d, %d, %d", p, q, r);
```

Printing out our final values:

- `p` is **4**
- `q` is **3**
- `r` is **4**

Output: **4, 3, 4**

---